

Programmable Logic

Programmable Logic

Digital design in VHDL, from a gate network to a CAN node in an FPGA

Erik Pihl

Programmable Logic

Copyright © 2026 QRTECH Academy.

This is the English edition of *Programmerbar logik*. The book is typeset from the course's own material: twenty lectures with their appendices, their exercises, the worked examples, the group project's specification and the two documents that form the contract with the driver course. The code in the book, and all the code in the accompanying course repository, may be used under the MIT licence, which is the repository's licence:

<https://opensource.org/licenses/MIT>

The course repository holds the lecture material, the worked examples, every testbench handed out, the build scripts and the Python sources behind the figures. Every path the book refers to, for example `lectures/L06/serial_rx8/serial_rx8.vhd`, is a path in that repository.

The VHDL code in the book is VHDL-93, simulated with GHDL and synthesised with Quartus Prime Lite. The C++ code in section 10.1 is C++17, compiled with GCC. DE0-CV, Cyclone V, Quartus, USB-Blaster, AVR and AVR32DB28 are trademarks belonging to their respective owners. CAN is standardised in ISO 11898; the controller this book builds is a deliberately simplified teaching design and no certifiable implementation, and section 19.7 lists the differences.

Text and figures are in English. So are all the code, all the comments in the code and all the output from tools and testbenches, exactly as in the course material.

Typeset in TeX Gyre Pagella and DejaVu Sans Mono with Lua^ATeX.

This edition: 13 September 2026.

Contents

Preface

xiii

I Digital design in VHDL

1	Combinational logic and first VHDL	3
1.1	Digital signals and logic gates	3
1.2	Boolean algebra as notation	4
1.3	Building and simulating a circuit in CircuitVerse	5
1.4	Exactly enough VHDL that you can read and write a gate network . .	6
1.5	The complete module: <code>or_gate</code>	9
1.6	A brake assistant, from requirement to circuit	10
1.7	Summary.	12
1.8	Exercises	13
2	Larger networks, multiplexers and submodules	21
2.1	Karnaugh maps: from truth table to minimised gate network	21
2.2	Translating the Karnaugh map's network to VHDL.	25
2.3	<code>std_logic_vector</code> : more than one wire.	26
2.4	Multiplexers	28
2.5	Multiplexers in VHDL: process and case.	30
2.6	Building a design out of submodules	33
2.7	Verifying your VHDL with a testbench	34
2.8	Summary.	38
2.9	Exercises	39
3	Sequential networks	51
3.1	From combinational to sequential networks	51
3.2	The D latch.	52
3.3	The D flip-flop.	53
3.4	Clocking	55

3.5	Registers: flip-flops in parallel	56
3.6	The synchronous process template in VHDL.	56
3.7	Edge detection.	58
3.8	Worked example: a D flip-flop in VHDL	59
3.9	Worked example: edge-detected LED toggle.	60
3.10	Summary	62
3.11	Exercises	63
4	Metastability and synchronisation	69
4.1	The problem: a button is not a testbench signal	69
4.2	Metastability: what actually happens inside the flip-flop	70
4.3	The two-flip-flop synchroniser	71
4.4	Intuition for the timing: why “almost certainly” will do	73
4.5	Synchronising a reset signal: assert asynchronously, release synchronously 73	
4.6	Reusing the chain for edge detection, and debouncing in passing	74
4.7	What a real hardware debounce adds	75
4.8	Generics: one module, several sizes	76
4.9	Worked example: <code>led_toggle_sync</code>	77
4.10	Summary	80
4.11	Exercises	81
5	Variables and the hardware underneath	93
5.1	What is left.	93
5.2	variable: process-local storage that updates immediately.	94
5.3	What the FPGA actually consists of	97
5.4	Why a clock has a speed limit.	98
5.5	The latch you get by accident	99
5.6	Summary	99
5.7	Exercises	100
6	Counters and shift registers	109
6.1	From register to counter	109
6.2	Shift registers	112
6.3	Serial in, parallel out (SIPO)	113
6.4	Parallel in, serial out (PISO).	114
6.5	Worked example: an 8-bit serial receiver	114
6.6	On the board: watching a counter wrap.	118
6.7	On the board: shifting in a byte by hand	118
6.8	Summary	119

6.9	Exercises	121
7	Timers	129
7.1	Timers: a counter with a target value	129
7.2	Building a timer by hand in CircuitVerse	130
7.3	The timer module in VHDL	132
7.4	Composing a complete timer circuit	133
7.5	walking_led: a shift register with a walking LED	136
7.6	Summary	138
7.7	Exercises	139
8	State machines	149
8.1	What a finite state machine is	149
8.2	Designing a Moore machine by hand	150
8.3	From gate network to CircuitVerse	153
8.4	The same machine in VHDL	154
8.5	Mealy machines: the one-clock-cycle difference	160
8.6	On the board	163
8.7	Summary	164
8.8	Exercises	166
9	Practical exam 1: sequential networks	183
9.1	Format	183
9.2	Permitted aids	184
9.3	Before the exam	184
9.4	What the exam measures	184
9.5	After the exam	185
II	The group project: a CAN node	
10	Project start: the CAN bus, the frame and can_def	189
10.1	Frames and serialisation	190
10.2	Introduction to CAN	196
10.3	Broadcast, not addressing	198
10.4	The CAN frame format	199
10.5	Bit stuffing	201
10.6	Arbitration, worked through in full	202
10.7	Bit timing and the sample point	202
10.8	The can_def package	203
10.9	Summary	205

10.10	Exercises	206
11	Architecture, the top level and the register map	217
11.1	The controller's architecture.	217
11.2	Following a frame through the blocks.	219
11.3	The controller's interface.	221
11.4	Writing <code>can_controller.vhd</code>	223
11.5	What the controller synchronises, and why that is the controller's job.	225
11.6	A first look at the register map	226
11.7	Summary	226
11.8	Exercises	227
12	Synchronisation and the bit timer	231
12.1	Designing <code>meta_prev</code>	231
12.2	Designing <code>bit_timer</code>	234
12.3	Summary	237
12.4	Exercises	239
13	The CRC-15 engine	243
13.1	Interface	243
13.2	The recursion	244
13.3	One engine, two jobs	245
13.4	A worked example, by hand	246
13.5	What the testbench pins down	246
13.6	Putting it into <code>can_controller</code>	246
13.7	Summary	247
13.8	Exercises	248
14	Transmit shift register and bit stuffing	251
14.1	Interface	251
14.2	What the module has to remember	253
14.3	The process, top to bottom	254
14.4	The outputs' timing, the part the testbench pins down exactly	258
14.5	One group, presentation by presentation	258
14.6	What the testbench pins down, and putting the module in	259
14.7	Summary	260
14.8	Exercises	261
15	Receive shift register and destuffing	265
15.1	Interface	265
15.2	What the module has to remember	267

15.3	The process, top to bottom	268
15.4	One group, sample by sample	271
15.5	What the testbench pins down	272
15.6	Putting it into <code>can_controller</code>	273
15.7	Summary	273
15.8	Exercises	275
16	The CAN controller I: the state machine and the transmit path	277
16.1	One node, two roles	278
16.2	The frame, field by field	280
16.3	What you add to the architecture.	282
16.4	The combinational process	284
16.5	The sequential process	287
16.6	Capturing the CRC	292
16.7	Feeding <code>crc15</code>	293
16.8	Clearing <code>crc15</code>	293
16.9	Summary	294
16.10	Exercises	295
17	The CAN controller II: reception and arbitration	299
17.1	Reading the bus back.	299
17.2	The receive path, state by state	300
17.3	A clean slate between frames	302
17.4	Feeding <code>crc15</code> on the receive side	303
17.5	Waiting for one extra sample: the decisive timing lesson.	304
17.6	Detecting lost arbitration	305
17.7	Completing the processes	306
17.8	Verifying it: <code>can_controller_tb</code>	312
17.9	Summary	313
17.10	Exercises	314
18	The register bank and SPI from the waveform	317
18.1	Why this layer exists	317
18.2	The interface	318
18.3	The three STATUS bits: one latch each	319
18.4	<code>TX_SEND</code> : a write becomes a pulse.	320
18.5	<code>TX_ABORT</code> : the escape hatch	320
18.6	The RX side: capturing at the pulse	321
18.7	Masking on write	321
18.8	Writing <code>register_bank.vhd</code>	322

18.9	SPI from the waveform upwards	322
18.10	Summary	328
18.11	Exercises	329
19	The SPI bridge, can_spi_node and bring-up	333
19.1	Verifying the whole system	334
19.2	The transaction bridge spi_reg_bridge	336
19.3	Designing can_spi_node	338
19.4	Getting the bus out onto a wire	340
19.5	Optional: a board-level wrapper for debugging	340
19.6	Bring-up	342
19.7	How production CAN controllers go further	344
19.8	The design, seen in the rear-view mirror	345
19.9	Summary	347
19.10	Exercises	348
20	Practical exam 2: CAN modules	353
20.1	Why an individual exam on a group project	353
20.2	Format	354
20.3	Permitted aids	355
20.4	Before the exam	355
20.5	What the exam measures	355
20.6	After the course	356
A	Simulation with GHDL	357
A.1	Why simulate	357
A.2	Installing GHDL	357
A.3	Where the files live	358
A.4	The three GHDL steps	358
A.5	Building everything at once	360
A.6	Cleaning up	361
B	Quartus Prime Lite and the DE0-CV	363
B.1	What you need	363
B.2	Installing Quartus Prime Lite	364
B.3	Creating a new project	365
B.4	Adding your VHDL	366
B.5	Assigning pins in the Pin Planner	366
B.6	Compiling the design	367
B.7	Programming the DE0-CV	367

B.8	Troubleshooting	368
B.9	Programming from Linux and from WSL.	368
C	The group project	371
C.1	Way of working	371
C.2	How the project is run	372
C.3	Project structure.	373
C.4	Naming conventions and coding style.	373
C.5	The shared package <code>can_def</code>	374
C.6	The modules in <code>controller/</code>	375
C.7	The modules in <code>bridge/</code>	379
C.8	Test plan	380
C.9	Examination and grades	381
C.10	Practical tips	383
D	The register map and the SPI protocol	385
D.1	The registers	386
D.2	<code>STATUS</code>	386
D.3	Semantics, register by register	387
D.4	Data byte order	388
D.5	Counterparts in the hardware	389
D.6	Electrical parameters and framing	389
D.7	Transactions	391
D.8	Timing requirements	391
D.9	Worked example	393
E	Further reading	395
E.1	The basics	395
E.2	The protocol reference	396
E.3	Standards and specifications	397
E.4	The tools	397
E.5	Further into digital design.	398

Preface

This book takes you from a gate network on paper to a CAN node in an FPGA that a microcontroller can talk to over SPI.

The first half builds up VHDL and synchronous design from the ground: combinational networks, sequential networks, metastability and synchronisation, counters, shift registers, timers and state machines, each one first reasoned about as a *circuit* and only then expressed in the language. The second half is a group project where that knowledge is put to something larger: a simplified CAN controller, designed module by module, and the two layers that make it reachable from the outside — a register bank and an SPI bridge.

The subject is VHDL and digital design in an FPGA, not the fundamentals of digital electronics. Gates, truth tables, Boolean algebra and Karnaugh maps are revised rather than taught, and they are revised where the language calls for them. What the book teaches from zero is the language and the toolchain.

What you are assumed to bring

- Logic gates and truth tables, and reading a sum of products out of a table.
- Boolean algebra: the basic identities and De Morgan's laws.
- Karnaugh maps, for functions of three or four variables.
- Registers, D flip-flops, and what a clock edge does.
- State machines as a concept: states, transitions, and the difference between Moore and Mealy.

If any of it needs brushing up, all of it is in *Digital Electronics* (<https://github.com/qrt ech-academy/digital-electronics>), a freely available book that goes from a single gate to a state machine: gates and Boolean algebra in chapters 1 and 2, Karnaugh maps in chapter 3, flip-flops and registers in chapter 5 and state machines in chapter 6. The chapters of this book point there wherever they revise the basics, and section E.1 has the whole map.

No programming language beyond general software experience is assumed. Familiarity with C or something like it helps, since several explanations set VHDL's semantics against it, and the one section that actually contains C++ (section 10.1) is a short introduction to framing rather than an exercise in the language.

The group project adds **Git and GitHub** at a basic level: clone, branch, commit, push and pull request.

What you should be able to do afterwards

- Realise a Boolean function as a gate network, and simulate it by hand.
- Read and write combinational and sequential VHDL: entities, architectures, signals, variables and processes.
- Explain metastability and apply the two-flip-flop synchroniser to every asynchronous input.
- Design counters, shift registers, timers and state machines, and compose a design out of modules you have already built rather than rewriting their logic.
- Design the hardware of a communication protocol: bit timing, checksums, bit stuffing, frame sequencing and arbitration.
- Design the interface between hardware and software: a register bank and an SPI transport, against a contract someone else writes to.
- Verify a design by running a self-checking testbench under GHDL, and trace a failure back to the code that caused it.
- Work in a group with feature branches, pull requests and code review.

How the book is laid out

The book is typeset from a course of twenty sessions, and **every chapter is one session**. The chapter's sections are the session's appendices, in the same order and under the same subheadings, so that a chapter here and a lecture directory in the course repository can be read side by side. The two parts match the course's two halves:

Chapter	Session	Subject
<i>Part I: digital design in VHDL</i>		
1	L01	Combinational logic and first VHDL
2	L02	Larger networks, multiplexers and submodules
3	L03	Sequential networks
4	L04	Metastability and synchronisation
5	L05	Variables and the hardware underneath
6	L06	Counters and shift registers
7	L07	Timers
8	L08	State machines
9	L09	Practical exam 1: sequential networks
<i>Part II: the group project, a CAN node</i>		
10	L10	Project start: the CAN bus, the frame and <code>can_def</code>
11	L11	Architecture, the top level and the register map
12	L12	Synchronisation and the bit timer
13	L13	The CRC-15 engine
14	L14	Transmit shift register and bit stuffing
15	L15	Receive shift register and destuffing
16	L16	The CAN controller I: the state machine and the transmit path
17	L17	The CAN controller II: reception and arbitration
18	L18	The register bank and SPI from the waveform
19	L19	The SPI bridge, <code>can_spi_node</code> and bring-up
20	L20	Practical exam 2: CAN modules

Every chapter opens with what is in it, works its way through the material, closes with a review of what you should be able to explain afterwards, and ends with its exercises. The appendices hold the tool references, the project specification, the shared contract with the driver course, and a closing note on where to read next.

Chapters 9 and 20 are the two examinations. They contain no new theory; they say what the exam measures, what shape the tasks take and how to prepare, and they are worth reading in advance rather than the day before.

One more book belongs to part II. This book describes how to *build* a CAN node; the protocol itself is described in *CAN: the bus, the frame and the controller*, a short book written for both halves of a CAN system, in English and in Swedish with the same chapter numbering. It is the protocol reference throughout the second half: its chapters 1–6 are the requirement specification for what is built here, bit by bit, and its chapter 7 describes exactly the simplified controller part II arrives at. Every chapter that adds a protocol rule says where in that book the rule is stated, and section E.2 has the full chapter map between the two. Its chapters 1 and 2 are worth reading before chapter 10; the rest works best as a reference.

Three layers of verification

The course checks its work at three levels, and keeping them apart is a point in itself:

Layer	What it checks
By hand	In CircuitVerse, before any VHDL exists. You derive what the circuit should do, build it, and simulate it against your own prediction. Nothing checks that step automatically, deliberately: predicting a behaviour and then confirming it is the skill the rest of the book rests on.
In the exercises	You write the VHDL and run a testbench handed out with it, self-checking, under GHDL. No FPGA board is needed, and nearly everything can be verified on an ordinary laptop.
On the board	The design is synthesised in Quartus Prime Lite and run on a Terasic DE0-CV. During the first half that step is demonstrated; in the group project's last session you do it yourself.

Section 2.7 is the full introduction to testbenches, and appendix A is the reference you will come back to. Appendix B is the Quartus flow.

What the exercises look like

The exercises come after every chapter, numbered within the chapter, and each one is tagged with its kind:

Kind	What it asks of you
CIRCUIT	Draw, build and simulate a circuit by hand in CircuitVerse, and check it against your own truth table or your own timing diagram.
VHDL	Write a module, with the entity name and the port order the exercise states, and run the testbench handed out with it.
SIMULATION	Change something and find out what GHDL reports. Several of them ask you to <i>predict</i> the outcome first and then run.
REASONING	Answer in prose or in numbers: what the code does, why one construction is better than another, or what a number works out to.
C++	The course's only C++ exercise, in chapter 10.

There are no printed solutions, and that is deliberate. Every exercise that asks you to write a VHDL module comes with a self-checking testbench, and that testbench is the answer key: it drives your module through every case and says exactly which check failed. The pen-and-paper exercises you check against the truth table or the timing diagram you derived yourself, which is the whole point of them. A list of answers would remove precisely the step the exercise exists for.

The port order is the contract. Every testbench handed out binds to the module it drives **positionally**, and so does every instantiation in the course. Nothing binds a port by name. The contract is therefore the *order* and the *types* of the ports, exactly as every exercise's interface table lists them, top to bottom. The *names* of the ports are yours.

That bargain is worth understanding rather than merely following. Positional binding is shorter to write and to read, and it makes the interface table the single source of truth. What it gives up is the compiler's help: swap two ports of the same type and nothing complains during analysis, the design simply misbehaves in simulation instead. The book keeps its port names all the same, so that the text and your code speak about the same signals.

Setting up the tools

Everything in this book runs in a Linux terminal: on Linux, or on Windows through WSL. Three things cover everything that is not a board:

- **GHDL**, the open-source VHDL analyser and simulator. It is the tool the course runs on: every module is verified against a testbench in GHDL, and nothing you are asked to produce needs anything else.
- **CircuitVerse**, a free logic simulator that runs in the browser (<https://circuitverse.org/simulator>). Nothing to install.
- **Git**, for the group project, plus g++ for chapter 10's framing exercise.

On WSL or Ubuntu:

```
sudo apt -y update
sudo apt -y install git make ghdl g++
ghdl --version           # 3.x or later
```

The course repository has the test framework as a submodule, so clone it with the submodules:

```
git clone --recurse-submodules <repo-url>
git submodule update --init      # if it is already cloned without them
```

Two commands at the root of the repository cover most of what the book asks for:

```
make build                # Build and simulate everything there is: examples, ↵
                           ↵ exercises, project.
make lint                 # Markdown links, duplicate check, no trailing whitespace↵
                           ↵ .
```

`make build-project` reports every project testbench as skipped in a freshly checked out repository. That is the expected state: `controller/` and `bridge/` hold only the files handed out, and the modules are the group's to write in its own repository. A testbench whose modules are missing is skipped, never failed.

Quartus Prime Lite and a DE0-CV board are recommended but not necessary. No exercise and no assessed element requires them; appendix B describes the flow for those

who want it, and for the group project's last session, where every group brings up its own node on hardware handed out to them.

Where to read next

The node built in part [II](#) is one half of a system. The other half, the C++ code that talks to it over SPI, is written in another course, against the same register map and the same protocol specification — and against the same protocol book, which is shared between the two. [Appendix E](#) describes it, the standards the design relates to, and four directions to take the subject further.

Part I

Digital design in VHDL

Combinational logic and first VHDL

From a truth table to a working gate network, in CircuitVerse and in VHDL.

IN THIS CHAPTER

- Logic gates and truth tables, and why digital logic tolerates noise that analogue does not.
- Boolean algebra: reading a sum of products straight out of a truth table.
- Building and simulating a gate network by hand in CircuitVerse.
- VHDL from the very first session: entity, architecture, `std_logic` and the concurrent assignment.
- A complete module, `or_gate`, from truth table to synthesisable file.
- A brake assistant, derived from a requirement and demonstrated on a DE0-CV board.

This chapter walks the path every circuit in the book walks: from a requirement, by way of a truth table and a Boolean equation, to a gate network built and simulated by hand, and only then to VHDL. The order is not a pedagogical detour. It is the working method the rest of the book rests on, and it is easiest to see on a circuit small enough to take in whole.

Work the chapter in order. Section 1.1 to 1.3 are the circuit; section 1.4 and 1.5 are exactly enough VHDL that you can read and write a gate network; section 1.6 is the requirement the session builds live, and it is worth arriving there with an answer of your own to compare against. The exercises in section 1.8 are where you write the code yourself.

Refresher: The gates, the truth tables and the Boolean algebra come with you from digital electronics. If they need brushing up, they are in sections 1.1–1.4 and 2.1–2.7 of *Digital Electronics*, with exercises and answers, and its appendix B describes CircuitVerse.

1.1 Digital signals and logic gates

A digital signal takes one of two values, 0 or 1, instead of the continuous range an analogue signal uses. Anything close enough to 0 or 1 reads as exactly 0 or 1, so small amounts

of electrical noise change nothing. That noise immunity is the reason nearly all modern computation, from microcontrollers to FPGAs, is digital.

A **logic gate** has one or more single-bit inputs, a single-bit output and a fixed Boolean function between them.

Combinational logic, this chapter's subject, produces outputs from the current inputs alone, with no memory. **Sequential networks** (chapter 3) depend on the current inputs *and* on state stored in flip-flops.

Every basic two-input gate, plus the unary NOT:

A	B	AND	OR	NAND	NOR	XOR	XNOR
0	0	0	0	1	1	0	1
0	1	0	1	1	0	1	0
1	0	0	1	1	0	1	0
1	1	1	1	0	0	0	1

A	NOT A
0	1
1	0

Worth committing to memory straight from the tables:

- NAND, NOR and XNOR are the inverses of AND, OR and XOR.
- XOR is 1 when the inputs differ. Beyond two inputs it generalises to **odd parity**: 1 when an odd number of inputs are 1 (Exercise 1.3).
- Every Boolean function can be built from AND, OR and NOT, and also from NAND alone or NOR alone.

1.2 Boolean algebra as notation

Three operators, which map straight onto the gates above:

Gate	Notation in Boolean algebra
AND	multiplication: A AND B is written AB
OR	addition: A OR B is written $A + B$
NOT	prime: NOT A is written A'

A function written as an OR of AND terms, for example $X = AB' + CD$, is in **sum-of-products** form.

Every truth table can be read off directly as a sum of products:

1. Find every row where the output is 1.

2. For each of them, write an AND term containing every input: plain if the input is 1 on that row, primed if it is 0.
3. OR those terms together.

For example:

A	B	X
0	0	0
0	1	1
1	0	1
1	1	0

The row $AB = 01$ contributes $A'B$ and the row $AB = 10$ contributes AB' , which gives $X = A'B + AB'$. Check it against the table: that is XOR, so the same network is a single XOR gate instead of two AND gates, two NOT gates and an OR.

The laws, as a reminder

You have met them before. They are here so that the book has a single notation for them and so that later chapters can refer to them, not because they need teaching:

Law	With OR	With AND
Identity	$A + 0 = A$	$A \cdot 1 = A$
Dominance	$A + 1 = 1$	$A \cdot 0 = 0$
Idempotence	$A + A = A$	$A \cdot A = A$
Complement	$A + A' = 1$	$A \cdot A' = 0$
Absorption	$A + AB = A$	$A(A + B) = A$
De Morgan	$(A + B)' = A'B'$	$(AB)' = A' + B'$

De Morgan is the law this book leans on, less for minimisation than as a rule for how an inversion moves through a gate: an OR with both inputs inverted *is* a NAND, and an AND with both inputs inverted *is* a NOR. That is why a NAND gate can build anything, which Exercise 1.6 lets you write in VHDL, and it comes back every time an active-low signal meets logic written for active-high. In a course where every reset and every button is active-low, that is often.

The gap between a directly read sum of products and the smallest network is what a **Karnaugh map** closes, and section 2.1 works through one from start to finish. Not because the technique is new to you, but because it is the minimised equation that gets written in VHDL, and chapter 8 derives its state machine's next-state logic in exactly that way.

1.3 Building and simulating a circuit in CircuitVerse

CircuitVerse (<https://circuitverse.org/simulator>) is a free, browser-based logic simulator. Every gate network in this book is built and tested there by hand before it is written in VHDL:

1. Open the simulator and start a new project.
2. Place one **input** element per input signal and one **output** element per output signal, renamed to match your variables.
3. Place the gates your equation calls for, from the *Gates* panel.
4. Wire the network up one gate at a time, following your derived equation. You should already know exactly which gates you need before you open the simulator, which is why section 1.2 comes first.
5. Toggle every input and check that the output matches your truth table on **every** row.
6. Save with Project -> Save Online, or Project -> Download as image/data for a re-importable .cv file.

Step 5 is not optional, and nothing else does it for you. Three rules make it a check rather than a formality:

- **Exhaustive, not sampled.** Two inputs are four rows, three are eight. At those sizes there is no excuse for sampling. For the sequential networks further on this becomes “step the clock and check at every edge”, the same discipline applied to time.
- **Derive first, simulate second.** Draw the circuit and *then* decide what it should do, and you will read your own expectations out of the simulation, which proves nothing.
- **A discrepancy is information.** It says the drawing and the equation disagree, and that one of them is wrong.

Every hardware bug you will chase is found by predicting a behaviour and then checking it. Doing it by hand on a circuit with four gates is how you learn to do it on one you cannot take in at a glance.

Once the design is VHDL, a self-checking testbench takes that job over. Every worked example and every exercise has one handed out with it; you run them rather than write them, which section 2.7 works through.

1.4 Exactly enough VHDL that you can read and write a gate network

VHDL describes hardware, not a sequence of instructions. Every statement below runs concurrently and continuously, exactly like the gates it describes. The language arrives as it is needed; this section is enough for a simple combinational network.

Every VHDL module has two parts:

- An **entity**, the view from outside: the ports, and nothing about what happens inside.
- An **architecture**, the implementation behind it.

That split is a contract, not ceremony. The entity is everything another design needs in order to use the module, which is what lets one design instantiate another without reading it, and what lets you rewrite an implementation without touching what depends on it.

The shape is a header beside a source file, or an interface beside its class; the difference is that VHDL requires it for every module, with no way to skip the interface or to leak the implementation through it.

Take `or_gate`: the inputs `a`, `b`, the output `x`, and $x = a + b$.

```
entity or_gate is
    port(a, b: in std_logic;
          x  : out std_logic);
end entity;
```

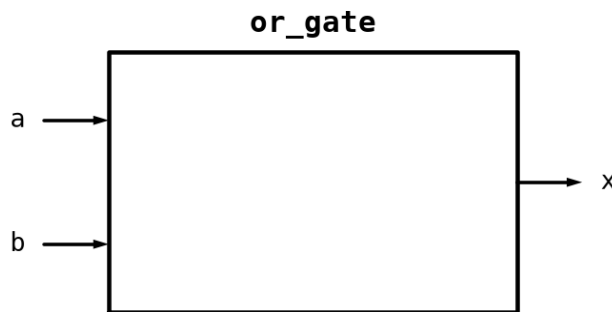


Figure 1.1 The entity `or_gate`: the view from outside, and nothing about what happens inside.

`std_logic` is the signal type for a single bit. It is not built into VHDL but comes from the package `std_logic_1164`, which is why every file in this book opens with:

```
library ieee;
use ieee.std_logic_1164.all;
```

It has nine values instead of two, because real hardware needs more than true and false:

- '0' and '1': driven low and driven high. The only two you will write in this book.
- 'Z': high impedance, nothing is driving the wire at all.
- 'U': uninitialised, what a signal holds before anything has assigned it a value.
- 'X': unknown, what a simulator shows when two things drive the same signal and disagree.
- '-', 'W', 'L', 'H': don't-care values and weakly driving values, rarely written by hand.

Recognising the seven you will not write still matters: they are how a simulator tells you something is wrong. A signal left sitting at 'U' or 'X' is a bug, not a value.

The architecture fills the box in:

```

architecture behaviour of or_gate is
begin
    x <= a or b;
end architecture;

```

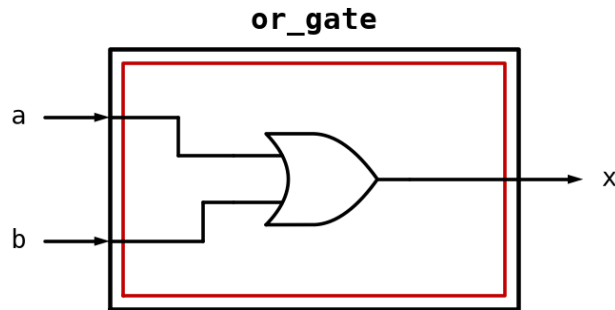


Figure 1.2 The architecture or_gate: the implementation behind the entity.

Together they make up the complete module:

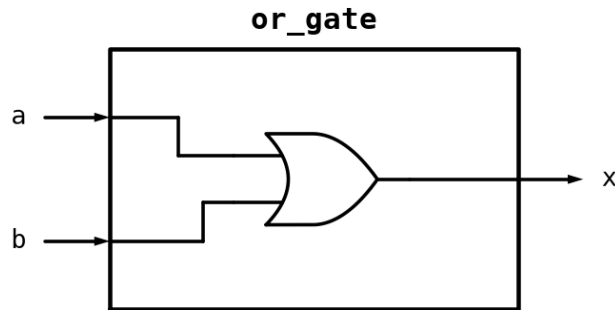


Figure 1.3 The module or_gate: entity and architecture together.

$x \leq a \text{ or } b$; is section 1.2's Boolean algebra almost verbatim: VHDL's and, or, not and xor work directly on `std_logic`, so every equation you derived by hand becomes VHDL with hardly any translation at all.

The assignment is **concurrent**. It does not run once; it holds continuously, like a wire between real gates.

The keyword `signal` declares an internal wire inside an architecture, as opposed to a port visible from outside. You use one in Exercise 1.7 to name an intermediate result instead of nesting it in place, and chapter 2 leans on the same idea throughout.

1.5 The complete module: or_gate

Section 1.4's parts together make a complete, synthesisable file. This is the whole of `or_gate.vhd`, and it is a real design that synthesises and runs on the DE0-CV:

```
lectures/L01/or_gate/or_gate.vhd

-----
-- Two-input OR gate: x = a or b.
--
-- Inputs:
--     - a, b: OR inputs.
--
-- Outputs:
--     - x: OR output.
-----

library ieee;
use ieee.std_logic_1164.all;

entity or_gate is
    port(a, b: in std_logic;
         x : out std_logic);
end entity;

architecture behaviour of or_gate is
begin
    x <= a or b;
end architecture;
```

- The library/use clauses pull in `std_logic`.
- entity declares the outside: two inputs, one output.
- architecture implements the inside, as a single concurrent assignment.

That is the smallest complete VHDL design there is, and the shape never changes. Every module in this book, all the way up to the CAN controller in chapter 17, is the same library/entity/architecture skeleton. What grows is the architecture body, never the frame.

Checking your work. `or_gate` comes with an `or_gate_tb.vhd`, which drives all four input combinations and checks the output:

```
cd lectures/L01/or_gate
ghdl -a --std=93 or_gate.vhd or_gate_tb.vhd
ghdl -e --std=93 or_gate_tb
ghdl -r --std=93 or_gate_tb --assert-level=error --stop-time=10ms
```

No output beyond a closing note means every check passed. Testbenches are treated properly in section 2.7; until then, take these three commands as the way you confirm that a module works.

1.6 A brake assistant, from requirement to circuit

The session builds a different circuit from `or_gate`, live, so that you get to see the same path walked twice: once here in writing, and once on something you have not already read the answer to.

The requirement: **the car brakes if the driver brakes, or if the assistance system decides to**. The assistance system decides to when either detector sees something, unless it is at the same time reporting a fault, in which case it is not trusted to brake at all.

Port	Direc- tion	Meaning
driver_brake	in	The driver is on the brake pedal.
sensor	in	The distance sensor sees an obstacle.
radar	in	The radar sees an obstacle.
error	in	The assistance system reports a fault.
engine_brake	out	Apply the brakes.

Three sentences together determine the whole structure:

- either detector is enough on its own, and neither of them outweighs the other.
- a fault raises no alarm; it *prevents* the assistance system from being able to brake.
- the driver can always brake, even when the assistance system is broken and doing nothing.

The last is a safety requirement, and it is the one to hold on to as you derive the network: the pedal is not an input to the assistance logic, it goes past it. Work out what changes if the pedal is instead fed through the fault logic, and you have found the bug the structure exists to prevent.

The truth table

Sixteen rows, in the order a four-bit counter would produce them. `adas_brake` is not a port; it is the assistance system's own decision, shown as an intermediate column so that you can see where the structure comes from.

driver_brake	sensor	radar	error	adas_brake	engine_brake
0	0	0	0	0	0
0	0	0	1	0	0
0	0	1	0	1	1
0	0	1	1	0	0
0	1	0	0	1	1
0	1	0	1	0	0
0	1	1	0	1	1
0	1	1	1	0	0
1	0	0	0	0	1
1	0	0	1	0	1
1	0	1	0	1	1
1	0	1	1	0	1
1	1	0	0	1	1
1	1	0	1	0	1
1	1	1	0	1	1
1	1	1	1	0	1

The lower half is 1 throughout: as soon as `driver_brake` is set, nothing else changes the answer. That is what “overrides everything” looks like in a table.

Four answers to bring to the session

Everything above is the specification. The derivation, the gate network and the VHDL code are what gets built live, and it is worth arriving there with an answer of your own to compare against rather than one you have already read. So, from the table above:

- Read a sum-of-products equation for `engine_brake` straight out of it, the same way section 1.2 did.
- Simplify it by hand. The requirement is three sentences long, so the minimal network is considerably smaller than what the row-by-row reading gives you. Work out how much smaller before you are told.
- Count the gates your simplified equation needs, and write the number down.
- Sketch the network, and decide which single gate the safety argument above rests on.

Bring those four answers to the session. Being wrong about one of them is worth more than not having guessed, because you then know exactly which step to look at again when the live derivation reaches it. It is section 1.3’s “derive first, simulate second” applied to a circuit you have not built yet, and it is the habit the rest of the book is built on. Afterwards you write the VHDL code yourself, in Exercise 1.7, and a testbench decides whether your equations and the truth table above agree.

Why there is a testbench for it

Sixteen combinations are few enough to check by hand once. They are not few enough to check every time someone edits the file, which is the situation every module in this book is in from here on.

A **testbench** is a second VHDL file that does the checking. It is not part of the design and never reaches the FPGA: it drives the inputs through every case, works out what the output should have been, compares and reports. For *adas*, which is written alongside it during the session, that is a loop over the sixteen combinations restating the braking rule independently.

Independently is the important word. A testbench that read its expected answer out of the design would agree with it by construction, even when the design is wrong. Restating the requirement separately is what makes it a check.

That is why nearly every directory in the course repository has a `_tb.vhd` beside the module: it is how you can be handed a module, change it, and know within a second whether you have broken it. Section 2.7 works through how they are run; you are not asked to write one anywhere in this book.

1.7 Summary

What you should be able to do now

- Derive a sum of products from a truth table, and then build and simulate the network in CircuitVerse.
- Apply De Morgan to move an inversion through a gate, and use that to realise a function in a gate set that lacks the operator you started from.
- Read and write a combinational VHDL module, an entity with `std_logic` ports and an architecture that drives its output, and say why VHDL keeps the two apart.
- Explain why a concurrent assignment describes a wire rather than a statement that runs once, and why a signal left sitting at 'U' or 'X' is a fault and not a value.
- Run a testbench handed out with an exercise and decide whether your module passed it.

Questions to test yourself with

1. What is the difference between an entity and an architecture, and why does VHDL separate them?
2. Why is `x <= a or b`; not “a statement that runs once”? What does it describe instead?
3. In the brake assistant the driver’s pedal is ORed in last, instead of going through the fault-handling logic. What breaks if you wire it the other way round?

Looking ahead

Chapter 2 goes on with:

- Karnaugh maps: finding the *smallest* gate network, not just a correct one.
- `std_logic_vector`, `process` and `case`.
- Submodules, demonstrated on a two-digit hexadecimal display driven by eight switches.
- Self-checking testbenches: verifying your own VHDL without a board.

1.8 Exercises

How to check your work. Exercise 1.1 is a short pen-and-paper warm-up on the notation the book uses throughout; it needs neither VHDL, GHDL nor a board, and it states what a good answer covers.

From Exercise 1.2 onwards, every exercise that asks you to write a VHDL module comes with a self-checking testbench under `lectures/L01/exercises/`. Write your module in its directory `exercises/<module>/`, with the entity name and the **port order** the exercise states, and then run it with GHDL — see section 2.7 for the three commands.

No FPGA board is needed for any exercise. The Quartus synthesis and board programming steps are demonstrated during the session; your job afterwards is to get the VHDL right, and the testbench is how you confirm it.

EXERCISE SET 1

Truth tables and gates

Exercise 1.1 A function that depends on less than it looks like it does

CIRCUIT

A function of two inputs A, B and one output X has the truth table below:

A	B	X
0	0	1
0	1	0
1	0	1
1	1	0

a) Derive the sum-of-products equation for X straight out of the truth table (section 1.2). Two rows have $X = 1$, so you get two AND terms.

b) Simplify it algebraically. Factor out what the two terms share, and apply $A + A' = 1$ and $1 \cdot B' = B'$ from section 1.2. You should end up with a single literal.

- c) Your answer to b) says something about A. What? Which single gate from the table in section 1.1 computes the function, and what happens to the input A when you draw it?
- d) Build that single-gate version in CircuitVerse and confirm that it reproduces every row of the truth table above. Leave the input A unconnected, or simply leave it out of the drawing; b) is the justification for doing so.

EXERCISE SET 2

Entities and architectures

Exercise 1.2 `and_gate` and `nand_gate`

VHDL

Write two small gates as complete VHDL modules, `and_gate` and `nand_gate`.

Neither gate is the point, and you should not expect either of them to be hard. The point is that this is where an entity and an architecture stop being something you read about in section 1.4 and become something you write, and that the two halves are worth writing in that order.

Both entities have the same ports:

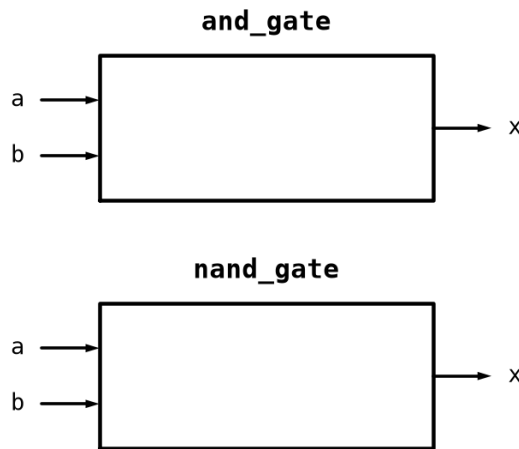
Port	Direction	Type
a	in	<code>std_logic</code>
b	in	<code>std_logic</code>
x	out	<code>std_logic</code>

a) Write the entity declaration for `and_gate`, and nothing else. In VHDL a module's interface is something you can write down and reason about before any implementation exists at all, which is the habit this part exists to build. Identify which ports are required, the direction of each, and the type of each.

b) Add an architecture that implements the AND gate with a single concurrent signal assignment.

c) Now write `nand_gate`: the same interface with an inverted output, using VHDL's `nand` operator directly. Do **not** write `not (a and b)`.

The two are equivalent as Boolean algebra, and on an FPGA they make no difference at all, since both end up in the same lookup table. It matters on the kind of hardware Exercise 1.6 below is about, where NAND is the gate you physically have and everything else must be built from it. Writing what you mean, and letting the tools decide what it costs, is the habit; reaching past an operator the language already gives you is what to avoid.



Self-check. Name your entities `and_gate` and `nand_gate`, each with the ports `a`, `b` (in) and `x` (out), declared in that order; their testbenches are in `exercises/and_gate/` and `exercises/nand_gate/`, and each checks all four rows of the corresponding truth table.

Exercise 1.3 xor3: odd parity with three inputs

VHDL

An XOR gate has three inputs, `a`, `b`, `c`, and one output, `x`. The output is 1 when an odd number of the inputs are 1 (three-input XOR, or the parity function).

- Write the truth table for `x` over all eight combinations of `a`, `b`, `c`, and confirm that it matches the description “an odd number of 1s”.
- Implement the gate in VHDL, as a module of its own with an entity whose ports are three `std_logic` inputs and one `std_logic` output, and an architecture that drives `x` with a single concurrent assignment. Section 1.4 shows the entity/architecture pattern and notes that VHDL’s `xor` operator works directly on `std_logic`; section 1.5 shows a complete module from start to finish.
- Build the same gate in CircuitVerse and confirm that it reproduces every row of your truth table.

Hint: `xor` chains from left to right, so the whole architecture body is a single line, `x <= a xor b xor c;`, with no need for either an internal signal or intermediate gates.



Self-check. Name your VHDL entity `xor3`, with the ports `a`, `b`, `c` (in) and `x` (out), declared in that order; its testbench is in `exercises/xor3/`, and section 1.5 lists the three commands that run it.

Exercise 1.4 majority3: a triple-redundant vote

VHDL

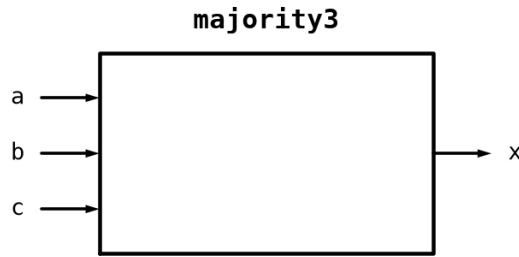
A **majority gate** has three inputs, `a`, `b`, `c`, and one output, `x`. The output is 1 when *at least two* of the three inputs are 1.

This is the circuit behind a triple-redundant vote: three sensors report the same measurement, and the system acts on the answer two of them agree on, so that a single broken sensor cannot decide anything on its own.

- a) Write the truth table for `x` over all eight combinations of `a`, `b`, `c`.
- b) Derive the sum-of-products equation for `x` straight out of the truth table (section 1.2):
 - Four rows have $x = 1$, so you get four AND terms.
 - Every term contains all three inputs, primed where the input is 0 on its row.
- c) Implement the gate in VHDL, as a module of its own:
 - An entity with three `std_logic` inputs and one `std_logic` output.
 - An architecture that drives `x` with a single concurrent assignment, translating your four AND terms directly (section 1.4).
- d) Build the same network in CircuitVerse and confirm that it reproduces every row of your truth table.

Hint: Write the equation out on paper before you open either the editor or CircuitVerse. Four three-input AND terms plus a four-input OR gate is a lot of wiring to correct after the fact.

Looking ahead: Count the gates your equation calls for, and then convince yourself by inspection that $x = ab + ac + bc$ computes the same function with three two-input AND gates instead of four three-input ones. Sum of products gave you a *correct* network, not the *smallest*; finding the smaller one systematically, rather than by staring at it, is what chapter 2 is for.



Self-check. Name your VHDL entity `majority3`, with the ports `a`, `b`, `c` (in) and `x` (out), declared in that order; its testbench is in `exercises/majority3/` and checks all eight rows. Either equation passes — the testbench checks the function, not which network you chose.

Exercise 1.5 `half_adder`: the first module with two outputs

VHDL

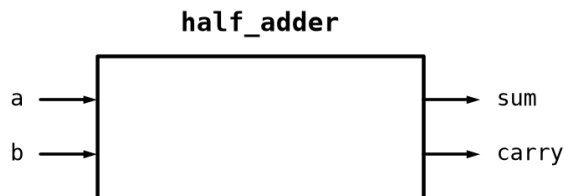
Write a **half adder**: sum and carry from two single bits. You know the circuit; the reason it is here is that it is the book's first module with **two outputs**, and that is a VHDL question rather than a logic question.

a) Implement it, with two concurrent assignments and no internal signal.

Two outputs do not mean two entities, two architectures or a process. Every output port is driven by its own concurrent assignment, and the two are not steps that happen in an order: they describe two separate pieces of wire, both live all the time. If you catch yourself reaching for a process in order to “return” two values, that instinct is the software instinct, and section 1.4's “a concurrent assignment describes a wire” is the correction.

b) Build it in CircuitVerse and confirm both outputs, so that you have seen the two-output shape as a drawing before you meet it as an entity with several out ports in chapter 2.

Looking ahead: This is the cell the counter in chapter 6 is built from. The four-bit adder there is four of these chained carry-to-carry, and the counter is that adder with its output fed back to its input, which is why the wraparound needs no logic at all.



Self-check. Name your entity `half_adder`, with the ports `a`, `b` (in) and `sum`, `carry` (out), declared in that order; its testbench is in `exercises/half_adder/` and checks `sum` and `carry` separately, so that a design that has one right and the other wrong is told which is which.

EXERCISE SET 3

Universal gates

Exercise 1.6 `or_from_nand`

VHDL

Section 1.1 states that every Boolean function can be built from NAND gates alone. This exercise has you prove it to yourself for the three operators you have been using.

a) Derive the OR case on paper before you build anything, since De Morgan (section 1.2) gives it to you directly. Start from $(A'B')'$ and apply the law; you should arrive at $A + B$. Now read that expression back as a circuit: what is $A'B'$ with an inversion on its output, and how many NAND gates is the whole thing?

b) Build each of the following in CircuitVerse from NAND gates alone, checking each against its truth table in section 1.1 as you go:

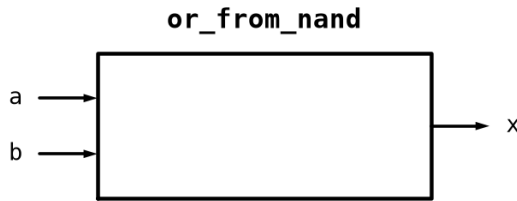
- NOT `a`: one NAND gate is enough, if you feed the same signal to both its inputs.
- `a AND b`: one NAND gate, followed by the NOT you just built.
- `a OR b`: the network you derived in a). Check your gate count against what you predicted.

c) Write the OR version in VHDL, as a module of its own:

- An entity with two `std_logic` inputs and one `std_logic` output.
- An architecture that drives `x` with a single concurrent assignment.
- Use VHDL's `nand` operator alone: no `or`, no `and`, no `not` anywhere in the expression.

d) Explain why this matters in practice. Bear in mind that a chip manufacturing process has to physically implement every gate type it offers, and that NAND is the cheapest gate to build in CMOS.

Hint: VHDL's `nand` operator does not chain the way `and` and `or` do: `a nand b nand c` is not valid. Parenthesise every NAND explicitly, which is what you want here anyway, since every pair of parentheses is a physical gate in your CircuitVerse drawing.



Self-check. Name your VHDL entity `or_from_nand`, with the ports `a`, `b` (in) and `x` (out), declared in that order; its testbench is in `exercises/or_from_nand/` and checks it against all four rows of the OR truth table.

EXERCISE SET 4

Putting it all together

Exercise 1.7 The brake assistant in VHDL

VHDL

Write the brake assistant from section 1.6 in VHDL.

Section 1.6 had you derive its equations, count its gates and sketch its network *before* the session, and the session then built it live. This is where you write it yourself, and where a testbench decides whether your reading of the requirement matches the one in the truth table.

The entity must have:

Port	Dir.	Type	Description
<code>driver_brake</code>	in	<code>std_logic</code>	The driver is on the brake pedal.
<code>sensor</code>	in	<code>std_logic</code>	The distance sensor sees an obstacle.
<code>radar</code>	in	<code>std_logic</code>	The radar sees an obstacle.
<code>error</code>	in	<code>std_logic</code>	The assistance system reports a fault.
<code>engine_brake</code>	out	<code>std_logic</code>	Apply the brakes.

a) Implement it from **your own** equations rather than from what was on the screen. If the two disagree, that disagreement is the most useful thing this exercise can give you, and the testbench says which of them the truth table sides with.

Use two concurrent assignments and one internal signal:

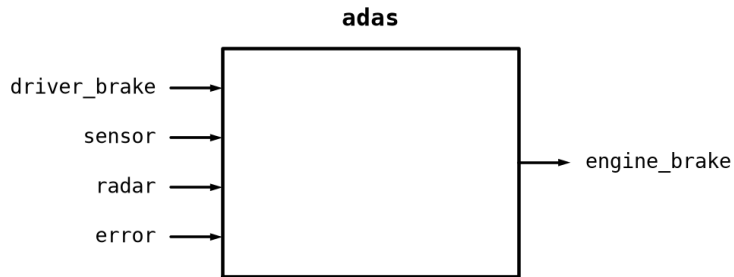
- The signal holds the assistance system's own decision, before the driver's pedal is weighed in.
- This is the book's first exercise with an internal signal, and it is the reason the keyword exists: a name for an intermediate result, so that every line still reads as the sentence in the requirement it came from.

b) Now break it deliberately, in the specific way section 1.6 warns about. Feed `driver_brake` *through* the fault logic instead of letting it go past:

```
engine_brake <= (driver_brake or sensor or radar) and (not error);
```

It is one gate simpler, and it looks reasonable. Run the testbench against it and answer:

- Which input combination does it fail on first, and what happens physically in the car at that moment?
- How many of the sixteen rows does it get wrong, and what do they all have in common?
- The rows that fail are exactly the ones the safety requirement was written for. What does that tell you about the value of a truth table derived from a requirement, as opposed to one read off a circuit you have already built?



Self-check. Name your entity `adas`, with the inputs `driver_brake`, `sensor`, `radar`, `error` and the output `engine_brake`, declared in that order; its testbench is in `exercises/adas/` and checks all sixteen rows of section 1.6's truth table.

Larger networks, multiplexers and submodules

Minimising a network, the first designs that call for a process, and the first design made of more than one entity.

IN THIS CHAPTER

- Karnaugh maps: why a sum of products read straight out of the truth table rarely gives the smallest network.
- `std_logic_vector`: bundling several wires under one name, indexing and slicing.
- Multiplexers, from 2-to-1 to 8-to-1.
- `process` and `case`: sequential statements inside otherwise parallel hardware.
- Submodules: writing a block once and instantiating it more than once.
- Self-checking testbenches, and the three GHDL commands that run one.

Chapter 1 took a circuit all the way from a requirement to a file. This chapter does three things to that path: it tightens the *derivation* with Karnaugh maps, it gives you the VHDL constructs a larger network needs, and it turns verification into something you run instead of something you do by eye.

Read section 2.1 to 2.6 in order; they build on each other. Section 2.7 is the book's guide to running testbenches, and every later chapter's exercises refer back here.

Refresher: The Karnaugh map has a chapter of its own in *Digital Electronics*, chapter 3, and the multiplexer is section 4.2 there.

2.1 Karnaugh maps: from truth table to minimised gate network

Reading a sum-of-products equation straight out of a truth table, the way section 1.2 did, always works but rarely gives the smallest network: every 1 row becomes an AND term of its own, even when several rows share most of their inputs and could be merged.

A **Karnaugh map** (K-map) makes the shared patterns visible by hand, for functions of 2 to 4 variables, without any formal manipulation of Boolean algebra. A worked example follows; more practice is in section 2.9, and classical minimisation theory (don't-cares, Quine-McCluskey) is outside the scope of this book.

The idea is to lay the truth table out as a grid instead of as a list, and to order the headings so that every cell differs from each of its neighbours across and down in exactly one input value, including around the left/right and top/bottom edges. That order is **Gray code**: for two bits 00, 01, 11, 10, where only one bit changes at each step, including from the last entry back to the first.

Then:

1. Write a 1 in every cell where the output is 1, and leave the rest empty.
2. Draw rectangular groups around the 1s:
 - Every group must be a rectangle whose width and height are each a power of two (1, 2 or 4), so that its size is a power of two as well.
 - a group of 4 is therefore either a 1x4 line or a 2x2 block, and both are allowed.
 - cells that meet only at a corner are not a rectangle, so they cannot be grouped.
 - Groups may overlap, and may wrap around an edge of the grid.
3. Make every group as large as possible, and cover every 1 at least once. A group of 1 keeps all its inputs in the term, a group of 2 drops one, a group of 4 drops two, and so on.
 - Then use as few groups as will do: drop every group all of whose cells are already covered by another. Nothing above forbids a redundant group, and a redundant group is a redundant AND term in the answer, correct but not minimal.
4. For every group keep only the input or inputs that have the same value in every cell of the group, drop the rest, and write that as an AND term:
 - write a kept input plain if the constant value is 1, and primed if it is 0.
 - a group covering only cells where $B = 0$ therefore contributes the term B' .
5. X is all the groups' terms ORed together.

Worked example

Three inputs A, B, C, one output X:

A	B	C	X
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Lay it out with AB in Gray code down the rows and C across the columns, and fill in a 1 everywhere $X = 1$:

		C	
		0	1
AB	00		1
	01		1
	11	1	1
	10		1

Figure 2.1 Karnaugh map for X, filled in from the truth table.

The whole $C = 1$ column is four 1s, a group of 4. Every cell shares $C = 1$ and nothing else is constant (A and B both vary), so it reduces to the single-variable term C :

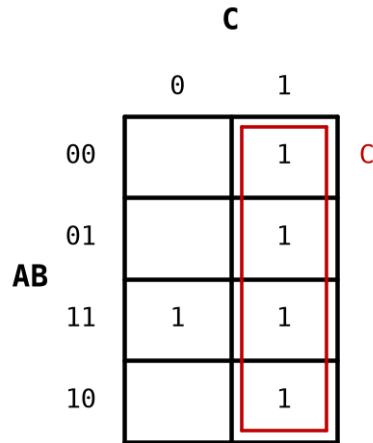


Figure 2.2 The same map with the four-cell $C = 1$ group marked.

One 1 is still uncovered, on the row $AB = 11$. Group it with its neighbour to the right, which the C group already covers. Both cells in the pair share $A = 1$ and $B = 1$, and C varies, so it reduces to AB :

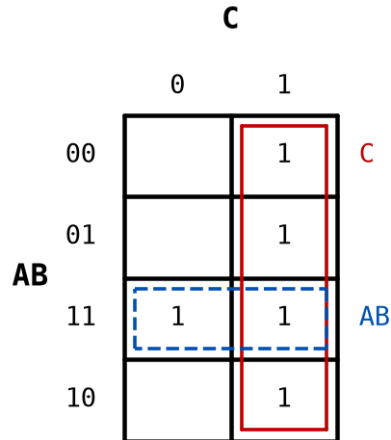


Figure 2.3 The same map with the two-cell $AB = 11$ group added.

Every 1 is now covered; the cell $AB = 11, C = 1$ lies in both groups, and overlap is fine. Summing up gives:

$$X = AB + C$$

Reading the truth table directly, chapter 1's way, would have given five AND terms, one per 1 row. The K-map found an equivalent equation with two terms, which is the whole value of doing this by hand.

Realising the network

$X = AB + C$ is two gates: an AND computing AB feeding an OR whose other input is C .

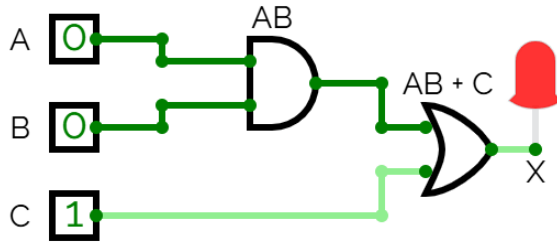


Figure 2.4 Gate network for the minimised expression.

You build this network by hand in CircuitVerse during the session, exactly as you built or_gate's network in chapter 1, and translate it to VHDL in section 2.2.

2.2 Translating the Karnaugh map's network to VHDL

$X = AB + C$ translates straight into VHDL. Three inputs, one output:

```

library ieee;
use ieee.std_logic_1164.all;

entity gate_network is
    port(a, b, c: in std_logic;
         x      : out std_logic);
end entity;
  
```

The architecture uses an internal signal `ab` to hold the AND gate's output, and mirrors the two-gate network one to one:

```

architecture behaviour of gate_network is
    signal ab: std_logic;
begin
    -- AND gate: AB.
    ab <= a and b;
  
```

```
-- OR gate: AB + C.
x <= ab or c;
end architecture;
```

Without the intermediate signal it collapses to $x \leq (a \text{ and } b) \text{ or } c$; . Both are equally valid and synthesise to the same hardware; use whichever is more readable for the network in front of you.

The buildable version uses the collapsed form, since at two gates there is nothing to gain from naming the intermediate signal:

```
lectures/L02/gate_network/gate_network.vhd

-----
-- Minimized gate network for the Appendix A Karnaugh example:  $x = ab + c$ .
--
-- Inputs:
--   - a, b, c: The three function inputs.
-- Outputs:
--   - x: Network output.
-----

library ieee;
use ieee.std_logic_1164.all;

entity gate_network is
    port(a, b, c: in std_logic;
         x      : out std_logic);
end entity;

architecture behaviour of gate_network is
begin
    x <= (a and b) or c;
end architecture;
```

Open it beside the version above and convince yourself that they describe the same circuit; that is the claim this section makes, and it costs nothing to check.

To run it on hardware, assign the ports to switches and an LED with the DE0-CV's pin planner (appendix B). Nothing here depends on specific pin numbers.

2.3 std_logic_vector: more than one wire

Everything so far has been `std_logic`, a wire carrying one bit. Bundling several wires under one name is what `std_logic_vector` is for.

```
signal count: std_logic_vector(3 downto 0);
```

That declares four wires: count(3) down to count(0). The range is 3 *downto* 0 rather than 0 *to* 3 so that the leftmost bit is the most significant, the way you would write the number on paper. The book uses *downto* everywhere, and so does nearly all the VHDL you will meet.

A vector is a port type like any other:

```
entity display is
    port(number: in std_logic_vector(3 downto 0);
          hex   : out std_logic_vector(6 downto 0));
end entity;
```

Values

A single bit takes single quotes, a vector double ones:

```
x      <= '1';           -- std_logic: one bit
count <= "1010";         -- std_logic_vector(3 downto 0): four bits
count <= (others => '0'); -- every bit '0', whatever the width
```

The last is an *aggregate*: (others => '0') means “all remaining bits are '0'”, and since none was named individually, that is all of them. It keeps working when the width changes, and it turns up throughout the rest of the book.

Indexing and slicing

One bit out of a vector is a *std_logic*; a contiguous run is a *slice*, which is itself a vector:

```
carry    <= count(3);
high_half <= input(7 downto 4);
```

Slicing is how one wide port feeds several narrower ones, which is what section 2.6’s two-digit display does with its eight switch inputs.

One detail is not what a software developer expects. input(7 *downto* 4) has the index range 7 *downto* 4, not 3 *downto* 0. Connected to a port declared *std_logic_vector*(3 *downto* 0), the two are matched **positionally, left to right**, not by index number: input(7) drives bit 3, input(6) drives bit 2, and so on. The lengths have to agree. The numbering does not.

A vector is a bundle of bits, not a number

`count <= "1010"` does not make `count` equal to ten. `std_logic_vector` says nothing about what the bits *mean*, so it offers only the operations that make sense for a bundle of wires: the logical operators applied bit by bit (`a and b`, not `a, a xor b`), comparison (`count = "1010"`) and concatenation (`"10" & "11"` is `"1011"`).

Arithmetic is not on that list. `count + 1` will not compile, because nothing has said whether the four wires are an unsigned number, a signed one, or four unrelated signals.

The book does not work around that with a numeric vector type. Anything that counts is an ordinary integer with a range:

```
signal count: natural range 0 to 9;
```

which you can add to and compare directly, and which synthesis turns into exactly the flip-flops the range needs. Vectors stay what they are: bundles of wires, for things that really are bundles of wires, such as eight switches or seven display segments.

The two worlds meet only at a boundary, and the conversions turn up where they are needed:

```
count_v <= std_logic_vector(to_unsigned(count, 4)); -- integer to vector
count    <= to_integer(unsigned(count_v));         -- vector to integer
```

Both come from `ieee.numeric_std`, which is why some files add a third use clause.

2.4 Multiplexers

A **multiplexer** (“mux”) connects exactly one of several data inputs to a single output, chosen by a separate set of **select inputs**. An N -to-1 mux has N data inputs and $\lceil \log_2 N \rceil$ select bits.

The simplest case, a 2-to-1 mux with the inputs A , B , the selector S and the output X :

S	X
0	B
1	A

As a Boolean equation, $X = S'B + SA$: for every combination of inputs exactly one AND term is active, and that term’s data input passes through.

Larger muxes are the same pattern with more select bits. An 8-to-1 mux with the data inputs A – H and the selector $S[2:0]$:

S[2:0]	X
000	A
001	B
010	C
011	D
100	E
101	F
110	G
111	H

Read as a sum of products with one term per row, the row 000 contributes $A S'_2 S'_1 S'_0$ and the rest follow the same shape, each gating its data input with the select combination that chooses it:

$$X = A S'_2 S'_1 S'_0 + B S'_2 S'_1 S_0 + \dots + H S_2 S_1 S_0$$

Worth writing out once by hand, not worth memorising. The pattern is the point, and it is the same at any size.

As gates: one four-input AND per data input (the data bit plus all three select bits), with the select bits inverted as required, and all eight AND outputs summed by a tree of OR gates.

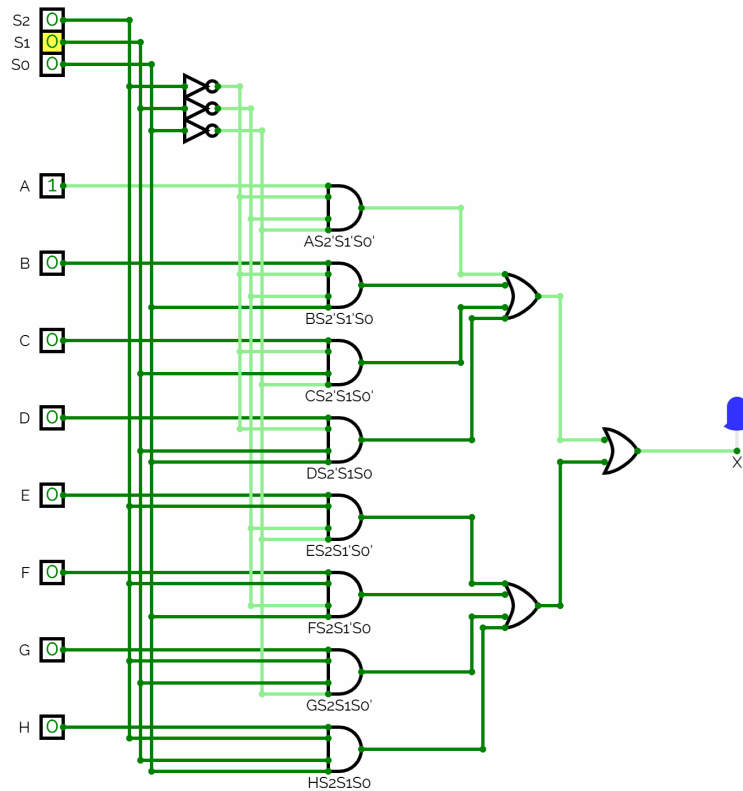


Figure 2.5 8-to-1 multiplexer as eight AND gates feeding an OR tree.

This is a real building block. The ATmega328P (the chip on an Arduino Uno and Nano) uses one internally so that its analogue-capable pins, plus an internal bandgap reference and a temperature sensor, can share a single analogue-to-digital converter, selected by the four MUX bits in its ADMUX register.

2.5 Multiplexers in VHDL: process and case

Every statement so far has been **concurrent**: a standing description of a wire, all of them live at once. A multiplexer is the first circuit that is awkward to write that way, and it needs two new constructs:

- a **process**, a block whose contents run sequentially, top to bottom, like ordinary code. It reruns every time any signal in its **sensitivity list** changes, and from the architecture's point of view it is still just one more concurrent statement. Every process in the book

has this shape; chapter 3 reuses it for clocked logic, with the clock in the sensitivity list instead of the inputs being decoded.

- a **case statement**, VHDL's switch. Like if and for it is a *sequential* statement, allowed only inside a process, and every value the selector can take must be covered, which when others guarantees.

"Choose one of several inputs based on a selector" is a case statement, almost word for word. The 2-to-1 mux from section 2.4:

```
MUX_PROCESS: process(a, b, sel) is
begin
    case (sel) is
        when '0'    => x <= b;
        when '1'    => x <= a;
        when others => x <= '0';
    end case;
end process;
```

Two things to note:

- Single-bit values take single quotes, '0'/'1', unlike the double quotes used for vectors such as "000".
- when others is mandatory rather than defensive. VHDL requires a case to cover every value of the selector's type, and std_logic has nine (section 1.4), so '0' and '1' alone never make a case exhaustive. It is a language rule, not a claim about the wire: a synthesised network carries a 0 or a 1 and nothing else, and the other seven values are the simulator's, where a signal left sitting at 'U' or 'X' is a bug rather than a level.

Worked example: the 8-to-1 mux

The same shape with a three-bit selector and eight branches, one per row of section 2.4's truth table. The data inputs A–H become the vector inputs(7 downto 0), with A as inputs(7) down to H as inputs(0), so the selector "000" connects inputs(7) through and "111" connects inputs(0).

```
lectures/L02/mux_8to1/mux_8to1.vhd
-----
-- 8-to-1 multiplexer: routes one of inputs(7:0) to x, selected by sel.
--
-- Inputs:
--   - inputs: Data inputs; sel "000" routes inputs(7), "111" routes inputs(0).
--   - sel   : Selector.
-- Outputs:
--   - x: Selected input.
```

```

-----
library ieee;
use ieee.std_logic_1164.all;

entity mux_8to1 is
    port(inputs: in std_logic_vector(7 downto 0);
          sel   : in std_logic_vector(2 downto 0);
          x      : out std_logic);
end entity;

architecture behaviour of mux_8to1 is
begin
    MUX_PROCESS: process (inputs, sel) is
    begin
        -- Check the selector combination, update the output accordingly.
        -- Safety default: drive output low if an invalid selector occurs.
        case (sel) is
            when "000" =>
                x <= inputs(7);
            when "001" =>
                x <= inputs(6);
            when "010" =>
                x <= inputs(5);
            when "011" =>
                x <= inputs(4);
            when "100" =>
                x <= inputs(3);
            when "101" =>
                x <= inputs(2);
            when "110" =>
                x <= inputs(1);
            when "111" =>
                x <= inputs(0);
            when others =>
                x <= '0';
        end case;
    end process;
end architecture;

```

Compare it with section 2.4's eight-term equation for the same circuit. The equation describes the gates; the case statement describes the *intent* and lets the synthesis tool produce the gates. Both are correct and both synthesise to the same hardware. That gap, between describing structure and describing behaviour, is most of what a hardware description language gives you.

2.6 Building a design out of submodules

Everything so far has been one entity with one architecture. That stops being enough the moment a design needs the same block twice.

The DE0-CV has six 7-segment displays, and you want to drive two of them, each showing one hexadecimal digit. The logic that turns a four-bit number into seven segment signals is the same for both. You could write it twice inside one architecture and nothing would stop you, but the second copy is a liability: every correction has to be made in both, and the day they drift apart is the day you lose an evening working out which of them is wrong.

Write the block once, as an entity of its own, and *instantiate* it twice.

The submodule

An ordinary module. Nothing about it says “I am a submodule”:

```
entity display is
    port(number: in std_logic_vector(3 downto 0);
          hex  : out std_logic_vector(6 downto 0));
end entity;
```

Whether it ends up as a whole design or as part of a larger one is decided by whoever instantiates it, not by the module.

Instantiating it

With entity work.<name>, inside another architecture:

```
entity hex_display is
    port(input      : in std_logic_vector(7 downto 0);
          hex1, hex0: out std_logic_vector(6 downto 0));
end entity;

architecture structure of hex_display is
begin
    display1: entity work.display
        port map(input(7 downto 4), hex1);

    display0: entity work.display
        port map(input(3 downto 0), hex0);
end architecture;
```

- display1 and display0 are **instance labels**, naming the two copies. They have to be unique within the architecture, and they are not the module’s name: both instances are display.

- entity `work.display` selects the module. `work` is the library your analysed files land in by default, in both GHDL and Quartus.
- port `map(...)` connects the instance's ports to signals in the enclosing architecture, **in the order the entity declares them**. `display` declares `number` and then `hex`, so the first item drives `number` and the second is driven by `hex`.
- `input(7 downto 4)` is a **slice**, which the instance sees as a `std_logic_vector(3 downto 0)`.

Note what `hex_display`'s architecture does *not* contain: any logic at all. It is wiring and nothing else. Every gate in the finished design comes from the two `display` instances.

Positional and named association

The port `map` above is **positional** and matches ports by order. VHDL also allows **named** association, which you will meet in other people's code:

```
display1: entity work.display
    port map(number => input(7 downto 4), hex => hex1);
```

The two are equivalent. The book uses positional everywhere, which means the order of an entity's port declarations is something you have to get right. That is no formality: it is exactly what the testbenches handed out rely on, which is why every exercise's self-check box spells out its ports *in order*.

What this costs in hardware

Nothing surprising. Instantiating a module twice puts two copies of its gates on the FPGA, exactly as writing the logic twice would. The saving is in the source, not in the silicon: one description, one place to correct, and a design you can read.

2.7 Verifying your VHDL with a testbench

Every VHDL module you write for an exercise comes with a **self-checking testbench**: a small VHDL file that drives your inputs, checks your outputs and says which case failed. Chapter 1's exercises already handed such things out, and section 1.5 gave you the three commands that run one.

You are not asked to write the testbench that verifies a module you build; every single one of them is handed out. Running them is enough to verify every exercise on your own laptop, with no FPGA board. (A couple of project exercises from chapter 13 onwards invite a short test rig of your own, but nothing you are assessed on rests on a testbench you wrote yourself.)

This section is the book’s introduction, and every later chapter’s exercises refer back here. Appendix A is the fuller version, written for the group project; come back here for the introduction and go there for the details.

2.7.1 What you need

GHDL, the open-source VHDL analyser and simulator. Check with `ghdl --version`; install on WSL or Ubuntu with `sudo apt -y install ghdl`.

Every command below uses `--std=93`, the VHDL standard the book targets.

2.7.2 What a testbench is

A testbench is itself a VHDL module, but a special one:

- Its entity has **no ports**: nothing connects to the outside world, since it is the top of the simulation.
- Its architecture **instantiates your design**, the “device under test”, labelled `dut`, and drives its inputs.
- It **checks** every output with an assert. If an output is wrong the assertion fires with a message and the simulation stops.

So instead of you reading a waveform diagram to decide whether the output is right, the testbench decides it, and speaks up only when something is wrong.

2.7.3 Running a testbench: analyse, elaborate, run

Three steps. Pass `--std=93` to all three, and give the **testbench’s entity name**, not a file name, to the last two:

```
ghdl -a --std=93 <module>.vhd <module>_tb.vhd      # analyze both files (module ↵
↳ first)
ghdl -e --std=93 <module>_tb                        # elaborate the testbench
ghdl -r --std=93 <module>_tb --assert-level=error --stop-time=10ms # run it
```

- `--assert-level=error` makes a failed check stop the run and return a **non-zero exit code**, a real pass/fail that you can check with `echo $?`. The testbenches in this book’s first part raise failures with severity `failure`, which stops the run by itself, so you get a real pass/fail even without the flag. Keep it on anyway: it is the habit that also works with testbenches reporting at a lower severity level, and the group project’s testbenches are exactly those (appendix A).
- `--stop-time=10ms` caps how long the simulation may run. A testbench often waits for your module to do something, with a line like `wait until tx = '0'`; . If your module never does, that wait never finishes, and without a stop time the simulation hangs: no output, no error, just a terminal sitting there waiting. 10 ms of *simulated* time is far more than any testbench here needs, and takes well under a second in real time.

- A passing run prints its “all checks passed” note and exits with 0. Every testbench in this book ends with that note, so if you do not see it the run did not finish: it either failed a check or hit the stop time.

2.7.4 Checking an exercise

Every VHDL exercise has its testbench in a directory of its own. This chapter’s Exercise 2.3, for example, has `lectures/L02/exercises/xyz_logic/xyz_logic_tb.vhd`. Write your module in the **same directory**, with exactly the name the exercise asks for, and then run the three commands from there:

```
cd lectures/L02/exercises/xyz_logic
ghdl -a --std=93 xyz_logic.vhd xyz_logic_tb.vhd
ghdl -e --std=93 xyz_logic_tb
ghdl -r --std=93 xyz_logic_tb --assert-level=error --stop-time=10ms
```

Every testbench’s file header lists exactly these commands.

The testbench connects to your module **positionally**, not by name, so your entity must declare its ports in the **same order** as the exercise lists them. The names are yours to choose, but use the given ones anyway, so that your module and the testbench’s error messages speak about the same signals.

A port declared in the wrong order fails in one of two ways, neither of which says “wrong order”:

- if the misplaced ports have **different** types or widths, analysis stops with can't associate "sel" with port "sel", pointing at the testbench’s port map line.
- if they have the **same** type, say several `std_logic` ports, it analyses and elaborates perfectly well, and then fails at run time as an ordinary assertion failure, which reads exactly like a logic bug in your design.

So when a testbench fails on the very first case it checks: reread your port order before you go through your architecture. The same goes for **generics**: a module that carries them must declare exactly the ones the exercise states, in order.

2.7.5 When the design needs more than one file

Some exercises build one module out of another, so the `ghdl -a` line names several files instead of two. The elaboration and run steps are unchanged: they still take only the testbench’s **entity name**.

```
ghdl -a --std=93 <subcomponent>.vhd <module>.vhd <module>_tb.vhd
ghdl -e --std=93 <module>_tb
ghdl -r --std=93 <module>_tb --assert-level=error --stop-time=10ms
```

- **Order matters on the analysis line:** name a module before anything that instantiates it, so that GHDL has already seen the entity when it reaches the entity `work.<name>` referring to it. Subcomponents first, your top level after them, the testbench last.
- No exercise directory has a subcomponent handed out with it. Every module a design instantiates is one you wrote in an earlier exercise and copy in yourself; the exercise says which, and from where.
- The exercises that need this are this chapter's `hex_display` (which instantiates your display), chapter 4's `led_toggle_sync2`, chapter 7's `blinker` and `walking_led`, and all of chapter 8's, where the receiver names five files and the transmitter four.
- Every testbench's file header lists the exact command for its own exercise.

2.7.6 Reading the result

A failed check looks like this:

```
xyz_logic_tb.vhd:38:13:@10ns:(assertion failure): xyz_logic: wrong X with a='0' b=
  = '0' c='0' d='0', expected '0' but got '1'!
ghdl:error: assertion failed
```

Read it in order: `file:line` is where the assertion is, `@time` is when it fired, and after that comes the message. It is the same shape as a compilation error from GHDL, so it maps onto something you already know. No error means every case matched.

2.7.7 A complete example

A combinational testbench in the form every testbench in this book uses, here for chapter 1's `or_gate`:

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity or_gate_tb is
end entity;

architecture behaviour of or_gate_tb is
signal a, b, x: std_logic;
begin
    dut: entity work.or_gate
        port map(a, b, x); -- positional: same order as or_gate's port clause.

    SIM_PROCESS: process is
begin
```

```

    for i in 0 to 3 loop
        (a, b) <= std_logic_vector(to_unsigned(i, 2)); -- apply inputs
        wait for 10 ns; -- let them settle
        assert x = (a or b) -- check AFTER the wait
    wait

        report "or_gate: wrong output with a=" & std_logic'image(a)
            & " b=" & std_logic'image(b)
            & ", expected " & std_logic'image(a or b)
            & " but got " & std_logic'image(x) & "!"
        severity failure;
    end loop;
    report "or_gate: all checks passed!" severity note;
    wait; -- end the simulation
end process;
end architecture;

```

Two rules keep a combinational testbench correct:

- **Apply, wait, check, in that order.** A signal assignment only *schedules* a value, so the inputs and the output are not updated until the wait. Checking before it tests the previous round's values.
- **End with wait;** so that the stimulus process suspends and the simulation stops by itself.

Clocked designs need a little more, a clock-generating process and a way to stop it, which is exactly the machinery the testbenches handed out already contain. For the exercises you write only the module.

Run one now. Against this chapter's first worked example:

```

cd lectures/L02/gate_network
ghdl -a --std=93 gate_network.vhd gate_network_tb.vhd
ghdl -e --std=93 gate_network_tb
ghdl -r --std=93 gate_network_tb --assert-level=error --stop-time=10ms

```

2.8 Summary

What you should be able to do now

- Derive a minimised equation from a Karnaugh map and turn it into a VHDL architecture.
- Read and write ports and signals of type `std_logic_vector`.
- Write a process with a correct sensitivity list and a case that covers every input value.
- Build a design out of more than one entity with a positional port map, and say why two instances of the same module put two copies of its gates on the FPGA.
- Verify a module by running its testbench, and read a failed assert.

Questions to test yourself with

1. Given a truth table, can you derive its equation both directly and by way of a Karnaugh map? What is the difference between the two results?
2. Why can a case not stand directly in an architecture body?
3. What does `when others` protect you from, when the selector “can only” be 0 or 1?
4. In `display1: entity work.display`, what does `display1` name, and what does `work.display` name? Could a second instance reuse the same label?
5. `hex_display`’s architecture contains no logic at all. Where do the gates in the finished design come from?
6. You instantiated `display` twice instead of writing its case twice. Does the FPGA get one copy of that logic or two? So what did you actually gain?
7. The book always writes `port map` positionally. What breaks if an entity’s ports are declared in a different order from the one the testbench expects, and when do you notice?

Looking ahead

Chapter 3 goes on with:

- Sequential networks: circuits that remember.
- The D latch, and the D flip-flop built from a pair of them.
- Clocking, and when a signal assignment actually takes effect.
- Edge detection: a button press that toggles an LED.

2.9 Exercises

How to check your work. Every exercise below that asks you to write a VHDL module comes with a self-checking testbench under `lectures/L02/exercises/`. Write your module in its directory `exercises/<module>/`, with the entity name and the **port order** the exercise states, and then run it with GHDL — see section 2.7 for the three commands.

No FPGA board is needed for any exercise. The Quartus synthesis and board programming steps are demonstrated during the session; your job afterwards is to get the VHDL right, and the testbench is how you confirm it.

EXERCISE SET 1

Karnaugh maps**Exercise 2.1 Three variables**

CIRCUIT

Derive a minimised equation for X from the Karnaugh map below, then realise the corresponding gate network and simulate it in CircuitVerse:

ABC	X
000	1
001	1
010	0
011	0
100	1
101	1
110	0
111	0

Hint: Put AB down the rows in Gray code order (00, 01, 11, 10) and C across the columns, exactly as in the worked example in section 2.1, before you start grouping 1s.

Exercise 2.2 Four variables

CIRCUIT

Derive a minimised equation for X from the Karnaugh map below, then realise the corresponding gate network and simulate it in CircuitVerse:

ABCD	X
0000	0
0001	1
0010	0
0011	1
0100	0
0101	0
0110	0
0111	0
1000	0
1001	1
1010	0
1011	1
1100	1
1101	0
1110	1
1111	0

Hint: With four variables you put two of them (in Gray code) on each axis, so that you get a 4x4 grid. Groups can still wrap around every edge of the grid, not just left-right; remember to check the top and bottom edges too.

EXERCISE SET 2

Circuits in VHDL**Exercise 2.3 xyz_logic: three outputs from four inputs**

VHDL

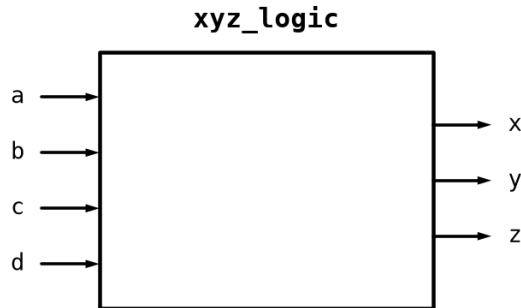
A gate network has four inputs, ABCD, and three outputs, XYZ. It is given by the truth table below:

ABCD	XYZ
0000	001
0001	010
0010	001
0011	011
0100	100
0101	110
0110	100
0111	111
1000	101
1001	110
1010	101
1011	111
1100	000
1101	010
1110	000
1111	011

- Treat X, Y and Z as three separate single-output functions of ABCD, and derive a minimised equation for each by way of a Karnaugh map.
- Realise the resulting gate network and simulate it in CircuitVerse; check all three outputs against the truth table above.
- Implement the design in VHDL, in a module of its own with four `std_logic` inputs and three `std_logic` outputs. Section 1.4 and section 2.2 show the pattern for turning a derived equation into concurrent VHDL assignments.
- Run the testbench and confirm that all 16 rows pass.
If a row fails, the assertion message names the input combination:
 - Go back to the Karnaugh map for whichever of X, Y, Z is wrong.

- A single wrong output on a single row is nearly always a mis-grouped 1 in a map.

Hint: Two of the three outputs here turn out to depend on only one or two of the four inputs once minimised. Do not assume every output needs all four.



Self-check. Name your VHDL entity `xyz_logic`, with the ports `a`, `b`, `c`, `d` (in) and `x`, `y`, `z` (out), declared in that order; its testbench is in `exercises/xyz_logic/` and checks all 16 rows of the truth table above.

EXERCISE SET 3

Multiplexers

Exercise 2.4 The 4-to-1 mux by hand

CIRCUIT

A 4-to-1 multiplexer has four data inputs, A–D, two select bits, $S[1:0]$, and one output, X.

Let the selector name its data input directly: $S[1:0] = "00"$ selects A, $"01"$ selects B, $"10"$ selects C and $"11"$ selects D. That is the mapping Exercise 2.6 implements.

- Write out the multiplexer's truth table (four rows, one per select combination).
- Derive the sum-of-products equation for X in terms of A–D and $S[1:0]$. Section 2.4 does the same for the 8-to-1 case; halve that here.
- Build and simulate the network in CircuitVerse, and confirm that X follows each data input in turn as you change $S[1:0]$.

Keep your answer: Exercise 2.6 below implements exactly this circuit in VHDL, once section 2.5 has given you the process and case statements it needs.

Exercise 2.5 mux2 with if/else

VHDL

Section 2.5 writes the 2:1 multiplexer with a case statement. Write the same circuit once more with an if/else instead, and convince yourself that the two describe identical hardware.

Write an entity named mux2 with:

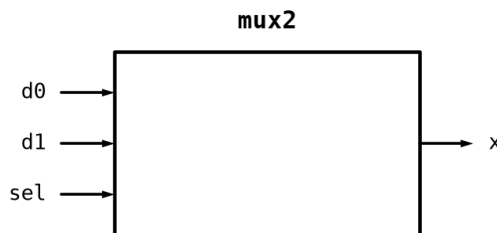
Port	Dir.	Type	Description
d0	in	std_logic	Data input.
d1	in	std_logic	Data input.
sel	in	std_logic	Selector.
x	out	std_logic	The selected data input.

The selector connects through the data input whose number it names:

- sel = '0' selects d0.
- sel = '1' selects d1.
- Note that this is the reverse of the case example in section 2.5, which selects its *second* input b at '0'. Which input a select value names is a convention, not a rule, so read it off the specification every time rather than assuming.

Implement the multiplexer with a process-based architecture:

- Include d0, d1 and sel in the sensitivity list.
- Use an if/else statement on sel.
- Assign x in every branch.
- Do not use:
 - A conditional concurrent assignment ($x \leq d1$ when sel = '1' else d0;).
 - A with...select statement: a concurrent form of case that the book does not cover, mentioned only so that you do not reach for it if you have met it elsewhere.



Self-check. Name your entity mux2, with the ports d0, d1, sel (in) and x (out), declared in that order; its testbench is in exercises/mux2/ and checks all eight combinations of the three inputs.

Exercise 2.6 mux4

VHDL

Extend Exercise 2.5 to a 4:1 multiplexer named mux4.

This is the circuit you derived, drew and simulated by hand in Exercise 2.4. Derive the case branches afresh yourself for four inputs and two select bits rather than copying the 8-to-1 example from section 2.5.

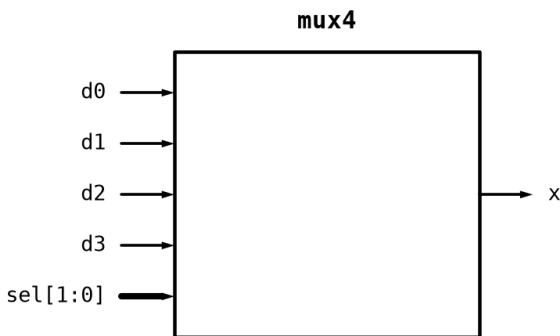
The entity must have:

Port	Dir.	Type	Description
d0	in	std_logic	Data input.
d1	in	std_logic	Data input.
d2	in	std_logic	Data input.
d3	in	std_logic	Data input.
sel	in	std_logic_vector(1 downto 0)	Selector.
x	out	std_logic	The selected data input.

As in Exercise 2.5, the selector names its data input directly: "00" selects d0, "01" selects d1, "10" selects d2 and "11" selects d3. Again this is the reverse mapping from section 2.5's worked 8-to-1 example, where "000" connects inputs (7) through. Derive your case branches from the four rows above, not from that listing.

Implement the multiplexer with a process:

- Include every data input and sel in the sensitivity list.
- Use a case statement on sel, one branch per select value above.
- Include a when others branch, and choose a safe output value for select inputs containing values such as 'X', 'U' or 'Z'.
- Do not replace the case statement with an if/elsif chain.



Self-check. Name your entity mux4, with the ports d0, d1, d2, d3, sel (std_logic_vector(1 downto 0)) and x (out), declared in that order; its testbench is in exercises/mux4/ and sweeps all four select values against all sixteen data input combinations.

EXERCISE SET 4

Submodules and 7-segment displays

Exercise 2.7 and 2.8 build one design in two halves: a module that drives a single 7-segment display, and a top level that uses two copies of it to show one hexadecimal digit on each. Do them in order: Exercise 2.8 instantiates what you write in Exercise 2.7.

A 7-segment display is seven LEDs, named a to g, arranged around a figure eight. Light the right subset and you get a digit. This is the layout, and it is what you need in order to work out the codes below:

```

      aaaa
    f    b
    f    b
      gggg
    e    c
    e    c
      dddd

```

So 1 lights b and c (the two on the right), and 7 lights a, b and c. Comparing the codes given for those two digits is a quick way to confirm that you have the layout the right way round before you start deriving A to F.

On the DE0-CV the displays are wired **active low**: driving a segment to '0' lights it, and '1' turns it off. That is why the code for 8, which lights every segment, is "0000000", and why a blank display is "1111111".

Throughout, hex(6 downto 0) maps onto the segments g, f, e, d, c, b, a: bit 6 is g and bit 0 is a.

Exercise 2.7 display

VHDL

Write a module named display that drives one 7-segment display with a hexadecimal digit.

The entity must have:

Port	Dir.	Type	Description
number	in	std_logic_vector(3 downto 0)	The value to show, 0000 through 1111.
hex	out	std_logic_vector(6 downto 0)	The segment code for that value.

Feeding number the value 0111 must therefore put the code for 7 on hex, and feeding it 1110 must put the code for E on hex.

Implement it with a process:

- Put number in the sensitivity list.
- Use a case statement on number, one branch per value.
- Include a when others branch, and blank the display in it.

You are given the codes for 0 to 9. Work out A to F yourself, from the segment layout above:

```
constant DISPLAY_0 : std_logic_vector(6 downto 0) := "1000000";
constant DISPLAY_1 : std_logic_vector(6 downto 0) := "1111001";
constant DISPLAY_2 : std_logic_vector(6 downto 0) := "0100100";
constant DISPLAY_3 : std_logic_vector(6 downto 0) := "0110000";
constant DISPLAY_4 : std_logic_vector(6 downto 0) := "0011001";
constant DISPLAY_5 : std_logic_vector(6 downto 0) := "0010010";
constant DISPLAY_6 : std_logic_vector(6 downto 0) := "0000010";
constant DISPLAY_7 : std_logic_vector(6 downto 0) := "1111000";
constant DISPLAY_8 : std_logic_vector(6 downto 0) := "0000000";
constant DISPLAY_9 : std_logic_vector(6 downto 0) := "0010000";
-- Todo: add DISPLAY_A through DISPLAY_F here.
constant DISPLAY_OFF: std_logic_vector(6 downto 0) := "1111111";
```

Two of the six are lower case on a 7-segment display, because their capital forms would be impossible to tell from a digit: b would read as 8 and d would read as 0. A, C, E and F are capitals.

All sixteen codes are also in `display_tb.vhd`, since a self-checking testbench cannot check an output without knowing what it should be. So this is no puzzle you can be locked out of. Derive the six anyway: it takes a couple of minutes with the segment diagram, and reading them out of the testbench teaches you nothing the case statement does not. The work in this exercise is the sixteen-branch case statement, not the lookup table.



Self-check. Name your entity `display`, with the ports `number` (`std_logic_vector(3 downto 0)`) and `hex` (`out, std_logic_vector(6 downto 0)`), declared in that order; its testbench is in `exercises/display/` and checks all sixteen values against the whole code table, naming the digit and both codes when one is wrong.

Exercise 2.8 hex_display

VHDL

Write a module named `hex_display` that shows a two-digit hexadecimal number on two displays, using two instances of your display from Exercise 2.7.

The entity must have:

Port	Dir.	Type	Description
input	in	<code>std_logic_vector(7 downto 0)</code>	Eight switches: the top four are the left digit, the bottom four the right.
hex1	out	<code>std_logic_vector(6 downto 0)</code>	Segment code for the left digit.
hex0	out	<code>std_logic_vector(6 downto 0)</code>	Segment code for the right digit.

When it works, the number on input (7 downto 4) appears on hex1, and the number on input (3 downto 0) on hex0.

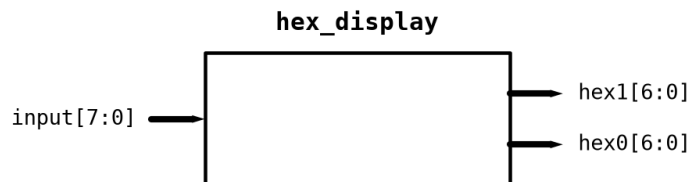
Build it out of submodules, following section 2.6:

- Create two instances of `display`, labelled `display1` and `display0`.
- Connect `input(7 downto 4)` and `hex1` to `display1`.
- Connect `input(3 downto 0)` and `hex0` to `display0`.
- Use a positional port map, as everywhere else in this book.
- Write no logic of your own in `hex_display`. If you catch yourself writing a case statement here, you are duplicating Exercise 2.7 instead of reusing it.

Copy your `display.vhd` from Exercise 2.7 into the exercise directory, since `hex_display` cannot be analysed without it. This is the book's first design built from more than one file, so name `display.vhd` first, before the file that instantiates it:

```
ghdl -a --std=93 display.vhd hex_display.vhd hex_display_tb.vhd
ghdl -e --std=93 hex_display_tb
ghdl -r --std=93 hex_display_tb --assert-level=error --stop-time=10ms
```

Section 2.7.5 explains why the order matters.



Self-check. Name your entity `hex_display`, with the ports `input` (`std_logic_vector(7 downto 0)`), `hex1` (out) and `hex0` (out, both `std_logic_vector(6 downto 0)`), declared in that order;

its testbench is in `exercises/hex_display/` and sweeps all 256 input values, checking each half independently so that a design swapping the two digits is caught instead of passing on the values where both halves happen to agree.

Exercise 2.9 One digit as a gate network

CIRCUIT

If you have time. Realise one digit as a gate network, the way chapter 1 and section 2.1 had you do:

- Take the four inputs and one segment, say *a*, and derive the Boolean equation for it from the 16-row truth table. A Karnaugh map is worth using here: most segments minimise considerably.
- Repeat for as many of the six remaining segments as you have patience for, and build the result in CircuitVerse.
- Then copy the whole gate network to drive a second display.

That copy is the point of the exercise. Duplicating a block of gates is exactly what instantiating display a second time did in Exercise 2.8, and it is worth having done once by hand to see what that single line of VHDL stands for.

EXERCISE SET 5

Naming an intermediate result

Exercise 2.10 `combo_logic`, written twice

VHDL

Write an entity named `combo_logic` that implements the Boolean function

$$x = ab + c'd$$

Section 2.2 claimed that naming a subexpression with an internal signal costs nothing and buys readability. This exercise is where you confirm that rather than take it on faith: you write the function twice, once as a single expression and once with the term $c'd$ factored out into an internal signal named *y*.

A signal declared inside an architecture is a wire. It is not a variable, it stores nothing, and naming a subexpression with one adds neither a gate, a delay nor a flip-flop to the circuit. It only gives part of the network a name, so that when you read it later you can see what that part is for. Both architectures below synthesise to the same gates, and the testbench handed out passes against either of them.

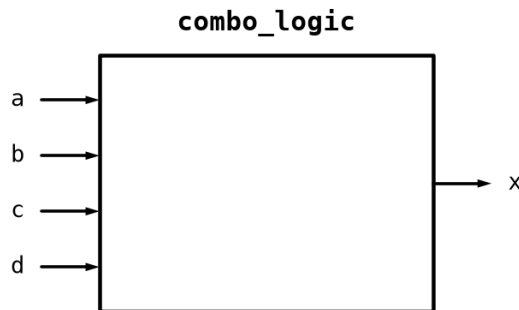
The entity must have the ports *a*, *b*, *c*, *d* (in) and *x* (out), all `std_logic`.

- Implement the whole function with a single concurrent signal assignment: declare no internal signals, and translate every Boolean operation into the corresponding VHDL operator.

b) Rewrite the architecture with an internal signal named y , representing the term $c'd$, and assign x the final expression. This is the same shape as `gate_network` in section 2.2, which names its AND term ab .

Confirm that the two architectures describe the same circuit.

Hint: Two descriptions represent the same combinational circuit when they produce the same output for every possible input combination. With four inputs there are 16 possible input combinations: write the full truth table and verify that both architectures produce the same value on x on every row.



Self-check. Name your entity `combo_logic`, with the ports `a`, `b`, `c`, `d` (in) and `x` (out), declared in that order; its testbench is in `exercises/combo_logic/` and checks all 16 rows. It passes against either architecture, which is the point of the comparison above.

Sequential networks

Storing state: the D latch, the D flip-flop, registers and clocking.

IN THIS CHAPTER

- Why combinational logic alone cannot remember anything, and what “state” means in a circuit.
- The D latch and the D flip-flop, and why synchronous designs are built from the flip-flop.
- Clocking: period, frequency, and rising versus falling edge.
- The synchronous process template in VHDL, and when a signal assignment actually takes effect.
- Edge detection: turning a level into a pulse one clock cycle long.
- `led_toggle`: a button press that toggles an LED and keeps it that way.

Two predictions are worth making before the respective simulation is run, and they are the whole reason the chapter builds the latch before it builds the flip-flop:

- for the latch, what Q does when enable goes low while D is still changing.
- for the flip-flop, what Q does when D changes *between* two clock edges.

Refresher: The latch, the flip-flop, the register and edge detection are in sections 5.1–5.6 of *Digital Electronics*, if they need brushing up before they are written in VHDL.

3.1 From combinational to sequential networks

Every circuit in chapter 1 and 2 was **combinational**: its output is a pure function of its current inputs, so the same inputs always give the same output, and there is no notion of “before” or “after” inside the circuit.

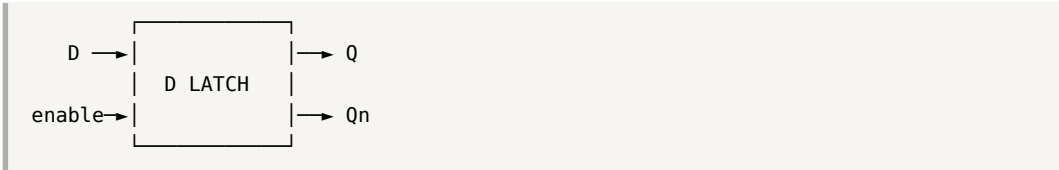
Most useful systems need more. A counter has to remember its count. A register has to hold a value after the signal that produced it has changed. An LED toggled by a button has to remember whether it is lit, since that is not encoded in the button’s momentary position at all.

All of these need **memory**: an output that depends on the circuit’s own *previous* output. A sequential network is combinational logic with part of the output fed back to the input,

and that **feedback loop** is what turns a stateless gate network into a circuit with state. This chapter builds the primitive that makes controlled feedback practical, the D flip-flop, and then the two things you build directly from it: registers and edge detectors.

3.2 The D latch

The simplest storage element is the **D latch**: two cross-coupled gates, gated by an enable input, storing a single bit.



D is the value to store, enable controls whether the latch is *open* (transparent) or *closed* (holding), Q is the stored value and Qn is always its inverse.

Each output feeds back into the other's equation, which is exactly section 3.1's feedback loop:

$$Q = (D' \cdot \text{enable} + Q_n)'$$

$$Q_n = (D \cdot \text{enable} + Q)'$$

enable	Behaviour	Result
1	the latch is open (transparent)	$Q = D, Q_n = D'$; the output follows the input immediately
0	the latch is closed (latched)	Q and Qn hold whatever they last had

Build it in CircuitVerse from a NOT gate, two AND gates and two NOR gates (a NOR is an OR followed by a NOT). Cross-couple the feedback so that the NOR producing Q feeds the NOR on the Qn side and the other way round. Drive D and enable from input switches and watch Q and Qn on LEDs.

Then test it:

- With enable = 1, change D and confirm that Q follows immediately. The latch is **transparent**.
- With enable = 0, confirm that Q is unchanged whatever D does. The latch is **latched**.
- Toggle D a few times while switching enable, and confirm that the latch always freezes whatever Q held at the moment enable went low. That is what **level-sensitive** means: transparent for the whole time enable is high, not just at a single instant.

That last point is also the latch's fundamental weakness. Because it is transparent for the whole time enable = 1, every glitch on D during that window passes straight through to Q.

In a large synchronous system, guaranteeing stable data for a whole clock phase is far harder than guaranteeing it around a single instant. That is the problem the D flip-flop solves.

3.3 The D flip-flop

A **D flip-flop** stores one bit the way a latch does, but is **edge-triggered** rather than level-sensitive: it samples D only at the *instant* the clock changes, not for the whole time the clock is high or low.

Feed both the same D and the same clock, and the difference shows immediately:

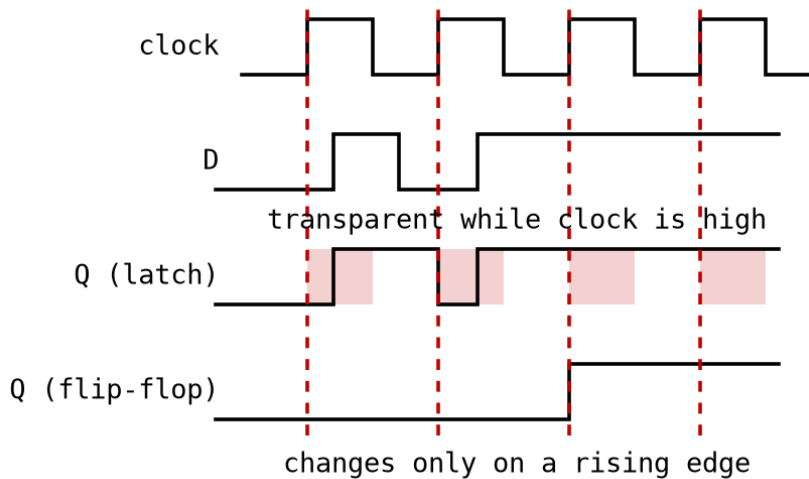
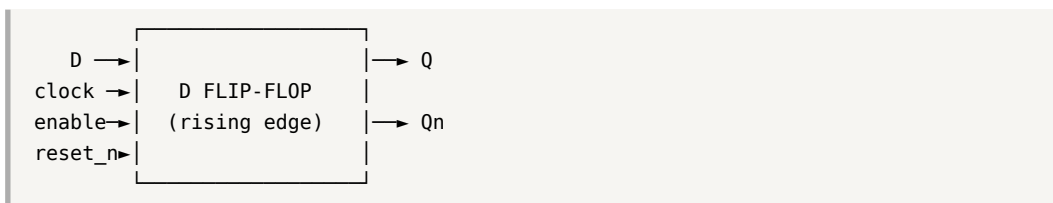


Figure 3.1 A latch follows D during every window where the clock is high, while a flip-flop changes only at the rising edges.

The latch changes three times: twice because D moved while the clock was high, and once because the clock went high against a D that had already changed. The flip-flop changes once, because only one of the four rising edges found D different from the value Q already held. Both behave correctly; they answer different questions.



clock is the time reference (section 3.4). enable gates whether the flip-flop takes a new value *at the triggering edge*. reset_n is an asynchronous, active-low reset: whenever it is 0, Q is forced to 0 immediately, regardless of the clock.

Condition	Result
reset_n = 0	$Q = 0, Q_n = 1$, immediately (asynchronous, ignoring the clock entirely)
reset_n = 1, rising edge on clock, enable = 1	$Q = D, Q_n = D'$, sampled at that instant
reset_n = 1, rising edge on clock, enable = 0	Q holds its previous value
reset_n = 1, no rising edge on clock	Q holds its previous value, whatever D does

That last row is worth seeing rather than reading:

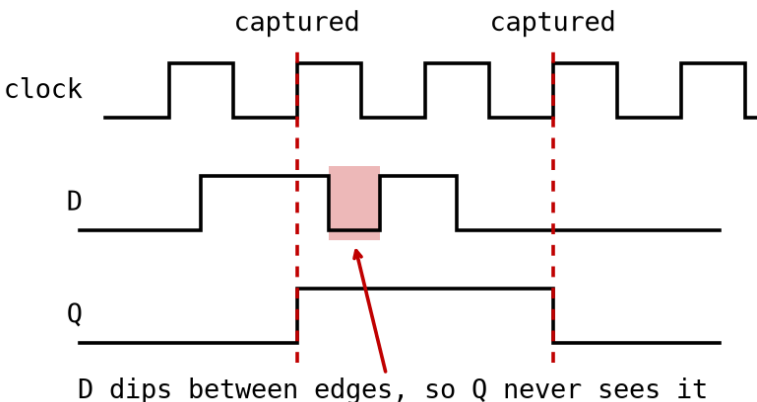


Figure 3.2 D goes high mid-cycle and is caught at the next rising edge, then dips and recovers entirely between two edges without Q ever moving.

D dips low and comes back within a single clock period, and Q never moves, because no edge occurred while it was low. A latch in the same position would have followed it down and up again.

Internally, an edge-triggered flip-flop is two D latches in series, a **master-slave** pair. One is transparent while the clock is low and captures D; the other is transparent while the clock is high and passes the captured value on to Q. The net effect is that Q changes only at the moment the clock goes from low to high, never while it sits at either level. The external behaviour is what VHDL's `rising_edge()` models (section 3.6).

Build it in CircuitVerse by building on your latch from section 3.2:

- Duplicate the latch so that two sit in series: the *master* feeds the *slave*, whose Q is the flip-flop's output.

- Add `clock`, `reset_n`, `D` and `enable` as inputs.
- Drive the two latches' transparency from the clock and its inverse. To keep the two meanings apart, call the internal per-latch control signal `gate`: the master's `gate` is `clock'` and the slave's is `clock`. The flip-flop's external enable is a different signal and is connected separately, below.
- Connect the external enable as a 2-to-1 mux (section 2.4) on the master's data input: when `enable` = 1 the mux passes `D` through, and when `enable` = 0 it passes the flip-flop's own `Q` back in, so that the edge recaptures the value already stored. Gate the *data*, never the clock.
- Add the reset to **both** latches. Forcing `Q` low takes two changes in each latch, not one:
 - feed an active-high reset (`reset_n` through a NOT) into the NOR producing `Q`, which forces `Q` to 0.
 - AND `reset_n` into the term `D AND gate` in the same latch, which frees `Qn` to go high.
- Set the clock period to something slow enough to watch, e.g. 1000 ms.

With the reset wired in, the equations for each latch become:

$$Q = (D' \cdot gate + Q_n + reset)'$$

$$Q_n = (D \cdot gate \cdot reset_n + Q)'$$

A tempting shortcut is to leave the `Qn` equation alone and break the loop instead, by ANDing `reset_n` into the `Qn` wire that feeds `Q`'s NOR. That is not enough. With `gate` = 1 and `D` = 1 it leaves `Qn` at 0, which is what holds `Q` at 1, so the reset silently does nothing in exactly the case you are most likely to test it in. Both changes, in both latches, or neither.

Confirm that `Q` updates only at the moment the clock goes high, and sits still between edges even if `D` changes, and that a low `reset_n` forces `Q` to 0 immediately, while releasing it changes nothing until the next rising edge.

3.4 Clocking

The **clock** is a periodic square wave, which every flip-flop in a synchronous circuit reads from the same source. Two numbers describe it: the **period** T , the time for one whole cycle, and the **frequency** $f = 1/T$ in Hz.

Every period has exactly two **edges**: a **rising edge** from 0 to 1, and a **falling edge** from 1 to 0. A synchronous design picks one of them, nearly always the rising edge, and updates every flip-flop at that edge and only that one. Using a single edge everywhere is what makes the timing of a large circuit analysable: every flip-flop changes at the same well-defined instant, so you can reason one clock cycle at a time instead of tracking every signal continuously.

Simulated and real clocks run on wildly different time scales, and the logic is identical in both cases:

- In CircuitVerse, set a period you can follow, e.g. 1000 ms.

- On the DE0-CV the system clock is a fixed 50 MHz, a period of 20 ns. Nobody perceives individual edges at that rate, so you reason in terms of “at the next rising edge”, never in real time.

3.5 Registers: flip-flops in parallel

A **register** stores more than one bit: N D flip-flops side by side, one per bit, all sharing the same clock, `reset_n` and `enable`.

```

D(0) → [D-FF] → Q(0)
D(1) → [D-FF] → Q(1)
D(2) → [D-FF] → Q(2)
    □      □
D(N-1) → [D-FF] → Q(N-1)

(clock, reset_n, enable are shared by every flip-flop above)

```

That is the whole idea: an 8-bit register is eight D flip-flops updating at the same clock edge.

In VHDL you rarely instantiate N flip-flops explicitly. Declare a `std_logic_vector` and assign the whole of it inside a single synchronous process (section 3.6): every bit synthesises to a flip-flop of its own, and all of them share the process’s clock and reset.

```

signal q: std_logic_vector(3 downto 0);
...
process (clock, reset_n) is
begin
    if (reset_n = '0') then
        q <= "0000";
    elsif (rising_edge(clock)) then
        if (enable = '1') then
            q <= d; -- d is a std_logic_vector(3 downto 0) input
        end if;
    end if;
end process;

```

Four bits, one process, four flip-flops in hardware: in principle no different from four `d_flip_flop` instances wired in parallel.

3.6 The synchronous process template in VHDL

Every synchronous element in this chapter, the flip-flop, the register and the edge detector, uses the same shape. Without a reset, a single D flip-flop is:

```

process (clock) is
begin
    if (rising_edge(clock)) then
        q <= d;
    end if;
end process;

```

q takes d's value at every rising edge and holds it the rest of the time. That single line inside the if statement is enough for synthesis to infer a real D flip-flop.

Rules

- **clock is the only signal in the sensitivity list**, unless there is an asynchronous reset. The process should run only when the clock changes; it has no reason to react to anything else.
- A process reacts to *every* change of a signal in the sensitivity list, not just rising ones, so `rising_edge()` specifically filters out the rising edge. All the logic that updates synchronously sits inside that if.
- Every signal assigned inside such a process synthesises to a flip-flop, one per bit.

With an asynchronous reset

An asynchronous reset must be in the sensitivity list too, since it can activate independently of the clock, and it takes precedence when it is active.

```

process (clock, reset_n) is
begin
    if (reset_n = '0') then
        q <= '0';
    elsif (rising_edge(clock)) then
        q <= d;
    end if;
end process;

```

Read it in priority order: check the reset first, and look for a rising edge only if it is inactive. If neither holds (a falling edge, or `reset_n` going from 0 back to 1, which also reruns the process) nothing is assigned and q keeps its previous value.

When the assignment actually takes effect

One rule underlies all of the above, and it is the one place where `<=` behaves unlike assignment in every language you have written software in.

A signal assignment does not take effect when its line runs. It **schedules** a value, which is applied only once the process has finished this round and suspends. Until then, every read

of that signal gives the value it held *before the round began*, however many assignments have already run.

That sounds like a technicality. It is the reason the template above is a flip-flop and not a wire:

```
process (clock) is
begin
    if (rising_edge(clock)) then
        b <= a;
        c <= b;
    end if;
end process;
```

Read as software, both *b* and *c* end up holding *a*. In hardware they do not. At each edge *b* is scheduled to take *a*'s value, and *c* is scheduled to take the value *b* held *going into that edge*, not the one just scheduled onto it. The result is two flip-flops in a chain: *a* reaches *b* after one edge and *c* after two. Swap the lines and nothing changes, because neither assignment can observe the other's result.

Two consequences worth taking away:

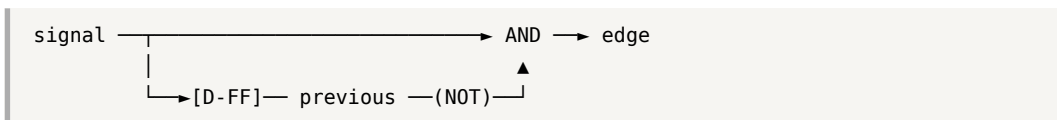
- **Order does not matter** between assignments to *different* signals in the same clocked process. All of them read the old values and all of them take effect at once. That is the opposite of the sequential reading the word “process” invites, and it is why a chain of flip-flops can be written in any order.
- **Assign the same signal twice in one round and only the last assignment survives.** The earlier ones are discarded without ever reaching the signal.

Section 3.7's edge detector is exactly the two-element chain above, and it works for exactly this reason. Chapter 5 returns to the rule from the other side, with a *variable*, which updates immediately and therefore behaves the way the software reading expects.

3.7 Edge detection

The flip-flop's other everyday job is detecting the *moment* a signal changes, rather than reading its level: store the signal's previous value in a flip-flop and then compare it with the current value using a gate.

- **Rising edge detection** (0 to 1): $edge = current \cdot previous'$
- **Falling edge detection** (1 to 0): $edge = current' \cdot previous$



▲
clock, reset_n

edge is high for exactly one clock cycle, the one immediately after the transition, because previous only catches up at the *next* edge. That is the mechanism behind “do something once per button press” instead of “do something every clock cycle the button is held down”, which is exactly what section 3.9 solves.

3.8 Worked example: a D flip-flop in VHDL

d_flip_flop.vhd implements section 3.3 and 3.6 together: an enable gating whether a rising edge captures d, and an asynchronous active-low reset.

```
lectures/L03/d_flip_flop/d_flip_flop.vhd

-----
-- Rising-edge D flip-flop with asynchronous reset and enable.
--
-- Inputs:
--   - clock : 50 MHz system clock.
--   - reset_n: Active-low asynchronous reset.
--   - d      : Data input.
--   - enable : Capture enable ('1' = load d on the edge, '0' = hold).
-- Outputs:
--   - q : Flip-flop output.
--   - q_n: Inverse of q.
-----

library ieee;
use ieee.std_logic_1164.all;

entity d_flip_flop is
    port(clock, reset_n, d, enable: in std_logic;
         q, q_n : out std_logic);
end entity;

architecture behaviour of d_flip_flop is
    -- Internal representation of q.
    signal q_s: std_logic;
begin
    q    <= q_s;
    q_n <= not q_s;

    -- Implement D flip flop logic.
    process (clock, reset_n) is
```

```

begin
    if (reset_n = '0') then
        q_s <= '0';
    elsif (rising_edge(clock)) then
        if (enable = '1') then
            q_s <= d;
        end if;
    end if;
end process;
end architecture;

```

Its single process is the reset-plus-enable variant of section 3.6's template: the reset takes precedence, then a rising edge captures `d` into an internal signal `q_s` if `enable = '1'`. `q_s` exists because an *output port* cannot be read back inside the same architecture; `q` and `q_n` are both driven combinatorially from it, outside the process.

To try it on hardware, assign `clock` to the 50 MHz system clock, `reset_n` to a push button, `d` and `enable` to two slide switches, and `q/q_n` to two LEDs. Then turn `enable` on and change `d`, and the LED follows; turn `enable` off and change `d`, and it does not.

There is no clock edge you can perceive at 50 MHz, so what you are really confirming is that the logic still holds when you can no longer see the individual edges. The flip-flop appears to respond to your switch; in fact it responded to one of fifty million clock edges that second, and the switch only decided which value that edge captured.

3.9 Worked example: edge-detected LED toggle

`led_toggle.vhd` combines section 3.5, 3.6 and 3.7 into a small system: two independent push buttons, each toggling its own LED exactly once per press.

```

lectures/L03/led_toggle/led_toggle.vhd
-----
-- Toggles led(i) on each falling edge of button_n(i); all LEDs off on reset.
--
-- Inputs:
--   - clock    : 50 MHz system clock.
--   - reset_n  : Active-low reset.
--   - button_n : Active-low push buttons.
-- Outputs:
--   - led: LED toggled by the matching button.
-----
library ieee;
use ieee.std_logic_1164.all;

```

```

entity led_toggle is
    port(clock, reset_n: in std_logic;
          button_n      : in std_logic_vector(1 downto 0);
          led            : out std_logic_vector(1 downto 0));
end entity;

architecture behaviour of led_toggle is
    -- Previous button values (for edge detection).
    signal button_prev_n: std_logic_vector(1 downto 0);

    -- Button pressdown indicators (falling edge).
    signal button_edge: std_logic_vector(1 downto 0);

    -- LED states (necessary for toggling, as output signals cannot be read).
    signal led_s: std_logic_vector(1 downto 0);
begin
    -- Detect a falling edge of button_n.
    button_edge <= (not button_n) and button_prev_n;
    led <= led_s;

    -- Update button_prev_n on system reset or on rising edge of the system clock.
    BUTTON_PROCESS: process(clock, reset_n) is
    begin
        if (reset_n = '0') then
            button_prev_n <= "11";
        elsif (rising_edge(clock)) then
            button_prev_n <= button_n;
        end if;
    end process;

    -- Update led_s on system reset or on rising edge of the system clock.
    LED_PROCESS: process(clock, reset_n) is
    begin
        if (reset_n = '0') then
            led_s <= "00";
        elsif (rising_edge(clock)) then
            for i in 0 to 1 loop
                if (button_edge(i) = '1') then
                    led_s(i) <= not led_s(i);
                end if;
            end loop;
        end if;
    end process;
end architecture;

```

Three 2-bit signals, two held in clocked processes and one computed between them:

- `button_prev_n` is a 2-bit register (section 3.5) holding last cycle's `button_n`, reset to "11" because the buttons are active low.
- `button_edge` $\leq$ (not `button_n`) and `button_prev_n` is section 3.7's falling-edge detector applied to both bits at once, high for one cycle exactly when a button goes from released to pressed.
- `led_s` is a second 2-bit register which, instead of capturing a new value every cycle, **toggles** bit `i` only when `button_edge(i) = '1'`. Without that gating it would toggle every clock cycle instead of once per press, which is the whole point of computing `button_edge`.

On the board. Create the Quartus Prime Lite project against the DE0-CV's 5CEBA4F23C7N exactly as in chapter 1, then assign `clock` to the 50 MHz oscillator, `reset_n` and `button_n` (1 downto 0) to three push buttons, and `led` (1 downto 0) to two LEDs. Every press should toggle exactly one LED, once, however long you hold it down.

One caveat, deliberately left open. `button_n` is a raw signal straight from a physical push-button pin. In simulation it is a clean, instantaneous transition; on real hardware it is neither instantaneous (contacts bounce) nor synchronised to `clock`. This edge detector is logically correct but *unsafe* to wire directly to a real button on an FPGA without more work first, and that work is exactly what chapter 4 takes up.

3.10 Summary

What you should be able to do now

- Say why sequential networks exist, explain the functional difference between a D latch and a D flip-flop, and why synchronous designs are built almost exclusively from the flip-flop.
- Read and write the synchronous process template, and recognise a register as several flip-flops sharing a clock, a reset and an enable.
- Say when a signal assignment takes effect: $\leq$ schedules rather than applies, so two assignments in a clocked process build a chain, and their order relative to each other does not matter.
- Build an edge detector from a flip-flop and a gate, and say exactly which signal it produces and for how long.

Questions to test yourself with

1. What is the functional difference between a D latch and a D flip-flop, and under what condition does each change its output?
2. Why do synchronous circuits update their state at a single clock edge instead of reacting continuously to their inputs?
3. Given a signal and a flip-flop holding its value from one cycle earlier, how do you combine them into a pulse that is high for exactly one clock cycle when the signal goes high?
4. Why must clock be the only signal in a flip-flop process's sensitivity list, unless the design has an asynchronous reset?
5. A clocked process contains `b <= a;` and after it `c <= b;`. What does c hold after a clock edge, and why is it not a? What changes if you swap the lines, and why?

Looking ahead

Chapter 4 goes on with:

- Why the raw button feeding this chapter's edge detector can drive a flip-flop into an undefined, unstable state, and why that is a different kind of problem from anything here.
- The two-flip-flop synchroniser, and what it does and does not do about a bouncing button.
- The same LED toggle, rebuilt so that it is actually safe on real hardware.

3.11 Exercises

How to check your work. Every exercise below that asks you to write a VHDL module comes with a self-checking testbench under `lectures/L03/exercises/`. Write your module in its directory `exercises/<module>/`, with the entity name and the **port order** the exercise states, and then run it with GHDL — see section 2.7 for the three commands. This is the first chapter whose exercises need that for a clocked design, so it is worth rereading before you start on register4.

EXERCISE SET 1

The D latch

Exercise 3.1 The first circuit that remembers

CIRCUIT

Build the first circuit in this book that can *remember* anything.

Everything you have built so far has been combinational: the outputs follow the inputs, and the moment the inputs change the outputs change with them. A latch is the smallest possible departure from that. It has a data input D and a control input *enable*, and it has two outputs, Q and its complement Q_n . As long as *enable* is high the latch is **transparent**: Q simply follows D , and it looks like a wire. What is interesting is what happens when *enable* goes low. The latch **latches**, holding whatever D was at that instant, and from then on D can do as it likes without Q noticing. That held value is one bit of memory, and it is built from nothing but the gates you already know, wired so that the output feeds back to the input.

Q_n is not decoration. The feedback loop is what makes the circuit remember, and the two outputs are what feed each other: Q is computed from Q_n and Q_n from Q . Build it and you see the loop; that is the whole point of drawing it by hand before you even meet the VHDL.

Use the equations from section 3.2:

$$Q = (D' \cdot \text{enable} + Q_n)' \quad Q_n = (D \cdot \text{enable} + Q)'$$

- a) Realise the corresponding gate network: draw the network by hand, and recreate and simulate it in CircuitVerse.
- b) Test the transparent and the latched state. Set *enable* = 1, change D , and observe what happens to Q and Q_n . Then set *enable* = 0, change D again, and observe what happens. Explain the difference between the two cases.
- c) Run D and *enable* through the combinations 00, 01, 10, 11, twice. Confirm that you observe both cases: the latch latching after being transparent with $D = 1$, and the latch latching after being transparent with $D = 0$. Explain how the value of Q depends on the value of D at the moment *enable* changes from 1 to 0.

EXERCISE SET 2

The D flip-flop

Exercise 3.2 From latch to flip-flop

CIRCUIT

Extend the D latch from Exercise 3.1 into a D flip-flop, following section 3.3.

The flip-flop should have the inputs *clock*, *reset_n*, D and *enable*, and the outputs Q and Q_n .

The reset signal is **asynchronous** (it takes effect without waiting for a clock edge) and **active low** (setting *reset_n* = 0 should force Q to 0 immediately).

Hints

- The reset has to do two things in each of the two internal latches, not one:
 - Feed an active-high reset (`reset_n` through a NOT gate) into the NOR gate producing `Q`. That forces `Q` low without waiting for a clock edge.
 - AND `reset_n` into the AND gate producing that latch's product of `D` and `enable`. That frees `Qn` to go high.
 - Gating only the feedback path for `Q` is not enough: with `enable = 1` and `D = 1` it leaves `Qn` low, and a low `Qn` is exactly what holds `Q` high, so the reset would do nothing.
 - Drive the two internal latches' enable inputs with `clock` and the inverse of `clock` respectively.
 - Follow section 3.3 for how the external `enable` input controls whether the flip-flop takes a new value.
- a) Realise the corresponding gate network: draw it by hand, recreate and simulate it in CircuitVerse, and set the clock period to 1000 ms, slow enough to observe the behaviour by eye.
- b) Test the clocked behaviour: change `D` during the clock's high phase, and then during its low phase, and observe `Q` and `Qn`. Confirm that the outputs change only at the active clock edge, never between clock edges. Explain how the two internal latches work together to give edge-triggered behaviour.
- c) Test the asynchronous reset: first set `Q = 1`, then activate the reset by setting `reset_n = 0`, and observe whether `Q` clears immediately or at the next clock edge. Explain the result by reference to the difference between an asynchronous signal and a synchronous one.

EXERCISE SET 3

Registers**Exercise 3.3** `register4`

VHDL

Write a complete VHDL entity named `register4`: four flip-flops side by side, sharing one clock, one reset and one enable.

This is the exercise where the flip-flop stops being a curiosity and becomes a building block. A lone `D` flip-flop stores one bit, which is not much use in itself; what a design actually wants is somewhere to keep a *number*. Put four flip-flops in a row, connect the same clock to all of them, and you have a four-bit register. Nothing about the individual flip-flop changes. Only the width does.

What is worth noticing is how little the VHDL has to change to say that. You write the same synchronous process template from section 3.6 that you already used for one bit, with `std_logic_vector(3 downto 0)` instead of `std_logic`. There is no loop, no repetition, no

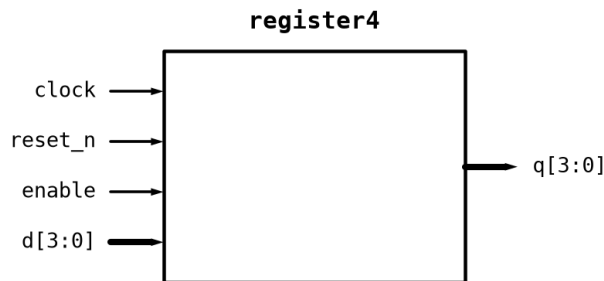
four copies of anything. That is not a shortcut the language is offering you; it is a fact about the hardware, and part c) below asks you to say why.

The entity should have:

Port	Dir.	Type	Description
clock	in	std_logic	System clock.
reset_n	in	std_logic	Asynchronous, active low. Clears the register to "0000".
enable	in	std_logic	'0' keeps the current value; '1' captures d at the next rising edge.
d	in	std_logic_vector(3 downto 0)	The value to capture.
q	out	std_logic_vector(3 downto 0)	The stored value.

Write the process out yourself rather than copying the worked flip-flop example in section 3.8. Typing it in is what makes the template stop being something you recognise and start being something you can produce from memory, and you will write it in every remaining chapter of this book.

- Write the entity and the synchronous process, adapting the template from a single bit to a four-bit vector.
- Check it with the testbench, as described at the top of this section.
- Explain why the same template scales from one flip-flop to four with no structural change, while a four-bit *counter* cannot simply be four one-bit counters side by side. What separates the two cases?



Self-check. Name your entity `register4`, with the ports `clock`, `reset_n`, `enable`, `d` (`std_logic_vector(3 downto 0)`) and `q` (`std_logic_vector(3 downto 0)`), declared in that order; its testbench is in `exercises/register4/`.

EXERCISE SET 4

Edge detection

Exercise 3.4 The toggle circuit by hand

CIRCUIT

Design a circuit that toggles an LED once for every sampled rising edge of a button signal. Follow section 3.7 and section 3.9.

The circuit should have the inputs button, clock and reset, and the output led.

In this exercise reset is active high, unlike the reset in the worked example. It is still **asynchronous**, like every reset in this book: activate it and led clears immediately, without waiting for a clock edge. In the template from section 3.6 that means the reset branch stays outside the `rising_edge(clock)` test, and only the polarity of the reset test changes.

Hints

Use two flip-flops. One stores the previous sampled value of button, and that value is used to detect a rising edge. The other stores the LED's current state, and is toggled only when a rising edge is detected.

a) Realise the corresponding gate network: draw it by hand, recreate and simulate it in CircuitVerse, and set the clock period to 1000 ms.

b) Test the edge detector: hold button high for several consecutive clock cycles, confirm that led toggles exactly once, and that it does not toggle once per clock cycle. Explain which stored signal stops the circuit from repeatedly detecting the same high level as a new rising edge.

Exercise 3.5 led_toggle_single

VHDL

Implement the circuit from Exercise 3.4 in VHDL.

a) Create an entity named `led_toggle_single` with:

Port	Dir.	Type	Description
clock	in	std_logic	System clock.
reset	in	std_logic	Asynchronous, and active high , unlike every other module in this book. Activate it and led clears without waiting for a clock edge, so the reset branch belongs outside the <code>rising_edge(clock)</code> test.
button	in	std_logic	Toggles led once per press.
led	out	std_logic	The LED's state.

Note the single-bit types: the worked example uses `std_logic_vector` ports for two buttons and two LEDs, while this one drives a single pair.

Do not copy `led_toggle.vhd` wholesale. Do this instead:

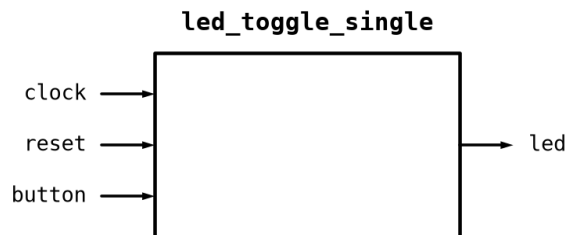
- Derive the two clocked processes yourself: one stores the previous value of `button`, the other stores and toggles the current value of `led`.
- Derive the edge-detection equation for a rising edge and an active-high button, that is, section 3.7's $edge = current \cdot previous'$.
- Do not use the falling-edge equation from the worked example, which assumes an active-low button.

b) Verify the design with its testbench.

The testbench drives the same sequence you would go through by hand on a board:

- It activates reset and checks that `led` is cleared.
- It pulls `button` high and checks that `led` toggles exactly once.
- It **holds button high** for several more clock cycles and checks, after each of them, that `led` does *not* toggle again.
- It releases and presses again, and checks that the second press toggles `led` once more.
- It activates reset with `led` lit and **with no clock edge afterwards**, and checks that `led` clears anyway. That is the check that fails if you put the reset inside the `rising_edge(clock)` branch.

The third check is the one that fails if your edge detector is wrong. An `led` toggling every clock cycle while the button is held down means you drove the toggle from button's **level** instead of from its edge. Reread the equation above: edge is high only when button is high *now* and its stored previous value is low.



Self-check. Name your entity `led_toggle_single`, with the ports `clock`, `reset`, `button` (in) and `led` (out), declared in that order; its testbench is in `exercises/led_toggle_single/`. Note that reset is **active high** here, unlike in the worked example.

Metastability and synchronisation

Making an asynchronous input safe to use in a synchronous system.

IN THIS CHAPTER

- Why an asynchronous input can wreck the clean edge detectors and registers built in chapter 3.
- Metastability: what it is, and why it cannot simply be designed away.
- The two-flip-flop synchroniser, and the “assert asynchronously, release synchronously” pattern for reset.
- Reusing the same chain for edge detection, and what it does and does not do about bounce.
- Generics: one module serving several sizes.
- `led_toggle_sync`, driven by a real, bouncing push button.

Before the synchroniser goes in: predict what a real, asynchronous button does to chapter 3’s edge detector. That prediction is the reason for the whole chapter.

The argument worth holding on to is why two flip-flops make the failure *unlikely* rather than impossible. That is the part that does not survive being remembered as a rule of thumb.

Refresher: Metastability, the two-flip-flop synchroniser and what it does not fix are also in sections 5.7–5.9 of *Digital Electronics*.

4.1 The problem: a button is not a testbench signal

In chapter 3 you built edge detectors and reasoned about them against a clean clock, where every input changed tidily between the edges, exactly as a simulator draws it. A push button, an external reset line, or any signal from another clock domain changes when the outside world decides to, entirely independently of your FPGA’s clock.

Every D flip-flop has two timing requirements around the active clock edge:

- a **setup time**: how long D must be stable *before* the edge.

- a **hold time**: how long D must remain stable *after* it.

If an input changes inside that window, the flip-flop is not guaranteed to capture a clean 0 or 1, and you can never promise that it lands outside, since you do not control when an external signal changes. The rest of this chapter solves that not by avoiding the problem but by containing it.

4.2 Metastability: what actually happens inside the flip-flop

A D flip-flop is built from cross-coupled gates that settle to 0 or 1 at the moment the clock samples the input. If D changes too close to the edge, the internal feedback loop can be left balanced between the two states, neither fully high nor fully low, for longer than expected. That is **metastability**.

Picture a ball balancing exactly on top of a hill. Given enough time it rolls down one side, but exactly when it makes up its mind, and which way, cannot be predicted from the moment it was placed there, and as long as it is still balancing its position is not a valid digital value at all.

A metastable output behaves the same way:

- it almost always settles to a valid 0 or 1 by itself, given enough time.
- “enough time” is not free, and is not the same every clock cycle.
- if the unsettled voltage reaches downstream logic before it has settled, different gates may quite legitimately read it differently: one sees 0, another 1, at the same instant.

That disagreement is where real systems misbehave: a counter that increments in one part of the circuit but not in another, or a state machine in an impossible state. Everything in this chapter exists to make sure the metastability settles *before* a signal touches the rest of your design.

In time, the whole problem looks like this:

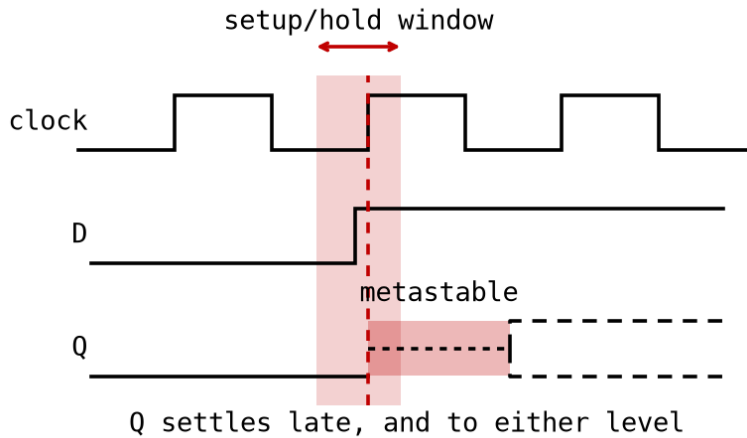


Figure 4.1 D changes inside the setup/hold window around a rising clock edge, leaving Q metastable before it settles late on either level.

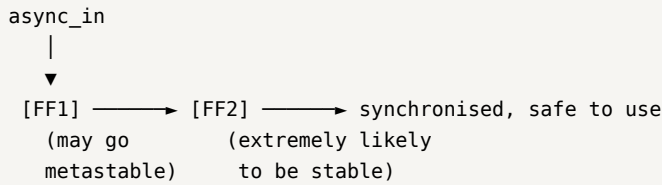
Read it left to right. The shaded band is the window from section 4.1, t_{su} before the edge and t_h after it. D changes inside it, which is exactly what an input you do not control is free to do. Q is then neither 0 nor 1 for a while, and once it does settle, both *when* it settles and *which level* it settles on are outside your control. Both dashed continuations are legitimate outcomes of the same input.

Note what the diagram does not show: a wrong answer. Sampling a signal in the middle of a transition is allowed to give you either level, and that in itself is harmless, since the input really was changing at that instant. The problem is the delay before you get an answer at all, and that is what the next section buys time for.

4.3 The two-flip-flop synchroniser

The standard solution is the **two-flip-flop synchroniser**:

- Put two D flip-flops in series on every asynchronous input: every input that changes with no fixed relationship to your clock. A push button, a reset line, a signal arriving from another clock domain.
- An input already clocked by *this* clock is not asynchronous, and must **not** be synchronised. Doing so only delays it, and shifts its timing relative to everything around it. `seq_detect_mealy` in chapter 8 is exactly that case: its `din` is synchronous serial data, so it goes straight into the machine while `reset_n` still gets a synchroniser.
- Never use anything but the *second* flip-flop's output elsewhere in the design.



The same two flip-flops, in time:

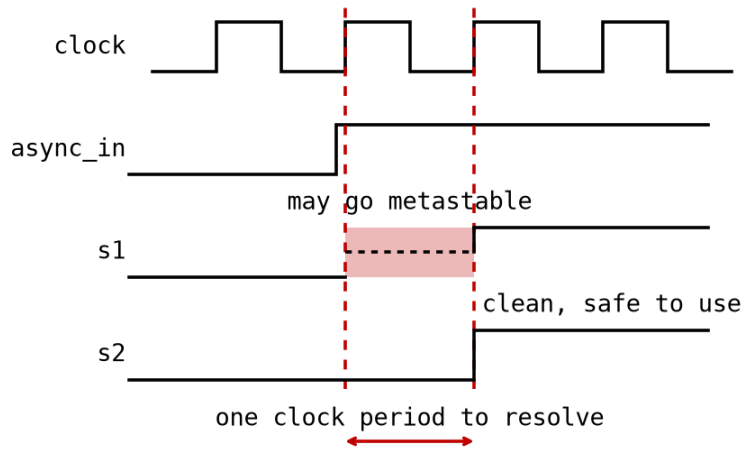


Figure 4.2 `async_in` changes just before a clock edge, `s1` goes metastable for one clock period, and `s2` comes out clean one edge later.

FF1 is the one exposed to the asynchronous input, so it is the one that can go metastable. Its output only has to reach FF2, and it has a whole clock period to do it. The probability that a metastable node is still unsettled after a whole period is small, and shrinks the longer it is given, so by the time FF2 samples on the next edge, FF1 has nearly always settled. FF2's output is what you build the rest of your synchronous logic on.

This is probabilistic, not absolute, and section 4.4 says what that means. At the speeds used in this book, tens of MHz, two stages are the usual, industry-accepted answer.

One rule ties it together: **every asynchronous input gets its own two-flip-flop chain before it is used anywhere else.** External reset buttons, push buttons, and later every **single-bit signal** crossing from another clock domain.

That last phrase is load-bearing, and it is the one limitation worth learning before you overuse the rule. A two-flip-flop chain synchronises **one bit**. Run a multi-bit bus through one chain per bit and every bit is safe on its own but they are not safe *together*: each settles independently to the old or the new value on the same edge, so the receiver can sample a combination that was

never on the bus. A counter stepping from 0111 to 1000 has every bit change at once, and one synchroniser per bit can give you 1111 or 0000, values the counter never held.

Busess crossing clock domains need a handshake, a FIFO, or Gray coding, which arranges things so that only one bit ever changes at a time. Nothing in part I crosses clock domains with a bus, so you will not need them there; the group project returns to the limitation in chapter 12, where the same synchroniser is written once and reused. Knowing that the two-flip-flop answer stops at one bit is enough for now.

4.4 Intuition for the timing: why “almost certainly” will do

A synchroniser does not buy “no metastability”. It buys metastability that has a whole clock period to settle before it can affect anything else, and that is a statistical argument rather than a proof. The correct stance is “reasonably safe, backed by an extremely low, well-characterised failure rate”, not “provably safe under all conditions”, which is why extra stages exist for extreme cases instead of two being declared sufficient by a law of nature.

The relationship has a standard form, worth seeing once even if you will not be computing with it here:

$$MTBF = \frac{e^{t_r/\tau}}{T_w \cdot f_{\text{clock}} \cdot f_{\text{data}}}$$

Read it for what it says rather than for numbers. t_r is the settling time one stage gets, roughly one clock period; τ and T_w are both properties of the flip-flop, where τ is how fast a metastable value decays and T_w the width of the window around the clock edge where the input can induce one at all. Adding a stage buys another t_r in the exponent, so **the numerator grows exponentially with every stage you add**, against one clock cycle of added latency. The denominator grows linearly with the clock frequency and, just as importantly, with f_{data} : how often the asynchronous input actually changes. A button pressed once a second and a data line toggling at 10 MHz differ by seven orders of magnitude in that term alone, which is why the argument for a third stage is a question about the input, not about the clock.

4.5 Synchronising a reset signal: assert asynchronously, release synchronously

Reset deserves a pattern of its own, which `reset_sync` demonstrates. There is no `reset_sync.vhd` to open in the course repository: it is Exercise 4.8, and you write it. The listing below is the module’s whole process.

```

process(clock, reset_n) is
begin
    if (reset_n = '0') then
        reset_s1_n <= '0';
        reset_s2_n <= '0';
    elsif (rising_edge(clock)) then
        reset_s1_n <= '1';
        reset_s2_n <= reset_s1_n;
    end if;
end process;

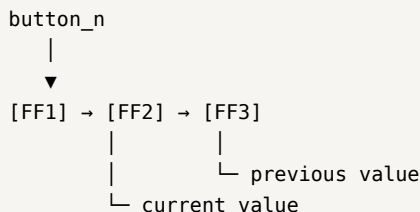
```

- **Assert asynchronously.** When `reset_n` goes low, both stages are forced to 0 immediately, with no dependence on the clock. A reset that waited for a clock edge would defeat its own purpose: it has to take effect the moment it is needed, with or without a clock.
- **Release synchronously.** When `reset_n` returns to 1, the reset condition does not vanish everywhere at once. It is shifted out through the two stages like any other asynchronous input, so `reset_s2_n`, the signal the design actually uses, releases cleanly on a clock edge along with everything else. Without this, different flip-flops could leave reset on different cycles and disagree for a moment about the state of the system, the same kind of failure metastability causes.

That is why `reset_n` is synchronised by a chain of its own before anything else depends on it, including the button synchroniser, whose reset input is `reset_s2_n` rather than the raw `reset_n`.

4.6 Reusing the chain for edge detection, and debouncing in passing

The button needs synchronising, and from chapter 3 you also need to detect *when* it is pressed rather than its level. `button_sync`, which is Exercise 4.9 and therefore also yours to write rather than a file in the course repository, gets both from a single chain of **three** flip-flops:



Flip-flops 1 and 2 are section 4.3's synchroniser, which makes `button_s2_n` the stable current value. Flip-flop 3 stores last cycle's `button_s2_n` as `button_s3_n`, the previous value. Flip-flops 1 and 2 are worth noticing separately: Exercise 5.4 factors that two-flip-flop chain

out into a reusable sync module, leaving the third flip-flop here as the edge detector's own. Comparing stage 2 against stage 3 gives edge detection for free:

```
button_edge_s2 <= (not button_s2_n) and button_s3_n;
```

which is true for exactly one clock cycle: the one where the button reads as pressed now and was not the cycle before.

Synchronisation by itself solves only metastability. It says nothing about a mechanical button's tendency to bounce: making and breaking contact several times during the first millisecond or so, where every bounce looks like a press of its own. What keeps this book's exercises from drowning in bounce is the *combination* of the chain and a detection window that is slow relative to the bounce:

- In this chapter's CircuitVerse exercises the clock period is a whole second. The bounce settles in far less than that, so at the next edge `button_s2_n` reads a single clean level. The chain looks like it debounces, but it really just samples too rarely to see the bounce.
- On the FPGA the clock is 50 MHz, one sample every 20 ns, while the bounce lasts from under a millisecond to a few tens of milliseconds, many thousands of cycles. At that rate `button_sync` faithfully detects *every* bounce transition as an edge of its own, and the LED may toggle several times for a single physical press.

So, precisely: the two-flip-flop synchroniser solves metastability reliably at the speeds used here. Whether it also *appears* to debounce depends entirely on the clock period being slow relative to the button's bounce time, which is true in CircuitVerse and not automatically true on the FPGA.

4.7 What a real hardware debounce adds

This book has no chapter of its own on debouncing, and the worked examples deliberately go on using section 4.6's circuit, bounce and all. It will do for a demo LED. A production design adds one of the following *on top*:

- **An analogue RC low-pass filter**, possibly with a Schmitt-trigger buffer, on the button input before it reaches an FPGA pin. The RC pair smooths the fast make-and-break transitions into a single gradual edge, and the Schmitt trigger, with its two switching thresholds, turns that back into a single clean digital transition. It solves the bounce before the signal is even digital, but costs components on the board and does not help for inputs that are not mechanical contacts.
- **A digital debounce counter**. Instead of acting on the next detected edge, require the input to hold a new level for some minimum time, a few milliseconds' worth of clock cycles, before accepting it. That builds your own artificially slow detection window on the chip, the same effect CircuitVerse's 1-second clock gives for free, without slowing the

system clock down. You meet counters in chapter 6 and timers, which this technique needs, in chapter 7.

Both sit on top of the synchroniser, not instead of it: even a perfectly debounced signal is still asynchronous to your clock and still needs a two-flip-flop chain.

4.8 Generics: one module, several sizes

reset_sync and button_sync are built from the same idea, but only one of them has a question to answer. A reset is one bit and always will be. How many *buttons* does a design have? This chapter's led_toggle_sync has one, chapter 8's fsm_led has two. Writing the same three-flip-flop chain out once per width is exactly the duplication that splitting things into modules was meant to avoid.

A **generic** answers it: a value an entity declares alongside its ports, fixed when the design is built rather than changeable while it runs.

```
entity button_sync is
  generic(COUNT: natural range 1 to 3 := 1);
  port(clock, reset_s2_n: in std_logic;
        button_n       : in std_logic_vector(COUNT-1 downto 0);
        button_edge_s2  : out std_logic_vector(COUNT-1 downto 0));
end entity;
```

- COUNT behaves like a constant inside the architecture. It cannot change while the circuit runs; it is decided before a single gate is placed.
- := 1 is its default value, used by every instantiation that says nothing.
- The port widths are written *in terms of it*, and so is every internal signal that follows.

Overriding it

An instantiation passes generics with a generic map, before its port map:

```
button_sync1: entity work.button_sync
  generic map(2)
  port map(clock, reset_s2_n, button_n, button_edge_s2);
```

Generics are matched **positionally**, in declaration order, exactly as ports are in section 2.6. Omitting the generic map applies the default; this book writes it out explicitly even when it agrees, since a reader should not have to open the module to find out how wide an instance is.

When a generic earns its place

button_sync has one and reset_sync has none, and that difference is the whole rule: **add a generic when something genuinely varies between instances, not on principle.**

button_sync is instantiated with COUNT = 1 by section 4.9's led_toggle_sync and with COUNT = 2 by chapter 8's fsm_led: two widths, one file. reset_sync is one bit everywhere, so a WIDTH generic on it would be a parameter nobody ever passes anything but 1 to, and one more thing for a reader to check. The second instantiation is what makes a generic worth having; a parameter with exactly one value in the whole book would just be an extra step in between. button_sync's own testbench instantiates it with *both* widths in the same simulation, precisely to prove that one file really does serve both.

You meet one more generic in chapter 7: timer's TICK_COUNT, which is a count rather than a width, and which is what lets the same file measure a fiftieth of a second in one design and a whole second in another.

4.9 Worked example: led_toggle_sync

This ties section 4.1–4.6 together with chapter 3's toggle circuit, made safe for a real, asynchronous button and reset.

Port	Dir.	Description
clock	in	50 MHz system clock on the DE0-CV board.
reset_n	in	Active-low asynchronous reset, connected to a push button.
button_n	in	Active-low push button; toggles the LED on its falling edge.
led	out	LED, toggled at every synchronised, edge-detected button press.

Internal signals: led_s holds the LED's state, reset_s2_n is the reset after the two-flip-flop synchroniser (the suffix s2 is the book's convention for "synchronised with two flip-flops"), and button_edge_s2 is the one-cycle pulse marking a synchronised falling edge on button_n.

Like chapter 2's two-digit display, this design is built from more than one entity, using the instantiation from section 2.6 and the generic from section 4.8. The difference is what the parts do: the display instantiated one module twice to do the same job on two halves of its input, while this one instantiates two *different* modules with one job each. Both are the same mechanism. Composing a design out of modules is not a special technique for repeated logic; it is how designs are built once they have outgrown a single file.

The two chains live in two subcomponents:

- reset_sync is section 4.5's two-flip-flop reset synchroniser. It takes the raw reset_n and returns reset_s2_n.
- button_sync is section 4.6's three-flip-flop synchroniser with edge detector. It takes the already synchronised reset_s2_n and pulses button_edge_s2 once per press.

Keeping them apart means more than it looks. A design with no button of its own still needs its reset synchronised, and with two modules it simply instantiates the one it wants,

as chapter 8's `seq_detect_mealy` does. Welded into one module, that design would have to instantiate the lot and disconnect the half it does not use.

`button_sync` takes a *vector* of buttons so that one file can serve a design with two buttons. This design has one, so the top level declares a pair of one-element vectors to connect to it; that is plumbing, not logic, two wires renamed. Full source:

```
lectures/L04/led_toggle_sync/led_toggle_sync.vhd
-----
-- Toggle an LED on each synchronized, edge-detected press of button_n.
-- The LED is cleared on reset.
--
-- Inputs:
--   - clock: 50 MHz system clock.
--   - reset_n: Active-low asynchronous reset.
--   - button_n: Active-low button.
-- Outputs:
--   - led: Toggled on the falling edge of button_n.
-----

library ieee;
use ieee.std_logic_1164.all;

entity led_toggle_sync is
    port(clock, reset_n, button_n: in std_logic;
         led                      : out std_logic);
end entity;

architecture behaviour of led_toggle_sync is
    -- Stable reset signal synchronized using a two-flip-flop synchronizer.
    signal reset_s2_n: std_logic;

    -- Pulse indicating a falling edge on button_n, synchronized using a two-flip-flop-
    --   synchronizer.
    signal button_edge_s2: std_logic;

    -- One-element vectors adapting the scalar button signals above to button_sync,
    -- whose button ports are COUNT-bit vectors.
    signal button_n_v, button_edge_s2_v: std_logic_vector(0 downto 0);

    -- Internal LED state.
    signal led_s: std_logic;

begin
    led <= led_s;
```



```

-- Adapt the scalar button signals to button_sync's vector ports.
button_n_v(0)    <= button_n;
button_edge_s2 <= button_edge_s2_v(0);

-- Synchronize the asynchronous reset.
reset_sync1: entity work.reset_sync
    port map(clock, reset_n, reset_s2_n);

-- Synchronize button_n and pulse on each press (a single button, so COUNT = 1).
button_sync1: entity work.button_sync
    generic map(1)
    port map(clock, reset_s2_n, button_n_v, button_edge_s2_v);

-- Toggle the LED on button press event.
LED_PROCESS: process(clock, reset_s2_n) is
begin
    if (reset_s2_n = '0') then
        led_s <= '0';
    elsif (rising_edge(clock)) then
        if (button_edge_s2 = '1') then
            led_s <= not led_s;
        end if;
    end if;
end process;
end architecture;

```

The top-level architecture contains nothing but the toggle process itself, and it never touches anything but the already synchronised `reset_s2_n` and `button_edge_s2`, never the raw inputs.

The two subcomponents are not in the course repository, because writing them is Exercise 4.8 and 4.9: `reset_sync` is section 4.5 above, and `button_sync` is section 4.6 and section 4.8, both written out in full. Copy your own into `led_toggle_sync/` to build it.

By hand, in CircuitVerse first. Before you touch VHDL, build it as a gate network: two D flip-flops in series for the reset synchroniser; two for `button_n` plus a third storing the previous synchronised value; an AND gate combining the inverted current value with the previous one to produce `button_edge_s2`; and a toggle flip-flop for `led_s`, with its inverted output fed back to its input, enabled only when `button_edge_s2 = 1`. Set the clock period to 1000 ms so that you can follow every step by eye.

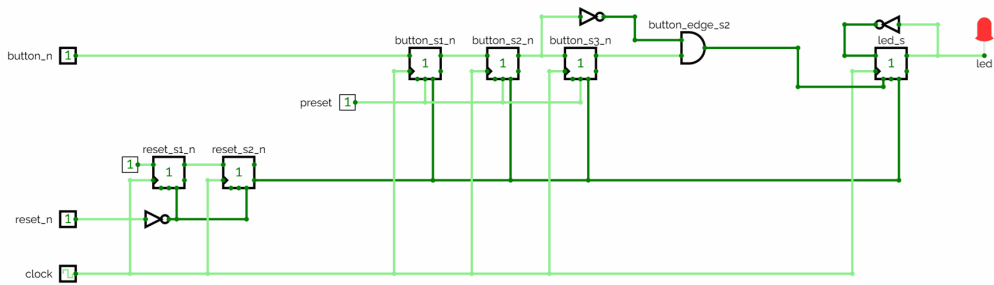


Figure 4.3 Synchronised LED toggle circuit: two-flip-flop synchronisers for reset and button, edge detection, and a toggle flip-flop for the LED.

One thing in the drawing needs reading closely: the flip-flops have a preset input, and on the button synchroniser’s three stages it is tied to 1. That is not decoration. In CircuitVerse, preset is the value a flip-flop loads when its reset is activated, so tying it to 1 is what makes button_s1_n, button_s2_n and button_s3_n come out of reset holding 1 rather than 0.

That agrees with button_sync, which resets the same three stages to (others => '1'), and it matters for the same reason: the buttons are active low, so 1 means “released”. Reset the chain to 0 instead and it starts out claiming the button is already held down. The reset flip-flops leave preset alone, since a reset to 0 there really is all the circuit asks for.

Press the button in the simulation and follow how it propagates. The LED does not toggle immediately: it waits for the synchronised, edge-detected pulse to reach the toggle flip-flop, which takes a small, fixed number of clock pulses, exactly the latency section 4.3 called the price of the safety.

On the FPGA. The same design is demonstrated on the Terasic DE0-CV (device 5CEBA4F23C7N) during the session, with clock on the 50 MHz oscillator, reset_n and button_n on two push buttons, and led on an LED. The LED toggles at every press, and per section 4.6 a real press may sometimes give more than one toggle because of contact bounce.

4.10 Summary

What you should be able to do now

- Explain what metastability is, why an asynchronous input can cause it, and why two flip-flops make it unlikely rather than impossible.
- Apply the two-flip-flop synchroniser to any asynchronous input, including the “assert asynchronously, release synchronously” pattern used for reset.
- Say why that chain also works as a practical but incomplete debounce, and what a real one would add.

- Give a module a generic, override it with a generic `map`, write a port's width in terms of it, and judge when a generic earns its place instead of being added on principle.

Questions to test yourself with

1. What exactly happens when an asynchronous input to a flip-flop changes too close to an active clock edge?
2. How does the two-flip-flop synchroniser handle that, and why is it the *second* flip-flop's output that is safe to use?
3. Why does the chain used here behave as a debounce in CircuitVerse but not fully debounce a real button at 50 MHz, and why is that a property of the clock period rather than of the circuit? What would you add to fix it?
4. `button_sync` has a generic and `reset_sync` has none. What is the rule, and what would be wrong with giving `reset_sync` a `WIDTH` generic “for symmetry”?

Further reading

Erik Pihl's video walkthrough of metastability and the two-flip-flop synchroniser is at <https://www.youtube.com/watch?v=KrssJRgF13I> as additional, optional viewing. <https://nandland.com/lesson-13-metastability/> and <https://vhdlwhiz.com/terminology/metastability/> cover the same concepts from a slightly different angle, useful if this chapter does not land on the first reading.

Looking ahead

Chapter 5 goes on with:

- `variable`: the one construct you have not yet met, and the sharp difference between how it updates and how a `signal` does.
- Why the same accumulation written with a `signal` and with a `variable` gives two different answers, and which of the two is the bug.
- What the toolchain actually builds: LUTs and flip-flops, what limits a clock's speed, and the latch you describe by accident.

4.11 Exercises

How to check your work. Every exercise below that asks you to write a VHDL module comes with a self-checking testbench under `lectures/L04/exercises/`. Write your module in its directory `exercises/<module>/`, with the entity name and the **port order** the exercise states, and then run it with GHDL — see section 2.7 for the three commands.

EXERCISE SET 1

Metastability concepts

Exercise 4.1 Metastability in your own words

REASONING

Explain metastability in your own words. In your explanation, describe:

- What happens inside a D flip-flop when its input changes too close to the active clock edge.
- Why the flip-flop's output can temporarily remain between a valid logic 0 and logic 1.
- Why the output can take an unpredictable amount of time to settle.
- Why two downstream gates can interpret the unsettled output differently: one as 0, the other as 1.

Explain why that disagreement can leave different parts of a digital circuit in inconsistent states.

Exercise 4.2 Setup, hold, and why an asynchronous input cannot be avoided

REASONING

Setup time and hold time define a window around the active clock edge during which a flip-flop's input must remain stable.

Explain:

- Why a synchronously generated signal can normally be designed to meet those timing requirements.
- Why an asynchronous input, for example a mechanical push button, has no fixed relationship to the system clock.
- Why an asynchronous input can therefore change during the setup and hold window regardless of how the surrounding synchronous logic is designed.

Conclude why the metastability risk from an asynchronous input must be managed rather than simply avoided.

EXERCISE SET 2

Designing a synchroniser

Exercise 4.3 The chain, drawn and explained

CIRCUIT

Draw a two-flip-flop synchroniser for an asynchronous input. The circuit should contain an asynchronous input, two D flip-flops connected in series, a shared system clock, and a synchronised output taken from the second flip-flop.

Explain in your own words:

- Why the first flip-flop is directly exposed to the asynchronous input.
- Why the first flip-flop can go metastable.
- How the interval between the first and second clock edges gives the first stage time to settle.
- Why the second flip-flop's output has a much lower probability of being metastable.
- Why it is the second stage, not the first, that the rest of the design should use.

A synchroniser does not make the probability of metastability exactly zero. Explain why it nonetheless lowers the risk to an acceptably low level in most designs.

Hint: Think about how much settling time the first flip-flop gets before its value is sampled by the second flip-flop.

Exercise 4.4 A third stage “just to be safe”

REASONING

A colleague suggests using three synchroniser flip-flops on every asynchronous input in a slow design clocked at a few megahertz, just to be safe.

A third stage buys another clock period for a metastable value to settle in, and costs another clock cycle of latency on every synchronised input. At a few megahertz that extra clock period is an enormous time in metastability terms, which is exactly why the suggestion sounds sensible, and also why it may already be unnecessary.

a) Reason through the trade-off in both directions. Under what circumstances does it come out in favour of the third stage, and under what circumstances is the second stage already enough? Your answer must hang on something other than the clock frequency, since frequency alone does not settle it: say what it hangs on instead.

b) Section 4.4 arrived at MTBF from the settling time available per stage. Without redoing that calculation, say which way each of these factors pushes the decision, and why:

- How often the asynchronous input actually changes.
- The metastability characteristics of the FPGA family you are targeting.
- What a synchronisation failure would cost if it happened.

c) Give your recommendation for an ordinary slow design of the kind found in this book. Then describe a design, real or invented, where you would argue for the third stage instead, and say which of the factors in **b)** does the work in that case.

Check that your answer covers the two things a third stage gives, another clock period of settling time and a lower probability of an unsettled value reaching the functional logic, and the two it costs, another cycle of latency and more flip-flops. If your answer to a) rests on the clock frequency rather than on the required mean time between failures, read section 4.4 once more before you move on.

Exercise 4.5 Assert asynchronously, release synchronously

REASONING

Explain the reset pattern **assert asynchronously, release synchronously** from section 4.5.

Describe the two halves separately:

- Asynchronous assertion: the reset takes effect immediately, and does not wait for a clock edge.
- Synchronous release: the reset is removed only at an active clock edge, and all affected logic leaves reset in a controlled relationship with the clock.

Explain:

- Why immediate assertion of reset can matter when the clock is stopped, when the clock is unstable, and when the system must enter its reset state without delay.
- Why releasing an asynchronous reset near a clock edge can violate a flip-flop's recovery or removal timing requirements.
- Why synchronising the release stops different flip-flops from leaving reset on different clock cycles.

EXERCISE SET 3

Building the circuit

Exercise 4.6 led_toggle_sync by hand

CIRCUIT

Build the `led_toggle_sync` circuit from section 4.9 by hand in CircuitVerse.

- Add the external ports: the inputs `clock`, `reset_n` (asynchronous, active low) and `button_n` (active low), plus the output `led`.
- Build the two-flip-flop reset synchroniser:
 - Connect the two flip-flops in series.
 - Assert both stages asynchronously with the raw `reset_n` input.
 - Let the reset propagate out of the chain synchronously.
 - Use the synchronised reset output, `reset_s2_n`, throughout the rest of the circuit, and do not use the raw `reset_n` input directly outside the reset synchroniser.
- Build the three-flip-flop button synchroniser and the edge detector:

- Use the first two flip-flops as a two-flip-flop synchroniser.
 - Use the third flip-flop to store the previous synchronised button value.
 - Compare the current and the previous synchronised value, and detect a button press when the current synchronised, active-low button value is 0 and the previous synchronised value is 1.
 - Use an AND gate with the current value inverted to generate `button_edge_s2`. The relationship is $button_edge_s2 = previous \cdot current'$.
- d) Build the LED toggle logic: add a flip-flop storing `led_s`, reset it with `reset_s2_n`, toggle it only when `button_edge_s2 = 1`, and connect the output `led` to `led_s`.
- e) Simulate the whole design with a clock period of 1000 ms. Assert and release `reset_n`, press and release `button_n`, and observe the synchronised button stages, `button_edge_s2` and `led_s`. Decide:
- Whether the LED toggles at the first clock edge after the physical button press.
 - How many clock cycles it takes for the button value to propagate through the synchroniser and the edge detector.
 - Whether the observed latency agrees with the behaviour this chapter predicts.

EXERCISE SET 4

Debouncing and timing

Exercise 4.7 What the chain does and does not do

REASONING

You have now built and watched the circuit, so this is a question about what you saw.

The three-flip-flop chain in Exercise 4.6 does three jobs at once: it reduces the risk of metastability, it stores the previous synchronised button value, and it detects a button press. Explain why it does **not** by itself guarantee that the button is debounced.

- a) Separate the three things that are easy to confuse:
- Synchronisation: turns an asynchronous input into a signal usable in the clock domain.
 - Edge detection: generates a pulse when the sampled signal changes.
 - Debouncing: stops several physical transitions from being read as several separate presses.

Which two of these does your circuit actually do?

- b) In your simulation one press gave a single clean toggle. That is a stronger claim than it looks: it holds only under a particular condition about where the bounce landed relative to the clock edges. State that condition exactly, in terms of what your circuit sampled and what it therefore saw. Then explain why that condition is an accident of your clock period rather than a property of your circuit. Be concrete about which of the two, the bounce or the sampling, your design actually controls.

c) Your simulation ran with a clock period of 1000 ms; the FPGA runs at 50 MHz. A push button's contacts typically bounce on the order of 1 ms. Work out the consequences yourself:

- How many times does a clock with a period of 1000 ms sample the button during that millisecond of bounce?
- How many times does a 50 MHz clock sample it?
- Every sampled transition looks like a new edge to your edge detector. From those two numbers, predict what the LED does on the board at a single physical press, and say why exactly the same circuit behaved impeccably in CircuitVerse.

d) Describe one remedy of each kind: a hardware remedy (for example an RC low-pass filter followed by a Schmitt-trigger input) and a purely digital remedy (for example requiring the synchronised signal to hold a new level for some minimum number of clock cycles before accepting it, or a counter that ignores further transitions for a fixed interval after a press — which is what chapter 7's timer would give you).

e) Explain why the two-flip-flop synchroniser must be kept even once a debouncing method has been added. Which problem does debouncing not solve?

EXERCISE SET 5

The synchronisers as modules of their own

The worked example puts each synchroniser in a module of its own, so that every later design instantiates the same two files instead of deriving them afresh. That is why both chapter 7's `walking_led` and chapter 8's `fsm_led` ask you to copy these two in, and why chapter 8's `seq_detect_mealy`, which has no button, takes the reset synchroniser alone and leaves the other one aside.

The two exercises that follow are you writing those modules. Section 4.5 and 4.6 give you the logic; what they do not give you is the entity around it, and, for the second, the step from a single button to a vector of them. Write each module yourself before you open the worked example's copy, and then compare. Exercise 4.10 then puts both together into a working design.

Exercise 4.8 reset_sync

VHDL

Write the reset synchroniser as a module named `reset_sync`.

The entity has:

Port	Dir.	Type	Description
clock	in	std_logic	50 MHz system clock.
reset_n	in	std_logic	Active-low asynchronous reset, straight from the outside world.
reset_s2_n	out	std_logic	Active-low synchronised reset, safe for the rest of the design to use. Goes low the moment <code>reset_n</code> does, with no clock edge involved, and returns high two rising edges after <code>reset_n</code> has.

a) Implement **assert asynchronously, release synchronously** (section 4.5):

- An internal signal for the first stage, plus the output for the second.
- One process, sensitive to both `clock` and `reset_n`.
- At `reset_n = '0'`, drive both stages low, outside the clocked branch.
- At a rising clock edge, shift a constant `'1'` through the two stages.

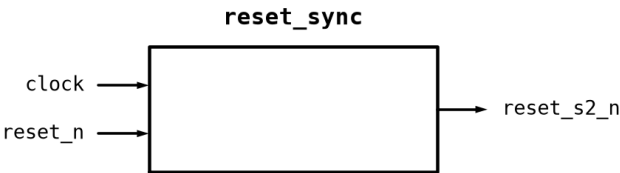
b) The value forced at reset and the value shifted in on the clock are each other's opposites, `'0'` and `'1'`. Explain in one sentence why that is no contradiction, in terms of what each branch is for.

c) Verify the design with its testbench. It checks both halves of the pattern separately: that a low `reset_n` pulls `reset_s2_n` low with no clock edge in between, and that releasing `reset_n` leaves `reset_s2_n` low until two rising edges have passed.

d) Answer in prose:

- The output is called `reset_s2_n` rather than `reset_n_sync`. What does `s2` say, and what would have to change in this module for that name to become wrong?
- Every other module in this book takes `reset_s2_n` as its reset. This one takes the raw `reset_n`. Why must exactly one module in a design be the exception?

Hint: The release is the interesting half. At reset the process does not care about the clock at all, so both stages go low together; on the clock it does not care about `reset_n`, so the `'1'` needs one edge per stage to reach the output. Two edges of latency at release is the price of the whole pattern.



Self-check. Name your entity `reset_sync`, with the ports `clock`, `reset_n` (in) and `reset_s2_n` (out), declared in that order; its testbench is in `exercises/reset_sync/`.

Exercise 4.9 **button_sync** VHDL

Write the button synchroniser and edge detector as a module named `button_sync`.

Exercise 4.10 handles two buttons by writing a two-bit synchroniser. This one is the same circuit with the width left open: a generic decides how many buttons it serves, so that `led_toggle_sync` can use it for one button and `fsm_led` for two without either file having to change. See section 4.8 for the syntax and for why this module has a generic where `reset_sync` has none.

The entity has one generic:

Generic	Type	Default	Description
COUNT	natural range 1 to 3	1	The number of buttons, and therefore the width of both vector ports.

and these ports:

Port	Dir.	Type	Description
clock	in	std_logic	50 MHz system clock.
reset_s2_n	in	std_logic	Active-low, already synchronised reset. This module does not synchronise its own reset; the module in Exercise 4.8 did that.
button_n	in	std_logic_vector(COUNT - 1 downto 0)	Active-low asynchronous push buttons.
button_edge_s2	out	std_logic_vector(COUNT - 1 downto 0)	A one-cycle pulse at every button's falling edge, two clock edges after the press. Every bit is independent of all the others.

- a) Build the three-stage chain from section 4.6, now over vectors instead of single bits:
- Three internal `std_logic_vector(COUNT - 1 downto 0)` signals, one per stage.
 - Stages 1 and 2 are the two-flip-flop synchroniser; stage 3 stores the previous value of stage 2.
 - At reset, set all three stages to `(others => '1')`, not `(others => '0')`.

- Derive the output with a single concurrent assignment, and note that the same expression works unchanged on vectors as on bits.

Keep this one in mind. The first two flip-flops are a synchroniser in their entirety and nothing else, and Exercise 5.4 has you factor them out again as a module of their own, generic in both width and the value they reset to. Writing them inline here first is what makes that exercise a consolidation rather than a new idea.

b) Explain why the reset value is all ones. All ones means “released”, since the buttons are active low, so a reset to all zeros starts the chain claiming that every button is already held down. Follow what that costs:

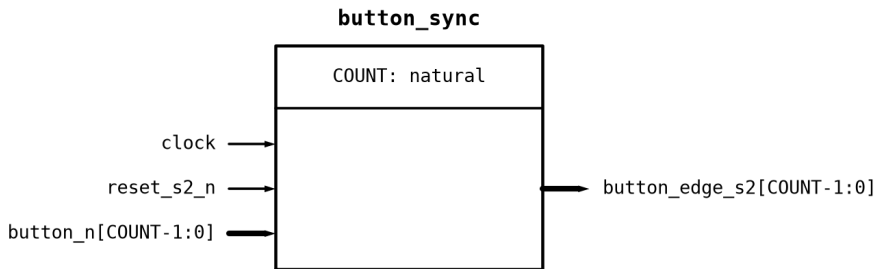
- With the buttons released and a reset to all zeros, does the module emit a false pulse as the released state propagates through the three stages? Walk through the output expression edge by edge before you answer, rather than guessing.
- Now the case that actually breaks. A user holds a button down at the moment the reset is released. What does the module report with a reset to all ones, and what does it report with all zeros? Which of the two is the behaviour you want, and why?

c) Verify the design with its testbench. It instantiates your module **twice**, once with `COUNT = 1` and once with `COUNT = 2`, and checks that a press gives exactly one one-cycle pulse, that a held button gives no further pulses, that a release gives none at all, and that pressing one button of a pair never pulses the other.

d) Answer in prose:

- `COUNT` is constrained to 1 to 3 rather than left as an unconstrained natural. What does that constraint buy, given that nothing in the architecture depends on the upper bound?
- This module is deliberately not merged with Exercise 4.8’s, even though a design that has a button always needs both. Name the design that justifies the split, and say what it would have had to do instead.

Hint: Write the architecture for `COUNT = 1` in your head first, then check that every line you wrote is already correct for a vector. and, not and the aggregate (others => '1') all work element by element, so the generalisation should cost you no extra code at all. If it does, that is worth another look.



Self-check. Name your entity `button_sync`, with the generic `COUNT` (natural range 1 to 3, default 1), the inputs `clock`, `reset_s2_n`, `button_n` (`std_logic_vector(COUNT - 1 downto 0)`) and the output `button_edge_s2` (`std_logic_vector(COUNT - 1 downto 0)`), declared in that order; its testbench is in `exercises/button_sync/`.

EXERCISE SET 6

Putting them together into a design

Exercise 4.10 led_toggle_sync2

VHDL

Now make chapter 3’s `led_toggle` metastability-safe, in VHDL, as a new module named `led_toggle_sync2`.

chapter 3’s `led_toggle` toggled two LEDs from two push buttons, but fed their raw, asynchronous signals straight into the edge detector — logically correct, but unsafe on real hardware. Here you rebuild it with every asynchronous input protected by this chapter’s two-flip-flop synchroniser.

The entity has:

Port	Dir.	Type	Description
<code>clock</code>	<code>in</code>	<code>std_logic</code>	50 MHz system clock.
<code>reset_n</code>	<code>in</code>	<code>std_logic</code>	Active-low asynchronous reset.
<code>button_n</code>	<code>in</code>	<code>std_logic_vector(1 downto 0)</code>	Two active-low asynchronous push buttons.
<code>led</code>	<code>out</code>	<code>std_logic_vector(1 downto 0)</code>	Two LEDs; <code>led(i)</code> toggles once per press of <code>button_n(i)</code> .

a) Synchronise every asynchronous input:

- Apply the two-flip-flop reset synchroniser from section 4.5.
- Give the buttons a two-flip-flop synchroniser plus a third stage for edge detection, as in section 4.6 — now over a `std_logic_vector(1 downto 0)`, so that both buttons are synchronised in parallel.
- Never let the raw `reset_n` or `button_n` reach the toggle logic; use only the synchronised reset and the per-button edge pulses.

Hint: You can put the synchroniser inline, or factor it out into a subcomponent taking a two-bit `button_n`, with the instantiation syntax from section 2.6. Inline is the shorter route, and then the build is a single file:

```
ghdl -a --std=93 led_toggle_sync2.vhd led_toggle_sync2_tb.vhd
```

If you factor it out instead, copy in the `reset_sync.vhd` and `button_sync.vhd` you wrote for Exercise 4.8 and 4.9 (and instantiate `button_sync` with `COUNT = 2`), and name them first

on the `ghdl -a` line, before your own module, so that GHDL sees them before the file that instantiates them:

```
ghdl -a --std=93 reset_sync.vhd button_sync.vhd \
        led_toggle_sync2.vhd led_toggle_sync2_tb.vhd
```

See section 2.7.5. Neither module is handed out anywhere in the course repository, because writing them is Exercise 4.8 and 4.9.

b) Build the toggle logic:

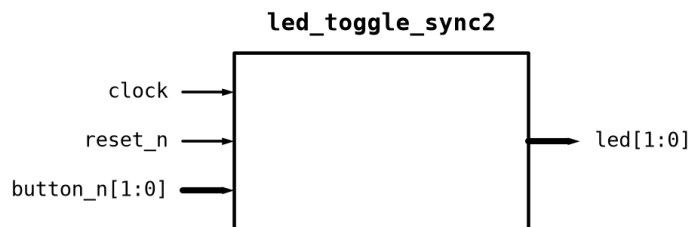
- One LED-state flip-flop per LED.
- At the synchronised reset, turn both LEDs off.
- Toggle `led(i)` only when button `i`'s synchronised falling-edge pulse fires.
- Confirm that `led(i)` responds to `button_n(i)` alone, never to the other button.

c) Verify the design with its testbench. The testbench presses each button in turn and checks that reset turns both LEDs off, that no press means no toggle, that pressing button 0 toggles **only** `led(0)` once the synchroniser latency has elapsed, that holding that button down does not toggle it again, and that pressing button 1 then toggles only `led(1)` and leaves `led(0)` as it was.

Pay attention to the **latency**: the testbench checks the *exact* edge, not merely that the LED eventually settles correctly. A press has to reach the toggle logic through two synchroniser flip-flops, so `led` is still unchanged after the first and the second rising edge and toggles at the third. Count the flip-flops a press travels through and convince yourself that the third edge is the right one, and that a defensive extra pipeline stage would now be caught rather than tolerated.

d) Answer in prose, comparing this design with your `led_toggle` from chapter 3:

- What does the synchroniser change about pressing a real, bouncing button?
- Why was the raw version unsafe to connect to a physical button at 50 MHz?
- Your testbench drives clean, perfectly timed transitions. Which of this chapter's two problems, metastability or contact bounce, can a testbench therefore never demonstrate to you?



Self-check. Name your entity `led_toggle_sync2`, with the inputs `clock`, `reset_n`, `button_n` (`std_logic_vector(1 downto 0)`) and the output `led` (`std_logic_vector(1 downto 0)`), declared in that order; its testbench is in `exercises/led_toggle_sync2/`. It presses each button in turn and checks that `led(i)` toggles on `button_n(i)`'s falling edge, that it does so at the right clock edge rather than merely settling on the right value eventually, and that it never fires the wrong LED.

Variables and the hardware underneath

The last construct left, and what the toolchain makes of all of it.

IN THIS CHAPTER

- `variable`: process-local storage, updating immediately where a signal schedules.
- Why the same accumulation gives two different answers depending on which one you reach for.
- What an FPGA physically consists of: lookup tables and flip-flops, and nothing that is a gate.
- The critical path and `Fmax`, and what puts a speed limit on a clock.
- The latch you describe by accident, and why it is only a warning.

This chapter's point is in its first worked example: code that looks correct, simulates without complaint, and is wrong. The only way to see why is to *trace* it rather than read it, one loop iteration at a time, all the way to the wrong answer. Do that on paper before you read the correction.

Refresher: The propagation delay behind a clock's speed limit is explained at transistor level in section 1.12 of *Digital Electronics*, and the D latch you can get by accident is in section 5.3 of the same book.

5.1 What is left

Every construct you need in order to describe hardware you have already used: entity and architecture in chapter 1, `std_logic_vector`, process and case in chapter 2, the clocked process and the scheduling rule in chapter 3, subcomponents and generics in chapter 2 and 4.

Two things are left, and they are the two hardest to work out on your own:

- The first is `variable`, the one construct the book has not needed until now. It looks like a signal and behaves nothing like one, and the difference is exactly the kind that gives you

a design which simulates plausibly and is silently wrong. Section 5.2 works through the case where it bites.

- The second is what any of this actually becomes. Since chapter 1 you have trusted the synthesis tool to build the circuit you described. It does, but not out of anything you would recognise. Section 5.3 opens the box. It turns out an FPGA contains no gates at all, which is why chapter 2’s Karnaugh maps bought you less than they appeared to. Your clock has a speed limit (section 5.4), and something specific sets it. And a case with a missing branch silently gives you a memory element nobody asked for (section 5.5).

Neither section answers “is my logic right?” You have been answering that question since chapter 1. They answer the two questions after it: is this what I meant, and will it fit and run.

5.2 variable: process-local storage that updates immediately

A variable is declared inside a process, between `process(...)` and `begin`. The decisive difference from a `signal`: variable assignment uses `:=` and takes effect **immediately**, so the very next statement that reads it sees the updated value, exactly like an assignment in C. There is no scheduling, none of the wait-until-the-process-suspends delay section 3.6 spelled out for signals, and no “last assignment wins”.

	signal (<=)	variable (:=)
Declared	In the architecture’s declarative part	In the process’s declarative part
Visible from	Anywhere in the architecture	Only inside its own process
Update time	Scheduled: applied when the process next suspends	Immediate: applied before the next statement
Repeated assignment in one round	Only the last survives; earlier ones are discarded	Each one takes effect, in turn

Why this actually matters: an XOR parity generator

An entity computing the XOR parity of an 8-bit input, ‘1’ if bits has an odd number of bits set:

```
entity parity_gen is
    port(bits : in std_logic_vector(7 downto 0);
          parity: out std_logic);
end entity;
```

Coming from software, the first instinct is to accumulate the running XOR in a loop, with a signal as the accumulator:


```
-- Don't do this: shown to demonstrate why it doesn't work.
architecture broken of parity_gen is
  signal parity_s: std_logic;
begin
  parity <= parity_s;

  process(bits) is
  begin
    parity_s <= '0';
    for i in bits'range loop
      parity_s <= parity_s xor bits(i);
    end loop;
  end process;
end architecture;
```

Trace one round, with the scheduling rule from section 3.6: every read of `parity_s` returns what it held *before this round began* (call it `old_parity`), and only the last assignment survives.

- `parity_s <= '0'`; schedules '0'. Not applied yet.
- `i = 7` (`bits'range` on (7 downto 0) runs from high to low): reads `parity_s`, gets `old_parity`, since the '0' has not been applied. Schedules `old_parity xor bits(7)`, discarding the '0'.
- `i = 6`: reads `parity_s`, again `old_parity`. Schedules `old_parity xor bits(6)`, discarding the previous one.
- ...and so on for every iteration ...
- `i = 0`, the last statement of the round: still reads `old_parity`, schedules `old_parity xor bits(0)`, discarding everything before it.
- The process suspends and `parity_s` becomes `old_parity xor bits(0)`, the only one of nine scheduled assignments that survived.

The result depends only on `bits(0)` and on whatever `parity_s` happened to hold already. Seven iterations and the initial '0' were computed and thrown away. This is no edge case: it is what a signal-as-accumulator *always* does, because every iteration reads the same frozen value and only the textually last assignment is kept.

The same computation with a variable, and this is the worked example on disk:

```
lectures/L05/parity_gen/parity_gen.vhd
-----
-- 8-bit XOR-parity generator.
--
-- Inputs:
--   - bits: The eight bits to compute parity over.
-- Outputs:
```

```

--      - parity: '1' if 'bits' holds an odd number of set bits, '0' otherwise.
-----
library ieee;
use ieee.std_logic_1164.all;

entity parity_gen is
    port(bits : in std_logic_vector(7 downto 0);
          parity: out std_logic);
end entity;

architecture behaviour of parity_gen is
-- Internal representation of 'parity'.
    signal parity_s: std_logic;
begin
    parity <= parity_s;

    PARITY_PROCESS: process(bits) is
        variable acc: std_logic;
    begin
        acc := '0';
        for i in bits'range loop
            acc := acc xor bits(i);
        end loop;
        parity_s <= acc;
    end process;
end architecture;

```

Now `acc := '0'` is applied immediately; `i = 7` reads `'0'`, XORs in `bits(7)` and updates `acc` at once; `i = 6` reads the just-updated value, and so on through every iteration. The loop ends with `acc` holding the true XOR of all eight bits, and `parity_s <= acc;` schedules the single correct value, the only assignment to `parity_s` in the round.

What that loop is not

It is not eight steps the hardware performs one after another. This is where a background in C misleads most, so it is worth saying outright: a `for` loop whose bounds are fixed at elaboration is **unrolled** by the synthesis tool. The eight iterations become eight XOR gates in a chain, all settling within one propagation delay, in the same clock cycle. There is no counter for `i`, no branching, no iteration in time. The loop writes eight gates in three lines; it does not do eight things in sequence.

That is also why `acc`, which reads like a running total, costs no storage at all. It never has to survive a clock edge: each of its values is just the wire between two of those XOR gates. A variable is the right tool precisely because it never survives a round.

The same goes for bits' range. The **attribute** ' range gives the index range of bits, here 7 downto 0, so that the loop covers every bit without the bounds being written twice. It is resolved at elaboration, like everything else in the loop's header, which is exactly why the tool can unroll it. `for i in 0 to 1 loop` in chapter 3's `led_toggle.vhd` means the same thing: both bits get gates of their own and both update in the same cycle.

The rule of thumb

- Use a variable for a temporary computation consumed entirely within one round: loop accumulators, temporary swaps, intermediate values handed over to a signal at the end, exactly like `acc`.
- Use a signal for anything another process or the outside world reads, and anything that has to hold its value across separate rounds (a counter, or a shift register's stored bits, both in chapter 6).

A variable *can* keep its value between rounds through the same process, behaving as static local storage rather than a fresh local variable each time, so it is possible to build state with one. But since its updates are immediate rather than scheduled, it behaves differently from a signal used the same way, particularly for anything modelling a chain of registers stepping one place per clock edge. If state genuinely has to persist and be read by other logic, reach for a signal: the scheduled-update semantics are what make chapter 6's counters and shift registers work.

5.3 What the FPGA actually consists of

An FPGA is not a blank slate that gates are etched into. It is a fixed grid of identical small blocks, manufactured before anyone wrote your VHDL, and "synthesis" is working out how to configure them so that they behave like the circuit you described. Two of them matter here:

- A **lookup table**, or **LUT**: a small memory, typically four to six inputs and one output. Load it with the truth table of any Boolean function of those inputs and it computes that function. That is the honest answer to "what does a gate become on an FPGA": nothing becomes a gate. A four-input LUT computes `a and b`, `a xor b` or `(c and not d)`, or any other function of up to four inputs at exactly the same cost, since in every case it is the same block holding a different table.
- A **flip-flop**: one bit of clocked storage, sitting beside every LUT.

On the DE0-CV's Cyclone V they are packaged together as an **ALM**, which holds a splittable six-input LUT, usable as one function of six variables or two smaller ones, plus four flip-flops. That is the unit Quartus's fitter report counts, which is why it talks about ALMs rather than gates.

That is almost the whole story. `register4` becomes four flip-flops, `mux_8to1` a handful of LUTs, and chapter 8's state machine LUTs feeding the flip-flops holding the state.

It also explains something from chapter 2 that would otherwise look like a broken promise: the book had you minimise expressions with Karnaugh maps to save gates, and then never mentioned the saving again. On an FPGA a function of four variables costs one LUT whether you minimised it or not. Minimisation still matters: it is how you understand a circuit, it is exactly right for the discrete logic and ASIC flows Karnaugh maps were invented for, and reducing a function below the LUT's input width is what keeps it from needing a second level of LUTs. But on this board it usually buys nothing, and it is better to know that than to wonder.

5.4 Why a clock has a speed limit

Signals take time through a LUT and along the wires between blocks. That **propagation delay** is why a clock cannot run arbitrarily fast.

Picture the shape every design in this book has, a flip-flop, some combinational logic, another flip-flop:

- At a rising edge the first presents a new value, which has to travel through every LUT and wire between the two and *settle* before the next edge arrives.
- If it has not, the second flip-flop captures a value that is still changing, and the design does the wrong thing reliably and mysteriously.

So the clock period has to cover three things:

- The first flip-flop's **clock-to-Q** delay: the time between the edge and its output actually changing.
- The **combinational delay** through the LUTs and wires between the two.
- The second flip-flop's **setup time**.

The slowest such path is the **critical path**, and one over that sum is **Fmax**.

Quartus computes Fmax for you, in the timing analysis step (appendix B), but only if you tell it what the clock is. That means an `.sdc` constraints file with a `create_clock` on the 50 MHz input. Without one the analyser has no period to compare anything against, so it reports every path as unconstrained and gives you no Fmax at all. Distinct from that, a design that “misses timing” is one where some path turned out slower than the period you actually declared.

None of this is part of this book. You will not write an `.sdc`, read a timing report, or meet a design that misses timing, since every design here fits inside 20 ns with room to spare. It is worth knowing that the step exists and what it requires, so that you recognise it the first time a design of yours approaches the limit.

Two consequences worth carrying:

- **Deep combinational logic costs you speed, not the number of flip-flops.** Chaining a long expression between two registers lengthens the critical path; splitting it so that part of the work happens on one cycle and the rest on the next shortens it. That trade-off, more registers against a faster clock, is the single most common move in fast digital design.
- **50 MHz is a budget of 20 ns.** That is the number every path in your design competes against, and it is plenty of time at these speeds.

5.5 The latch you get by accident

Chapter 3 built a D latch, showed that it was transparent while enabled, and then set it aside in favour of the flip-flop. Here it comes back uninvited.

Section 2.5 required a when others on every case, and gave the language's reason: a case must cover every value of its selector's type. The general rule behind it is that a combinational process must assign its output on *every* path through it, and that reason can now be stated:

- VHDL requires a signal to keep its old value if nothing assigns a new one.
- If some path through your process leaves x unassigned, you have said that x has to *remember* its previous value on that path, and remembering is not something a combinational network can do.
- So the tool builds the only thing that can: a latch, exactly the one from section 3.2.

That is an **unintended latch**, and it is almost never what anyone meant. It is a warning rather than an error, the design will often appear to work, and the message is worth recognising:

```
Warning: Inferred latch(es) for signal "x"
```

The fix is never to add a latch deliberately. It is to assign the signal on every path, which is the rule you were already following.

5.6 Summary

What you should be able to do now

- Say what separates a signal from a variable, predict the different result each gives from the same code, and choose between them deliberately.
- Explain what an FPGA physically consists of, why two differently written descriptions of the same circuit give the same hardware, and why a Karnaugh map buys less here than on discrete logic.
- Explain what limits a design's clock frequency, and why more registers can make a design faster rather than slower.

- Recognise an unintended latch from its warning, say what caused it and fix it, and say why “it compiled without errors” promises less for an FPGA design than for a C program.

Questions to test yourself with

1. A signal is assigned twice, with different values, in one round through a process. What ends up on it, and when does that update actually happen?
2. The same question for a variable.
3. Why does accumulating over a loop with a signal give a different, usually incorrect, result?
4. Nothing on an FPGA becomes a gate. What does a Boolean function become instead, and why does a function of four variables cost the same whether or not you minimised it first?
5. What is the critical path, and why can more flip-flops make a design run faster?
6. A combinational case assigns its output in three branches out of four. What does the synthesis tool build for the fourth, and why is that the only thing it *can* build?
7. Why is an unintended latch a warning and not an error, and why does that make it more dangerous rather than less?

Looking ahead

Nothing in chapter 6 to 8 introduces a new idiom. Counters, shift registers and timers all hold state across clock edges using chapter 3’s clocked process and the scheduling rule above, and chapter 8’s state machines merely add an enumerated type and a case on it. What changes is what the state is used for.

Two things to watch for. Chapter 6’s `serial_rx8` is the first design whose correctness you cannot check by eye on an LED, which is where the testbenches handed out stop being a formality. And section 4.8’s generic is put to work straight away: chapter 7’s timer uses one for its tick count, and `walking_led` instantiates that timer alongside `reset_sync` and `button_sync`, writing almost no logic of its own.

5.7 Exercises

How to check your work. Every exercise below that asks you to write a VHDL module has a self-checking testbench under `lectures/L05/exercises/`. Write your module in its directory `exercises/<module>/`, with the entity name and the **port order** the exercise states, and then run it with GHDL — see section 2.7 for the three commands.

EXERCISE SET 1

variable versus signal**Exercise 5.1** Trace the accumulator bug

REASONING

The broken parity_gen architecture from section 5.2 is reproduced below:

```
architecture broken of parity_gen is
  signal parity_s: std_logic;
begin
  parity <= parity_s;

  process(bits) is
  begin
    parity_s <= '0';
    for i in bits'range loop
      parity_s <= parity_s xor bits(i);
    end loop;
  end process;
end architecture;
```

Assume bits has the range 7 downto 0, that bits = "10110110", and that parity_s holds '0' before the process starts.

a) Trace the process by hand. Note, for the initial assignment and for every loop iteration: the current index *i*, the value of bits(*i*), the value read from parity_s, the value scheduled for parity_s, and whether that scheduled assignment survives until the process suspends.

Remember that a signal assignment does not update the signal immediately, and that assignments to the same signal during one and the same round replace earlier assignments scheduled for the same simulation time.

b) Determine the final value assigned to parity_s after the process suspends. Then compute the correct XOR parity of "10110110" by hand, compare it with the result from the broken architecture, and explain why repeated assignments to the signal do not create an accumulator.

c) Trace the variable-based architecture behaviour from parity_gen.vhd with the same input. Note, for every loop iteration: the value of bits(*i*), the value of acc before the XOR operation, and the value of acc immediately after it. Confirm that acc keeps each iteration's result so that the next iteration can use it, and that the final value assigned to parity is the correct XOR parity.

Exercise 5.2 **min_of_two**

VHDL

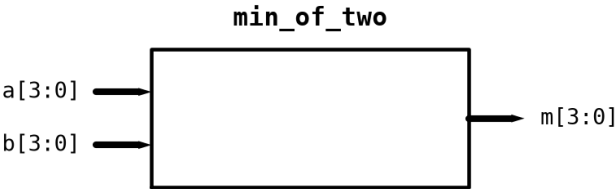
Write an entity named `min_of_two`.

Port	Dir.	Type	Meaning
a	in	<code>std_logic_vector(3 downto 0)</code>	First value.
b	in	<code>std_logic_vector(3 downto 0)</code>	Second value.
m	out	<code>std_logic_vector(3 downto 0)</code>	The smaller of the two.

Treat `a` and `b` as unsigned four-bit values. Implement the circuit with a process:

- Include `a` and `b` in the sensitivity list.
- Declare a process-local variable holding the smaller value.
- Compare `a` and `b` as unsigned numbers, using the facilities in `ieee.numeric_std` per section 2.3.
- Assign the variable exactly once per round, and assign it to `m` after the comparison.

This exercise needs no loop. Its point is to use a variable as temporary process-local storage for something *other* than an accumulator: section 5.2’s worked example builds a value up across the iterations of a loop, which is the case a variable is most often introduced for, and it is easy to leave thinking that is the only thing they are good for. Here the variable simply holds an intermediate result for the few lines separating the computation from its use, which is the more common of the two uses in real code.



Self-check. Name your entity `min_of_two`, with the ports `a`, `b` (in) and `m` (out), all `std_logic_vector(3 downto 0)`, declared in that order; its testbench is in `exercises/min_of_two/`.

Exercise 5.3 ones_count8

VHDL

Write an entity named `ones_count8` that counts how many bits in an 8-bit input are set. Exercise 5.1 had you trace the accumulator bug on paper, and Exercise 5.2 used a variable that was no accumulator at all. This one is the case section 5.2 is actually about: a value built up across the iterations of a loop. It is the only exercise in the book where you write one.

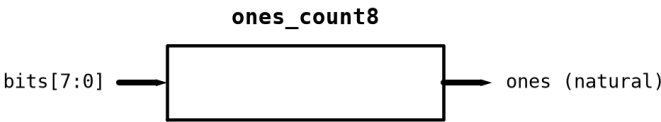
Port	Dir.	Type	Meaning
bits	in	std_logic_vector(7 downto 0)	The value to count over.
ones	out	natural range 0 to 8	The number of bits in bits that are '1'.

Implement it with a single combinational process:

- Put bits in the sensitivity list.
- Declare a process-local variable for the running count, and set it to 0 at the top of every round. A variable keeps its value between rounds, so a count that is never cleared keeps growing.
- Loop over bits' range and add 1 for every bit that is set.
- Assign the variable to ones once, after the loop.

Then answer, in one sentence each:

- Write the same thing with a signal as the accumulator instead, run the testbench, and note what it reports for the input 00000010. Explain the number you get using the scheduling rule, not by guessing.
- The loop runs eight times. How many adders does this design put on the FPGA, and how many clock cycles does it take to produce an answer? If either answer surprises you, reread section 5.2.
- ones is a natural range 0 to 8 rather than a std_logic_vector(3 downto 0). Both hold the answer. What does the range buy you, and what would GHDL do if your count ever left it?



Self-check. Name your entity `ones_count8`, with the input bits (`std_logic_vector(7 downto 0)`) and the output ones (`natural range 0 to 8`), declared in that order; its testbench is in `exercises/ones_count8/` and sweeps all 256 input values.

EXERCISE SET 2

A module worth having on its own

Exercise 5.4 sync: the synchroniser as a module of its own

VHDL

Write the two-flip-flop synchroniser as a module of its own, named sync.

You have already built this circuit. It is the first two flip-flops of chapter 4’s button_sync, written inline there because at that point it was the *idea* that mattered: an asynchronous input needs two flip-flops before anything else gets to look at it. Here you factor those two out into a module, and the exercise is not the logic, which you already know, but everything around it.

Two things make it worth doing again. The first is that a bare synchroniser is what most asynchronous inputs actually need: button_sync adds an edge detector because a button press is an event, but a serial line, a status flag from another clock domain, or a switch whose level is all you ever read, need the two flip-flops and nothing more. Chapter 8’s serial receiver instantiates exactly this on its rx pin. The second is that it takes **two generics**, and one of them is not a number.

Generic	Type	Default	Meaning
SIZE	natural range 1 to 15	1	How many independent signals to synchronise, and therefore the width of both vector ports.
PRESET	std_logic	'0'	The value both stages take at reset.

and these ports:

Port	Dir.	Type	Meaning
clock	in	std_logic	50 MHz system clock.
reset_s2_n	in	std_logic	Active-low, already synchronised reset. A synchroniser does not synchronise its own reset; chapter 4’s reset_sync does that.
async_in	in	std_logic_vector(SIZE - 1 downto 0)	The asynchronous input or inputs, straight from outside the clock domain.
sync_out	out	std_logic_vector(SIZE - 1 downto 0)	The same signals, safe to use, two rising edges later. Every bit is synchronised independently.

a) Implement it:

- Two internal signals of type std_logic_vector(SIZE - 1 downto 0), one per stage.
- One process, sensitive to clock and reset_s2_n.
- At reset, drive both stages to PRESET, outside the clocked branch.
- At a rising clock edge, shift async_in through the two stages.
- Drive sync_out from the second stage with a concurrent assignment.

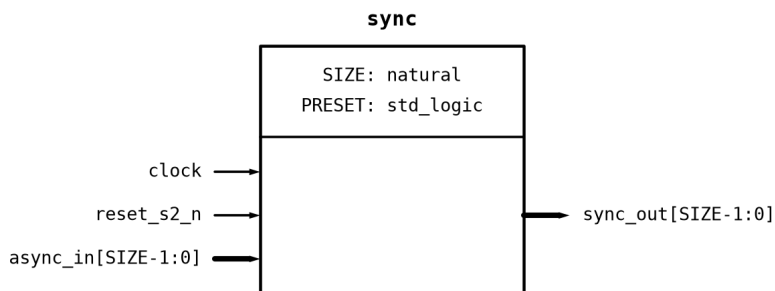
b) PRESET is a generic of type `std_logic`, the first in this book that is not a number, and it exists because the safe reset value depends on what the signal *means*. Only the instantiating design knows that. Answer:

- `button_sync` synchronises active-low buttons, so it would pass '1' in. What would '0' claim about the buttons during the two cycles after every reset?
- A serial line idles high, so chapter 8's receiver also passes '1' in. What would '0' look like to a receiver watching for a start bit?
- Why is the *default* '0' rather than '1' all the same, given both of those, and what does that say about whose job it is to get it right?

c) This is also a question about what a generic costs. `SIZE` is natural range 1 to 15 and `PRESET` is `std_logic`, and both are fixed before synthesis runs. Use section 5.3's picture of what the FPGA actually consists of and say how many flip-flops an instance with `SIZE = 4` uses, how many an instance with `SIZE = 1` uses, and where in the built circuit `PRESET` ends up. Neither of the two turns up as logic anywhere; say what happened to them instead.

d) Verify it with its testbench. It instantiates your module twice, once one bit wide with `PRESET = '1'` and once three bits wide with `PRESET = '0'`, so a design that hardcodes either generic fails. It checks that `sync_out` follows `async_in` after **exactly** two rising edges rather than after one or eventually, that every bit in a vector is synchronised separately, and that reset loads `PRESET` with no clock edge involved.

Hint: Write the architecture for `SIZE = 1` in your head first, then check that every line is already correct for a vector. The aggregate (`others => PRESET`) and an ordinary vector assignment both work element by element, so the generalisation should cost no extra code.



Self-check. Name your entity `sync`, with the generics `SIZE` (natural range 1 to 15, default 1) and `PRESET` (`std_logic`, default '0'), the inputs `clock`, `reset_s2_n`, `async_in` (`std_logic_vector(SIZE - 1 downto 0)`) and the output `sync_out` (`std_logic_vector(SIZE - 1 downto 0)`), declared in that order; its testbench is in `exercises/sync/`.

Keep this file. Chapter 8's serial receiver capstone instantiates it on the `rx` pin, and asks you to copy it in.

EXERCISE SET 3

From VHDL to hardware

The Quartus flow is demonstrated on a real DE0-CV board during the sessions — first with the brake assistant back in chapter 1 — and is written down in appendix B for reference. You need neither Quartus nor a board yourself; the exercises below are about understanding what that flow does with the VHDL you write.

Exercise 5.5 What the toolchain does

REASONING

Answer each of the following, with reference to appendix B and to what you saw demonstrated. One or two sentences each.

- a) A Quartus project needs a chosen **top-level entity** before it compiles. What does the tool do differently with the top-level entity than with any other entity in the project? `min_of_two` from Exercise 5.2 and its testbench are both valid VHDL. Why can only one of them ever be the top-level entity in an FPGA project?
- b) Pin assignment connects a port name to a physical pin. Why must the pins be assigned *before* compilation rather than afterwards? What would `m` physically be connected to if you compiled `min_of_two` having assigned no pins at all?
- c) Synthesis turns your architecture into gates on the chip. Chapter 2's exercise `combo_logic` had you write $x = ab + c'd$ twice: once as a single expression, and once with $c'd$ named by an internal signal. What should synthesis produce for each, and why is your answer the same for both? Which of the two would you rather read in six months' time?
- d) Compilation gives warnings as well as errors. Why is "it compiled without errors" a much weaker claim about an FPGA design than "it compiled without errors" is about a C program? Name one kind of warning you would never ignore.

Exercise 5.6 Four predictions about the parity

SIMULATION

`parity_gen` from section 5.2 is the chapter's worked example, and it comes with a reference testbench.

- a) Predict, before you run anything, parity by hand for each of these inputs: "00000000", "10110110", "11111111" and "10000000".
- b) Run the reference testbench against the worked example and confirm your four predictions:

```
cd lectures/L05/parity_gen
ghdl -a --std=93 parity_gen.vhd parity_gen_tb.vhd
ghdl -e --std=93 parity_gen_tb
ghdl -r --std=93 parity_gen_tb --assert-level=error --stop-time=10ms
```

c) Take any of those inputs and flip exactly one bit, leaving the other seven unchanged. What happens to parity? Explain why changing exactly one input bit *always* has to invert the XOR parity result, whichever bit you pick and whatever the starting value. That property is what makes a parity bit capable of detecting a single-bit error in transmission. What kind of error does it miss, and why?

Exercise 5.7 A decoder that compiles cleanly and is still wrong REASONING

A colleague sends you this combinational decoder for review. It compiles, and Quartus reports no errors.

```
architecture behaviour of level_decode is
begin
  process(sel) is
  begin
    case sel is
      when "00" => leds <= "0001";
      when "01" => leds <= "0010";
      when "10" => leds <= "0100";
      when others => null;
    end case;
  end process;
end architecture;
```

a) Quartus gives Warning: Inferred latch(es) for signal "leds". Using section 5.5, explain what the tool built and why. What did the VHDL ask for that a combinational network cannot do?

b) The author's defence is that sel is two bits, so "11" is the only value when others can match, and the design never generates it. Give two separate reasons why that argument does not hold: one about what sel can actually hold (section 1.4), and one that would apply even if sel really could only be "00", "01" or "10".

c) Fix it, in two different ways: by changing only the when others branch, and by adding a single line *before* the case statement and leaving every branch untouched. Which of the two would you rather maintain, and why? The second scales to a case with twenty branches; the first does not.

d) This is a warning, not an error, and the design will often appear to work on the board. Why does that make it more dangerous than an error rather than less?

Counters and shift registers

Building blocks for everything that needs to count or move bits.

IN THIS CHAPTER

- Counters: a register, an adder and a constant 1, and why the wraparound comes free.
- Shift registers: moving data one bit per clock edge, in a direction used consistently.
- SIPO and PISO, and which suits which job.
- An 8-bit serial receiver: a shift register plus a bit counter, which knows when a byte has arrived.
- Two demonstrations on the board: a row of LEDs counting and wrapping, and a byte shifted in one button press at a time.

Before the counter reaches 1111: predict what it will show next, and then find the element in the drawing that makes it happen. There is none, which is the point: nothing in the circuit performs the wraparound, and it is a consequence of the register's width rather than logic someone put there.

`serial_rx8` in section 6.5 is the book's first design whose correctness you cannot check by eye at full speed: continuously enabled, it swallows a byte in 160 ns. That is why it has a reference testbench, and why the board demonstration in section 6.7 drives `shift_enable` from a button instead. Slowing the *clock* down would be the software instinct and the wrong move; the module already has the control that lets you take one bit at a time.

Refresher: Counters and shift registers built from flip-flops and gates are in sections 6.1–6.2 of *Digital Electronics*.

6.1 From register to counter

A register (chapter 3) is a bank of D flip-flops storing whatever value is fed in at the next rising edge. A **counter** is the same idea with a twist: instead of a new value from outside, you feed the register its *own current value plus one*. At every edge it captures `current_value + 1`, so the stored number increases once per clock cycle with no further input.

Build it by hand first

As with every circuit in this book: build it before you write it. A 4-bit counter is three elements, and watching it run is the whole point:

- A **4-bit register**: four D flip-flops sharing a clock, exactly the register from section 3.5.
- An **adder** and a **constant 1**: the register's four outputs to one of the adder's inputs, the constant to the other.
- The adder's sum wired back to the register's four D inputs. That feedback is the whole circuit: the register holds a number, the adder computes that number plus one, the next edge stores it. It is section 3.1's feedback loop again, with an adder in the way instead of a plain wire.

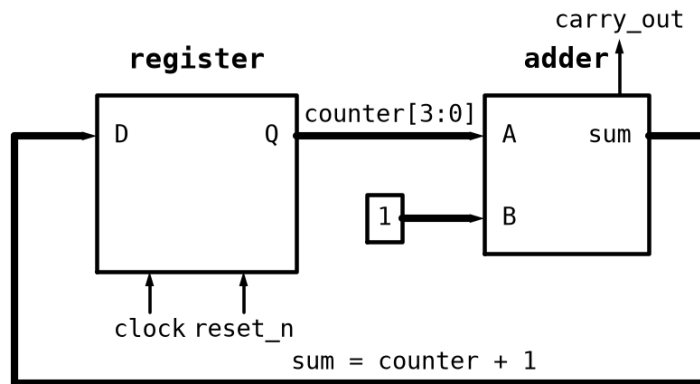


Figure 6.1 A 4-bit counter: a register whose own output, plus one, is wired back to its input.

Then confirm, one clock edge at a time, with a period of 1000 ms:

- Watch the outputs step 0000, 0001, 0010, 0011, . . ., one count per rising edge.
- **Let it run past 1111.** It returns to 0000 by itself, and nothing in your circuit tells it to: there is no comparison, no clearing logic, no “if the count is at the maximum” anywhere. The adder produced 10000, the fifth bit had nowhere to go, and the four bits that survived are 0000. That is **overflow**, and in a counter it is not a bug to work around but the property the rest of this chapter and the whole of chapter 7 rest on.
- **Keep an eye on carry_out on the adder while it happens.** That is the fifth bit, and the drawing shows exactly where it goes: high during the single edge where the count wraps, connected to nothing, out of the adder and no further. A wider counter would feed it into a fifth stage; here it is thrown away, and throwing it away *is* the wraparound. So “the count wraps because the bits ran out” is not a figure of speech. You can point at the bit that ran out.

Keep the project. Chapter 7 starts from exactly this circuit and adds two things to it, and reopening it beats redrawing it.

The same counter in VHDL

One signal and one process, exactly like chapter 3’s register, except that the right-hand side refers to the signal itself:

```
signal counter: natural range 0 to 15;
...
process(clock, reset_s2_n) is
begin
    if (reset_s2_n = '0') then
        counter <= 0;
    elsif (rising_edge(clock)) then
        counter <= counter + 1;
    end if;
end process;
```

`natural` is a predefined integer subtype in VHDL: a non-negative integer, unlike the `std_logic` types which model bits directly. VHDL’s arithmetic operators work on it the way ordinary integer maths does, which is why `counter + 1` needs no extra machinery. Synthesis turns it into a register of flip-flops all the same, exactly as a `std_logic_vector` would become; the difference is purely in how you write and reason about the value.

The range constraint `0 to 15` is not just documentation. It tells the synthesis tool that this counter needs four bits, so counting past 15 falls back to 0 for free, purely as a consequence of the bits running out, with no explicit “if counter = 15” logic in either the drawing or the VHDL.

The free wraparound has one condition worth saying out loud: it happens only when the upper bound is $2^N - 1$. `0 to 15` wraps for free; `0 to 9` does not, since 10 is not a power-of-two boundary, and a modulo-10 counter needs an explicit comparison. That is exactly what Exercise 6.4 asks you to write.

One place where the simulator disagrees with the hardware. The free wraparound is a property of the *synthesised* circuit, and the drawing shows that it is real. A VHDL *simulator* is stricter than a wire:

- `natural range 0 to 15` is range-checked, so GHDL treats $15 + 1 = 16$ as an error and aborts instead of wrapping.
- The synthesised hardware has no such notion. There is no sixteenth value for the bits to take, so they land on 0000, exactly as your four LEDs did.

The gate network and the chip both wrap; only the simulator objects, since a range-constrained integer is a promise you made and 16 breaks it. The rule that follows is not “trust the hardware”:

it is that anything you intend to verify should say what it means. Write the comparison out, as timer and serial_rx8 do, and the drawing, the simulator and the chip all agree.

This wraparound that follows from the bit width is the counter's single most important property, and it is what chapter 7 builds a timer out of: a counter watching for a particular count value and raising a flag when it arrives.

6.2 Shift registers

A **shift register** is, again, N D flip-flops, wired so that each flip-flop's output feeds the *next* flip-flop's input instead of its own. At every clock edge every bit moves one position down the chain, simultaneously:

```
serial_in -> [D Q]-->[D Q]-->[D Q]-->[D Q] -> serial_out
           FF0      FF1      FF2      FF3
           bit 0    bit 1    bit 2    bit 3
           ^clock   ^clock   ^clock   ^clock   (all share the same clock)
```

After one edge FF0's contents are in FF1, FF1's in FF2, and so on; a new bit comes into FF0 from serial_in, and what was in the last flip-flop falls off the end as serial_out. Run it for N edges and N serial bits have travelled all the way through.

One direction, used throughout this book. A shift register can move bits either way, and both are equally valid; it is mixing them within one design that creates confusion. So every shift register here follows the diagram above:

- the chain maps onto a std_logic_vector with FF0 as **bit 0** and FF($N-1$) as bit $N-1$.
- serial data **enters at bit 0** and **leaves from bit $N-1$** .
- every bit moves one position towards the **most significant** bit per edge.

In VHDL that direction is a single expression, identical in every example below:

```
shift_reg <= shift_reg(N-2 downto 0) & serial_in;
```

shift_reg($N-2$ downto 0) is everything but the top bit, moved up one position; the top bit has nowhere to go and falls off as serial_out; serial_in fills the vacated bit 0.

When you meet a shift register outside this book: check its direction before you assume anything. The mirror image, serial_in & shift_reg($N-1$ downto 1), is exactly as common and at a glance looks almost the same.

Shift registers are named after how data comes in and leaves:

Form	Data in	Data out	Typical use
SISO	one bit per clock	one bit per clock	a plain delay line
SIPO	one bit per clock	all N bits at once	receiving a stream of serial bits, e.g. an incoming byte on a single wire
PISO	all N bits at once (loaded)	one bit per clock	sending N bits out over a single wire
PIPO	all N bits at once	all N bits at once	in practice an ordinary register (section 3.5)

This chapter covers the two practically useful ones, SIPO and PISO.

6.3 Serial in, parallel out (SIPO)

A SIPO register is what you reach for when bits arrive one at a time on a single wire and you need the accumulated value all at once. The classic is deserialising an incoming byte, which is fundamentally what the receive path of any serial protocol does.

Which end of the byte arrives first is each protocol's own decision. SPI and CAN send the most significant bit first by default, which is what section 6.2's direction assumes: the first bit to arrive ends up in the MSB once all N have been shifted in. (Bit order in SPI is configurable on most controllers, so check the device.) A protocol sending the least significant bit first, like UART, shifts the mirrored way, and its idiom is the mirror image too.

The idiom is one line inside a clocked process, section 6.2's shift expression verbatim:

```

signal shift_reg: std_logic_vector(7 downto 0);
...
process(clock, reset_s2_n) is
begin
    if (reset_s2_n = '0') then
        shift_reg <= (others => '0');
    elsif (rising_edge(clock)) then
        shift_reg <= shift_reg(6 downto 0) & serial_in;
    end if;
end process;
parallel_out <= shift_reg;

```

After 8 edges the received bits have entirely replaced the register's original contents, and `parallel_out` exposes all 8 at once.

6.4 Parallel in, serial out (PISO)

A PISO register reverses SIPO's job: load N bits at once, then send them out one at a time. It is what you reach for when a fixed pattern has to go out over a single wire, for example to a chain of LEDs or a display driven by an external shift register chip.

It needs one control signal more than SIPO: something separating *loading* a new parallel value from *shifting* the existing one out.

```
process(clock, reset_s2_n) is
begin
    if (reset_s2_n = '0') then
        shift_reg <= (others => '0');
    elsif (rising_edge(clock)) then
        if (load = '1') then
            shift_reg <= parallel_in;           -- Parallel load.
        elsif (shift = '1') then
            shift_reg <= shift_reg(N-2 downto 0) & serial_in; -- Shift toward the-
-- MSB.
        end if;
    end if;
end process;
serial_out <= shift_reg(N-1);
```

Bit N-1 is always the next bit to leave, so a loaded value goes out most significant bit first: load "1010" into a 4-bit register and serial_out presents 1, 0, 1, 0 over the following cycles.

Note how little separates this from section 6.3's SIPO. **The shift expression is identical**, same direction, same line of VHDL. Only two things differ: PISO adds a load branch, and it taps shift_reg(N-1) as a serial output where SIPO exposes the whole vector. "SIPO" and "PISO" name how you *wire* a shift register, not two kinds of hardware, which is why section 6.2 could describe the shifting once before either name came up.

Whether a design needs a genuinely new serial_in at every shift, or is content to recirculate what falls off the end, is decided by how it is wired. That recirculation is exactly the trick the walking LED example in chapter 7 uses, once it has a timer clocking the shifting.

6.5 Worked example: an 8-bit serial receiver

The chapter's two halves meet in one module. A shift register can receive serial bits forever but cannot say *when* a complete byte has arrived, since it has no notion of "how many bits so far". That is a counting problem, and section 6.1 already solved it. serial_rx8 combines the two, plus a one-cycle data_ready pulse when the eighth bit lands.

This is no toy: it is the front half of every serial receiver there is. The receive paths of SPI, CAN and UART all start with a shift register and a bit counter, and differ only in what decides when a bit is valid.

Port	Dir.	Type	Meaning
clock	in	std_logic	System clock.
reset_s2_n	in	std_logic	Active-low, already synchronised reset; clears the register and the bit count. Asynchronous, so it clears them with no clock edge involved.
shift_enable	in	std_logic	High for exactly one clock cycle per incoming bit.
serial_in	in	std_logic	The incoming bit, sampled at every enabled rising edge.
data_out	out	std_logic_vector(7 downto 0)	The byte received so far, most significant bit first.
data_ready	out	std_logic	One-cycle pulse when the eighth bit lands.

The reset port is the synchronised one, and _s2 says so. It is called `reset_s2_n` rather than `reset_n` because what belongs on it is the output of a `reset_sync` (section 4.5), the module you wrote in Exercise 4.8: assert asynchronously, release synchronously. **Instantiate one in the design wrapping this receiver and connect its `reset_s2_n` here**, which is exactly what section 6.7's board wrapper does. Wire a push button straight into this port instead and you get the race chapter 4 exists to prevent: the shift register and the bit counter are a dozen flip-flops, and an asynchronously *released* reset lets them leave reset at different edges, so the receiver can start counting a byte one cycle before it starts shifting one in.

counter, `sipo8` and `piso8` in the exercises take `reset_s2_n` for the same reason. None of them is a design you put on a board by itself: a counter, a shift register and a receiver are parts, and a part is entitled to assume that the design around it has handled the synchronisation. So this whole chapter takes the synchronised reset signal, and only the wrapper modules in section 6.6 and 6.7, which own the pins, ever see a raw `reset_n`.

A **top level** is named the other way round: it takes `reset_n` and instantiates `reset_sync` itself, since it is the one holding the pin. `blinker` and `walking_led` in chapter 7, and both capstones in chapter 8, are written that way. Get into the habit of reading `_s2` as a requirement on the caller, because from here on most of what you write is a subcomponent.

```
lectures/L06/serial_rx8/serial_rx8.vhd
```

```
-----
-- 8-bit serial receiver: a SISO shift register paired with a bit counter, so it
-- reports a complete byte rather than shifting forever. Bits arrive most
-- significant first, one per enabled clock edge. See Appendix A.5.
--
-- Inputs:
--   - clock      : 50 MHz system clock.
```

```

--   - reset_s2_n : Active-low, already-synchronized reset: drive it from a
--                 reset_sync (L04 A.5), never straight from a pin. ↪
--   ↪ Asynchronous
--                 here, so it clears the register and the bit count with no
--                 clock edge involved.
--   - shift_enable: High for one clock cycle per incoming bit; holds both the
--                 shift register and the bit count when low.
--   - serial_in   : Serial input bit, sampled on each enabled rising edge.
-- Outputs:
--   - data_out    : Received byte, valid while data_ready is high.
--   - data_ready: One-cycle pulse as the eighth bit lands.
-----
library ieee;
use ieee.std_logic_1164.all;

entity serial_rx8 is
    port(clock, reset_s2_n, shift_enable, serial_in: in std_logic;
          data_out                                     : out std_logic_vector(7 downto↪
            ↪ 0);
          data_ready                                     : out std_logic);
end entity;

architecture behaviour of serial_rx8 is

    -- Bit width.
    constant BIT_WIDTH: natural := 8;

    -- Most significant bit.
    constant MSB: natural := BIT_WIDTH - 1;

    -- Shift register.
    signal shift_reg: std_logic_vector(MSB downto 0);

    -- Bit counter.
    signal bit_count: natural range 0 to MSB;

begin
    data_out <= shift_reg;

    process(clock, reset_s2_n) is
    begin
        if (reset_s2_n = '0') then
            shift_reg <= (others => '0');
            bit_count <= 0;

```

```

        data_ready <= '0';
    elsif (rising_edge(clock)) then
        data_ready <= '0';
        if (shift_enable = '1') then
            shift_reg <= shift_reg(MSB - 1 downto 0) & serial_in;
            if (bit_count >= BIT_WIDTH - 1) then
                data_ready <= '1';
                bit_count <= 0;
            else
                bit_count <= bit_count + 1;
            end if;
        end if;
    end if;
end process;
end architecture;

```

At every rising edge where `shift_enable = '1'`, `serial_in` is shifted into bit 0 with section 6.3's idiom unchanged, so the first bit to arrive has travelled up to bit 7 by the time the eighth lands, and an internal bit counter of type `natural` range 0 to 7 increments. At the edge where that counter already stands at 7, `data_ready` pulses high for one cycle and the counter clears back to 0, ready for the next byte. `data_out` is driven by a concurrent assignment from the shift register, so the complete byte is on it during the cycle where `data_ready` is high.

Note that the counter is *cleared* here rather than left to wrap: 0 to 7 is a power-of-two range so it would wrap for free, but writing it out keeps the simulator in agreement with the hardware, per section 6.1.

Two details are worth dwelling on, since both are patterns you have met more than once:

- **data_ready is a pulse, not a level.** It is driven low unconditionally at the top of the clocked branch and raised only at the edge completing a byte: the same one-cycle shape as the edge detector in section 3.7. A consumer that misses it has missed the byte, which is exactly why it is paired with a `data_out` that *holds* its value afterwards. A real peripheral would put a holding register or a small FIFO here so that a slow consumer cannot lose data.
- **shift_enable gates the bit counter, not just the shifting.** Both advance together or neither does, so a gap in the incoming stream pauses the receiver mid-byte instead of corrupting it, and it resumes on exactly the bit it was waiting for.

Checking it. The example comes with a `serial_rx8_tb.vhd`, which shifts in two bytes with a pause between them and checks that `data_ready` pulses once per byte, never early:

```
cd lectures/L06/serial_rx8
ghdl -a --std=93 serial_rx8.vhd serial_rx8_tb.vhd
ghdl -e --std=93 serial_rx8_tb
ghdl -r --std=93 serial_rx8_tb --assert-level=error --stop-time=10ms
```

Clocked at 50 MHz and continuously enabled, this receiver swallows a byte in 160 ns, which no LED can show you. That is why the testbench matters here in a way it did not for a blinking LED, and that is how most of real digital design goes. But it is no reason to skip the board, because `shift_enable` is precisely the way out: see section 6.7.

6.6 On the board: watching a counter wrap

The counter is demonstrated on the DE0-CV during the session, built the same way as every design since chapter 1: a Quartus Prime Lite project against 5CEBA4F23C7N, the .vhd file added, compile, assign every port a physical pin in the Pin Planner, then recompile and program the board.

One thing has to change first, and it is worth seeing why. Clocked at 50 MHz, a 4-bit counter runs through all sixteen states and wraps every 320 ns, about three million times a second. An LED driven straight from it does not blink, it simply glows at half brightness. To see a wraparound you have to watch a much slower bit: count in a wider register, say 28 bits, and assign the **top** four bits to the LEDs.

The demonstration is therefore a wrapper around two modules rather than the exercise's entity on its own: a `reset_sync` taking the raw `reset_n` from the pin, and the widened counter taking its `reset_s2_n` from there. The counter's port is called `reset_s2_n` precisely so that this connection is the only one that type-checks, and section 6.7 repeats the pattern with a third module.

- clock to the 50 MHz oscillator, `reset_n` to a push button (into `reset_sync`, never straight into the counter), count (27 downto 24) to four LEDs.
- Bit 24 changes once every 2^{24} clock cycles, about a third of a second, so the four LEDs count visibly from 0000 up to 1111 and then, with nothing in the circuit doing it, back to 0000. A full round takes about five seconds.

Dividing a clock by watching a high bit is the crudest timer imaginable, and its limitation is worth naming now: the period is locked to a power of two, and the only way to change it is to pick a different bit. Chapter 7 replaces it with a counter compared against a target value you choose, which is all a timer really is.

6.7 On the board: shifting in a byte by hand

Section 6.5 said that a byte arrives too fast to follow. The solution is not to slow the clock down, which would be the software instinct and is the wrong move in hardware: the clock

is the one thing in a synchronous design you do not fiddle with. `serial_rx8` already has the right control, and it is `shift_enable`, which “gates the bit counter, not just the shifting”. Hold it low and the receiver stops dead, mid-byte, for as long as you like. Raise it for exactly one cycle and exactly one bit goes in.

So drive `shift_enable` from a **button** instead of holding it high. One press, one bit.

The demonstration is a small wrapper around three modules you already have, and no new logic beyond a single flip-flop:

- `reset_sync` (section 4.5), turning the raw `reset_n` into `reset_s2_n` for everything else.
- `button_sync` (section 4.6) with `COUNT = 1`, turning a press into the one-cycle pulse `shift_enable` wants. This is no optional garnish: a raw button connected to `shift_enable` would shift in a random handful of bits per press, one per clock cycle for as long as the contacts bounce.
- `serial_rx8` itself, unchanged, with its `reset_s2_n` taken from `reset_sync` above rather than from the button. That is the whole reason the port is called `reset_s2_n`: the wrapper owns the raw `reset_n`, and the receiver only ever gets to see the synchronised one.
- A flip-flop latching `data_ready`, set when the pulse arrives and cleared by reset. Without it there is nothing to see: `data_ready` is high for 20 ns, and no LED can show you that.

On the board: clock to the 50 MHz oscillator, `reset_n` to a push button, `serial_in` to a switch (that is the bit you are about to send), the shift button to a second push button, `data_out(7 downto 0)` to eight LEDs, and the latched `data_ready` to a ninth.

Now set the switch, press the button, and watch one bit come in at the rightmost LED and the whole pattern step left. Send 10110010 one bit at a time and the byte builds up across the LEDs in front of you, most significant bit first, exactly as section 6.3’s idiom says it must. At the eighth press the ninth LED lights and the bit counter starts over.

Three things this makes visible that the testbench can only assert:

- **The bit order.** The first bit you send ends up in bit 7. Reading that off a waveform is a chore; watching it travel across a row of LEDs is not.
- **What `shift_enable` is for.** Let go of the button for a minute mid-byte and nothing moves, nothing is lost, and the eighth bit still completes the byte. That is the “pauses mid-byte instead of corrupting it” property, demonstrated rather than claimed.
- **Pulse versus level, once more.** The reason a flip-flop has to be added for `data_ready` is that whole distinction, and it is the same reason the edge detector in section 3.7 existed.

6.8 Summary

What you should be able to do now

- Build a 4-bit counter as a gate network, and explain, with your finger on your own drawing, why nothing in it makes the count return to zero.
- Implement the same counter in VHDL, explain the wraparound as a consequence of the register's bit width, and say why a simulator objects to a wraparound that the hardware performs without blinking.
- Implement a shift register in both SIPO and PISO form, know which suits which job, and know which direction the bits shift in and why sticking to one matters.
- Combine a counter with a shift register to detect that a complete word has arrived, and verify a clocked design by running its testbench.

Questions to test yourself with

1. Why does a counter of type `natural` range 0 to 15 return to 0 after 15 with no explicit “if at the maximum, clear” logic? What has to hold for the range? Which element performs the wraparound?
2. A VHDL simulator aborts on the wraparound your CircuitVerse drawing performs without complaint. Which of the two models the hardware, and what does that say about trusting either one alone?
3. What is the practical difference between a SIPO and a PISO shift register? Which would you reach for to receive a stream of serial bits, and which to drive a chain of LEDs from a fixed pattern?
4. A shift register and a counter are both “N flip-flops sharing a clock”. What separates them is what feeds each flip-flop's input. Describe that difference for each.
5. Why is `data_ready` in `serial_rx8` a one-clock-cycle pulse rather than a level that stays high once a byte has arrived?
6. Why must `shift_enable` gate the bit counter and not just the shifting?
7. The board demonstration drives `shift_enable` from a button instead of slowing the clock down. Why is a slowed clock the wrong answer, and what would break if the button were connected to `shift_enable` without passing through `button_sync` first?
8. `data_ready` needs a flip-flop of its own before an LED can show it. Which chapter's problem is that, in another guise?

Looking ahead

Chapter 7 goes on with:

- Timers: a counter with a target value, which is what turns clock cycles into real time.
- Built by adding exactly two things to the counter you drew here, a comparator and a clear, so keep that CircuitVerse project.

- The walking LED: this chapter’s shift register, clocked by that timer and started by a synchronised button.

6.9 Exercises

How to check your work. Every exercise below that asks you to write a VHDL module has a self-checking testbench under `lectures/L06/exercises/`. Write your module in its directory `exercises/<module>/`, with the entity name and the **port order** the exercise states, and then run it with GHDL — see section 2.7 for the three commands.

EXERCISE SET 1

Counters

Exercise 6.1 Width and wraparound point

REASONING

A signal is declared as:

```
signal counter: natural range 0 to 1023;
```

- How many bits wide is the register this synthesises to?
- What is the largest value counter can hold before it wraps back to 0 at the next increment?
- Rewrite the declaration for a counter that instead has to count from 0 up to 63.
Follow the counter pattern in section 6.1.

Exercise 6.2 Trace a wraparound by hand

REASONING

Trace a counter of type `natural range 0 to 3` by hand (no VHDL needed). Assume the counter starts at 2, that it is incremented once per clock edge for five consecutive edges, and that there is no reset in between.

Note, clock edge by clock edge: the value held before the edge, and the value captured at the edge. Explain why the counter returns to 0 with no explicit “if the count reaches its maximum, clear it” logic.

Note: This is the *synthesised* behaviour, the one that matters for hardware. Section 6.1 flags the simulation caveat that goes with it: GHDL range-checks a signal of type `natural range 0 to 3`, so a simulation of this counter aborts at the wraparound instead of rolling over. Trace it on paper as hardware, and reach for a full-width unsigned if you ever need a counter that wraps in simulation too.

Exercise 6.3 The counter by hand, and the bit that runs out

CIRCUIT

Build the 4-bit counter from section 6.1 by hand in CircuitVerse, and use it to see the overflow rather than take it on trust.

- a) Build it: a 4-bit register (four D flip-flops sharing a clock), an adder with the register's outputs on one input and a constant 1 on the other, the adder's sum wired back to the register's D inputs, and four output elements or LEDs, one per bit.
- b) Run it with a clock period of 1000 ms and note what you see:
 - Write down the sequence of counts over sixteen consecutive clock edges, starting from 0000.
 - Keep an eye on carry_out on the topmost adder as the count rolls from 1111 to 0000: during how many of the sixteen edges is it high, and what is connected to it?
 - Explain, in one sentence, where the fifth bit goes, and why the register stores 0000 rather than 10000.
- c) Now reason about what you did *not* build: point at the part of your drawing that decides the counter should return to 0000. There is none. Say what does the job instead. What would you have to add to make it count 0 to 9 and then wrap? (Exercise 6.4 asks you to write exactly that in VHDL, so keep your answer.)
- d) Change the width to three bits by removing the top flip-flop and narrowing the adder. Predict the new wraparound point before you run it, confirm it, and state the rule connecting a counter's width to the value it wraps at.

Keep this circuit. Chapter 7 builds its timer by adding two things to it, a comparator and a clear, and starting from a counter you have already seen working is the whole point of the exercise there.

Exercise 6.4 counter: a counter with an arbitrary radix

VHDL

Write an entity named counter that counts from 0 up to one below its radix, and then wraps back to 0. With the default radix that is a decimal counter, 0 to 9.

The entity has one generic:

Generic	Type	Default	Meaning
RADIX	natural range 1 to 2**16	10	How many values the counter goes through before repeating. It counts 0 to RADIX-1 inclusive, so RADIX is the length of the cycle, not the largest count.

and these ports:

Port	Dir.	Type	Meaning
clock	in	std_logic	System clock.
reset_s2_n	in	std_logic	Active-low, already synchronised : it comes from your reset_sync, never straight from a pin. Asynchronous, so it clears with no clock edge involved.
count	out	natural range 0 to RADIX-1	The running count. 0 is a value it takes, so the range starts there: reset drives it to 0, and it returns there at every wraparound.
tick	out	std_logic	A one-cycle pulse at the edge where count wraps from RADIX-1 back to 0.

Implement it with a single synchronous process:

- Include clock and reset_s2_n in the sensitivity list.
- At reset, set count to 0 and tick to '0'.
- At every rising clock edge: drive tick low by default; when count has reached RADIX-1, wrap count to 0 and pulse tick high; otherwise increment count.

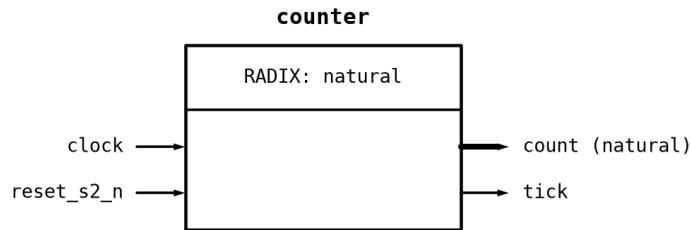
Write the comparison against RADIX-1, never against 9. The generic is the whole difference between a module and a module you can use twice: RADIX is fixed before synthesis runs, so the comparator it builds is no larger than a hardcoded one, and the counter's register is sized for the radix you asked for. That is the same argument section 4.8 makes for button_sync's COUNT, and the reason chapter 7's timer takes its target value as a generic rather than as a constant.

Unlike the free wraparound of a counter of type natural range 0 to 15, a counter with an arbitrary radix needs an explicit comparison, since RADIX-1 is in general not a power-of-two boundary. Note what happens when it is: with RADIX = 16 the comparison is still written out, but the hardware underneath no longer needs it.

Hint: The process needs to read the running count (to compare it and to increment it), so hold the count in an internal signal count_s: natural range 0 to RADIX-1;; run the logic on count_s, and drive the output with count <= count_s; outside the process, exactly

as `d_flip_flop.vhd` uses `q_s` in chapter 3. That is no stylistic choice: every ghdl command in this book passes `--std=93`, and under VHDL-93 reading an out port does not compile at all. Later standards relax that, which is why you may have seen it done elsewhere.

Follow the counter pattern in section 6.1, and the drive-low-then-pulse shape `data_ready` has in section 6.5.



Self-check. Name your entity `counter`, with the generic `RADIX` (`natural` range 1 to 2^{16} , default 10), the inputs `clock`, `reset_s2_n` and the outputs `count` (`natural` range 0 to `RADIX-1`) and `tick` (`std_logic`), declared in that order; its testbench is in `exercises/counter/`. It elaborates **two** instances, one with `RADIX = 10` and one with `RADIX = 4`, and runs each through two full cycles, checking that `tick` is a one-cycle pulse rather than a level and that it never fires halfway through. An architecture that silently hardcodes 10 passes the first instance and fails on the second, which is the point of checking two.

EXERCISE SET 2

Shift registers

Exercise 6.5 Four bits into a SIPO register

REASONING

An 8-bit SIPO shift register starts at `00000000` and uses the idiom from section 6.3:

```
shift_reg <= shift_reg(6 downto 0) & serial_in;
```

It receives the serial bit stream 1, 0, 1, 1 (one bit per clock edge, in that order) over four consecutive rising clock edges.

- Write out the whole 8-bit contents of `shift_reg` after each of the four clock edges.
- After the four edges: which four bits of `shift_reg` hold the received data, and in what order do they sit, relative to the order the bits arrived in?

Exercise 6.6 Four bits out of a PISO register

REASONING

A 4-bit PISO register, following the idiom from section 6.4, is parallel-loaded with `parallel_in = "1010"` (load = '1' for one clock edge), then shifted with `shift = '1'` for the following four clock edges, with `serial_in` held at '0' throughout.

Remember the direction from section 6.2: bits move towards the MSB, `serial_out` is bit 3 (the value about to leave), and `serial_in` enters at bit 0.

- a) Write out the register's contents and `serial_out` at every step, one row per clock edge: the row for the load edge, before any shifting, and then one row for each of the four shift edges.
- b) In what order do the four loaded bits appear on `serial_out`, relative to their positions in `parallel_in`?
- c) What are the register's final contents after all four shifts, and why?

Hint: You need no real hardware to check Exercise 6.5 and 6.6 by hand. Work through them exactly as you would trace a truth table: one clock edge, one row, at a time.

Exercise 6.7 sipo8 and piso8

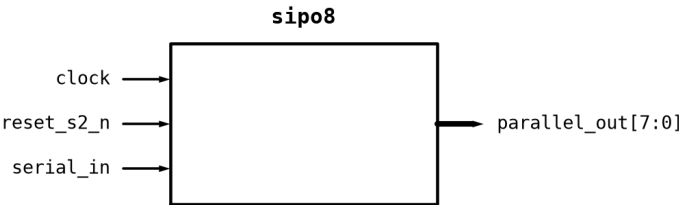
VHDL

Implement the chapter's two useful shift register forms in VHDL.

- a) **sipo8.** Write an entity named `sipo8`, a serial-in, parallel-out register.

Port	Dir.	Type	Meaning
clock	in	std_logic	System clock.
reset_s2_n	in	std_logic	Active-low, already synchronised . Asynchronous. Clears the whole register, so <code>parallel_out</code> reads "00000000" while it is held low.
serial_in	in	std_logic	The incoming bit.
parallel_out	out	std_logic_vector(7 downto 0)	The register's contents.

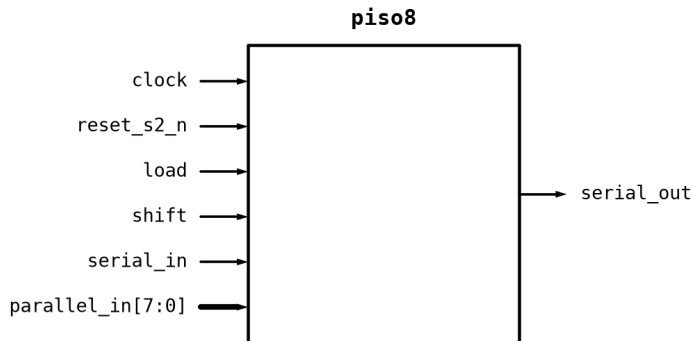
Follow the SIPO idiom from section 6.3.



- b) **piso8.** Write an entity named `piso8`, a parallel-in, serial-out register.

Port	Dir.	Type	Meaning
clock	in	std_logic	System clock.
reset_s2_n	in	std_logic	Active-low, already synchronised . Asynchronous. Clears the whole register, so serial_out reads '0' while it is held low.
load	in	std_logic	Capture parallel_in at the next rising edge.
shift	in	std_logic	Shift one position at the next rising edge.
serial_in	in	std_logic	The bit fed in at bit 0 on a shift.
parallel_in	in	std_logic_vector(7 downto 0)	The value captured on a load.
serial_out	out	std_logic	The register's top bit.

Follow the PISO idiom from section 6.4: at a load edge (`load = '1'`), capture `parallel_in`; at a shift edge (`shift = '1'`), move every bit one position towards the MSB and feed `serial_in` in at bit 0; drive `serial_out` from bit 7, so that a loaded byte leaves most significant bit first; and let `load` take precedence over `shift` when both are high.



c) For each of the four shift register forms (SISO, SIPO, PISO, PIPO), name a task it is the natural choice for. Follow the summary table in section 6.2.

Derive each idiom afresh from the chapter rather than copying `serial_rx8`'s shift line.

Self-check. `sipo8`: the ports `clock`, `reset_s2_n`, `serial_in` (in) and `parallel_out` (`std_logic_vector(7 downto 0)`, out), declared in that order; testbench in `exercises/sipo8/`. `piso8`: the ports `clock`, `reset_s2_n`, `load`, `shift`, `serial_in`, `parallel_in` (`std_logic_vector(7 downto 0)`) and `serial_out` (out), declared in that order; testbench in `exercises/piso8/`.

Keep piso8. Chapter 8's transmitter capstone does not instantiate it, but it is worth rereading there for the idiom's sake, since that machine shifts the byte out inline.

EXERCISE SET 3

Reading the serial receiver

The worked example comes with a reference testbench. Run it from the example's own directory before you start:

```
cd lectures/L06/serial_rx8
ghdl -a --std=93 serial_rx8.vhd serial_rx8_tb.vhd
ghdl -e --std=93 serial_rx8_tb
ghdl -r --std=93 serial_rx8_tb --assert-level=error --stop-time=10ms
```

Exercise 6.8 The receiver's behaviour, and three real bugs

SIMULATION

Account for the receiver's behaviour, with reference to section 6.5.

- a) The bit stream 1, 0, 1, 1, 0, 0, 1, 0 arrives, one bit per enabled clock edge. What is on `data_out` during the cycle where `data_ready` pulses? Which arriving bit ends up in `data_out(7)`, and which in `data_out(0)`?
- b) `shift_enable` goes low for three clock cycles after the fourth bit, and then high again for the remaining four. Does the receiver still produce the same byte? Explain what the bit counter does during the gap. What would go wrong instead if `shift_enable` gated only the shift register and not the counter?
- c) Make each of the following changes in your own copy of `serial_rx8.vhd`, one at a time, and **predict** what the reference testbench will report before you run it. Then run it and account for the result:
 - Change the shift line to the mirrored direction, `shift_reg <= serial_in & shift_reg(7 downto 1);`.
 - Remove the unconditional `data_ready <= '0';` at the top of the clocked branch.
 - Change the bit counter's comparison from 7 to 6.

Each of them is a real bug that someone has shipped. For each, say which of the testbench's checks catches it, and what the failure would have looked like on a real serial link if nothing had caught it.

Timers

A counter with a target value, drawn by hand before it is written.

IN THIS CHAPTER

- Timers: a counter raising a flag when it reaches a target value.
- Turning clock cycles into real time, and the equality comparator that does it.
- The same timer in VHDL, reusable thanks to a generic rather than through rewiring.
- Composing a design out of finished modules instead of rewriting their logic.
- `walking_led`: a single lit bit walking along a row of LEDs.

Before the timer runs: predict at which count value `timeout` fires, and why that is every eleventh edge rather than every tenth. That off-by-one is the whole reason the module's generic is called `TICK_COUNT` and not `PERIOD`.

`walking_led` in section 7.5 is the payoff from the last four chapters: three modules, two of them unchanged from chapter 4 and one written here, composed into something you can see from the back row.

Refresher: The timer as a counter with a comparison is in section 6.3 of *Digital Electronics*.

7.1 Timers: a counter with a target value

A **timer** answers a different question from the counter in section 6.1. A counter answers “what is the count right now?”; a timer answers “has a certain amount of time passed?”

It does so with three parts: an internal counter invisible from outside, a target value `TICK_COUNT` fixed per instance, and a comparison which, when true, clears the counter and pulses the output flag `timeout` for one clock cycle.

Because the system clock ticks at a known, fixed frequency, counting ticks is the same thing as measuring elapsed time. On the DE0-CV that is 50,000,000 ticks per second:

$$TICK_COUNT = seconds \times 50\,000\,000$$

- `TICK_COUNT = 50_000_000`: timeout once per second.
- `TICK_COUNT = 25_000_000`: twice per second.

- `TICK_COUNT = 5_000_000`: ten times per second.

Strictly speaking the counter runs 0 through `TICK_COUNT` inclusive, so a full period is `TICK_COUNT + 1` cycles. That extra tick is between 0.02 and 0.2 ppm at these values, far below the board oscillator's own tolerance, so the round numbers above are used for clarity; for an exact cycle count, use $TICK_COUNT = seconds \times 50\,000\,000 - 1$.

This is the opposite of a blocking `_delay_ms()`. A blocking delay stops the whole program to wait; a timer runs permanently in parallel with everything else and simply raises a flag, and nothing in the design is ever blocked waiting for it.

7.2 Building a timer by hand in CircuitVerse

Before any VHDL is written: build a timer as a gate network, exactly as you did for chapter 1 and 2's combinational networks and chapter 3's flip-flops. A timer is small enough to draw in its entirety, and drawing it is what makes section 7.3's VHDL read as a description of something you already understand rather than a new idiom to learn by heart.

Build it in a size you can follow: a **4-bit counter** with `TICK_COUNT = 10`, which is 1010. The real design counts to 50,000,000; nothing in the structure changes, only the width.

Start from the counter you have already built in section 6.1: a 4-bit register Q0 to Q3 sharing a clock, an adder and a constant 1 with the sum fed back to the D inputs, and nothing more. If you did not keep it, it is three elements and about five minutes to redraw. Confirm that it still counts and wraps before you add anything, so that you know the counter underneath is sound when the timer later misbehaves.

Everything here adds **two things on top of that**: a comparison saying “the count has arrived”, and a clear making it repeat. That is the whole difference between a counter and a timer.

Add the comparison. You want timeout high exactly when the four bits spell 1010, and the clean way to build “are these two numbers equal?” is one **XNOR gate per bit**. An XNOR gives 1 when its inputs are *the same*, which is section 1.1's XNOR column read as an equality test. Wire in four, each comparing one counter output against the corresponding target bit (Q3 against 1, Q2 against 0, Q1 against 1, Q0 against 0), and drive the target side from a constant in CircuitVerse rather than from a wire you might later mistake for a signal. Then combine the four outputs with AND; that single AND gate is timeout. With $\odot$ for XNOR, the one operator section 1.2 gave no symbol:

$$timeout = (Q_3 \odot 1) \cdot (Q_2 \odot 0) \cdot (Q_1 \odot 1) \cdot (Q_0 \odot 0)$$

All four bits have to match simultaneously, which is what “the counter has reached 1010” means. This is a **general equality comparator**, worth recognising as a building block of its own: the same four XNORs plus an AND compare against any 4-bit value, and only the constants change.

Add the clear. So that the timer restarts instead of carrying on: feed timeout back to force the counter to 0000 at the next edge, by gating every flip-flop's D input so that timeout = 1 takes precedence over the adder's output. Without it the counter carries on to 1111 and wraps by itself, so timeout fires every sixteenth edge instead of every eleventh. Both are periodic; only one has the period you asked for.

Then confirm, one clock edge at a time.

- The counter goes from 0000 through 1010, and timeout is low for every count value except 1010. Keep an eye on the four XNOR outputs as it counts and note how often three are high at once. Three bits matching is no hit; it is the AND gate insisting on all four.
- At the edge after timeout fires, the counter is back at 0000 and timeout is low, so it is high for exactly one clock cycle, never two.
- The whole cycle is therefore eleven edges, count values 0 through 10, repeated forever. That is section 7.1's TICK_COUNT + 1 off-by-one, and this is the book's easiest place to see it, since you can count the edges by eye.

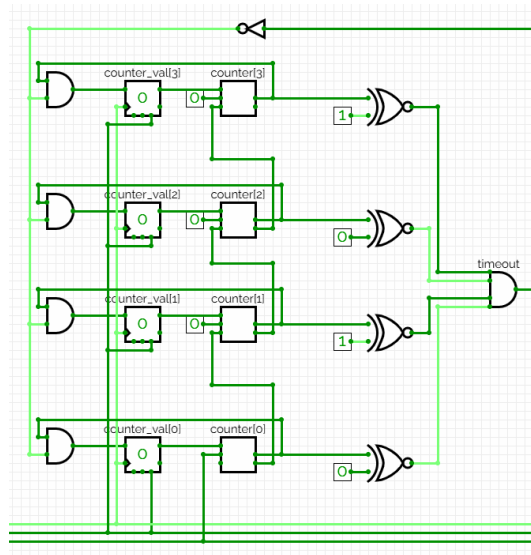


Figure 7.1 The comparison logic: four XNORs against 1010, ANDed into timeout.

The drawing is cropped to the part this section adds. It labels the counter's bits counter[3] down to counter[0] where the text above calls them Q3 to Q0, and the increment feeding them is drawn as gate-level carry logic rather than the single adder block section 6.1 used, since that is what CircuitVerse gives you when you build it one bit at a time. The four XNOR gates, their constants 1, 0, 1, 0, and the AND combining them are all that is new here.

Two things are worth noticing before the VHDL, since both are what the language buys you:

- **The comparator is hardwired to one target value.** Changing TICK_COUNT from 10 to 12 means going back into the drawing and flipping two constants. It is a small change, but it is a change to the *circuit*, and every instance needs its own copy. In section 7.3 the target value becomes a generic.
- **The width is hardwired too.** Counting to 50,000,000 takes 26 flip-flops and a 26-bit comparator, 26 XNORs over 52 inputs: the same circuit, an unreasonable drawing. In section 7.3 it is the same six lines of VHDL either way.

That is the trade-off this book makes throughout: draw the circuit once, at a size you can see, to know what the tool builds for you, and then let the tool build it at a size you could not have drawn.

7.3 The timer module in VHDL

The same circuit as a reusable module. It is built live during the session, and Exercise 7.4 asks you to write it yourself afterwards, so it is printed here in full rather than handed out as a file:

```
entity timer is
    generic(TICK_COUNT: natural := 50_000_000);
    port(clock, reset_s2_n, enable: in std_logic;
         timeout                : out std_logic);
end entity;

architecture behaviour of timer is
    -- Internal tick counter.
    signal counter: natural range 0 to TICK_COUNT;
begin
    process(clock, reset_s2_n) is
    begin
        if (reset_s2_n = '0') then
            timeout <= '0';
            counter <= 0;
        elsif (rising_edge(clock)) then
            timeout <= '0';
            if (enable = '1') then
                if (TICK_COUNT > counter) then
                    counter <= counter + 1;
                else
                    timeout <= '1';
                    counter <= 0;
                end if;
            end if;
        end if;
    end process;
end architecture;
```

```

        end if;
    end if;
end if;
end process;
end architecture;

```

- `TICK_COUNT` is a **generic**, not a constant baked into the architecture, so the same entity serves any frequency by being instantiated with a different generic `map`. The internal counter's range is written in terms of it, so the register is sized for the target value rather than for whatever an unconstrained `natural` would give, the same point section 6.1 makes about `natural` range 0 to 15. A generic can size a signal precisely because it is fixed before synthesis runs.
- `reset_s2_n` is the already *synchronised* reset signal, never the raw asynchronous `reset_n`. A timer never touches a raw asynchronous input directly.
- `enable` **freezes** the timer rather than clearing it. While it is '0', counter holds its value and `timeout` stays low, and a disabled timer carries on where it left off rather than restarting. Chapter 8's `fsm_led` depends on that.
- `timeout` is a one-cycle pulse. The `else` branch setting it is reached at only one edge: the edge where counter already stands at `TICK_COUNT` and is cleared back to 0. At the following edge the unconditional `timeout <= '0'`; runs with nothing after it to overwrite it, so the pulse is exactly one cycle wide.

Compare this with section 7.2's gate network. The logic is the same, and section 7.2 already named what the generic buys. One difference it did not name: section 7.2's `timeout` is a combinational decode, high during the cycle where the count shows 1010, while this one is registered, high one cycle later, during the cycle where counter shows 0. Same period, different phase, and different glitch behaviour, which is the distinction chapter 8's discussion of Moore versus Mealy rests on.

7.4 Composing a complete timer circuit

A timer on its own only pulses `timeout`; something has to react to that pulse, and a button has to control the timer. With chapter 4's two-flip-flop synchroniser reused, a complete button-controlled, timer-driven LED circuit looks like this:

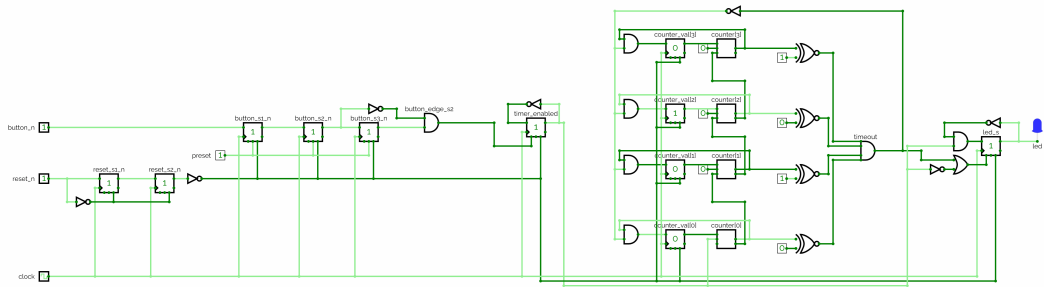
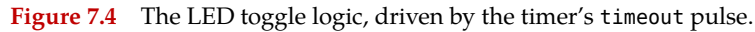
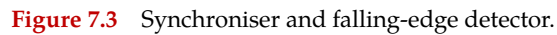


Figure 7.2 The whole circuit at once: synchronised button, timer and toggled LED. The details are legible in the two close-ups below.

Read it as a floor plan rather than as a wiring diagram. It is a wide circuit, so at page size its labels come out small; what it is for is showing how few parts there are and roughly where each sits. The two close-ups below are where the signal names are readable, and together they cover everything in it.

From left to right:

- **Synchronisation and edge detection.** `button_n` passes through two flip-flops for metastability protection, and a third then lets the circuit compare “now” against “previous” and detect a falling edge, giving a one-cycle `button_edge_s2` pulse.
- **The timer** (section 7.3), enabled or disabled by `timer_enabled`, which in turn is toggled by `button_edge_s2`.
- **Toggling the LED.** The timer’s timeout pulse flips the LED’s state every time it fires, and the LED is forced off as soon as the timer is disabled or the circuit is reset.



- **Every asynchronous input is synchronised before it touches any other logic**, and every enable or toggle signal derived from a button is a synchronous signal from that point on. You reuse that unchanged in section 7.5 and again for chapter 8’s state machines.
- **One clock, and everything else is an enable.** timeout clocks the LED by being a one-cycle *enable* for a flip-flop running on the system clock, not by being wired into a clock input. It is tempting to run a counter or timer output into a clock port to “slow things down”; do not. A design with one clock is a design whose timing the tool can analyse, and this is

the single most common mistake a programmer makes when a baud generator has to be written.

7.5 walking_led: a shift register with a walking LED

walking_led is the first design composing three chapters' building blocks into a circuit that can be demonstrated in hardware: a single lit LED walking along a row, one position per timer tick, started and stopped with a button. It is built live during the session, and Exercise 7.6 asks you to write it yourself afterwards, so it is printed here in full rather than handed out as a file.

It writes almost no new logic, reusing reset_sync and button_sync unchanged (section 4.5 and 4.6) to synchronise and edge-detect the button, timer (section 7.3) to clock the speed, and the shift register from section 6.2–6.4 for the walking itself. That is what the last four chapters have accumulated: a small set of composable modules.

You write all three yourself: reset_sync and button_sync in Exercise 4.8 and 4.9, timer in Exercise 7.4 in this chapter. Copy your own into the exercise directory before you build this one. Reusing a module presupposes having one, and this is the first design that asks you for the ones you have already built.

```
entity walking_led is
    generic(LED_COUNT : natural := 8;
           TICK_COUNT : natural := 25_000_000);
    port(clock, reset_n, button_n: in std_logic;
         led : out std_logic_vector(LED_COUNT-1 downto 0));
end entity;

architecture behaviour of walking_led is
    signal reset_s2_n, button_edge_s2 : std_logic;
    signal button_n_v, button_edge_s2_v: std_logic_vector(0 downto 0);
    signal shift_enable, shift_tick   : std_logic;
    signal shift_reg: std_logic_vector(LED_COUNT-1 downto 0);

begin
    led <= shift_reg;

    button_n_v(0) <= button_n;
    button_edge_s2 <= button_edge_s2_v(0);

    reset_sync1: entity work.reset_sync
        port map(clock, reset_n, reset_s2_n);

    button_sync1: entity work.button_sync
```

```

    generic map(1)
    port map(clock, reset_s2_n, button_n_v, button_edge_s2_v);

timer1: entity work.timer
    generic map(TICK_COUNT)
    port map(clock, reset_s2_n, shift_enable, shift_tick);

SHIFT_ENABLE_PROCESS: process(clock, reset_s2_n) is
begin
    if (reset_s2_n = '0') then
        shift_enable <= '0';
    elsif (rising_edge(clock)) then
        if (button_edge_s2 = '1') then
            shift_enable <= not shift_enable;
        end if;
    end if;
end process;

SHIFT_PROCESS: process(clock, reset_s2_n) is
begin
    if (reset_s2_n = '0') then
        shift_reg <= (0 => '1', others => '0');
    elsif (rising_edge(clock)) then
        if (shift_tick = '1') then
            shift_reg <= shift_reg(LED_COUNT-2 downto 0) & shift_reg(LED_COUNT-
-1);
        end if;
    end if;
end process;
end architecture;

```

Design choices worth noting:

- button_n_v and button_edge_s2_v only bridge a width difference: walking_led's button signals are single bits while button_sync's ports are vectors of width COUNT. The two concurrent assignments connect the scalar to element 0 and back. They cost no hardware.
- This is a PISO register (section 6.4) in its simplest form: parallel-loaded exactly once, at reset, with a single lit bit at position 0, and shifting only thereafter. There is no load at run time, since this design never loads a new pattern.
- It is a **circular** shift register. The shift is section 6.2's expression unchanged, so the lit bit walks towards the MSB; the only difference from an ordinary PISO is what feeds bit 0, which is the bit falling off the top rather than a serial_in from somewhere else. A one-shot register that "shifts a pattern out" becomes an endlessly repeating pattern generator for free, just by recirculating its own output.

- `shift_enable` is section 7.4’s toggle flip-flop reused verbatim: the button’s synchronised edge pulse flips a single bit, and that bit becomes the timer’s enable.
- `shift_tick` is the timer’s timeout pulse, reused as the shift register’s clock enable instead of driving an LED toggle. Same building block, a different consumer, and exactly the “one clock, everything else is an enable” rule from section 7.4.
- Both `LED_COUNT` and `TICK_COUNT` are generics, so generic `map(4, 25_000_000)` gives a row of 4 LEDs and generic `map(8, 5_000_000)` a faster 100 ms step, without touching the module.

On the board, demonstrated during the session, the ports are assigned via the Pin Planner (appendix B): `clock` to the 50 MHz oscillator, `reset_n` and `button_n` to two push buttons, and `led` to a row of LEDs on the board. Press the button once to set the bit moving, once more to freeze it.

You can see the same behaviour without a board by running the reference testbench, which overrides both generics so that a full round takes a handful of clock cycles:

```
cd lectures/L07/exercises/walking_led
cp ../../L04/exercises/reset_sync/reset_sync.vhd . # the three you wrote ↵
↳ yourself
cp ../../L04/exercises/button_sync/button_sync.vhd .
cp ../timer/timer.vhd .
ghdl -a --std=93 reset_sync.vhd button_sync.vhd timer.vhd \
    walking_led.vhd walking_led_tb.vhd
ghdl -e --std=93 walking_led_tb
ghdl -r --std=93 walking_led_tb --assert-level=error --stop-time=10ms
```

7.6 Summary

What you should be able to do now

- Explain what separates a timer from an ordinary counter, and why the difference is a comparison.
- Build a timer as a gate network: an equality comparator of one XNOR per bit, and the clear that makes it repeat instead of wrapping.
- Implement a timer in VHDL as a reusable module, convert a period in seconds into a `TICK_COUNT`, and say why timeout is a one-clock-cycle pulse rather than a level.
- Compose a design out of finished modules instead of rewriting them, and say why a button controlling a timer still has to pass through a synchroniser first.

Questions to test yourself with

1. In what sense is a timer “just a counter with a comparison”?
2. Why must the internal counter be cleared to 0 when it reaches its target, instead of being left to count on?
3. Why is timeout a single-cycle pulse rather than a level that stays high?
4. In the CircuitVerse version, changing the target value means rewiring the comparator. What replaces that rewiring in the VHDL version, and when is its value fixed?
5. Why is it still necessary to run a button through a synchroniser before its edge is used to enable a timer, even though the button has nothing to do with the counting?
6. `walking_led` writes almost no logic of its own. Name the three modules it composes and which chapter each came from.

Looking ahead

Chapter 8 goes on with:

- State machines, the book’s last building block, first designed by hand: state diagram, state table, Karnaugh-derived logic and a gate network in CircuitVerse, exactly as this chapter’s timer was drawn before it was written.
- Then in VHDL, where an enumerated type and a case replace the state table and the Karnaugh maps entirely.
- A timer is itself a state machine with two states and a single purpose. Chapter 8 generalises that to circuits with many states and many transitions between them.

7.7 Exercises

How to check your work. Every exercise below that asks you to write a VHDL module comes with a self-checking testbench under `lectures/L07/exercises/`. Write your module in its directory `exercises/<module>/`, with the entity name and the **port order** the exercise states, and then run it with GHDL — see section 2.7.

From this chapter onwards the designs reuse modules you wrote earlier, and they are not handed out a second time: every exercise says which of your own files to copy in, and writes out the `ghdl -a` line that needs them.

EXERCISE SET 1

Timers by hand

Exercise 7.1 A timer with a different target value

CIRCUIT

Build a second timer by hand in CircuitVerse, following section 7.2, but with the target value `TICK_COUNT = 12` instead of the section's 10.

a) Retarget the comparator:

- Write 12 in binary with 4 bits.
- The four XNOR gates stay exactly where they are. Say which of their constant inputs change, and to what.
- Say, in one sentence, why an equality comparator built from XNORs needs no gate added, removed or rewired in order to compare against a different value.

b) Build it: reuse your 4-bit counter and comparator from the session, changing only the constants. Set the clock period to 1000 ms and confirm that `timeout` fires once every thirteenth clock edge, for exactly one edge each time.

c) Now remove the clear, so that `timeout` no longer forces the counter back to 0000. Predict, before you run it, how often `timeout` fires now. Run it and confirm. Explain the result in terms of the counter's own wraparound (section 6.1).

d) Break the comparator deliberately: disconnect one XNOR's output from the AND gate and hold that AND input high instead. Predict which count values now raise `timeout`, before you simulate, and confirm by stepping the counter through a whole cycle. This is what "three of the four bits match" looks like, and it is the most common way for a hand-drawn comparator to be wrong.

Hint: Counting 0 through 12 inclusive is thirteen edges, not twelve. Section 7.1's remark about `TICK_COUNT + 1` is the same off-by-one, and it is worth sorting out here, where you can count the edges by eye, rather than later at 50 MHz.

Exercise 7.2 Four questions about the module

REASONING

Answer each of the questions below in one or two sentences, with reference to the timer module in section 7.3:

- Why is `timeout` a one-cycle pulse rather than a signal that stays high once the target value has been reached?
- While a running timer's `enable` input is held low, what happens to its internal counter from one clock edge to the next: is it cleared, does it pause, or does it carry on counting?
- Why does `timer` take `reset_s2_n` (the synchronised reset signal) rather than the raw `reset_n` as its reset input?

d) The CircuitVerse timer in Exercise 7.1 had to be rewired for the target value to change, while the VHDL module needs no change at all. What replaces the rewiring, and when is its value fixed?

EXERCISE SET 2

Timers in VHDL

Exercise 7.3 Three tick counts

REASONING

The DE0-CV board's system clock is 50 MHz. For each target below, work out the `TICK_COUNT` generic you would pass to the timer module:

- a) A timeout pulse once per second.
- b) A timeout pulse every 250 ms.
- c) A timeout pulse twenty times per second (20 Hz).

Hint: The relationship between a period and its tick count is $TICK_COUNT = seconds \times 50\,000\,000 - 1$. The -1 term is the same off-by-one Exercise 7.1 circled around: the counter runs 0 through `TICK_COUNT` inclusive, so a period is `TICK_COUNT + 1` clock cycles, not `TICK_COUNT`.

Exercise 7.4 timer

VHDL

Write the timer module itself, as an entity named `timer`.

It is built live during the session and printed in full in section 7.3, so this is no puzzle. Write it anyway, without copying: every design from here to the end of the book instantiates this module, both capstones included, and it is worth having built what you are about to reuse six times.

The entity has one generic:

Generic	Type	Default	Meaning
TICK_COUNT	natural	50_000_000	Number of ticks to count before a timeout. One second at the DE0-CV's 50 MHz clock, rounded: by Exercise 7.3's rule the exact value is 49_999_999, and 20 ns of error over a second is not worth the less readable constant.

and these ports:

Port	Dir.	Type	Meaning
clock	in	std_logic	System clock.
reset_s2_n	in	std_logic	Active-low, already synchronised reset. Asynchronous: it clears the counter and timeout with no clock edge involved.
enable	in	std_logic	The timer's enable. While it is low the counter holds its value, and carries on from there rather than starting over.
timeout	out	std_logic	One-cycle pulse, firing every TICK_COUNT + 1 cycles while the timer is enabled.

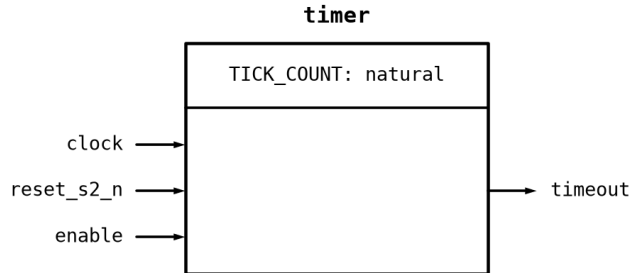
The architecture should:

- Declare the internal counter as `natural` range 0 to TICK_COUNT, so that the register is sized by the generic rather than left unconstrained.
- Use a single process, sensitive to `clock` and `reset_s2_n`, following the clocked process template from section 3.6.
- Clear both the counter and `timeout` at reset, outside the clocked branch.
- Drive `timeout` low unconditionally at the top of the clocked branch, and high only at the edge that clears the counter. That pattern, a default followed by an override, is what makes the pulse exactly one cycle wide rather than two.

Then answer, in one sentence each:

- `enable = '0'` must **pause** the counter, not clear it. Suppose you cleared it instead. Predict what the reference testbench reports, and which of the two properties above it is measuring when it does. Then run it and see. (Chapter 8's `fsm_led` depends on the pause, which is why the testbench cares.)
- The counter counts 0 through TICK_COUNT inclusive, so a period is TICK_COUNT + 1 cycles. Where in your code is that "+ 1" actually written? It does not appear as a literal anywhere.

- Section 7.2's hand-drawn timeout is a combinational decode of the count; yours is registered. Both have the same period. Which cycle does each fire on, and why does the difference matter to something downstream sampling timeout?



Self-check. Name your entity `timer`, with the generic `TICK_COUNT` (`natural`), the inputs `clock`, `reset_s2_n`, `enable` and the output `timeout`, declared in that order; its testbench is in `exercises/timer/`. It runs three full periods, so a timer firing only once does not pass, and it pauses mid-count to confirm that the count continued rather than starting over.

Keep this file. Exercise 7.5 below reuses it, so does `walking_led` in Exercise 7.6, and so do both of chapter 8's capstones.

Exercise 7.5 blinker

VHDL

Write an entity named `blinker` that blinks an LED on and off once per second, by reusing the `timer` module you just wrote rather than counting clock cycles yourself.

The entity has one generic:

Generic	Type	Default	Meaning
<code>TICK_COUNT</code>	<code>natural</code>	<code>25_000_000</code>	Half a second at the DE0-CV's 50 MHz clock. Expose it as a generic instead of baking the count in, so that a testbench can shorten the blink; the FPGA build only uses the default.

and these ports:

Port	Dir.	Type	Meaning
<code>clock</code>	in	<code>std_logic</code>	System clock.
<code>reset_n</code>	in	<code>std_logic</code>	Active-low reset, raw from the outside world. Asynchronous: activate it and <code>led</code> goes out immediately, without waiting for a clock edge.
<code>led</code>	out	<code>std_logic</code>	Cleared to '0' while the reset signal is active, and left there until the first timeout after the reset signal is released.

Note that this entity takes the **raw** `reset_n` rather than an already synchronised `reset_s2_n`. Synchronising it is part of the exercise, and it is what section 4.5 means by “exactly one module in a design is the exception”: here that module is `blinker` itself.

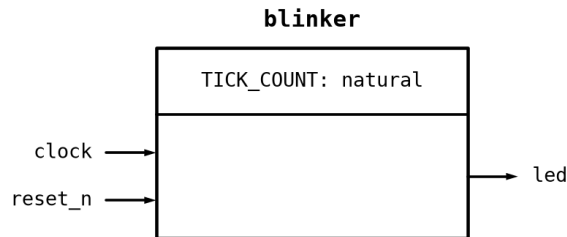
The architecture should:

- Instantiate your `reset_sync` from Exercise 4.8, and use its output `reset_s2_n` as the reset for everything inside.
- Instantiate the timer module from section 7.3, passing your own generic `TICK_COUNT` straight through to the timer’s `TICK_COUNT`, and holding its enable input high.
- Toggle `led` in a synchronous process every time the timer’s timeout pulse fires.

Do not reimplement the counting logic, or the synchroniser, yourself. Reuse `timer` and `reset_sync` unchanged, via a generic `map` and a port `map`, following the composition pattern in section 7.4. Explain why a timer period of **half** a second gives a blink cycle of **one** second (a full period of on and then off).

Then answer, in one sentence each:

- The LED has to be off during reset, and the testbench checks that it goes out with no clock edge in between. Which half of the assert-asynchronously-release-synchronously pattern does that check, and which half does it not check?
- Your timer is reset by `reset_s2_n`, two edges after `reset_n` is released. What would go wrong if you fed it the raw `reset_n` instead? Name the failure, not just the rule.



Self-check. Name your entity `blinker`, with the generic `TICK_COUNT` (`natural`), the inputs `clock`, `reset_n` and the output `led`, declared in that order; its testbench is in `exercises/blinker/`. **Copy your own `reset_sync.vhd` and `timer.vhd` into that directory** before you build; neither is handed out there, since you wrote both:

```

cd lectures/L07/exercises/blinker
cp ../../L04/exercises/reset_sync/reset_sync.vhd .    # the two you wrote ↪
  ↳ yourself
cp ../timer/timer.vhd .
ghdl -a --std=93 reset_sync.vhd timer.vhd blinker.vhd blinker_tb.vhd
  
```

The testbench overrides `TICK_COUNT` with a small value and checks that the LED toggles over and over, stays put for exactly one timer period between toggles, and that the first toggle is late by your synchroniser's two edges.

EXERCISE SET 3

The walking LED

A single lit LED walking along a row, one position per timer tick, started and stopped with a button. It is built live during the session and described in section 7.5.

This is the payoff from the last four chapters, and it writes almost no logic of its own: three modules, two of them unchanged from chapter 4 and one written here, composed. Exercise 7.6 builds it, and Exercise 7.7 and 7.8 change what you built.

Exercise 7.6 walking_led

VHDL

Write an entity named `walking_led`.

The entity has two generics:

Generic	Type	Default	Meaning
<code>LED_COUNT</code>	<code>natural</code>	8	How many LEDs the bit walks along. The row is this wide, and the walk wraps after this many steps.
<code>TICK_COUNT</code>	<code>natural</code>	<code>25_000_000</code>	Passed straight through to your timer, so that the bit moves one position per timer period: one step per half second at the DE0-CV's 50 MHz clock.

and these ports:

Port	Dir.	Type	Meaning
<code>clock</code>	in	<code>std_logic</code>	System clock.
<code>reset_n</code>	in	<code>std_logic</code>	Active-low reset, raw from the outside world. Synchronising it is your job, exactly as it was in blinker.
<code>button_n</code>	in	<code>std_logic</code>	Active-low push button, raw. Every press starts or stops the walk.
<code>led</code>	out	<code>std_logic_vector(LED_COUNT-1 downto 0)</code>	The LED row. Exactly one bit is lit at a time; at reset that is bit 0.

The architecture composes three modules you have already written, and adds two processes:

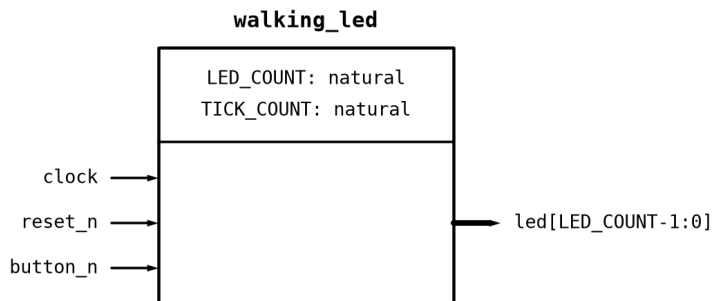
- Instantiate your **reset_sync**, and use its `reset_s2_n` as the reset for everything else in the design, the two instances below included.
- Instantiate your **button_sync** with generic `map(1)`, for a one-cycle pulse per press. Its button ports are vectors of `COUNT` bits while `button_n` here is a scalar, so bridge the width

with a one-element `std_logic_vector(0 downto 0)` in each direction. Section 7.5 shows it.

- Instantiate your **timer** with your own `TICK_COUNT`, drive its enable from the shift flag below and take its timeout as the shift tick.
- A process toggling **shift_enable** at every pulse from the button, cleared to '0' at reset. The walk therefore starts stopped.
- A process holding the **shift register** that drives led: at reset, load a single lit bit at position 0 and clear the rest; at every timeout from the timer, rotate one step towards the MSB and let the top bit wrap back into bit 0. That is a PISO register (section 6.4) with its own serial output wired back to its serial input, which is what turns a one-shot shift-out into an endless walk.

Then answer, in one sentence each:

- `shift_enable` drives the timer's enable, so stopping the walk stops the timer too. Exercise 7.4 had you show that enable **pauses** the count rather than clearing it. What does an observer at the board see, at the press that restarts the walk, if your timer clears instead?
- `button_sync` is instantiated with `COUNT = 1` here and with `COUNT = 2` in chapter 8's `fsm_led`, from the same file with no change. What would you have had to write instead if the width were fixed in the module rather than passed in as a generic?



Self-check. Name your entity `walking_led`, with the generics `LED_COUNT` and `TICK_COUNT` (both natural), the inputs `clock`, `reset_n`, `button_n` and the output `led`, declared in that order; its testbench is in `exercises/walking_led/`. It overrides both generics (`LED_COUNT = 4`, `TICK_COUNT = 3`) so that a full round takes a handful of clock cycles rather than two seconds, and it checks the reset pattern, that nothing moves before the first press, and that the number of shifts over a fixed window matches the rate the timer sets. Copy in the three modules it composes first; the command is in section 7.5.

Exercise 7.7 Changing rate and width

SIMULATION

Make the following changes one at a time, and rerun the testbench after each. Parts **a)** and **b)** change only the testbench's constants — that they require no change at all in your `walking_led.vhd` is the whole point of them. Part **c)** is the one that touches the module.

a) Change the walking speed:

- Change the constant `TICK_COUNT` in the **testbench** from 3 to 6.
- It should still pass, this time because the testbench adapts with you: its observation window is a fixed `PULSES * 6` cycles, and it derives the number of shifts it expects from `TICK_COUNT`, one shift per `TICK_COUNT + 1` cycles.
- Now find where that adaptation stops meaning anything. Keep raising `TICK_COUNT` past the length of the window and work out what the expected number becomes, and what the check then actually proves. A test that passes for the wrong reason is worth being able to recognise.
- Then explain why a changed tick count cannot break the *design*, only slow it down.

b) Change the width: change the testbench's constant `LED_COUNT` from 4 to 8. It should still pass, with no change at all in your `walking_led.vhd`. Explain what the module would have had to look like for this to have required a change.

c) Load two lit bits on opposite sides of the register at reset (e.g. "10001000" for `LED_COUNT = 8`), and let them walk in step:

- Point out the single line in your `walking_led.vhd` you need to change.
- **Predict** what the reference testbench will report, before you run anything.
- Then run it and see. Explain the result: is the design wrong, or is the testbench checking something that is no longer the specification?

Hint: Both `LED_COUNT` and `TICK_COUNT` are generics, so parts **a)** and **b)** require no change at all in the module's body. That is the whole point of a generic: one module, many sizes, no changes.

Exercise 7.8 A second button that reverses the direction

VHDL

The current design walks in one direction only (towards the MSB, where the top bit wraps back to the bottom). Change your `walking_led.vhd` so that a **second** button reverses the walking direction.

a) Add a second, synchronised button input: run it through the same `button_sync` instance, and pass 2 to its generic `COUNT` (generic `map(2)`, positionally, as everywhere else in this book).

b) Add a signal `direction`, and toggle it at the new button's edge, exactly as `shift_enable` is toggled by the first button.

c) Make the rotation in `SHIFT_PROCESS` depend on `direction`. When `direction = '0'`, rotate towards the MSB (the book's convention from section 6.2, and what you wrote in Exercise 7.6):

```
shift_reg <= shift_reg(LED_COUNT-2 downto 0) & shift_reg(LED_COUNT-1);
```

When `direction = '1'`, rotate the other way, towards bit 0:

```
shift_reg <= shift_reg(0) & shift_reg(LED_COUNT-1 downto 1);
```

This is the one place in the book where a shift register deliberately goes against the convention: reversing the direction is the whole point of the exercise. Note that the second expression is exactly the mirror image section 6.2 warns you to watch for in other people's code.

d) Your entity's ports have now changed: `button_n` is a two-bit vector rather than a single bit.

- Explain why `walking_led_tb.vhd` can no longer be run against your version, and exactly what GHDL will complain about when you try.
- This is the ordinary cost of changing an interface, and it is worth feeling once: a testbench is written against a particular entity, and widening a port breaks every instantiation of it, including the ones you did not write.
- Describe, in prose, the sequence of presses you would use to convince yourself that the design works, and what you would expect the LEDs to do at each step. That sequence is exactly what a testbench for the new entity would have to drive.

State machines

First designed by hand, then expressed in the language.

IN THIS CHAPTER

- Finite state machines: states, transitions, inputs and outputs.
- Designing a Moore machine by hand: state diagram, state table, Karnaugh-derived logic.
- State encoding as a design decision, and why the worked machine uses Gray code.
- The same machine in VHDL: an enumerated type and the case pattern for transitions.
- Why Moore is the default here and in the CAN project.
- fsm_led: buttons switching the LED between off, blinking and on.

This is the last chapter of the book's first part, and its building block is the last one missing. The payoff is in seeing the two halves against each other: every row you derive by hand in the state table in section 8.2 becomes a branch of a case in section 8.4, and seeing that correspondence is worth more than either step on its own.

Mealy machines (section 8.5) are read about rather than presented, and that matters more than it may look. Everything presented is Moore, and Moore is the default both here and in the CAN project. Section 8.5 is where you find out what the alternative costs and buys, which is the only way for that default to become a choice rather than a habit.

Refresher: The design procedure by hand, from state diagram via state table and Karnaugh maps to a gate network, and the difference between Moore and Mealy are in sections 6.4–6.7 of *Digital Electronics*.

8.1 What a finite state machine is

A finite state machine (FSM) is a circuit whose behaviour depends not only on its current inputs but on a finite amount of *history*, summarised in a single value called its **state**. At every instant the machine is in exactly one state; at every clock edge it may move on to a new one, determined by its current state and inputs; and its output is computed from that state.

None of this is new hardware. A state machine combines two things you already know: a **register** (chapter 3) holding the current state, one flip-flop per state bit, and **combinational logic** computing the next state and the output from the current state and the inputs.

What is new is the discipline. Instead of ad hoc logic feeding an ad hoc set of flip-flops, you work in a fixed order: first draw a **state diagram**, a few named states with labelled transitions between them, and derive the state table only once that is right, then the next-state logic, then the gate network or the VHDL.

Every synchronous circuit so far is technically a trivial state machine: a counter's state is its count, a shift register's is its bit pattern. What distinguishes a state machine is that the state is neither a count nor a shifted pattern but an arbitrary label (STATE_OFF, STATE_BLINK, ...) whose meaning you choose and whose next value may depend on the input however you like.

There are two varieties, and the difference decides what an output may look at:

- **Moore machines:** the output depends *only* on the current state. That is the kind designed by hand here.
- **Mealy machines:** the output depends on the current state *and* the current input. Section 8.5 covers them briefly, in VHDL only, so that you recognise one when you meet it and know the one-clock-cycle difference.

8.2 Designing a Moore machine by hand

The same process as for every gate network in chapter 1 and 2, applied to a circuit with memory. The worked example is a small, closed Moore machine with four states: a circular counter through a Gray code sequence, stepped one place per button press.

Specification

- Four states, each represented by two flip-flops $\{Q_1, Q_2\}$:

State	$\{Q_1, Q_2\}$
STATE_0	00
STATE_1	01
STATE_2	11
STATE_3	10

- A system clock `clock`, and an active-low reset `reset_n` returning the machine to STATE_0.
- An active-low push button `button_n`, from which a synchronised, edge-detected pulse `X` is derived in exactly the way chapter 4's two-flip-flop synchroniser does. `X` is high for exactly one clock cycle at a falling edge on `button_n`.
- An output `Y`, connected to an LED, high only in STATE_3.
- The machine is **closed**: the state after STATE_3 is STATE_0.

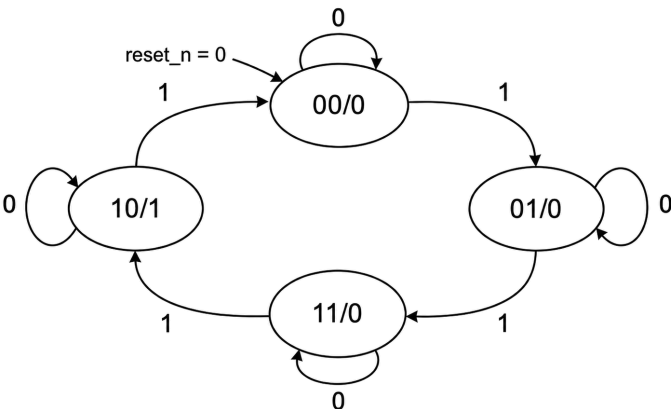
- It is a **Moore** machine: Y depends only on $\{Q_1, Q_2\}$, never on X .

The encoding is a Gray code ($00 \rightarrow 01 \rightarrow 11 \rightarrow 10 \rightarrow 00$), where consecutive states differ in exactly one bit. That is a deliberate choice, not an accident:

- With an ordinary binary count ($00 \rightarrow 01 \rightarrow 10 \rightarrow 11$) a transition such as $01 \rightarrow 10$ flips two bits at once, and the two flip-flops will not switch at *exactly* the same instant.
- That skew is harmless to the machine itself. Both flip-flops update at the same clock edge, and nothing samples the state register in between, so the machine never acts on the intermediate value. It is the same argument as section 5.4's critical path: as long as everything settles before the next edge, what happens in between does not matter.
- What the skew *can* disturb is anything reading the state combinationaly. An output decoded directly from the state bits, like Y below, can glitch briefly while they are skewed. The same goes for a value sampled by *another* clock domain, which is the case section 4.3 warned about.
- A Gray code removes both, since only one bit ever changes per transition. It therefore matters most for a counter crossing clock domains, and least for a machine whose output is registered.

Encoding is a design decision in general, and this is the only place in the book where you make it by hand. Binary uses the fewest flip-flops; Gray minimises the number of bits changing per transition; **one-hot** puts one flip-flop per state so that the next-state logic becomes a single wide OR gate per state, which is often fastest on an FPGA where flip-flops are plentiful. As soon as you write the machine in VHDL as an enumerated type (section 8.4), you leave the decision to the synthesis tool, which picks an encoding for you and re-encodes freely unless you say otherwise.

State diagram



State label format: Q1Q2/Y
Example: 00/0 means Q1Q2 = 00 (current state in binary) and Y = 0 (output).
Arrow labels 0 and 1 denote X, the input that changes the state.

Figure 8.1 State diagram for the four-state, Gray-coded Moore machine.

Every state bubble shows the current state and the output as Q1Q2/Y, and every arrow is labelled with the input X that triggers it. Every state returns to itself as long as X=0, and a synchronised pulse from the button (X=1) steps on to the next state, wrapping from STATE_3 back to STATE_0.

State table

Every combination of current state and input, with the next state $\{Q_1^+, Q_2^+\}$ and the output Y:

Q_1	Q_2	X	Q_1^+	Q_2^+	Y
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	1	0
0	1	1	1	1	0
1	0	0	1	0	1
1	0	1	0	0	1
1	1	0	1	1	0
1	1	1	1	0	0

Karnaugh-derived logic

Treating Q_1^+ and Q_2^+ separately as Boolean functions of $\{Q_1, Q_2, X\}$ and running each through a Karnaugh map, the same technique as in chapter 2:

$$Q_1^+ = Q_1X' + Q_2X \quad Q_2^+ = Q_2X' + Q_1'X$$

Note the shape of both:

- each is “hold when X is 0, take something from the other bit when X is 1”.
- the holding halves, Q_1X' and Q_2X' , really are mirror images of each other.
- the stepping halves are Q_2X and $Q_1'X$, and the complement on the second is not a typo: it is what makes the sequence a Gray code rather than a simple rotation.

The output Y simplifies further, straight from the state table, with no dependence on X at all, which confirms that this is a valid Moore machine:

$$Y = Q_1Q_2'$$

Realising it as a gate network

$\{Q_1, Q_2\}$ are one D flip-flop each, exactly like chapter 3’s register: Q_1^+ and Q_2^+ feed the D inputs and Q_1, Q_2 are read back from the Q outputs. Y is a single two-input AND gate with one input inverted, and it reads the current state. That is the whole machine: two flip-flops and a handful of gates computing Q_1^+, Q_2^+ and Y from $\{Q_1, Q_2, X\}$.

8.3 From gate network to CircuitVerse

Before any of this is written in VHDL, realise it by hand in CircuitVerse, exactly as for every gate network in chapter 1: two-flip-flop synchronise `button_n` and `reset_n`, edge-detect the synchronised button to produce X , wire Q_1^+ and Q_2^+ to a pair of D flip-flops, and wire Y straight into the LED.

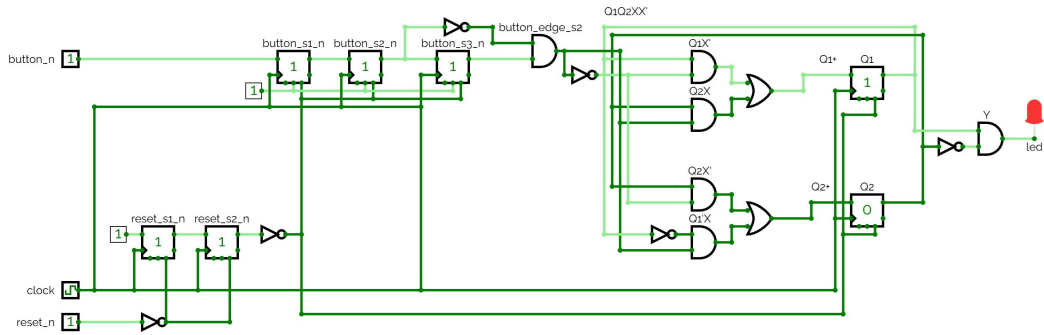


Figure 8.2 Gate network realising the four-state Moore machine, including two-flip-flop synchronisation and edge detection for button_n.

Read left to right:

- button_n passes through three synchronisation flip-flops (button_s1_n, button_s2_n, button_s3_n).
- An AND gate derives button_edge_s2 (this circuit's X) from the second and third stages.
- The four vertical wires labelled Q1Q2XX' carry the state bits and the edge pulse on to the gates on the right, which implement section 8.2's equations for Q_1^+ and Q_2^+ .
- The two labelled flip-flops store Q_1 and Q_2 and feed the final AND and NOT gates producing Y, connected to led.
- reset_n is synchronised by its own pair of flip-flops at the bottom left, exactly like every other synchronised reset in this book.

Open the runnable circuit yourself: `lectures/L08/appendix/images/fsm_gate_network.cv`. Step it by hand, one button press at a time, and confirm that it visits $\text{STATE}_0 \rightarrow \text{STATE}_1 \rightarrow \text{STATE}_2 \rightarrow \text{STATE}_3 \rightarrow \text{STATE}_0$ in order, with the LED lit only in STATE_3 .

8.4 The same machine in VHDL

That was the last time in this book you design a machine by hand. Everything from here writes it directly in VHDL, which is worth having earned rather than having started with: the synthesis tool does exactly what you just did on paper, and when a state machine misbehaves it is the diagram you debug, not the VHDL.

The translation replaces the explicit state table and the Karnaugh equations with an **enumerated type** naming every state, a **case statement** describing every state's transitions, and a synthesis tool left to work out the flip-flop encoding and the gate-level logic.

The enumerated state type

Take a three-state machine controlling an LED: STATE_OFF, STATE_BLINK (blinking at a fixed rate) and STATE_ON. VHDL declares exactly those three values as a new type:

```
type state_t is (STATE_OFF, STATE_BLINK, STATE_ON);
signal state: state_t;
...
state <= STATE_ON;
```

The case statement pattern

The machine is nearly always one synchronous process updating state at a rising edge via a case, one branch per state, with an asynchronous reset returning it to a known starting state. Assume to_next_state and to_prev_state are synchronised, edge-detected pulses produced in exactly the same way as X in section 8.2 and 8.3:

```
process(clock, reset_s2_n) is
begin
    if (reset_s2_n = '0') then
        -- On reset, go to STATE_OFF.
        state <= STATE_OFF;
    elsif (rising_edge(clock)) then
        case (state) is
            when STATE_OFF =>
                if (to_next_state = '1') then
                    state <= STATE_BLINK;
                elsif (to_prev_state = '1') then
                    state <= STATE_ON;
                end if;
            -- STATE_BLINK and STATE_ON follow the same shape.
            when others =>
                state <= STATE_OFF;
        end case;
    end if;
end process;
```

case (state) is ... end case; has to cover every value state_t can take, and when others keeps it exhaustive if you add a state later and forget a branch.

What when others is *not* is a safety net against a corrupted state register. Every value state_t can take is already named above it, so the branch is unreachable in the source and synthesis normally optimises it away rather than building recovery logic. Once Quartus has re-encoded the machine, most bit patterns are illegal, and getting out of one takes Quartus's Safe State Machine setting rather than anything you can write in VHDL.

Worked example: fsm_led

fsm_led.vhd implements exactly that three-state machine, extended to two buttons: button_n(0) steps forward, button_n(1) goes back, the states are closed in a loop, and STATE_BLINK toggles the LED every 100 ms.

Three subcomponents are reused, all modules you wrote yourself:

- reset_sync: chapter 4's reset synchroniser, unchanged (Exercise 4.8).
- button_sync: chapter 4's button synchroniser, also unchanged, but instantiated with generic map(2) so that it handles both buttons at once and produces a two-bit pulse vector. That is the payoff from that generic: the file is identical to the one chapter 4 and 7 instantiate with 1 (Exercise 4.9).
- timer: chapter 7's timer, unchanged (Exercise 7.4). fsm_led declares a generic TIMER_TICK_COUNT of its own and passes it straight on, with the default 5_000_000 for 100 ms at 50 MHz, and enables the timer only in STATE_BLINK.

None of the three is handed out in fsm_led/: copy your own in before you build the example. By this point in the book that is four chapters' own modules composed into a single design, which is what is worth noticing.

```
lectures/L08/fsm_led/fsm_led.vhd

-----
-- Moore FSM cycling an LED through OFF / BLINK / ON via two push buttons.
-- (Moore: the LED depends only on the state, never directly on the buttons;
-- contrast seq_detect_mealy. See Appendix A.5.)
--
-- Generics:
--   - TIMER_TICK_COUNT: The timer's tick count, so the interval between LED
--     toggles in STATE_BLINK (default = 5 000 000, i.e. about 100 ms at
--     50 MHz; override to shorten it in a testbench).
--
-- Inputs:
--   - clock: 50 MHz system clock.
--   - reset_n: Active-low asynchronous reset.
--   - button_n: Active-low buttons; (0) = next state, (1) = previous.
--
-- Outputs:
--   - led: The controlled LED.
-----

library ieee;
use ieee.std_logic_1164.all;

entity fsm_led is
    generic(TIMER_TICK_COUNT: natural := 5_000_000);
```

```

    port(clock, reset_n: in std_logic;
          button_n      : in std_logic_vector(1 downto 0);
          led            : out std_logic);
end entity;

architecture behaviour of fsm_led is
-- Enumeration of states.
type state_t is (STATE_OFF, STATE_BLINK, STATE_ON);

-- Synchronized input signals after two-flip-flop metastability protection.
signal reset_s2_n      : std_logic;
signal button_edge_s2: std_logic_vector(1 downto 0);

-- Timer signals.
signal timer_enable, timer_timeout: std_logic;

-- Internal LED state.
signal led_s: std_logic;

-- Current state.
signal state: state_t;

-- Signals indicating whether to change state.
signal to_prev_state, to_next_state: std_logic;
begin
-- Assign the internal LED state to the output port.
led <= led_s;

-- Enable the timer in STATE_BLINK only.
timer_enable <= '1' when STATE_BLINK = state else '0';

-- Indicate a change to the previous state on a falling edge of button_n(1).
-- Each direction also requires the other button's edge to be '0', so two
-- presses landing on the same clock cycle move the machine nowhere.
to_prev_state <= '1' when button_edge_s2(1) = '1' and button_edge_s2(0) = '0'
↳ else '0';

-- Indicate a change to the next state on a falling edge of button_n(0).
to_next_state <= '1' when button_edge_s2(0) = '1' and button_edge_s2(1) = '0'
↳ else '0';

-- Synchronize the asynchronous reset (Appendix A.4, reusing L04's reset_sync)↳
↳
reset_sync1: entity work.reset_sync

```

```

    port map(clock, reset_n, reset_s2_n);

-- Synchronize both buttons and pulse on each press (Appendix A.4, reusing
-- L04's button_sync at COUNT = 2: the same file, a wider vector).
button_sync1: entity work.button_sync
    generic map(2)
    port map(clock, reset_s2_n, button_n, button_edge_s2);

-- Instantiate the 100 ms timer (Appendix A.4, reusing L07's timer unchanged).
timer1: entity work.timer
    generic map(TIMER_TICK_COUNT)
    port map(clock, reset_s2_n, timer_enable, timer_timeout);

-- State-transition process:
--   - On reset: reset the current state to STATE_OFF.
--   - On rising clock edge: update the current state when a synchronized,
--     edge-detected button press is seen.
STATE_PROCESS: process(clock, reset_s2_n) is
begin
    if (reset_s2_n = '0') then
        state <= STATE_OFF;
    elsif (rising_edge(clock)) then
        case (state) is
            when STATE_OFF =>
                if (to_next_state = '1') then
                    state <= STATE_BLINK;
                elsif (to_prev_state = '1') then
                    state <= STATE_ON;
                end if;
            when STATE_BLINK =>
                if (to_next_state = '1') then
                    state <= STATE_ON;
                elsif (to_prev_state = '1') then
                    state <= STATE_OFF;
                end if;
            when STATE_ON =>
                if (to_next_state = '1') then
                    state <= STATE_OFF;
                elsif (to_prev_state = '1') then
                    state <= STATE_BLINK;
                end if;
            when others =>
                state <= STATE_OFF;
        end case;
    end if;
end process;

```



```

        end if;
    end process;

    -- LED (Moore output) process, depending only on the current state:
    --   - On reset: disable the LED.
    --   - On rising clock edge, based on the current state:
    --       - STATE_OFF : disable the LED.
    --       - STATE_BLINK: toggle the LED on every timer timeout.
    --       - STATE_ON  : enable the LED.
    LED_PROCESS: process(clock, reset_s2_n) is
    begin
        if (reset_s2_n = '0') then
            led_s <= '0';
        elsif (rising_edge(clock)) then
            case (state) is
                when STATE_OFF =>
                    led_s <= '0';
                when STATE_BLINK =>
                    if (timer_timeout = '1') then
                        led_s <= not led_s;
                    end if;
                when STATE_ON =>
                    led_s <= '1';
                when others =>
                    led_s <= '0';
            end case;
        end if;
    end process;
end architecture;

```

Two synchronous processes implement the machine:

- STATE_PROCESS is the case transition logic above, driven by `to_next_state` and `to_prev_state`, which are derived combinatorially from `button_edge_s2`. Each requires the *other* button's pulse to be '0' in the same cycle, so that pressing both close enough in time for their pulses to land on the same clock cycle moves the machine nowhere rather than arbitrarily picking a winner. An ambiguous input deserves a defined answer, and “do nothing” is the one that cannot surprise you.
- LED_PROCESS drives `led_s` from the current state alone: off in STATE_OFF, on in STATE_ON, toggling at every timeout from the timer in STATE_BLINK. Since it never reads `button_n` or `button_edge_s2` directly, only `state`, this is a genuine Moore machine: the LED responds to a press only indirectly, through the state the press caused.

8.5 Mealy machines: the one-clock-cycle difference

Everything so far has been a **Moore** machine, deliberately: Moore is the safer default and the form you design by hand. A **Mealy** machine relaxes the rule defining a Moore machine: the output may depend on the current state *and* the current input, computed combinatorially. It is not stored in a register, and it changes within the same clock cycle as the input does.

The behaviour: detecting two 1 bits in a row

A serial input `din` carries one bit per clock cycle, and `y` should indicate that the two most recent bits were both 1, with overlapping hits counted, so 111 reports a hit twice.

As a Mealy machine this needs only two states, since the output can react to the input *before* it shows up in the next state: `STATE_IDLE` (the previous bit was 0, or this is the first) and `STATE_ONE` (the previous bit was 1).

State	din	Next state	y
STATE_IDLE	0	STATE_IDLE	0
STATE_IDLE	1	STATE_ONE	0
STATE_ONE	0	STATE_IDLE	0
STATE_ONE	1	STATE_ONE	1

The state process is section 8.4's case pattern; only the output differs:

```
lectures/L08/seq_detect_mealy/seq_detect_mealy.vhd
-----
-- Mealy FSM detecting two consecutive '1's on serial input din (overlapping
-- matches allowed, e.g. "111" reports two). y is combinational in (state, din),
-- so it asserts the same cycle the second '1' arrives - one cycle earlier than
-- the Moore equivalent. See Appendix A.5.
--
-- Inputs:
--   - clock: 50 MHz system clock.
--   - reset_n: Active-low asynchronous reset, from a push button.
--   - din: Synchronous serial data (already clocked by clock, so it is NOT run
--         through a synchronizer - doing so would shift the sequence being detected).
--
--
-- Outputs:
--   - y: Asserted during the cycle the second '1' of a "11" is seen on din.
-----
library ieee;
use ieee.std_logic_1164.all;
```

```

entity seq_detect_mealy is
    port(clock, reset_n: in std_logic;
        din      : in std_logic;
        y        : out std_logic);
end entity;

architecture behaviour of seq_detect_mealy is
    -- Enumeration of states. Only two are needed: this is the state count the
    -- Mealy design saves relative to the three-state Moore equivalent.
    type state_t is (STATE_IDLE, STATE_ONE);

    -- Synchronized reset signal after two-flip-flop metastability protection.
    signal reset_s2_n: std_logic;

    -- Current state.
    signal state: state_t;
begin
    -- Instantiate reset_sync (Appendix A.5 / L04) to synchronize reset_n. This
    -- example has no push button of its own, so the button synchronizer is
    -- simply not instantiated.
    reset_sync1: entity work.reset_sync
        port map(clock, reset_n, reset_s2_n);

    -- State-transition process:
    --   - On reset: reset the current state to STATE_IDLE.
    --   - On rising clock edge: move to STATE_ONE whenever din = '1', back to
    --     STATE_IDLE whenever din = '0'. Note this process only ever reads din
    --     to decide the *next* state; it never assigns y.
    STATE_PROCESS: process(clock, reset_s2_n) is
    begin
        if (reset_s2_n = '0') then
            state <= STATE_IDLE;
        elsif (rising_edge(clock)) then
            case (state) is
                when STATE_IDLE =>
                    if (din = '1') then
                        state <= STATE_ONE;
                    else
                        state <= STATE_IDLE;
                    end if;
                when STATE_ONE =>
                    if (din = '1') then
                        state <= STATE_ONE;
                    else

```

```

        state <= STATE_IDLE;
    end if;
    when others =>
        state <= STATE_IDLE;
    end case;
end if;
end process;

-- Mealy output: a combinational (unregistered) function of the current
-- state AND the current input, assigned outside any clocked process.
-- Asserted the same cycle the second '1' arrives, one cycle earlier than
-- a Moore machine could report the same event.
y <= '1' when (state = STATE_ONE and din = '1') else '0';
end architecture;
```

The output assignment is a single concurrent statement, entirely outside any clocked process:

```
y <= '1' when (state = STATE_ONE and din = '1') else '0';
```

That is a **conditional signal assignment**, and it is the last bit of VHDL syntax this book introduces. It is a concurrent statement, so it belongs in the architecture body rather than inside a process, and it describes a multiplexer: take the first value whose condition holds, otherwise the value after else. It is the same hardware you would get from an if/else inside a combinational process, written in one line instead of five, and it is worth having because a one-line output assignment makes it obvious at a glance what the output depends on.

```

-- These two describe exactly the same multiplexer.
x <= a when sel = '1' else b;

process(sel, a, b) is
begin
    if (sel = '1') then
        x <= a;
    else
        x <= b;
    end if;
end process;
```

Always give it an else. Omit one and you have described a signal keeping its old value when no condition holds, which is a latch, exactly as in section 5.5. `fsm_led` uses this form three times, and Exercise 8.4 asks you to use it.

That single line is the whole difference. It reads `din`, so it is Mealy. Move the same condition into the clocked process, or take `din` out of it, and you have a Moore machine instead.

What it costs and what it buys

The Moore version of this detector needs a **third** state, `M_TWO`, existing purely to remember for one cycle that the hit occurred, so that the output has something to read that does not depend on the input. Feed both machines 1, 1, 0, 1, 1, 1:

Cycle	din	Mealy state (before)	Mealy y	Moore state (before)	Moore y
1	1	STATE_IDLE	0	M_IDLE	0
2	1	STATE_ONE	1	M_ONE	0
3	0	STATE_ONE	0	M_TWO	1
4	1	STATE_IDLE	0	M_IDLE	0
5	1	STATE_ONE	1	M_ONE	0
6	1	STATE_ONE	1	M_TWO	1

Moore's `y` column is exactly Mealy's `y` column shifted down one row. That is the whole trade-off: Mealy reports the hit in the *same* cycle the second 1 arrives where Moore takes one more, and Mealy needs 2 states where Moore needs 3.

Which to choose

Moore, nearly always, and that is what the rest of the book uses, the group project included:

- its output does not depend on any primary input, so it cannot follow a noisy one.
- a Mealy output can in principle glitch combinationally if its inputs are not already clean.
- note that “Moore” does not by itself mean glitch-free. An output decoded combinationally from several state bits, like section 8.2's $Y = Q_1 Q'_2$, can still glitch while those bits are skewed at a transition. What makes an output glitch-free is *registering* it, as `fsm_led`'s `LED_PROCESS` does, and that is what you want on anything driving a wire out of the circuit. A serial tx line decoded straight from the state bits will send out spurious edges.
- a machine whose output depends on fewer things is a machine with fewer ways to be wrong.

Reach for Mealy only when you specifically need the lower latency or the smaller state count, and be aware that you are then trading a synchronous output for a combinational one.

8.6 On the board

`fsm_led` is synthesised and run exactly as every VHDL design has been since chapter 1: create a Quartus Prime Lite project with the DE0-CV's 5CEBA4F23C7N as the target device,

add the .vhd files for the example and its subcomponents, compile, assign every port a physical pin in the Pin Planner, then recompile and program the board.

For fsm_led, assign clock the 50 MHz oscillator's pin, reset_n and button_n(1 downto 0) three push buttons, and led an LED on the board. Then confirm that repeated presses of "next" make the LED cycle off → blinking → on → off, that "previous" goes the other way, and that reset returns it to off.

seq_detect_mealy is deliberately **not** demonstrated on the board. It consumes one bit per clock cycle, and a switch flipped by hand holds its value for millions of cycles, so all an LED can show is the degenerate case: din held high, y high from the second cycle on. The one-clock-cycle difference the whole of section 8.5 is about happens far too fast to see. Run its testbench instead, which is the general lesson rather than an exception:

```
cd lectures/L08/seq_detect_mealy
cp ../../L04/exercises/reset_sync/reset_sync.vhd .      # the one you wrote in L04
ghdl -a --std=93 reset_sync.vhd seq_detect_mealy.vhd seq_detect_mealy_tb.vhd
ghdl -e --std=93 seq_detect_mealy_tb
ghdl -r --std=93 seq_detect_mealy_tb --assert-level=error --stop-time=10ms
```

Exact pin numbers depend on which buttons and LEDs on the DE0-CV you pick, assigned the same way as for every design since chapter 1. Look up the DE0-CV's own documentation for its physical pin configuration if you need to confirm which header pin corresponds to which labelled button, switch or LED.

8.7 Summary

What you should be able to do now

- Explain what a finite state machine is, and point out its states, transitions, inputs and outputs in a given circuit.
- Design a Moore machine by hand, from specification through state diagram and state table to Karnaugh-derived equations, and build the result in CircuitVerse.
- Model the same machine in VHDL with an enumerated type state_t and a case process.
- Recognise a Mealy machine on sight and say what its output costs and buys, one clock cycle of latency against one extra state, and why Moore is the default here and in the CAN project.
- Compose a state machine with previously built subcomponents instead of rewriting them. In the capstone that means three earlier chapters' modules around a machine that is the only new logic, with shift and counter idioms from one more.

Questions to test yourself with

1. What makes a circuit a state machine, rather than just a register with some logic around it?
2. What exactly is `X` in the hand-designed machine, and why does it have to be a one-clock-cycle pulse rather than the button's level?
3. Why is that machine's state encoding a Gray code? What can go wrong with an ordinary binary count?
4. A counter and a shift register are technically state machines too. What separates them from the machine designed here?
5. Which signals may a Moore output depend on, and which may a Mealy output depend on? Given a single line of VHDL driving an output, how do you decide which of them you are looking at?
6. Why does the Moore version in section 8.5's comparison need an extra state, and why is its output exactly one clock cycle behind the Mealy version's?
7. Why does `LED_PROCESS` in `fsm_led.vhd` read only `state`, never `button_edge_s2` or `button_n` directly? What would change if it did?
8. Why does `din` in `seq_detect_mealy` not go through a synchroniser, unlike `reset_n`?
9. What does `when others` buy you in a case for state transitions, when `state_t` only has the values you declared?
10. Why does the timer in the capstone receiver run sixteen times as fast as the baud rate rather than once per bit, and why does sampling mid-bit make the design tolerate a transmitter whose clock differs slightly from yours?
11. The capstone transmitter uses a full-bit timer and needs nothing mid-bit. What does it have that the receiver does not, which makes the difference?

Looking ahead

This is the last chapter of the book's first part. Over eight chapters we have gone from gates and Karnaugh maps, by way of registers, metastability and the VHDL language itself, to counters, shift registers and timers, and now finite state machines, at every step reasoning about the circuit by hand before writing it in synthesisable VHDL and running it on a real FPGA. Every pattern built along the way, the two-flip-flop synchroniser, the process and case coding style, and now the Moore machine, has been reused in this chapter's own examples, and carries over unchanged into what comes next.

Next is chapter 9, the first practical exam: three hours, individual, at the computer with GHDL, over everything the first eight chapters have built. After it the group project begins, where a CAN controller is designed in VHDL over ten chapters and finishes against a real microcontroller over SPI.

8.8 Exercises

How to check your work. Exercise 8.1 and 8.2 are pen and paper: they are about getting a state machine right *before* it becomes code, and nothing in them needs VHDL, GHDL or a board.

From Exercise 8.4 onwards, every exercise that asks you to write a VHDL module has a self-checking testbench handed out under `lectures/L08/exercises/`, and nothing else; Exercise 8.3 changes a module you already have and is checked by hand. Assembling the rest of the directory is part of the work: the modules you wrote in earlier chapters (`reset_sync`, `button_sync`, `sync`, `timer`) are copied in from wherever you wrote them. Every exercise writes out the exact commands it needs.

Exercise 8.6 and 8.7 are the book's capstones: the two ends of one and the same serial link, a receiver and a transmitter. Each composes modules from four different chapters. Set aside real time for them, and design each state diagram on paper before you write any VHDL. Do the receiver first: it is the harder of the two, since it has to recover the bit timing it is given rather than defining it itself.

EXERCISE SET 1

State diagrams and state tables

Exercise 8.1 A machine with binary state ordering

CIRCUIT

Design a new four-state Moore machine, $\{Q_1, Q_2\}$, controlling an LED via a single synchronised, edge-detected pulse X from a button (produced exactly like X in the worked example in section 8.2).

It differs from the worked example in two ways:

- The state ordering is the ordinary binary count $00 \rightarrow 01 \rightarrow 10 \rightarrow 11 \rightarrow 00$, **not** the Gray code used in section 8.2.
- The LED (Y) is lit in **two** states rather than one: whenever $Q_1 = 1$ (that is, in both 10 and 11).

a) Draw the state diagram: four states, one transition arrow per state for $X = 1$, and one self-loop per state for $X = 0$.

b) Fill in the complete state table (all eight rows of $\{Q_1, Q_2, X\}$).

c) Derive Q_1^+ , Q_2^+ and Y with Karnaugh maps, the same way section 8.2 derives them for the Gray-coded version.

d) Realise the gate network by hand: two D flip-flops for $\{Q_1, Q_2\}$, gates computing Q_1^+ , Q_2^+ and Y , and build and simulate it in CircuitVerse, with two-flip-flop synchronisation and edge detection of the button input exactly as in section 8.3.

e) Verify it in CircuitVerse, against your own state table. There is no testbench for this machine, and there is not meant to be: the state table you derived in b) *is* the specification, and checking a circuit against a specification you worked out yourself is the whole point of the exercise.

Step the clock by hand and check, in this order:

- **The stepping path.** Pulse X once per step and go a full round, $00 \rightarrow 01 \rightarrow 10 \rightarrow 11 \rightarrow 00$. Read the expected next state off your table before every edge. Read $\{Q_1, Q_2\}$ off the flip-flops' outputs after the edge. All four transitions must match.
- **The holding path.** Step the clock with $X = 0$ in each of the four states and confirm that the machine stays put. That is four more checks, and it is the half people skip; a machine that steps correctly but does not hold is a machine whose X' terms are wrong.
- **The output.** Check Y against your table in every state: high in 10 and 11, low in 00 and 01. Do it as a pass of its own, not while you are looking at the state bits.

Where they differ tells you which gate to look at. A wrong Q_1^+ shows up only on the transitions where Q_1 should have changed. A wrong Y shows up with the state sequence still entirely correct, which is why it is worth a pass of its own.

Hint: Since the state ordering changed from Gray code to binary, expect one transition ($01 \rightarrow 10$) where *two* state bits flip at once. Per section 8.2 that is harmless to the machine itself, since nothing samples the state register between edges, and CircuitVerse's ideal simulation shows you nothing either way. What it does affect is Y , which here is decoded

combinationally from the state bits: on real hardware Y can glitch briefly while the two bits are skewed. Note which transition it is, and what you would do about it if Y drove something that cared.

EXERCISE SET 2

Sequence detection

Exercise 8.2 A Moore detector for 101

REASONING

Design a **Moore** machine detecting the sequence 1, 0, 1 as it arrives one bit per clock cycle on an input d_{in} , raising its output y as soon as the hit's third bit has been seen. Non-overlapping will do here: after a complete hit, start looking for a new 1, 0, 1 from the beginning rather than reusing the closing bits.

- Draw the state diagram: one state per how much of the pattern has matched so far, one arrow out of every state for $d_{in} = 0$ and for $d_{in} = 1$ (a sequence detector is never idle, it either steps forward or falls back), and mark which state drives $y = 1$.
- Fill in the state table, one row per combination of (state, d_{in}).
- Trace it. Pick a six-bit sequence for d_{in} containing at least one hit, and note, cycle by cycle: the state before the edge, the value of d_{in} , and the output y .
- The trap in this machine is the arrows back, not the ones forward. For every state, say what happens on the input that does *not* carry the hit forward: which of them return to the start, and is there any input that should send you back to a *partially* matched state instead of all the way to the beginning? Justify your answer for this pattern.

Hint: Count your states before you draw them. “Nothing matched yet”, “seen 1”, “seen 10”, “seen 101” is the natural split, and the last of them is what drives y .

Keep your answer. Exercise 8.4 implements the same detector in VHDL, as a Mealy machine — which, per section 8.5, needs one state fewer than the one you just drew. Working out which of your states disappears, and why, is part of that exercise.

EXERCISE SET 3

Finite state machines in VHDL

Exercise 8.3 A fourth state in fsm_led

VHDL

Extend `fsm_led` with a fourth state, `STATE_FAST_BLINK`, blinking at 20 ms instead of `STATE_BLINK`'s 100 ms. Follow the worked example in section 8.4.

Work on a **copy** of the module, not on `lectures/L08/fsm_led/fsm_led.vhd` itself. First copy the whole directory somewhere of your own, together with the three modules of yours that it instantiates:

```
cp -r lectures/L08/fsm_led /tmp/fsm_led_fast
cp lectures/L04/exercises/reset_sync/reset_sync.vhd /tmp/fsm_led_fast # the ↵
↵ three you wrote
cp lectures/L04/exercises/button_sync/button_sync.vhd /tmp/fsm_led_fast # ↵
↵ yourself
cp lectures/L07/exercises/timer/timer.vhd /tmp/fsm_led_fast
cd /tmp/fsm_led_fast
```

There is no testbench to run here, but the four-state machine should still analyse, and that is worth doing after every change: it catches a missing case branch, or a `state_t` value you added in one process but not the other.

```
ghdl -a --std=93 reset_sync.vhd button_sync.vhd timer.vhd fsm_led.vhd
```

The reason for working on a copy is worth knowing, since it is a real pitfall and not busywork for its own sake. `fsm_led` has a reference testbench handed out with it, `fsm_led_tb.vhd`, and Exercise 8.5 asks you to run it. That testbench checks the **three**-state cycle: it presses the button once from `STATE_BLINK` and checks that the LED is steadily on, since the next state in the machine as written is `STATE_ON`. In your four-state version that press lands in `STATE_FAST_BLINK`, the LED is still blinking, and the assertion fails. Nothing in the failure says which of the two exercises it belongs to. Keep the reference machine intact and that question never arises.

There is no testbench for the four-state machine. Check it the way section 8.2 had you check a state diagram: predict the state sequence for a series of button presses, then read your case branches back and confirm that they agree.

You need to:

- Add the new value to `state_t`.
- Add a branch in every case statement, in both `STATE_PROCESS` and `LED_PROCESS`.
- Handle the two blink rates, either by adding a second timer instance, or by having the existing timer's tick count switch on the current state.

Exercise 8.4 seq_detect_101_mealy

VHDL

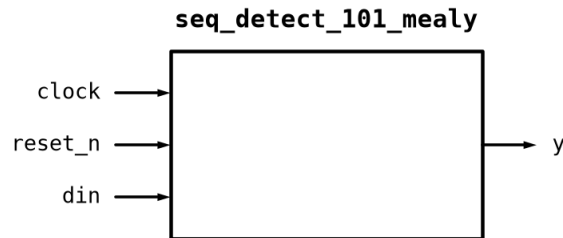
Implement the 1, 0, 1 detector from Exercise 8.2 in VHDL, in a new directory `seq_detect_101_mealy` — this time as a **Mealy** machine.

Start from your state diagram for the Moore machine and convert it:

- Your last state, “seen 101”, exists only to hold the output high for one cycle.
- A Mealy machine does not need it: the output condition can read `din` directly in the state that has already matched 1, 0.
- Remove that state, and move its output into the condition for `y`.

Then follow the type `state_t is (...)` and case pattern from section 8.4 and 8.5:

- Reuse `reset_sync` for `reset_n`, exactly as `seq_detect_mealy.vhd` does. There is no button in this design, so `button_sync` is simply not instantiated.
- Make sure the assignment of the output `y` is a single concurrent statement, outside any clocked process, depending on both state and `din`, not just state. Otherwise you have silently built a Moore machine instead, and its output comes one cycle too late — which is exactly what the testbench checks.



Self-check. Name your entity `seq_detect_101_mealy`, with the ports `clock`, `reset_n`, `din` (in) and `y` (out), declared in that order to match `seq_detect_mealy`. A testbench is in `exercises/seq_detect_101_mealy/`; it checks a non-overlapping 101 detector. Your module instantiates `reset_sync`, which is not handed out there since you wrote it in Exercise 4.8: copy your own in.

```
cd lectures/L08/exercises/seq_detect_101_mealy
cp ../../L04/exercises/reset_sync/reset_sync.vhd .      # the one you ↵
↳ wrote in L04
ghdl -a --std=93 reset_sync.vhd seq_detect_101_mealy.vhd ↵
↳ seq_detect_101_mealy_tb.vhd
ghdl -e --std=93 seq_detect_101_mealy_tb
ghdl -r --std=93 seq_detect_101_mealy_tb --assert-level=error --stop-time=10↵
↳ ms
```

EXERCISE SET 4

Reading the worked machine

Exercise 8.5 Blink rate, testbench and state sequence

SIMULATION

fsm_led is demonstrated on the DE0-CV (section 8.6). This exercise checks that you can account for what you saw, and it comes with a reference testbench you can run yourself. It reuses your two synchronisers from chapter 4 and your timer from chapter 7, and none of them is handed out, so copy them in first:

```
cd lectures/L08/fsm_led
cp ../../L04/exercises/reset_sync/reset_sync.vhd .      # the three you wrote ↵
↵ yourself
cp ../../L04/exercises/button_sync/button_sync.vhd .
cp ../../L07/exercises/timer/timer.vhd .
ghdl -a --std=93 reset_sync.vhd button_sync.vhd timer.vhd \
        fsm_led.vhd fsm_led_tb.vhd
ghdl -e --std=93 fsm_led_tb
ghdl -r --std=93 fsm_led_tb --assert-level=error --stop-time=10ms
```

- a) Work out the blink rate from the source, not from the demonstration. fsm_led instantiates timer with TIMER_TICK_COUNT defaulting to 5_000_000, on a 50 MHz clock. How long is one timeout period? STATE_BLINK toggles the LED at every timeout. What **blink** frequency does that give, and why is it half of what you might first write down?
- b) The testbench overrides TIMER_TICK_COUNT with a much smaller value than the default. Find that override in fsm_led_tb.vhd. Explain why a testbench cannot reasonably use the real value, and roughly how long a simulation of one real blink period would have to run.
- c) Account for the state machine's behaviour:
 - All three states are reachable in both directions. Trace the presses of button_n(0) and button_n(1) that visit every state and return to STATE_OFF.
 - Press both buttons during the same clock cycle. What does fsm_led do, and which two lines in the architecture decide that?
 - timer_enable is driven from state, so the timer runs only in STATE_BLINK. Note that timer is **frozen** rather than cleared when it is disabled (section 7.3). What does that mean for the moment you return to STATE_BLINK after leaving it, and would it be better or worse to hold timer_enable permanently high?

EXERCISE SET 5

Capstone: a serial link

Exercise 8.6 `uart_rx8`: the receiver

VHDL

This is the first half of the book's capstone, and it composes four chapters into one design. Exercise 8.7 builds the other end of the same serial link, so do not stop here. Nothing in it is a new idea; it is all about wiring together modules you already have, around a state machine you design yourself:

- chapter 4's `reset_sync`, synchronising reset.
- chapter 5's `sync`, synchronising the rx line, which comes from somebody else's crystal and is asynchronous in exactly the sense chapter 4 means.
- chapter 7's timer, generating the oversampling tick.
- This chapter's state machine, framing all of it and doing its own shifting and bit counting. You built that as a module in chapter 6; here it is four lines inside a machine that already has the counters it needs, which is the more common way a receiver is written.

Write an entity named `uart_rx8`: a receiver for one byte of asynchronous serial data, in the format a UART sends.

The line's format

The rx line carries frames, and between frames it idles:

- **Idle:** the line is held high.
- **Start bit:** the line goes low for exactly one bit period. That is what marks the beginning of a frame; there is no separate clock line, which is what "asynchronous serial" means.
- **Eight data bits**, most significant first, one bit period each.
- **Stop bit:** the line goes back high for one bit period.

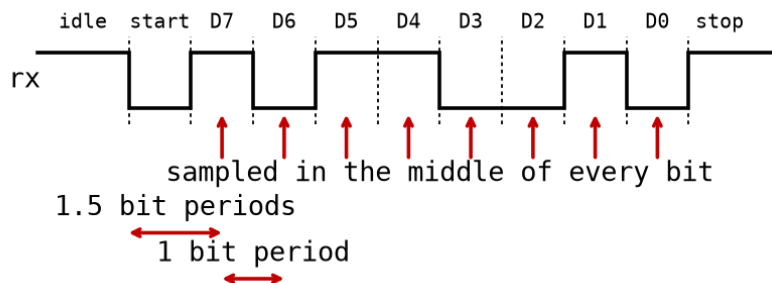


Figure 8.3 A UART frame: a line idling high, a start bit, eight data bits with the most significant first, and a stop bit, with the receiver sampling mid-bit.

The frame above carries 10110010, which is the first byte the testbench sends. The arrows are the eight moments where the receiver picks up a **data bit**: one and a half bit periods after the falling edge, and then once per bit period thereafter. It looks at the line in between as well — after the falling edge, mid start bit to check that it is still low, and mid stop bit — but only the eight arrows contribute bits to the received byte.

The receiver has to recover the bit timing from the start bit's falling edge alone. That is the whole problem this exercise solves, and it is why the timer is here.

One deliberate departure from a real UART. A real UART sends its data bits **least** significant first, as section 6.3 points out. This exercise sends them most significant first:

- the framing and the timing, which are what these two exercises are actually about, are identical either way.
- it is the direction section 6.3's SIPO idiom shifts in, so the receiver's shift is one line, and the transmitter in Exercise 8.7 is its mirror image.
- the testbenches handed out send and expect MSB first to match, so the pair is consistent with itself.
- to talk to a real UART you would reverse the shift direction to section 6.2's mirrored form and change nothing else. Worth knowing before you connect this to a computer's serial port and wonder why every byte arrives bit-reversed.

Oversampling: sixteen ticks per bit

The receiver has to find the middle of a bit whose beginning it can only infer. The technique is called **oversampling**: run timer at **sixteen times** the baud rate, so that there are sixteen ticks per bit period, and count them.

- The start bit's falling edge clears the tick count to 0. The timer runs continuously, before, during and after a frame; it is the *count* that is cleared, never the timer.
- **Tick 8** is half a bit later, in the middle of the start bit. Check that the line is *still low* there. If it has gone high it was a glitch and no start bit, and the receiver returns to idle. That recheck is what keeps noise on an idle line from being read as data.
- From there, every **sixteenth** tick is the middle of the next bit: sample the line and shift it in. Eight of them are the data bits, and the ninth is the stop bit.

Sampling mid-bit instead of at the edges is what gives the receiver its timing margin. The transmitter's clock is a different physical crystal from yours, so the two drift apart over a frame; a receiver sampling near the boundaries would read the wrong bit as soon as they did. Sixteen ticks is the conventional choice, and the transmitter in Exercise 8.7 runs on the same tick.

The entity

Generic	Type	De- fault	Description
OVERSAMPLE_TICK_COUNT	natural	325	The tick count for timer for one sixteenth of a bit period. 9600 baud at 50 MHz; part b) asks you to derive it.

Port	Dir.	Type	Description
clock	in	std_logic	System clock.
reset_n	in	std_logic	Asynchronous, active low; goes through reset_sync, as always.
rx	in	std_logic	The serial line, straight from a pin and therefore asynchronous . Synchronise it to rx_s2 before anything else reads it.
data_out	out	std_logic_vector(7 downto 0)	The received byte, valid during the cycle where byte_valid is high.
byte_valid	out	std_logic	A one-cycle pulse: a complete, correctly framed byte is on data_out.
frame_err	out	std_logic	A one-cycle pulse: the stop bit was not high, so the frame is broken and no byte is handed over.

a) Design the state machine **on paper first**, exactly as section 8.2 had you do. Four states:

- STATE_IDLE, waiting for the line to go low.
- STATE_START, checking at tick 8 that the line is still low.
- STATE_DATA, sampling one bit every sixteenth tick, eight times.
- STATE_STOP, sampling the stop bit sixteen ticks after the last data bit and choosing between byte_valid and frame_err.

Draw it before you write any VHDL, and on the diagram:

- Give every state its **self-loop**: the arrow for “the tick I am waiting for has not come yet, stay put”. That is the arrow that usually goes missing, and a state without it falls through to wherever else happens to point.
- Write beside every state **how many ticks it waits for**, so that the tick bookkeeping is on the paper instead of in your head.
- Mark where ticks is **cleared**. The timer itself never stops, so those clears are the only places where the receiver decides what “the beginning of a bit” means.
- Label the arrow out of STATE_IDLE with an **edge**, not a level: rx_s2 low *and* high at the previous tick. That takes one bit of history, so note down where you intend to keep it.

b) Work out the timing before you implement it:

- The DE0-CV's clock is 50 MHz and 9600 baud means 9600 bits per second. How many clock cycles is one bit period? How many is one sixteenth of one?
- timer pulses every `TICK_COUNT + 1` clock cycles (section 7.1). Which `OVERSAMPLE_TICK_COUNT` does that give you? The exact answer is not an integer, so there are two candidates either side of it: work out the error each of them leaves, and say which of them the default above is and why.
- That error is a fraction of a percent, and it accumulates over a frame. How far has your sampling point drifted from the middle of the stop bit ten bit periods after the start edge, measured in oversampling ticks? How many ticks of margin did it have to begin with?
- And now the reason the answer is comfortable: you resynchronise at every start bit. What would have to be true of the frame length for that to stop being enough?

c) Implement `uart_rx8`. It instantiates three modules and does the rest itself:

- `reset_sync` for reset. Everything else in the design takes its `reset_s2_n`.
- **sync**, the module you wrote in Exercise 5.4, on the rx pin, giving the `rx_s2` the state machine reads. Instantiate it with `SIZE = 1` and `PRESET = '1'`: an idle line is high, and a synchroniser coming out of reset reading '0' would look to this machine like a start bit. rx comes from a pin driven by a transmitter with its own crystal, so it is asynchronous in exactly the sense chapter 4 means, and every rule from there applies unchanged. Note that it takes a one-element `std_logic_vector` on each side, so you declare a pair of those and pick `rx_s2` out of the output.
- timer, with `OVERSAMPLE_TICK_COUNT` passed straight through, enable tied to '1', and its timeout as your only notion of elapsed time. Call it `baud_tick`. The timer free-runs: it is never stopped, never cleared, and no state ever touches it. What the machine rephases instead is its own counter ticks.
- No shift register module. The machine needs a bit counter anyway, so it shifts the line straight into a signal frame itself, one line of VHDL per sampled bit.

The state machine, state by state

Two counters: ticks counting 0 to 15 within a bit, and `bit_count` counting the eight data bits. Everything below happens **only when `baud_tick = '1'`**; at every other clock edge the machine stands still. Since the timer free-runs, a tick is always on its way, in every state including idle.

State	Waits for	Then	To
STATE_IDLE	falling edge on <code>rx_s2</code>	clear ticks	STATE_START
STATE_START	ticks = 8	<code>rx_s2</code> still low? clear ticks <code>rx_s2</code> high? it was a glitch	STATE_DATA STATE_IDLE
STATE_DATA	ticks = 15	shift <code>rx_s2</code> into frame, clear ticks	STATE_DATA until 8 bits, then STATE_STOP
STATE_STOP	ticks = 15	<code>rx_s2</code> high? <code>data_out</code> $\leq$ frame, pulse <code>byte_valid</code> <code>rx_s2</code> low? pulse frame_err, discard the byte	STATE_IDLE STATE_IDLE

There is no “done” state. The stop bit’s sample is where the frame is judged, so that is where the byte is handed over, and the machine goes straight back to idle.

Why the idle state waits for an edge instead of a level

Keep one tick of history for `rx_s2`, and leave the idle state only at the transition from high to low. Testing the level instead, “if `rx_s2` is low, start a frame”, looks equivalent and handles everything except the one case that matters.

Consider a **break**: the line held low for far longer than a frame, which is what a transmitter losing power looks like. A level test rearms the moment the previous frame’s stop bit is judged, so the receiver spends the whole break marching through frame after frame of all zeros. Each ends on a low stop bit and is correctly rejected, and that part is fine. The problem is the frame in the air when the break *ends*: its stop bit is sampled after the line has gone back high, so it is a perfectly well-formed frame as far as the receiver can tell, and a byte made of nothing is handed over as real data. The edge test does not rearm at all during the break, since after the first falling edge there is no second one until the line has recovered.

That is the whole difference, it costs one flip-flop, and it is why a receiver that “works” on clean data can still hand over rubbish the first time a cable is pulled out.

The shift

Eight data bits arrive most significant first, so the SIPO idiom from section 6.3 handles it with no index arithmetic at all:

```
frame <= frame(6 downto 0) & rx_s2;
```

`bit_count` then only counts *how many* bits have arrived, never *where* they land. Storing with `frame(bit_count) <= rx_s2` instead puts the first-arrived bit in bit 0, which reverses the byte: that is the least-significant-first order a real UART uses, and it would be the right choice if this receiver had to talk to one. Here the MSB comes first on the line, so the shift is the one that matches.

Four things decide whether this works:

- **Everything is conditioned on `baud_tick`.** A state acting on an ordinary clock edge runs sixteen times per tick and about five thousand times per bit.
- **Every state needs its “not yet” path.** If the tick you are waiting for has not arrived, or the count has not reached its target, the machine must **stay where it is**. Sending it somewhere else in the `else` is by far the most common way this design fails, and it fails completely rather than subtly: the machine ends up bouncing back and forth between two states and never reaches the third.
- **Watch the off-by-one in the counter.** With ticks starting at 0 and incrementing at every tick, the n th tick is `ticks = n - 1` at the moment you test it. Whether you compare before or after the increment decides which tick you actually sample on, and being one or two ticks late is survivable while eight is not.
- **The timer free-runs, so your phase comes from ticks, never from the timer.** The start edge arrives when the transmitter sends it, which is somewhere inside a tick period the receiver did not choose, so the first tick after that edge can come anywhere from a whole tick to almost no time later. Clearing ticks on the way out of the idle state is what rephases the sampling against the start bit. The leftover fraction of a tick is the price of never stopping the timer, and it is cheap: one sixteenth of a bit, against the half bit of margin part b) asks you to work out.

d) Run the testbench. It needs four design files, the subcomponents first, and none of the three subcomponents is handed out in the exercise directory: all three are modules you wrote.

```
cd lectures/L08/exercises/uart_rx8
cp ../../L04/exercises/reset_sync/reset_sync.vhd .      # the three you wrote ←
  ← yourself
cp ../../L05/exercises/sync/sync.vhd .
cp ../../L07/exercises/timer/timer.vhd .
ghdl -a --std=93 sync.vhd reset_sync.vhd timer.vhd uart_rx8.vhd uart_rx8_tb.vhd
ghdl -e --std=93 uart_rx8_tb
ghdl -r --std=93 uart_rx8_tb --assert-level=error --stop-time=10ms
```

It plays transmitter, and checks:

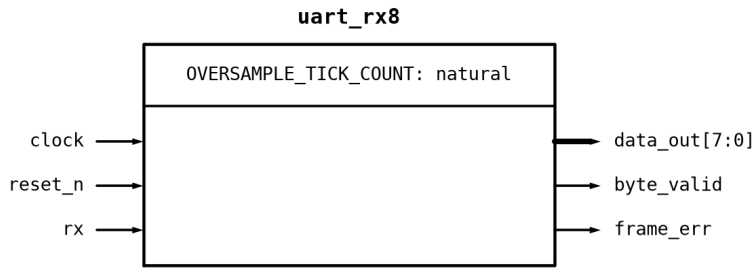
- two clean frames, arriving as the right byte, with no `frame_err`.

- an idle line, before, between and after them, producing nothing.
- a frame whose stop bit is **low**: `frame_err` must pulse and no byte may be handed over.
- the line held low for far longer than a frame, which is a **break**: every frame the receiver thinks it sees there ends on a low stop bit, so none of them is a byte.
- a frame where every data bit carries its value only in the **middle** of the bit period and the opposite level either side. A receiver sampling near tick 8 reads it perfectly; one sampling near a boundary reads the neighbouring bit.
- that `byte_valid` and `frame_err` are each never high for two cycles in a row.

Hint: Build it in stages and run the testbench after each, rather than writing all four states and debugging them at once. First get the machine to leave the idle state on a start bit and reach `STATE_DATA`; if it never gets there it is the “not yet” path to look at. Then sample one data bit and check that it is the right bit before you worry about the other seven. Then the stop bit and the two pulses.

e) Now break it deliberately, and think about what the result proves and does not prove:

- Remove the stop bit state, and hand the byte over as soon as the eighth data bit is in. Predict what should go wrong, then run it. Two checks catch this, and it is worth knowing which fires first: the frame with the low stop bit no longer raises `frame_err`, and it hands over a byte that should have been discarded.
- Now the other half of the lesson. Passing is the weaker signal, so find a sabotage the testbench does **not** catch. Here is one: **remove the mid-start-bit recheck**, so that `STATE_START` moves on to `STATE_DATA` at tick 8 without confirming that the line is still low. The testbench never puts a glitch on an idle line, so nothing notices, and on real hardware a single spike of noise starts a phantom frame. Read `uart_rx8_tb.vhd` and say what you would add to catch it.
- Here is a second: **sample at tick 3 instead of tick 8**. Work out why the testbench still passes, and what you have given away. The answer is in the margin calculation in part b).
- And a third, which is the most important of the three: **remove the synchroniser for rx** and read the pin directly. Every check still passes, and it always will, whatever is added to the testbench. Say why, in terms of what a simulator models and what it does not, and then reread section 4.4. This is the one kind of bug in the book that testing cannot reach, which is exactly why chapter 4 gives you a rule to follow rather than a symptom to look for.
- In a few sentences: what does “passes its testbench” actually promise, and what does it not?



Self-check. Name your entity `uart_rx8`, with the generic `OVERSAMPLE_TICK_COUNT` (`natural`), the inputs `clock`, `reset_n`, `rx`, and the outputs `data_out` (`std_logic_vector(7 downto 0)`), `byte_valid` and `frame_err`, declared in that order. Its testbench is in `exercises/uart_rx8/`, and nothing else: **copy in your own reset_sync, sync and timer**, as part d) shows. A project directory contains everything it is built from, and assembling it is part of the exercise.

Exercise 8.7 `uart_tx8`: the transmitter

VHDL

The other half of the link: write `uart_tx8`, a transmitter sending a byte in the same format Exercise 8.6 receives.

A transmitter is the easier of the two, and the reason is worth understanding before you start:

- The receiver has to *recover* the bit timing from a start edge it did not control, which is why it sampled mid-bit and why its timer ran at one sixteenth of a bit period.
- The transmitter **defines** the timing. Nothing has to be recovered, so the timer runs at a whole bit period and the machine simply holds every bit on the line for one timeout.

It sends the most significant bit first, which matches the receiver in Exercise 8.6 rather than a real UART, for the reason given there.

The entity

Generic	Type	De- fault	Description
BIT_TICK_COUNT	natural	5207	The tick count for timer for a whole bit period, 9600 baud at 50 MHz. Sixteen of Exercise 8.6's oversampling ticks, give or take the rounding its part b) asks you to work out, so that a uart_tx8 and a uart_rx8 built with the matching defaults talk at the same rate.

Port	Dir.	Type	Description
clock	in	std_logic	System clock.
reset_n	in	std_logic	Asynchronous, active low; through reset_sync, as always.
data_in	in	std_logic_vector(7 downto 0)	The byte to send.
send	in	std_logic	A request to send data_in. Ignored while the transmitter is busy.
tx	out	std_logic	The serial line. Idles high , including during reset.
busy	out	std_logic	High from the moment a frame starts until the stop bit is over.

a) Draw the state machine first. Four states are the natural split: idle, start bit, data bits, stop bit, and each holds the line at a defined level for one bit period.

- Mark what drives tx in every state. Note that tx never depends on send or any other primary input: it is the state, plus the shift register's current output. That makes this a Moore machine, like everything else in this book.
- Mark where busy comes from. It is a single comparison against the state.
- Decide where the byte is captured. It has to be latched when the frame starts, not read continuously out of data_in: the producer is free to change data_in the moment busy goes high, and a real one will.

b) Shift the byte out most significant bit first, in the machine itself, the mirror image of what the receiver does: latch data_in into a signal frame when the frame starts, drive tx from the top bit of frame while in the data state, and shift frame one place up per bit period, counting the eight bits going out.

You wrote this as a module in Exercise 6.7, and piso8 is worth rereading for the idiom's sake. Instantiating it here would work, but the machine already owns the bit counter a PISO register needs, so the shift becomes a single line inside the state where it belongs. That is how both halves of this link are built, and the same trade-off comes back in chapter 14, where tx_shift_reg is a module of its own precisely because can_controller does *not* already own the counter it needs.

c) Implement it, with `reset_sync` instantiated for reset and timer with `BIT_TICK_COUNT` passed straight through.

Here the timer's `enable` is driven from the state, high only while a frame is in the air, and that is the one place where this design deliberately differs from Exercise 8.6. The receiver has to let its timer free-run because it does not control when a frame starts: it can only rephase its own tick count once the start edge has arrived. The transmitter has the opposite problem. It *decides* when the frame starts, so it can start the timer at that moment and get a first bit period exactly as long as all the others. Let it free-run here instead and the start bit is short by precisely as much of a tick period as happened to remain when send arrived, which is a defect in the one thing a transmitter is responsible for.

Two requirements the testbench is strict about:

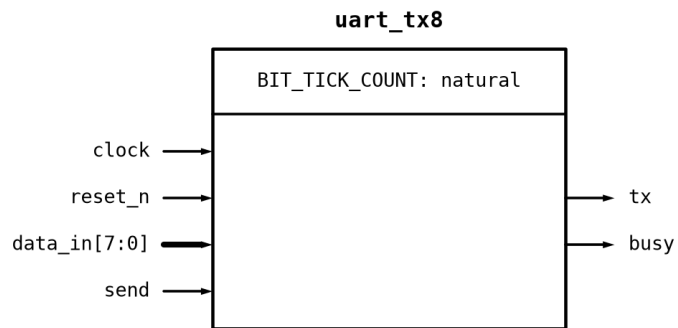
- **send is ignored while the transmitter is busy.** Reloading mid-frame would restart the byte and corrupt what is already on the line. A request arriving during a frame is discarded, not queued.
- **busy stays high through the whole stop bit**, not just the data bits. The frame is not over until the line has been held high for the last bit period.

d) Run the testbench. It plays receiver: it waits for your start bit, samples the middle of every bit period, and checks the framing, the byte, the busy handshake, and that a send applied two bit periods into a frame is ignored.

```
cd lectures/L08/exercises/uart_tx8
cp ../../../../L04/exercises/reset_sync/reset_sync.vhd .      # the two you wrote ↵
↵ yourself
cp ../../../../L07/exercises/timer/timer.vhd .
ghdl -a --std=93 reset_sync.vhd timer.vhd uart_tx8.vhd uart_tx8_tb.vhd
ghdl -e --std=93 uart_tx8_tb
ghdl -r --std=93 uart_tx8_tb --assert-level=error --stop-time=10ms
```

e) Answer, in prose:

- A transmitter skipping the stop bit entirely still leaves the line high afterwards, since idle and stop are the same level. What actually goes wrong, and when does it first show? The testbench catches this one: which of its checks does it, and why could it never have been caught by looking at tx alone?
- Your receiver in Exercise 8.6 samples mid-bit; your transmitter changes the line at the edges. If both ran on crystals differing by 1 %, roughly how many bits into a frame would the receiver's sampling point drift into the wrong bit? What does that tell you about why serial formats have a start bit per byte, rather than a single one at the beginning of a long message?



Self-check. Name your entity `uart_tx8`, with the generic `BIT_TICK_COUNT` (`natural`), the inputs `clock`, `reset_n`, `data_in` (`std_logic_vector(7 downto 0)`), `send`, and the outputs `tx`, `busy`, declared in that order. Its testbench is in `exercises/uart_tx8/`, and nothing else: copy in your own `reset_sync` and `timer`, as part **d**) shows.

Practical exam 1: sequential networks

Three hours, individual, at the computer. Covers the whole of part I, with the emphasis on sequential networks.

IN THIS CHAPTER

- The exam's format: independent tasks, each with a self-checking testbench handed out.
- What you are allowed to use, and what you are not.
- How to prepare, and what the best preparation there is.
- What the exam actually measures.

This is the course's first examination. It contains no new theory; the chapter says what the exam measures, what shape the tasks take and how to prepare, and it is worth reading in advance rather than the day before. Marks and grade boundaries are in appendix C.

The exam covers chapter 1 to 8 in full, with the emphasis on sequential networks: registers, counters, shift registers, timers, synchronisers and simple Moore machines.

9.1 Format

The exam consists of a number of independent tasks. Every task gives you a **self-checking testbench, handed out**, and a description of the module it drives: port order, types and behaviour. Your job is to write the module so that the testbench passes.

- The tasks are independent of each other. A module you cannot get working stops no other.
- The tasks are ordered roughly by increasing difficulty, but read all of them before you start.
- A module that is not entirely finished is marked on what is there. Partially correct logic earns partial marks, so never hand in an empty file because it did not get finished.

- The testbench is the answer key during the exam, not after it. `all checks passed!` means that task is done; the marking reads the code as well.

A practice paper with tasks of the same kind and the same scope is handed out in advance.

9.2 Permitted aids

Allowed:

- All the course material: this book, the lectures' READMEs, every appendix, and all the VHDL you wrote yourself during part 1, including your solutions to the exercises.
- Your own computer, with GHDL. See section 2.7 if you need reminding how a testbench is run.
- CircuitVerse, if you want to draw a circuit before you write it. That is often faster, not slower.

Not allowed: communication with others in any form; language models and other tools that generate or explain VHDL for you; and searching the internet for ready-made solutions. The line is not drawn at who presses the keys but at whose understanding is being tested. Looking up GHDL's or VHDL's documentation is allowed; searching for a finished module is not.

The full and authoritative list of permitted aids is in appendix C; if anything here says otherwise, that one applies.

9.3 Before the exam

- Work through the practice paper, under time pressure and without looking at the solutions first.
- Exercise 8.6 and 8.7, the book's two capstones, are the best preparation there is: they force you to design a state diagram yourself and compose earlier modules around it, which is exactly what the exam measures.
- Make sure GHDL works on your machine *before* you arrive. A broken toolchain is no reason for extra time.

9.4 What the exam measures

- That you can write the synchronous process template correctly, without looking it up.
- That you can build a counter, a timer and a shift register and know what separates them.
- That you synchronise every asynchronous input, without being reminded.
- That you can design a small Moore machine from a specification in words, and write it as an enumerated type and a case.

- That you can read a testbench's error output and find your way back to the line that caused it.

9.5 After the exam

The next chapter starts the **group project**, which runs from chapter 10 to 19 and accounts for half the course's marks. Read the project specification in appendix C before chapter 10; the groups are put together there.

Chapter 10 takes up the project start: the CAN bus, the frame and arbitration, and the project's first VHDL, the shared package `can_def`.

Part II

The group project: a CAN node

Project start: the CAN bus, the frame and `can_def`

Why protocols need frames, what CAN actually is, and the project's first VHDL.

IN THIS CHAPTER

- Why protocols need frames at all: payload, length field, checksum, and why sender, receiver and sequence numbers appear as soon as a link is shared by more than two nodes.
- CAN's differential bus and multi-master design.
- Dominant and recessive bits, and why arbitration replaces a bus master.
- Broadcast plus ID filtering instead of addresses.
- The CAN frame field by field: SOF, identifier, RTR, control field, DLC, data, CRC-15, ACK, EOF.
- Bit stuffing: the five rule, and why it exists.
- Wired-AND arbitration, worked through by hand.
- Bit timing and the sample point, conceptually.
- `can_def`: the shared package holding every constant the chapter defines.

This is the first chapter of the group project. Read the project specification in appendix [C](#) before you start: it describes the way of working, the modules and what is assessed. The groups are put together in this session.

Section [10.1](#) builds up a small frame format of its own from nothing and motivates every field CAN then has. It is the book's only C++ material, and it exists to make the rest of the chapter a comparison rather than an enumeration: CAN answers the same four questions, but answers one of them entirely differently.

The protocol reference. This book is about *building* a CAN node, not about CAN as a standard. The protocol itself is described in *CAN: the bus, the frame and the controller*, a short book written for both halves of a CAN system — the one building the controller in hardware and the one writing the driver in software — and it is the protocol reference throughout part II.

This chapter gives the working understanding needed to write `can_def` and get started; if you get lost in a field, a rule or a timing relationship, that is where you look it up. For whoever is building the controller, its **chapters 1–6 are the requirement specification, bit by bit**, and its chapter 7 describes exactly the simplified controller part II builds: what it implements of the standard and what it deliberately leaves out. Appendix E has the full chapter map between the two books.

10.1 Frames and serialisation

Chapter 1 to 8 ended with state machines controlling a single board: a blinking LED, a UART on an FPGA. The project starts from a different problem: getting independent pieces of hardware to exchange information reliably over a shared wire, where neither side can see inside the other.

This is central to embedded systems because the communication happens as raw byte streams, RAM and flash are limited, and robustness is critical. Nodes usually communicate over serial interfaces such as UART, SPI, I2C or RS-485. Data is sent as a sequence of bytes, and interpreting that byte stream requires a structure. That structure is called a **frame**.

A frame is a structured representation of data that:

- Contains clearly defined fields.
- Has a defined length.
- Can be validated (for example via a checksum).
- Can be serialised into a byte array (`std::uint8_t[]`).
- Can be deserialised from a byte array.

Consider node A sending the value `0x2A` to node B over a shared wire, with nothing else agreed in advance:

- How does B know when a new value begins, in a stream of bits that looks the same in the middle of a value as it does between two values?
- How does B know how long (how many bytes) the value is?
- How does B know that the bits it sampled were not corrupted by electrical noise?
- What happens if two nodes transmit at the same time?

Every real protocol answers these with some combination of the following, each named here after the field this book's frame carries it in:

- **Framing (SOF)**: a recognisable start marker (and often an end marker), so that a receiver can decide where a message begins and where it ends.
- **A length field (LEN)**: so that the receiver knows how much payload to expect.
- **A checksum (CHK)**: extra bits, computed from the payload, letting the receiver detect (not necessarily correct) many kinds of transmission error.

- **A media access rule:** what happens when more than one node wants to transmit at the same time.

A TYPE field joins them to say what the message means, so that the receiver can act on it.

A small but realistic frame

To make it concrete: design a frame for sending short messages between nodes on a shared, byte-oriented link (UART, RS-485, ...):

Example Frame Format							
SOF (2 bytes)	LEN (1 byte)	TYPE (1 byte)	DST (1 byte)	SRC (1 byte)	SEQ (2 bytes)	DATA (N bytes)	CHK (2 bytes)
Field	Size	Description					
SOF	2 bytes	Start-of-frame sync word, 0xA5F7					
LEN	1 byte	Payload length, in bytes					
TYPE	1 byte	Message type					
DST	1 byte	Destination address					
SRC	1 byte	Source address					
SEQ	2 bytes	Sequence number (correlates a response with its request)					
DATA	N bytes	Payload					
CHK	2 bytes	Checksum					

Figure 10.1 The frame format the rest of this section builds, field by field.

Three rules matter as soon as you start working these fields out by hand:

- **Byte order:** every field wider than a byte (SOF, SEQ, CHK) is big-endian, most significant byte first.
- **LEN** counts only the DATA field, never the header and never CHK.
- **CHK** is a 16-bit sum of every byte from SOF through the last payload byte, wrapping on overflow. It is computed over the *serialised* byte stream rather than over the field values as integers, so SOF contributes $0xA5 + 0xF7$, not $0xA5F7$ treated as a single 16-bit number.

Why DST, SRC and SEQ in particular. SOF, LEN, TYPE and CHK already answer the four questions above. These three exist because a shared link with more than two participants raises new ones:

- **DST and SRC:** who the frame is for and who it is from, so that a node can ignore traffic addressed elsewhere and a reply can find its way back. CAN answers this entirely differently, as section 10.3 shows.
- **SEQ:** matches a reply back to *that particular* request, not some other one in the air.

Choosing SOF

- A single sync byte is easy to hit by accident: any payload byte that happens to equal it looks like the beginning of a frame.
- Two bytes are therefore common, and this protocol uses 0xA5F7.
- SOF is the *first* thing a parser checks when reading a frame. If it does not match the data is discarded, and the parser goes on looking for the next SOF.
- Note what that gives and does not give: it makes a false synchronisation much less likely, but not impossible, since payload bytes can still happen to spell A5 F7. The remaining risk is why LEN and CHK still have to be checked afterwards.

Worked example: PING and PONG

Suppose node 0x01 pings node 0x02, with sequence number 0x0020 and no payload:

- SOF = 0xA5F7, LEN = 0x00, TYPE = 0x00 (Ping), DST = 0x02, SRC = 0x01, SEQ = 0x0020.
- CHK = 0xA5 + 0xF7 + 0x00 + 0x00 + 0x02 + 0x01 + 0x00 + 0x20 = 0x01BF.

SOF	LEN	TYPE	DST	SRC	SEQ	DATA	CHK
A5 F7	00	00	02	01	00 20	--	01 BF

Node 0x02 replies with a PONG:

- SOF = 0xA5F7, LEN = 0x00, TYPE = 0x01 (Pong), DST = 0x01, SRC = 0x02, SEQ = 0x0020.
- CHK = 0xA5 + 0xF7 + 0x00 + 0x01 + 0x01 + 0x02 + 0x00 + 0x20 = 0x01C0.
- Note that DST and SRC are swapped, SEQ is unchanged, and only TYPE and the two addresses differ from the PING: identical arithmetic with different inputs.

That covers three of the four opening questions, plus the extra ones a shared link with several nodes raises. The fourth is still open: what happens if two nodes transmit at the same time? CAN answers it differently, and that is the subject of section [10.2](#).

Frame types

TYPE is one byte on the wire, so a frame type travels as an unsigned integer. Every type is assigned an id of its own by convention: Ping = 0, Pong = 1, StatusReq = 2, StatusResp = 3. Further types the protocol acquires simply continue the sequence.

Two consequences worth noting before any code is written:

- The received byte is untrusted input. Every value the receiver does not recognise must be rejected rather than acted on, exactly like a wrong SOF or a wrong checksum.
- Reserving an ID immediately above the last real type, an “unknown” value, makes that check a single comparison. Without it the validation needs a list of valid IDs that has to be kept in step every time a type is added.

A simple 16-bit checksum

For this book's purposes a checksum only has to be simple to implement, simple to test, and good enough to catch many random bit errors. A 16-bit sum meets all three: start at 0, and for every byte in the range, `sum += byte`. The result is the checksum, modulo 2^{16} .

```
/**
 * @brief Compute a simple 16-bit checksum (sum of bytes modulo 2^16).
 *
 * @param[in] data Pointer to the bytes to checksum.
 * @param[in] dataLen Number of bytes to checksum.
 *
 * @return The 16-bit checksum.
 */
std::uint16_t checksum16(const std::uint8_t* data, const std::size_t dataLen) noexcept
{
    if ((nullptr == data) || (0U == dataLen)) { return 0U; }
    std::uint16_t checksum{};
    for (std::size_t i{}; i < dataLen; ++i) { checksum += data[i]; }
    return checksum;
}
```

A checksum's limitations, worth being explicit about. A checksum detects only errors that change the arithmetic sum. A single flipped bit always changes the sum, so single-bit errors are always caught. The real weaknesses lie elsewhere: two whole bytes swapping places leave the sum unchanged, and *some* combinations of several flipped bits cancel each other out and slip through undetected. CRCs catch a much broader, mathematically well-understood class of errors, and are therefore used by CAN instead of a simple sum. They are introduced in section 10.4 and implemented in chapter 13.

This is not as strong as a CRC, but it is enough for practising framing, parsing and robust error handling, which is what this introduction is for.

Serialisation and deserialisation

Serialisation turns structured data into a sequence of bytes (`std::uint8_t[]`) so that it can be sent over an interface. **Deserialisation** interprets a received byte array and extracts the structured data again: the fields are validated first (SOF, length, type, checksum), and only once every check has passed is the data extracted.

Validating before extracting matters because UART and RS-485 deliver bytes as a stream, so a receiver routinely sees rubbish between frames, dropped bytes and corrupted frames.

In embedded systems this is nearly always done with C arrays (`std::uint8_t buffer[Size]`), pointers to those arrays at call boundaries (`std::uint8_t*`), and an explicit length (`std::size_t`).

Both directions need the same byte order handling on every multi-byte field, which is repetitive and easy to get subtly wrong. Two function templates remove the repetition. Both constrain `T` with `std::is_unsigned`, so wherever they end up they need `<type_traits>`:

```
#include <type_traits>

/**
 * @brief Write a value to a buffer, most significant byte first.
 *
 * The caller is responsible for the buffer being large enough.
 *
 * @tparam T The value type. Must be unsigned.
 *
 * @param[out] buf The buffer to write to.
 * @param[in] offset Value offset, i.e. the index at which to write the value.
 * @param[in] value The value to write.
 */
template <typename T>
void writeToBuf(std::uint8_t* buf, const std::size_t offset, const T value) noexcept
{
    static_assert(std::is_unsigned<T>::value, "Failed to write to buffer: T must be unsigned!");
    constexpr std::size_t len{sizeof(T)};
    constexpr std::size_t msb{len - 1U};
    constexpr std::size_t bitsPerByte{8U};

    for (std::size_t i{}; i < len; ++i)
    {
        const std::size_t shift{bitsPerByte * (msb - i)};
        buf[offset + i] = static_cast<std::uint8_t>((value >> shift));
    }
}

/**
 * @brief Read a value from a buffer, most significant byte first.
 *
 * @tparam T The value type. Must be unsigned.
 *
 * @param[in] buf Buffer to read from.
 * @param[in] offset Value offset in the buffer.
 */
```

```

*
* @return The retrieved value.
*/
template <typename T = std::uint8_t>
[[nodiscard]] T readFromBuf(const std::uint8_t* buf, const std::size_t offset) →
    ↳ noexcept
{
    static_assert(std::is_unsigned<T>::value, "Failed to read from buffer: T must →
    ↳ be unsigned!");
    constexpr std::size_t len{sizeof(T)};
    constexpr std::size_t msb{len - 1U};
    constexpr std::size_t bitsPerByte{8U};
    T value{};

    for (std::size_t i{}; i < len; ++i)
    {
        const std::size_t shift{bitsPerByte * (msb - i)};
        value |= static_cast<T>(buf[offset + i]) << shift;
    }
    return value;
}

```

A pitfall worth naming, since the compiler does not catch it. The number of bytes written is `sizeof(T)`, derived from the *argument*, not from the field. Passing in a `std::size_t` where the field is one byte writes **eight** bytes and silently overwrites the fields that follow:

```

constexpr std::size_t payloadLen{0U};
writeToBuf(buf, FrameOffset::Len, payloadLen); //→
    ↳ Writes 8 bytes (error).
writeToBuf(buf, FrameOffset::Len, static_cast<std::uint8_t>(payloadLen)); //→
    ↳ Writes 1 byte.

```

Always cast to the field's own width at the call site.

From paper to code

This exact frame is made runnable in C++ under `lectures/L10/appendix/code/`: a `Frame` struct serialising itself into a byte buffer, deserialising itself back, and rejecting anything with a wrong SOF, length, type or checksum. Serialising the PING above produces exactly the bytes worked out above, `A5 F7 00 00 02 01 00 20 01 BF`. `comm/def.hpp` already provides the field offsets and field lengths, so you never have to count byte positions by hand.

Turning the type IDs above into a C++ enumeration, declaring the struct and implementing both methods yourself is Exercise 10.2. A set of unit tests comes with the code, in

`test/frame_test.cpp`: they check both methods against the frames worked out on paper, byte by byte. You do not write them; `make` runs them, and your job is to make them pass.

What comes next

One question from the beginning of this section is still open: **what happens if two nodes transmit at the same time?**

Nothing in this frame format answers it. A sync word cannot help, since both transmitters send one; a checksum cannot help, since it only reports the damage after the fact. Answering it takes a different kind of protocol, one where the bus's *electrical* behaviour decides who continues and who drops out, before any data is lost. That protocol is CAN.

10.2 Introduction to CAN

CAN was designed for automotive and industrial systems: many independent controllers (engine, brakes, instrument cluster, sensors) exchanging short messages reliably, with no single point of failure.

Point-to-point wiring stops scaling almost immediately. The framed protocol in section 10.1 assumed a single shared link; one controller talking to an instrument cluster, an ABS unit and three sensors would need five separate links, each with its own cabling, connectors and transceiver. A single shared bus replaces all of them: every node connects to the same pair of wires, and every node can both listen and transmit on it.

The physical bus

A real CAN bus is a twisted differential pair, `CAN_H` and `CAN_L`. A node signals a bit by driving the two wires apart (**dominant**) or letting them float together through a resistor (**recessive**). An external transceiver chip handles the analogue layer.

This book works entirely above it: one logic value per bit, 0 for dominant and 1 for recessive. That is exactly what a transceiver presents to, and accepts from, the digital logic on either side.

Dominant and recessive

The whole arbitration scheme rests on a single property:

- **Recessive (1)** is the bus's idle state. Every node releases the bus and the transceivers' bias networks pull both wires to the same level.
- **Dominant (0)** is asserted by actively driving the wires apart. If *any* node drives dominant while the rest are recessive, the bus reads dominant, regardless of how many nodes are involved or which asserted it first.

That is exactly a **wired-AND**: treat every node's output as an input to a bus-wide AND gate, and 0 dominates. Real transceivers implement it in hardware with open drain; the

system testbench in chapter 17 models identical behaviour digitally by combining every node's pair (`tx_bus`, `bus_en`) in the same way.

Multi-master, and why that requires arbitration

Most embedded buses answer the question “who may transmit?” by decree: SPI has a single master, and I2C, which does define a wired-AND arbitration much like the one below, is also nearly always run with a single master. CAN makes multi-master the normal case: **any node may transmit as soon as the bus is idle**, and if two start at the same instant the bus itself decides who carries on. There is no retransmission back-off and a collision is not a failure at all. This is CAN's signature idea, **arbitration**.

The mechanism in brief; the worked example comes in section 10.6:

1. Every message starts with its identifier, sent bit by bit, MSB first.
2. Every transmitting node also monitors the bus while it sends each identifier bit.
3. As long as the bit it sent matches what the bus reads, it goes on transmitting.
4. The moment a node transmits recessive (1) but observes dominant (0), some other node is transmitting a 0 where it itself sent a 1. It has lost arbitration: it stops transmitting immediately and becomes a receiver. The node that carried on driving dominant wins, without having noticed anything.

Two consequences of that:

- **Lower identifiers always win**, since they carry more leading dominant bits. Hence “priorities” in CAN folklore.
- **Every identifier has to be unique**. CAN puts that requirement on the network designer; the controller does not enforce it in hardware.

A simplification to flag early. Step 4 describes a *production* controller, and seamlessly becoming a receiver mid-frame is part of why real CAN treats a collision as a non-event. This book's `can_controller` (chapter 16–17) stops driving at exactly the same moment, which is the part that matters to the bus, but then abandons the frame and sets error instead of decoding the winner's message. Section 19.7 returns to it in its list of differences from a production controller.

Chapter 1 of *CAN: the bus, the frame and the controller* takes the bus on its own: the topology, the dominant and recessive asymmetry, the wired-AND, the two termination resistors, and the relationship between bit rate and bus length — 1 Mbit/s reaches about 40 m, and the reason is arbitration, which requires a bit to travel out to the bus's furthest node and back before the bit is over.

10.3 Broadcast, not addressing

The framed protocol in section 10.1 needed DST and SRC because more than one node could listen. CAN faces the same problem on the same kind of bus and answers it entirely differently: **a CAN frame carries no destination address at all.**

Every node sees every frame. The identifier winning arbitration names no receiver; it names the *kind of message*, broadcast to everyone and relevant to whoever cares. Every node decides for itself whether an identifier means anything, typically by comparing it against a small set it is interested in, in software or in the controller's filtering hardware. A node that is not interested simply discards the frame; nothing at the protocol level steered it away.

Example. Suppose a design reserves identifiers by convention, $StatusReq = 0x300 + nodeId$ and $StatusResp = 0x400 + nodeId$. Sensor node 3 is asked for status on 0x303 and replies on 0x403. Every node receives the frame 0x303; only the one recognising it as its own request identifier replies. No field said "this is for node 3".

It is a genuinely different design choice. The protocol in section 10.1 scales addressing by widening DST and SRC as the number of nodes grows; CAN scales by convention for how identifiers are assigned and filtered, and there is no address field to widen.

10.4 The CAN frame format

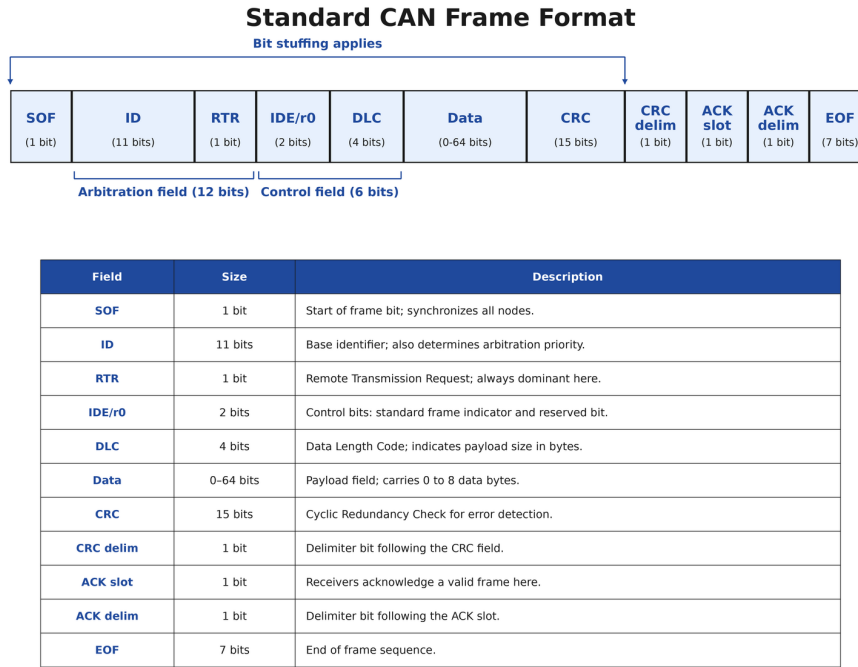


Figure 10.2 CAN's standard data frame, field by field, in transmission order.

This book implements CAN's **standard data frame (base format)**, bit by bit as the standard defines it: an 11-bit identifier and up to 8 data bytes. Remote frames, extended 29-bit identifiers and everything the standard defines *around* data frames (error and overload frames, and the 3-bit intermission separating one frame from the next) are outside the book's scope, but every frame this controller puts on the wire is a fully standard-compliant data frame. It is the same simplification real designs often start from, and it is more than enough to build a genuine, working controller.

The diagram gives the transmission order and every field's width. Bit stuffing applies to every field from **SOF through the CRC field**, and not the CRC delimiter, the ACK field or EOF.

Field by field

SOF, 1 bit, always dominant. Sent out on an otherwise idle, recessive bus, so it produces the single recessive-to-dominant edge every node waits for. That edge marks both that a

frame is starting and resynchronises every node's bit timing against it; `bit_timer.resync` in chapter 12 is exactly this.

Identifier, 11 bits, MSB first. Doubles as the message's arbitration priority. Every transmitting node compares every bit it sends against what the bus actually reads, and drops out the moment it sends recessive but observes dominant.

RTR, 1 bit, always dominant here. The Remote Transmission Request bit ends the 12-bit arbitration field: dominant marks a data frame, recessive a *remote frame*, a request for whoever owns the identifier to send its data. This book supports data frames only, so its controller always sends RTR dominant, and a receiver sampling it recessive abandons the frame and sets error (chapter 17 implements that check). Formally, arbitration spans the identifier *and* RTR; in real CAN that is what lets a data frame beat a remote frame with the same identifier. With every node here sending RTR dominant, the identifier always decides on its own.

Control field, 6 bits: IDE, r0, DLC. IDE is always 0 here; 1 would mark an extended-identifier frame, outside the book's scope. r0 is reserved, always 0. DLC is the 4-bit-wide length field, 0–8 bytes encoded as 0000–1000. It is the length field from section 10.1 in a new guise: without it a receiver could not tell whether the data field ends after this byte or carries on.

Data field, 0 to 8 bytes. Sent MSB first, byte 0 first, with the length DLC announced. When DLC is 0 the field is absent entirely and the CRC follows directly on the control field.

CRC field, 15 bits plus a delimiter. The CRC covers every bit from SOF through the end of the data field, and is far more robust than the summed checksum in section 10.1. One detail matters later on: it covers only the *real* bits. Stuff bits are inserted after the CRC has been computed on the way out and discarded before it is checked on the way in, so they never enter the computation (chapter 14 and 15 are where that gating is built). Chapter 13's engine computes it bit-serially, one bit at a time while the frame is being sent or received, with no separate "compute the checksum" step at the end.

The **CRC delimiter** is the 16th bit, always recessive and never stuffed. It exists so that a bit independent of the bit stuffing rule always follows the CRC value, which is what lets a receiver find the boundary between the CRC and ACK fields.

ACK field, 2 bits. The transmitter sends both recessive. Every *other* node that received the frame correctly (framing intact, stuffing respected, CRC valid) pulls the ACK slot dominant. The transmitter never checks its own CRC against itself, so the dominant bit from somebody else is its only confirmation that the frame arrived intact somewhere. The **ACK delimiter** is recessive and unstuffed, and marks the boundary for the same reason as the CRC delimiter.

EOF, 7 recessive bits, unstuffed. Seven identical bits in a row are far more than the bit stuffing rule would ever allow inside a stuffed field, which is exactly what makes EOF unmistakable. That holds only if every node agrees not to stuff it, which is why EOF and the two delimiters are defined as exempt from stuffing.

Chapter 2 of *CAN: the bus, the frame and the controller* goes through the same frame field by field, and includes what lies outside this book's controller: the RTR semantics of remote frames, what the ACK slot means to a transmitter reading it back, and the three bits of intermission separating one frame from the next.

10.5 Bit stuffing

The rule. Throughout every stuffed field, SOF through the CRC field: after **five consecutive bits of the same value** the transmitter inserts an extra bit of the *opposite* value. That bit carries no data, and every receiver discards it by counting the same way.

Why it exists, for two related reasons:

1. **Clock recovery.** Every node's bit timer runs on its own oscillator. A long run of identical bits contains no transitions, so a receiver has nothing confirming that it is still aligned with the transmitter's bit boundaries. Guaranteeing a transition at least every fifth real bit bounds how far its timing can drift. Real CAN resynchronises on the edges bit stuffing guarantees; this book's controller is simpler and resynchronises only at SOF (`bit_timer.resync`, chapter 12), which makes the bounded drift more important still.
2. **Telling real data from EOF.** EOF is seven identical recessive bits, unstuffed. A stuffed field can never accidentally produce that run, which is what makes seven unstuffed recessive bits an unambiguous end of frame rather than something the identifier or the data field could have produced by chance.

Worked example. To send the 8-bit sequence 11111000 the rule fires after the fifth consecutive 1: insert a 0, then carry on with the real data. Every column below is a bit actually on the wire, so the real-bits row leaves a gap where the stuff bit sits:

Real bits:	1	1	1	1	1	-	0	0	0
Transmitted bit stream:	1	1	1	1	1	0*	0	0	0
Run length:	1	2	3	4	5	1	2	3	4
						^ stuff bit - not real data			

Read the run-length row against the transmitted row, not against the real-bits row: the rule counts bits on the wire. The stuff bit is itself the first bit of the new run, which is why the count reads 1 under it and 2 under the first real 0 that follows. Chapter 14 builds exactly this counter in hardware.

A receiver counts the same five 1s, expects the next bit to be a stuff bit, and discards it if it inverts the run. If a *sixth* consecutive 1 turns up instead where the stuff bit should have been, that is a stuffing violation: a detected framing error, treated as fatal, reported by `rx_shift_reg.stuff_error` in chapter 15.

A subtlety worth mentioning now, since it surprises people later. The run count is *not* cleared at field boundaries. A run beginning in one field and carrying on into the next is counted straight through, and can trigger a stuff bit right at the boundary. Stuffing cares only about the bits actually on the wire, never about which named field they belong to.

Chapter 3 of *CAN: the bus, the frame and the controller* derives the five rule from the clock recovery problem and takes the CRC-15 in the same chapter, for the reason chapter 13 and 14 then live with: stuffing and the checksum share a bit stream but not bits, and whoever understands one without the other gets the wrong CRC.

10.6 Arbitration, worked through in full

Nodes P (identifier 0x0F0) and Q (identifier 0x100) start transmitting at the same SOF. Both send SOF as 0, so no contest arises yet. Then the identifiers, MSB first:

bit:	1	2	3	4	5	6	7	8	9	10	11	
P (0x0F0) sends:	0	0	0	1	1	1	1	0	0	0	0	
Q (0x100) sends:	0	0	1	-	-	-	-	-	-	-	-	(loses at bit 3, stops driving)
bus reads:	0	0	0	1	1	1	1	0	0	0	0	

Bits 1–2 match for both nodes, so nothing happens. At bit 3, P sends dominant and Q recessive, so the bus reads dominant. Q, monitoring while it transmits, sees its own 1 against a bus reading 0: **Q has lost arbitration** and stops driving immediately. P sees 0 on a bus reading 0, exactly what it expected, and carries on without ever finding out that a contest took place.

In a production controller Q would now receive the rest of P’s frame. This book’s `can_controller` stops driving at the same moment but abandons the frame, as flagged above and as section 19.7 returns to.

The mechanism generalises to any number of simultaneous transmitters: the numerically lowest identifier wins (equivalently: the longest run of leading dominant bits), and every loser drops out the moment its recessive bit is overruled, never later than exactly that bit.

Chapter 4 of *CAN: the bus, the frame and the controller* works through both the two-node and the three-node case, and then takes the two questions this chapter does not ask: what starvation and priority inversion mean on a bus where low identifiers always win, and what a transmitter never gets to find out about why it lost.

10.7 Bit timing and the sample point

Arbitration works only if every node agrees on *when* a bit is on the bus, which raises a question: at what point within a bit period does a node read it?

Every node divides its own clock into bit periods of fixed length using its own local bit timer. Because those oscillators are independent, and because signals take time to propagate along the bus, a node does **not** sample at the start of the period. It waits until a **sample point** further into the period, by which time the transmitted level has had time to settle.

This book samples 70 % into every bit period, a reasonable and simple choice for a short bus on a single board at 1 Mbit/s.

Real CAN controllers make the sample point configurable and divide the bit period into named segments (propagation segment, phase segments) tuned against bus length, propagation delay and oscillator tolerance. That lies outside this book's fixed sample point, but not outside chapter 5 of *CAN: the bus, the frame and the controller*, which divides the bit period into its segments, works out the timing from a clock frequency and a bit rate, and describes the resynchronisation at every edge that this controller replaces with a single pulse at SOF.

Section 10.8 turns those 70 % into the concrete VHDL constant `SAMPLE_TICK`, derived from the clock frequency and the bit rate rather than hardcoded.

10.8 The `can_def` package

Why use a VHDL package?

- Part I never needed one: every example there was self-contained, and where an exercise reused a module you had already written — `reset_sync`, `button_sync`, `timer` — you copied the file into the directory that needed it.
- This project is different. `TICKS_PER_BIT`, `SAMPLE_TICK`, `ID_WIDTH`, `CRC_WIDTH` and more are each needed by *several* of the blocks built from chapter 12 onwards, and they have to agree exactly, everywhere, or the design breaks silently. That much shared information is worth defining once.
- Every constant in it comes from something this chapter has just taught: the field widths are the ones you lay out by hand in the exercises, and `SAMPLE_TICK` is the 70 % point section 10.7 reasoned about. This package is where those numbers stop being prose and become the single source of the definitions every module uses.
- It also changes the layout of the *files*. Every module here is part of **one** design, so all of them live in `controller/` and read the single `can_def.vhd` beside them. Nothing is duplicated, and there is exactly one file to change when a constant changes.

Reading `can_def.vhd`

Live-coded together during the session, and written **once**, to `controller/can_def.vhd`. This is the first VHDL the book writes. Every module you write from chapter 11 onwards lands in the same directory, and all of them except `meta_prev` read this one file.

The bit timing constants, derived rather than hand-picked, so that they stay consistent with each other:

```

constant CLOCK_FREQ_HZ: natural := 50_000_000;           -- System clock ↪
    ↪ frequency: 50 MHz.
constant BIT_RATE_HZ   : natural := 1_000_000;           -- CAN bit rate: 1↪
    ↪ Mbit/s.
constant TICKS_PER_BIT : natural := CLOCK_FREQ_HZ / BIT_RATE_HZ; -- Clock ticks per↪
    ↪ CAN bit: 50.
constant SAMPLE_TICK   : natural := (TICKS_PER_BIT * 7) / 10;  -- Sample point: ↪
    ↪ 35 (70%).

```

TICKS_PER_BIT has to divide evenly for this simple design to hit an exact bit rate: 50 000 000/1 000 000 is exactly 50. SAMPLE_TICK is deliberately computed as a *percentage* of TICKS_PER_BIT, not as a hardcoded tick number, so that a later change of bit rate does not silently leave the sample point wrong. This is section 10.7’s 70 % sample point, turned into a number bit_timer can count to in chapter 12.

The frame field widths, straight from the frame diagram in section 10.4:

```

constant ID_WIDTH  : natural := 11;           -- Standard CAN identifier width in ↪
    ↪ bits.
constant DLC_WIDTH : natural := 4;           -- Data Length Code field width in ↪
    ↪ bits.
constant CRC_WIDTH : natural := 15;          -- CAN CRC-15 sequence width in bits.
constant DLC_MAX   : natural := 8;           -- Maximum payload length in bytes.
constant DATA_WIDTH: natural := DLC_MAX * 8; -- Maximum data-field width in bits.

```

The bit stuffing rule’s run length, from section 10.5’s rule about five identical bits:

```

constant MAX_RUN: natural := 5; -- Maximum run of identical bits before a stuff ↪
    ↪ bit is inserted.

```

Both shift registers count runs against this single number (chapter 14, 15): the transmitter to decide when a stuff bit must be *inserted*, the receiver to predict where one must *appear*. That both ends of the link agree on it exactly is this whole package’s argument in miniature — one definition, several modules, no way to drift apart.

Common subtypes, so that every module names identically shaped signals the same way:

```

subtype byte_t is std_logic_vector(7 downto 0);           -- 8-bit byte.
subtype id_t   is std_logic_vector(ID_WIDTH-1 downto 0);  -- Standard CAN ↪
    ↪ identifier.
subtype dlc_t  is std_logic_vector(DLC_WIDTH-1 downto 0); -- Data Length Code ↪
    ↪ field.
subtype crc_t  is std_logic_vector(CRC_WIDTH-1 downto 0); -- CAN CRC-15 sequence.
subtype data_t is std_logic_vector(DATA_WIDTH-1 downto 0); -- CAN data field.

```

Every module except `meta_prev` starts with `use work.can_def.all;` and uses these names directly: `bit_timer`'s counter range, `crc15`'s register width, and `tx_shift_reg/rx_shift_reg`'s byte ports. All of it goes back to this one file.

The CRC-15 polynomial, as a 15-bit-wide constant. Chapter 13's `crc15` uses it directly; the mathematics is covered there, not here. For now treat it as a fixed standard constant that every CAN controller uses:

```
constant CRC_POLY: crc_t := "100010110011001";
```

It corresponds to $x^{15} + x^{14} + x^{10} + x^8 + x^7 + x^4 + x^3 + 1$.

What comes next

Chapter 11 maps this chapter's protocol onto a block architecture and writes the controller's top level, `can_controller.vhd`, whose ports are typed from this package (`id_t`, `dlc_t`, `data_t`). It also introduces the simulation flow and leaves that top level's architecture empty. Chapter 12 builds the first two blocks to go into it, `meta_prev.vhd` and `bit_timer.vhd`.

10.9 Summary

What you should be able to do now

- Say why communication protocols use frames, checksums and arbitration.
- Describe the CAN frame's format field by field, and construct a complete frame by hand.
- Explain how wired-AND arbitration resolves contention without a bus master, and decide which of two nodes wins.
- Apply the bit stuffing rule to a bit sequence, by hand.
- Write `can_def.vhd`: the constants, the subtypes and the CRC polynomial the rest of the project reads.
- Account for the group's way of working: branching, PRs and review.

Questions to test yourself with

1. Why does a protocol need a length field when the frame has a clear beginning anyway?
2. Why does CAN have no destination address, and what does it use instead?
3. Two nodes start transmitting at the same time. Which wins, and what does the loser do?
4. Why must a transmitter insert a bit after five identical ones in a row, and why does the rule not apply to the whole frame?
5. What happens to the CRC computation for a stuffed bit?
6. Why is the sample point around 70 % into the bit period instead of in the middle?

Target

`can_def.vhd` should be written and merged by the end of the session. It is a short file and the rest of the project reads it, so it is worth finishing straight away.

The targets in these chapters are just that, targets. They are not milestones, they are not graded, and the group decides for itself in which order the modules are built; see appendix C.

Looking ahead

Chapter 11 goes from protocol to block diagram: the controller's architecture, the top level every later module is instantiated in, the register map the driver class will consume, and the project's simulation flow in GHDL.

10.10 Exercises

The exercises are laid out like this. Exercise 10.1 and 10.2 belong to section 10.1: the first builds frames by hand, on paper, and the second turns the same frame into a working C++ implementation. Exercise 10.3 bridges that frame format and CAN. Exercise 10.4 to 10.8 consolidate section 10.2 to 10.7; they are done on paper and build up to assembling a whole frame by hand. Exercise 10.9 writes the `can_def` package and then works against it at the keyboard.

EXERCISE SET 1

Frames by hand and in C++

Exercise 10.1 Constructing two frames by hand

REASONING

Start from the frame structure from section 10.1:

Field	Size	Description
-----	-----	-----
SOF	2 bytes	0xA5F7
LEN	1 byte	Payload length
TYPE	1 byte	Message type
DST	1 byte	Destination address
SRC	1 byte	Source address
SEQ	2 bytes	Sequence number
DATA	N bytes	Payload
CHK	2 bytes	Checksum (sum of the preceding bytes)

The checksum is the sum of every byte from SOF through the last payload byte, as a `std::uint16_t` (so it wraps on overflow). Fields wider than one byte (SOF, SEQ, CHK) are big-endian. The frame types are Ping = 0x00, Pong = 0x01, StatusReq = 0x02 and StatusResp = 0x03.

A device at address 0x17 sends a status request to a sensor at address 0x25, with sequence number 0x7F05. That gives TYPE = 0x02, DST = 0x25, SRC = 0x17, SEQ = 0x7F05, and an empty DATA.

- a) Draw the corresponding StatusReq frame on paper, and fill in every field: SOF, LEN, TYPE, DST, SRC, SEQ and CHK. Compute the checksum by hand.
- b) The sensor replies with a 16-bit sensor value, 0x3201, carried in the payload. Draw the corresponding StatusResp frame on paper. The rules: TYPE changes to StatusResp, DST and SRC swap places, SEQ is unchanged, and DATA is 0x3201 (2 bytes). Compute this frame's checksum by hand as well.

Exercise 10.2 The frame in C++

C++

Implement the frame in C++ as a struct `comm::Frame`. Your implementation is validated by an existing set of unit tests under `lectures/L10/appendix/code/`.

1. Examine the file structure

- `test/frame_test.cpp` contains the unit tests validating your frame implementation. You do not write them; you make them pass.
- `include/comm/def.hpp` contains the frame fields' offsets (index positions), the field sizes and more. The enumeration class `comm::FrameType` is to be implemented here.
- `include/comm/frame.hpp` contains the declaration of the struct `comm::Frame`.
- `source/comm/frame.cpp` is to contain the implementation details of the struct `comm::Frame`, that is, its method definitions.

2. Create the frame types

Implement the enumeration class `comm::FrameType` in `comm/def.hpp`, giving each of the four types the id listed in Exercise 10.1. Add a final enumerator `Unknown`, one step beyond the last real type, so that an out-of-range type can be rejected during deserialisation with a single comparison.

3. Create the frame struct

Add the member variables of the struct `comm::Frame` in `comm/frame.hpp`, where `@todo` marks the place: `std::uint8_t data[MaxDataLen]`, `std::uint16_t seq`, `std::uint8_t dst`, `std::uint8_t src`, `std::uint8_t len` and `FrameType type`.

Value-initialise each of them with braces: the numeric members with `{}`, and type with `{FrameType::Unknown}`, so that a default-constructed frame never holds a type that happens to be valid.

The widths are not a matter of taste. The unit tests compare a member against a literal of the field's own width through a template deriving **one** type from both arguments, so a `len` declared as `std::size_t` or a `seq` declared as `std::uint32_t` does not merely waste space; the tests stop compiling.

`serialize()` and `deserialize()` are **already declared** in that file, documented and correctly qualified (`const noexcept` and `noexcept` respectively). Leave those declarations alone; you write their definitions in steps 4 and 5.

4. Implement the serialisation method

Section 10.1's `checksum16()`, `writeToBuf()` and `readFromBuf()` are **not** part of the code handed out, and both methods below need all three. Copy them into `comm/frame.cpp`, in an anonymous namespace above the method definitions, so that they stay private to that file:

```
namespace
{
    // checksum16(), writeToBuf() and readFromBuf() from the framing introduction go ↵
    ↵ here.
} // namespace
```

Two things about that copying, both of which the compiler only tells you about after the fact:

- `writeToBuf()` and `readFromBuf()` are constrained with `std::is_unsigned`, and nothing in `frame.cpp`'s include chain pulls in `<type_traits>`. Add that include at the top of the file.
- Keep section 10.1's warning in mind when you call them: `writeToBuf()` writes `sizeof(T)` bytes derived from the *argument*, not from the field, so cast to the field's own width at every call site.

Then implement the method `comm::Frame::serialize()` so that it writes the following into the given buffer: SOF = `0xA5F7` (big-endian), LEN, TYPE, DST, SRC, SEQ (big-endian), DATA in the same order as it sits in the frame's payload buffer, and CHK, computed over all the preceding fields, byte by byte.

Return the frame's total length if serialisation succeeded, and `0U` on failure, which means a buf that is null or a `bufLen` smaller than the frame to be written. Both have a test of their own; neither is optional.

5. Implement the deserialisation method

In the same file, implement the method `comm::Frame::deserialize()` so that it validates the given data, **in this order**, since every check makes the next one safe to perform:

- `buf` is not null, and `bufLen` is at least `MinFrameLen`. Nothing has been read yet at this point, so this is what makes reading the header permissible at all. Skip it and the reads of SOF and LEN below run off the end of a too-short buffer before any other check gets a chance.
- SOF is correct.
- LEN is within bounds, that is, at most `MaxDataLen`.

- The given buffer holds the **whole** frame the header claims. This is a second, stricter length check, and the first does not replace it: a buffer truncated after the payload still leaves *something* where CHK should be, and reading that as the checksum makes a truncated frame look complete.
- The frame type is valid, that is, the received byte is below Unknown.
- The checksum matches.

Store the given data in the frame, but only once all of it is valid.

Two of those points are checks the tests punish hardest. A missing null check does not make a test fail, it **crashes the whole run**, so you see a segfault and no test name. A buffer length check that stops at the header fails `DeserializeRejectsShortBuffer` and `DeserializeRejectsTruncatedPayload` and nothing else, which is a considerably friendlier way of being told.

6. Validate your implementation

The unit tests are behind a macro, so nothing runs until you turn it on. Add `-DL01` to `CXX_FLAGS` in the makefile, where a `Todo` comment marks the place:

```
CXX_FLAGS := -Wall -Werror -std=c++17 -Iinclude -DL01
```

Then build and run them from a Linux terminal:

```
cd lectures/L10/appendix/code
make
```

Until you do, the program reports that the tests are disabled and exits successfully, whether or not your implementation works.

If the build stops immediately with this, it is not your code:

```
make[1]: *** No rule to make target 'lib'. Stop.
make: *** [Makefile:26: ../../../../libs/test/libqacademy_test.a] Error 2
```

The test framework is a submodule in `libs/test`, and a clone without `--recurse-submodules` leaves the directory empty. That it nonetheless *exists* is why the error is about a missing make target and not about a missing directory: it is the framework's makefile that has not been fetched yet. Fetch it once, from the repository root, with `git submodule update --init`. The exercise also needs a C++ compiler.

The tests check both methods against the frames you built by hand in Exercise 10.1: the PING bytes from section 10.1 and both frames from that exercise, byte by byte; a round trip through serialisation and deserialisation that has to preserve every field; and the rejection cases (wrong SOF, wrong checksum, invalid type, truncated buffer, null pointer).

Every passing test is printed in green; a failing one is printed in red and names the expression, both values and the line, so that you can compare directly against your own paper calculation. Expect to see all of them fail at first, and go green a group at a time as you work through steps 2–5.

Two things worth knowing in advance:

- The tests read `frame.type`, `frame.dst`, `frame.src`, `frame.seq`, `frame.len` and `frame.data` directly, so those member names have to match. If you have named yours something else you are free to change the tests to match.
- `DeserializeLeavesFrameUntouchedOnFailure` gives `deserialize()` a corrupt buffer and then checks that the frame's fields are *unchanged*. A `deserialize()` copying fields into the frame as it parses them passes every other test and fails this one.

EXERCISE SET 2

From the frame format to CAN

Exercise 10.3 The same frame, as CAN

REASONING

Take the `StatusResp` frame you built by hand in Exercise 10.1 b), and send it as a CAN frame instead. Its fields, as a reminder: `TYPE = StatusResp`, `DST = 0x17`, `SRC = 0x25`, `SEQ = 0x7F05`, payload 2 bytes with the value `0x3201`.

Start from the convention in section 10.3: the identifier combines a per-type base with the id of the sensor node the message concerns, that is, the node being asked in a request and the node replying in a response (node 3 is asked on `0x303` and replies on `0x403`):

$$ID = TYPE_BASE + nodeId$$

The type bases are `StatusReq = 0x300`, `StatusResp = 0x400`, `Ping = 0x500` and `Pong = 0x600`.

- Write out the corresponding CAN frame in the form ID | DLC | DATA.
- State which fields of section 10.1's frame have no counterpart in the CAN frame. For every missing field, say what CAN does instead, or explain why CAN needs no equivalent field.
- Suppose two nodes start transmitting on the CAN bus at exactly the same moment. Explain briefly how the bus decides which node carries on transmitting, and how the losing node detects that it has lost. This is the arbitration mechanism from section 10.6; restate it in your own words.
- Explain why there is no separate checksum field to fill in when constructing a CAN frame, unlike section 10.1's frame format.

EXERCISE SET 3

Bit stuffing, arbitration and timing**Exercise 10.4 Bit stuffing by hand**

REASONING

- a) Apply the bit stuffing rule to the following 10-bit sequence:

```
0011111111
```

Write the complete transmitted bit stream, with all the original bits and every inserted stuff bit clearly marked.

- b) A receiver observes the following 9-bit transmitted stream:

```
101111101
```

The receiver knows that bit stuffing is in use. Destuff the stream: identify any stuff bits, remove them, and recover the original sequence of real data bits.

- c) A receiver observes six dominant (0) bits in a row at a place where a stuff bit should have broken a run of five. Explain what the receiver concludes, and what it should do next.
- d) Section 10.5 points out that the run counter is not cleared at field boundaries. Suppose a field ends with three identical bits and the next begins with two more of the same value. Apply the rule and say where the stuff bit lands, counted from the field boundary, and explain why neither field on its own would have caused it.

Exercise 10.5 Decoding a real bit stream

REASONING

A receiver observes the following 21-bit stream:

```
0 00000010101 0 00 0010 11
```

The stream is already destuffed, so that this exercise keeps the focus on the fields' division rather than on the stuffing. The gaps are only there for readability; the real CAN bus has no gaps between the fields.

- a) Split the stream into its named fields, with the widths from section 10.4: SOF, ID, RTR, IDE, r0, DLC. Then determine the identifier in hexadecimal, the DLC value, and which field has begun in the two bits left over once all the named fields are accounted for.
- b) From the DLC in a), determine how many data bits the receiver expects in total, and how many further data bits remain before the CRC field begins.
- c) Suppose the DLC had instead been 0000. Which field would then follow immediately after the control field?

Exercise 10.6 Arbitration between three nodes

REASONING

Three nodes try to transmit at the same SOF: node R with ID 0x155, node S with ID 0x154, and node T with ID 0x1D5.

- a) Work through the arbitration bit by bit, in the same style as section 10.6's worked example. Determine which node wins, and at which identifier bit each losing node drops out.
- b) The two losing nodes drop out at different bit positions. Explain why by stating where each losing node's identifier first differs from the winner's identifier, and why that first differing bit in particular decides when the node loses arbitration.
- c) A losing node has to stop driving the bus, and the interesting question is *when*. (Chapter 11 introduces the port pair `tx_bus/bus_en` that the book's controller stops driving with, and chapter 16–17 build the logic; here you reason at the protocol level.) Explain, in one or two sentences, what each losing node's `can_controller` has to do on the very first clock cycle after it detects the discrepancy, and why that timing matters.

Clue: Think through what “the moment” means in section 10.6's worked arbitration example.

Exercise 10.7 Reasoning about the sample point

REASONING

This book samples every bit 70 % into the bit period, not at 0 % (the very start of the period) or 100 % (the very end).

- a) Explain what could go wrong if a receiver sampled at 0 %. Start from signal propagation, why a bit period needs a minimum duration, and whether the transmitted value has had time to settle.
- b) Explain what could go wrong if a receiver sampled at 100 %, on the last possible clock pulse. Make the explanation specific to the fact that in the book's design every node has its own local bit timer, that the timer is resynchronised only at SOF, and that it is never resynchronised mid-frame (real CAN resynchronises on edges mid-frame as well; the book's controller does not).
- c) Arbitration requires every competing node to read the bus during the *same* bit period as it drives it. Explain why a sample point too near the start of the period is risky, why a sample point too near the end of the period is risky, and why the danger grows the further into the frame you get.

Exercise 10.8 Constructing a CAN frame by hand

REASONING

Exercise 10.4 to 10.7 worked with individual rules. This one assembles a complete frame from the frame diagram in section 10.4.

A remark about the CRC. You are given the CRC-15 value for every frame below. Computing a 15-bit CRC over some 30-odd bits by hand is mechanical rather than instructive; in chapter 13 you work through the recursion properly, over a 4-bit sequence, when the engine itself is the subject.

- a) Take the CAN frame you produced in Exercise 10.3: ID = 0x425, DLC = 2, data 32 01. Write out every field as bits, in transmission order, with the widths from section 10.4: SOF; ID (11 bits, MSB first); RTR; IDE and r0; DLC; data (MSB first, byte 0 first); CRC (given: 010101100100011); CRC delimiter, ACK slot, ACK delimiter and EOF.
- b) How many bits is that in total, before any stuffing?
- c) Now apply the bit stuffing rule from Exercise 10.4 over SOF through the CRC field, remembering that the run counter is not cleared at field boundaries. How many stuff bits are inserted, and how many bits does the frame occupy on the wire?
- d) At the book's bit rate of 1 Mbit/s, how long does it take to send that frame?
- e) Repeat a) to d) for a frame **with no payload**: ID = 0x100, DLC = 0, CRC 011100000001010. Note which parts of the frame are structurally unchanged despite the missing data field, and where the CRC field now begins.
- f) Compare your two answers to c). The second frame carries no data at all, and yet its stuffed region needs *more* stuff bits than the first. Explain why, in terms of what the bit stuffing rule actually reacts to.

EXERCISE SET 4

Working with the package

Exercise 10.9 `can_def` and its derivations

VHDL

Every constant in `can_def` can be traced back to something this chapter defined. These questions write the package, check that the derivations hold when the inputs change, and extend it with values section 10.4 named but the package does not yet hold.

a) Write `controller/can_def.vhd`, with the constants and subtypes section 10.8 sets out. If you were at the session you already have it; if you were not, this is where it comes from, and every chapter from chapter 11 onwards assumes it exists.

This is the only module in the book handed out in full rather than as a specification, and for a reason worth mentioning: a package contains no behaviour to design, so there is nothing between the values and the code for you to work out. What is new here is the VHDL itself, since a package declaration is a construct the book has not used before. Write it out by hand rather than pasting it in. The names are the next eight chapters' vocabulary, and writing them once is cheaper than looking them up eight times.

b) Suppose the DE0-CV's clock were 40 MHz instead of 50 MHz, with the bit rate still 1 Mbit/s. Recompute `TICKS_PER_BIT` and `SAMPLE_TICK` (still 70 %).

c) Suppose instead that the clock stays at 50 MHz but the bus runs at 125 kbit/s, a real CAN "low speed" rate. Recompute both constants.

d) `TICKS_PER_BIT` is `CLOCK_FREQ_HZ / BIT_RATE_HZ`, with VHDL's integer division for natural, which truncates. Give a pair of clock and bit rate where the division does *not* come out even, and explain in one sentence why that would be a problem for this design, even though the VHDL would still compile and elaborate without complaint.

e) Add a constant `BIT_PERIOD_NS`, the length of a CAN bit period in nanoseconds, derived from `BIT_RATE_HZ` rather than hardcoded. Then state the relationship between it, `TICKS_PER_BIT` and the 50 MHz system clock's period of 20 ns, and confirm that the three agree.

f) Section 10.4 states the CRC delimiter, the ACK field and EOF as 1, 2 and 7 bits respectively, but the package holds none of them: `can_controller` will use those numbers as bare literals. Add `CRC_DELIM_BITS`, `ACK_BITS` and `EOF_BITS`, and say which of the two forms a later reader is better served by.

g) Check that the package actually compiles. Install GHDL if you have not already (appendix A), and then run `make build-project` from the root of the group's repository. You should see:

```
--> controller/can_def.vhd analyzes cleanly
```

followed by the two `bridge/` files handed out, and after that every testbench reported as skipped, since the modules they check do not exist yet:


```
0 testbench(es) run, 8 skipped.
```

Now break something deliberately, a missing semicolon or a misspelled type, rerun, and read what GHDL says about it. Put it back.

That is the whole verification story for a package: a package has no behaviour to simulate, so “it analyses” is all there is to check. Chapter [11](#) introduces the tool properly, and from there every module gets a testbench too.

Architecture, the top level and the register map

From protocol to block diagram, and the top level every later module is instantiated in.

IN THIS CHAPTER

- Mapping CAN concepts onto hardware blocks: bit timer, CRC engine, shift registers, and the frame-sequencing state machine controlling them.
- Who owns what: `bit_timer` owns *when*, the shift registers own the bit stuffing, `crc15` knows nothing about frames, and `can_controller` is the only block that knows the frame format.
- A first look at the register map the driver class will consume.
- Writing `can_controller.vhd` from scratch: its fifteen ports, and an architecture that starts empty and grows chapter by chapter.
- Which of the ports are asynchronous, and why the controller synchronises them itself.

Chapter 10 gave the protocol. This chapter turns it into an *architecture*: five blocks with one responsibility each, an entity holding them together, and an interface that stays fixed for the rest of the book. None of the parts is built here. What is written is the entity they are all to live in, and its architecture is deliberately left empty.

11.1 The controller's architecture

Every rule from chapter 10 maps onto one of five blocks:

- **bit_timer** owns nothing about *what* is sent, only *when*. It is the only thing in this design that even borders on a clock domain crossing: it free-runs continuously once enabled, and resynchronises exactly once per frame, at SOF (section 10.5's clock recovery argument, applied).
- **tx_shift_reg** and **rx_shift_reg** own the bit stuffing and nothing else: they serialise or deserialise up to 8 real bits at a time and transparently insert or remove stuff bits. They

have no idea whether the bits they shift are an ID, a data byte or a CRC value; that is `can_controller`'s job.

- **crc15** owns the CRC-15 computation, one bit at a time, with no notion of “frames” or “fields” either; it merely accumulates the bit it is given, when it is told to.
- **can_controller** is the only block that knows the actual CAN frame format: it sequences through SOF, arbitration, the control field, data, CRC, ACK and EOF, in order, drives the other four blocks and reads their outputs. Detecting lost arbitration also lives here, since it is specifically about comparing what this controller decided to send against what the bus actually shows.

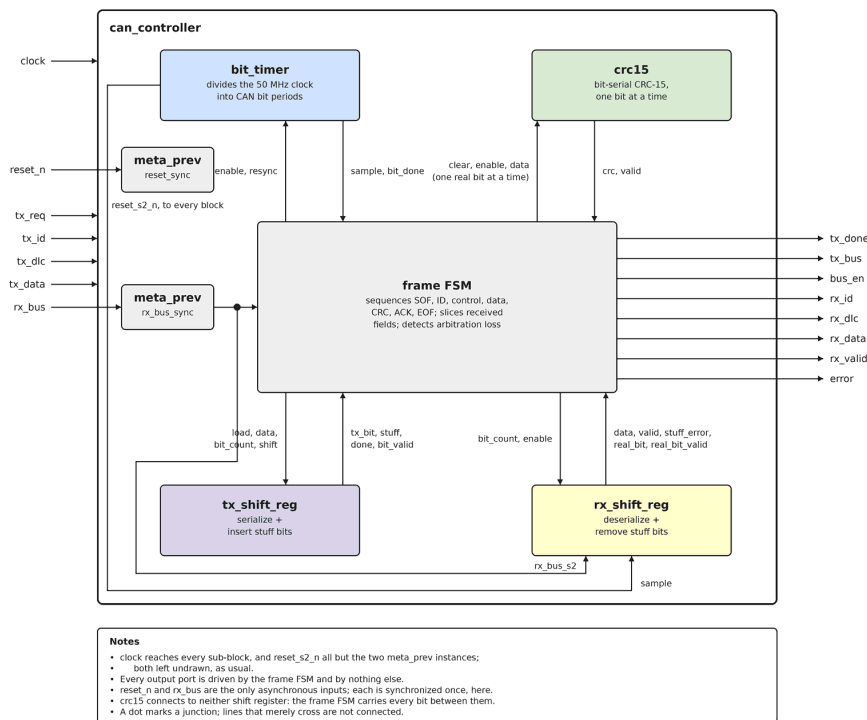


Figure 11.1 `can_controller`'s block architecture.

Two conventions keep the picture readable. `clock` reaches every subblock, and the synchronised `reset_s2_n` reaches all but the two `meta_prev` instances producing it and its bus counterpart; both are left undrawn, as is usual in a block diagram. Every output port (`tx_done`, `tx_bus`, `bus_en`, `rx_id`, `rx_dlc`, `rx_data`, `rx_valid`, `error`) is driven by the frame state machine and by nothing else.

Three of the eleven internal connections are worth reading twice, since they are the ones people redraw wrongly from memory:

- The two lines at `crc15` run in **opposite** directions. `clear`, `enable` and `data` go up into it, one real bit at a time; `crc` and `valid` come back down out of it. (`clear` is chapter 13's per-frame reset — the engine has to start every frame at zero, and an aborted frame would otherwise leave residue behind; chapter 16 drives it from `STATE_START`.)
- `rx_shift_reg` has **three** lines arriving: `sample` from `bit_timer`, `rx_bus_s2` from the synchroniser chapter 12 adds, and `bit_count` plus `enable` from the frame state machine. Without the first two it has no data source at all.
- `tx_shift_reg` and `rx_shift_reg` are never connected to each other, and neither is connected to `crc15`. The frame state machine carries every bit between them. `rx_shift_reg` is likewise the only subblock that reads the bus at all, and even that reads the synchronised copy rather than the port; see section 11.5.

How they nest. The diagram reads as a layer picture, with the cheapest concern on top, but the actual VHDL hierarchy is containment, not a pipeline: `can_controller` instantiates all four of `bit_timer`, `crc15`, `tx_shift_reg` and `rx_shift_reg` inside itself, alongside the `meta_prev` synchroniser chapter 12 adds. `can_controller` is therefore the only entity you ever instantiate by hand, and the whole controller is one component from outside, exactly as the register bank in chapter 18 will see it.

Two more layers then sit above it: `register_bank` (chapter 18), turning those register-level ports into the register map a driver polls, and `can_spi_node` (chapter 19), the top level adding the SPI transport and the board's pins. Both belong to reaching the controller, not to speaking CAN, which is why they turn up in the last two chapters rather than here.

This split — one block per *responsibility*, not one block per *frame field* — is deliberate: `tx_shift_reg` is reused for the ID, the control field, every data byte and the CRC value, simply reloaded with different data and a different bit count each time, instead of writing nearly identical shift logic five separate times.

11.2 Following a frame through the blocks

Naming the five blocks is not the same as knowing how they cooperate. This section takes a concrete frame, ID 0x123, DLC 2, data bytes A1 B2, and follows it through the architecture above. There is no VHDL here; the question is only which block does what, and what passes between them.

First, the only block that counts time. `bit_timer` divides the 50 MHz clock into bit periods and emits two pulses per period: `sample`, 70 % into the period, once the bus level has had time to settle, and `bit_done`, at the very end, when the current bit is over. Every other block acts on one of the two pulses, and none of them counts clock cycles against the bus itself. Transmission moves on `bit_done`, at the boundary where a new bit begins. Reception moves

on sample, in the middle of a bit already sitting there. That difference in particular is why a receiver reads the value a transmitter put out, instead of catching it mid-change.

Transmission: 60 bits, 50 of them through a register. With the field widths from chapter 10, this frame is 1 (SOF) + 11 (ID) + 1 (RTR) + 2 (IDE, r0) + 4 (DLC) + 16 (data) + 15 (CRC) = **50 bits in the stuffed region**, followed by an unstuffed tail of $1 + 1 + 1 + 7 = 10$ bits. Sixty bits before any stuff bit is inserted.

Those 50 bits reach the bus as **eight loads of a single tx_shift_reg**:

#	Group	Bits
1	SOF	1
2	Identifier, high part	8
3	Identifier, low part + RTR	4
4	IDE, r0, DLC	6
5	Data byte 0 (A1)	8
6	Data byte 1 (B2)	8
7	CRC, high part	8
8	CRC, low part	7

can_controller reloads the same 8-bit register eight times, each time with a different value and a different bit count. That is the “one block per responsibility” claim above, made concrete: the identifier is no field to the shift register, it is group 2. The remaining 10 tail bits never reach a shift register at all; they are fixed recessive levels, and can_controller drives them directly while counting bit_done pulses.

Within a group, four things happen per bit:

- tx_shift_reg produces the next bit. The load presents the first one immediately; every bit_done after that shifts out the next. The register also decides, on its own, whether a stuff bit should be inserted, and announces on stuff which kind of bit this was.
- can_controller turns the bit into bus levels. Dominant means bus_en = '1' with tx_bus = '0'; recessive means bus_en = '0', which releases the bus. A node never drives recessive actively (section 10.2's wired-AND).
- crc15 is fed the same bit, but only when it was a real one. Stuff bits are skipped. crc15 has no idea what a stuff bit is; being told when *not* to accumulate is its entire interface on that side.
- can_controller compares rx_bus against the bit it has just driven. That comparison is the arbitration, and it stays here rather than in a subblock because it needs both what this node intended and what the bus actually shows, and can_controller is the only block holding both.

After group 6, crc15 holds the CRC over everything sent so far. can_controller reads it and feeds it straight back into tx_shift_reg as groups 7 and 8. Note what that means: the engine computing the CRC and the register sending it are never connected to each other.

`can_controller` carries the value across, which is why neither block has to know that a CRC field exists.

Reception: the same path, mirrored. `rx_shift_reg` samples `rx_bus` at `bit_timer.sample`, removes stuff bits and reports every real bit on `real_bit` and `real_bit_valid`. Those two signals feed `crc15` exactly as `tx_shift_reg`'s do during transmission: the same instance, the same rule about accumulating real bits only, no mode switch anywhere (which is chapter 13's point about one engine doing both jobs). Completed groups arrive on `data`, and `can_controller` slices them back into `rx_id`, `rx_dlc` and `rx_data`. A stuff bit that fails to appear raises `stuff_error`, and `rx_shift_reg` stops there: it reports the violation without deciding what it means. Deciding is `can_controller`'s job, and it decides to abort the frame and raise error.

What the walkthrough shows. No subblock knows which field it is handling. Each reports facts about *bits*: one is done, that one was inserted, a stuff bit was missing, this group is complete. `can_controller` supplies every bit of the meaning. Three things follow, and all three are worth having:

- The four subblocks can be tested without CAN. Their testbenches in chapter 12 to 15 drive bit patterns and check bit patterns; not one of them needs a whole frame to be meaningful.
- `can_controller` is the only block that would change if the frame format did. Extended 29-bit identifiers, for example, would mean different groups, not different blocks.
- Every connection between blocks is a one-bit or one-byte signal with exactly one writer. There is no shared bus or common structure two blocks both reach into, which is what keeps the wiring work in chapter 12 to 15 to one port map at a time rather than a redesign every chapter.

11.3 The controller's interface

`can_controller`'s ports are what the outside world sees, and they stay fixed for the rest of the book: chapter 12 to 17 add subblocks *inside* this entity without changing a single port. Two things connect to it, and every port belongs to one of them.

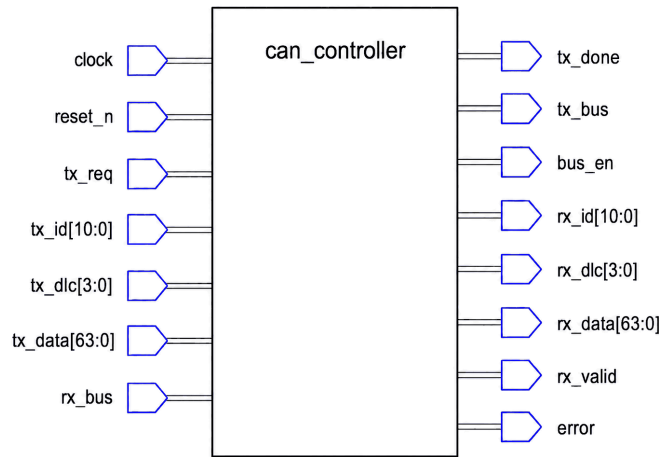


Figure 11.2 can_controller's fifteen ports.

The bus side, five ports. These are the ports reaching the outside world as physical pins; the first three face the CAN transceiver:

- rx_bus is the current bus level, one bit, straight from the transceiver and asynchronous to clock. The controller synchronises it itself, with a meta_prev instance chapter 12 builds, so that the board's top level can connect this port to a pin and think no more about it. The controller reads it to receive, and also to monitor its own transmissions during arbitration.
- tx_bus is the bit this node wants to drive, and bus_en says whether it is driving at all. The pair models open-drain behaviour: bus_en = '1' with tx_bus = '0' pulls the bus dominant, and bus_en = '0' releases it so that another node can. A node never drives recessive actively; it simply lets go. That is what makes section 10.2's wired-AND work.
- clock is the 50 MHz system clock, and reset_n a **raw**, active-low reset, straight from a pin. Like rx_bus it is asynchronous, and like rx_bus the controller synchronises it itself. The synchronised copy is an internal signal called reset_s2_n, the name the reset_sync exercise gave it in chapter 4 and the name every subblock in the project still takes.

The transmit side, five ports. These are the ones a caller fills in to send a frame:

- tx_id, tx_dlc and tx_data carry the frame itself, typed as id_t, dlc_t and data_t from can_def. tx_data holds byte 0 in its most significant bits, in the same order the bus sends them.
- tx_req requests transmission. The controller acts on it only when it is itself idle **and the bus has been idle too**, since a node must not start driving on top of a frame already in progress (chapter 16). A request raised while the bus is busy is discarded rather than queued, so the caller has to ask again.

- `tx_done` pulses high for one cycle as soon as this node's own frame has finished sending.

The receive side, five ports. They mirror the transmit side exactly:

- `rx_id`, `rx_dlc` and `rx_data` hold the most recently received frame, the same types and the same byte layout as their `tx_` counterparts.
- `rx_valid` pulses high for one cycle when a complete, valid frame has arrived.
- `error` is the only port that is *not* a pulse: it goes high on lost arbitration, a stuffing violation, a failed CRC check or a received remote frame (a recessive RTR bit), and stays high until the next frame begins. A caller polling it now and then still sees the error.

Four things about this interface are worth noticing now, since they shape everything that follows:

- **There is no serialisation in it.** The caller hands over an identifier, a length and bytes. Every bit of the framing, the stuffing and the CRC that chapter 10 described happens inside, on the way to `tx_bus`. That is the whole point of building a controller.
- **Transmission and reception are symmetric but not simultaneous.** A node either sends its own frame or receives somebody else's, never both, so the interface's two halves are never active at the same time.
- **`tx_done`, `rx_valid` and `error` are the whole status interface.** Every status bit the register map offers a driver (chapter 18) is derived from one of the three.
- **Every port on the register side is assumed synchronous to `clock`.** `rx_bus` is the exception, and the controller handles that itself with `meta_prev`. The rest, the transmit ports a caller drives and the status and receive ports it reads, assume a caller in the same clock domain, which is exactly what `register_bank` (chapter 18) is. A caller in a *different* domain needs a handshake in front of them, not a wider `meta_prev`: running 64 bits of `tx_data` through a bit-by-bit synchroniser removes the metastability but still lets the controller latch a value that was never written, the same skew problem chapter 19 sets out for the switch bank. A synchronisation chain is the right tool for one bit whose neighbours are independent, and the wrong tool for a value whose bits have to agree.

11.4 Writing `can_controller.vhd`

This chapter writes the file from scratch, to `controller/can_controller.vhd`, beside the `can_def.vhd` written in chapter 10.

The entity declares the fifteen ports described above, in exactly this order, which is what `can_controller_tb` binds to:

1 <code>clock</code>	4 <code>tx_id</code>	7 <code>rx_bus</code>	10 <code>bus_en</code>	13 <code>rx_data</code>
2 <code>reset_n</code>	5 <code>tx_dlc</code>	8 <code>tx_done</code>	11 <code>rx_id</code>	14 <code>rx_valid</code>
3 <code>tx_req</code>	6 <code>tx_data</code>	9 <code>tx_bus</code>	12 <code>rx_dlc</code>	15 <code>error</code>

Every input first, then every output, which is this book's convention everywhere and is why rx_bus sits seventh rather than beside the bus outputs it conceptually belongs with. Read the block diagram above straight down its left column and then its right, and you get 1 to 15 in order. The types come from can_def, so the file opens with use work.can_def.all; alongside the usual ieee.std_logic_1164, and the port list mixes plain std_logic with the package's subtypes. The first lines set the pattern for all fifteen:

```
library ieee;
use ieee.std_logic_1164.all;
use work.can_def.all;

entity can_controller is
  port(clock    : in std_logic;
        reset_n : in std_logic;
        tx_req  : in std_logic;
        tx_id   : in id_t;
        ...
```

Get the **order** exactly right, since it is what everything binds to: can_controller_tb associates its ports positionally, and so does every instantiation in this book. The names are yours. A swapped pair analyses perfectly well today, and unlike a misspelled name it will not fail to analyse in chapter 17 either. It will silently connect the wrong signals together, and the testbench will report behaviour that does not add up, six chapters later.

The architecture starts empty. Not a placeholder to be replaced later, but genuinely empty:

```
architecture behaviour of can_controller is
begin
end architecture;
```

That analyses and elaborates cleanly, and make build confirms that it does. Write it, check it, and look at it for a moment: an entity with no behaviour at all is still a complete design unit. Then the rest of the project puts the first thing inside it.

Why write it now, empty?. Because the interface is a decision, and it is one this chapter is in a position to make. You know what a CAN frame contains (chapter 10), you know which blocks will do the work (section 11.1), and you know what a driver eventually needs to read and write (section 11.6). All that chapter 12 onwards adds is *behaviour behind ports that already exist*.

The alternative, writing the subblocks first and discovering the top level's interface at the end, is how the design would emerge if nobody had thought about it in advance. It also

means that the shape of what you are building stays a mystery right up until the very last chapter.

What each chapter adds. From here on every chapter builds one block and instantiates it here, so that the architecture fills in from the inside. Chapter 12 does two, since both are small:

Chapter	Adds to <code>can_controller</code>
12	<code>meta_prev</code> twice, synchronising <code>reset_n</code> and <code>rx_bus</code> , then <code>bit_timer</code>
13	<code>crc15</code> , and the signals feeding it one bit at a time
14	<code>tx_shift_reg</code>
15	<code>rx_shift_reg</code>
16	The frame-sequencing state machine, the transmit path
17	The receive path and the arbitration, completing the design

The file has to analyse cleanly at every one of those steps, not just at the end. `make build` checks every module separately as soon as it exists, so an entity that does not compile is caught in the chapter that wrote it.

11.5 What the controller synchronises, and why that is the controller's job

One decision follows directly from the last point in section 11.3, and it is worth making explicit here since every later chapter depends on it. `rx_bus` arrives from a transceiver on another chip, and `reset_n` from a button or a supervisor, so nothing in this FPGA has ever controlled either of them. Sampling either straight into `clock`'s domain can leave a flip-flop metastable, the problem chapter 4 solved with two flip-flops in series.

Those two are the controller's *only* asynchronous inputs, and they are asynchronous whoever instantiates it, so **the controller synchronises them itself** rather than leaving it to the board's top level. Whoever integrates it connects both straight to pins and thinks no more about it. Everything else on the entity is the register interface, which is a different contract: one a caller in this clock domain satisfies for free, and one a caller in another domain has to satisfy with a handshake rather than a wider synchroniser.

Internally the synchronised copies are called `reset_s2_n` and `rx_bus_s2`, following chapter 4's convention from `reset_sync`, and those are the names every subblock from chapter 12 onwards takes. **No block ever reads `reset_n` or `rx_bus` directly**, the arbitration comparison in chapter 17 included. The module producing them, `meta_prev`, is chapter 12's, alongside the bit timer; this chapter establishes that the controller owns the problem, and the next builds what solves it.

11.6 A first look at the register map

A parallel class writes a C++ driver against this controller, and talks to it through **registers accessed over SPI** rather than via VHDL ports directly. You build the layers presenting those registers: `register_bank` in chapter 18 and `spi_reg_bridge` in chapter 19. The whole map and the transaction format are in appendix D, and together they are the entire contract between the two classes.

It is worth a preview here, seven chapters in advance, because it is exactly the same information `can_controller`'s ports already carry — the later layers change the information's *shape*, never its content:

Concept (<code>can_controller</code>)	Register in the map
<code>tx_id</code> , <code>tx_dlc</code> , <code>tx_data</code>	<code>TX_ID</code> , <code>TX_DLC</code> , <code>TX_DATA_LO/HI</code>
<code>tx_req</code> , <code>tx_done</code>	<code>TX_SEND</code> , one bit in <code>STATUS</code>
<code>rx_id</code> , <code>rx_dlc</code> , <code>rx_data</code>	<code>RX_ID</code> , <code>RX_DLC</code> , <code>RX_DATA_LO/HI</code>
<code>rx_valid</code>	one bit in <code>STATUS</code> , cleared via <code>RX_ACK</code>
<code>error</code>	<code>ERROR_FLAGS</code>

Keep this correspondence in mind through chapter 19: everything the parallel class's driver reads or writes is just this controller's own port-level interface, wrapped in registers.

11.7 Summary

What you should be able to do now

- Draw the controller's block diagram and say what every block is responsible for, and just as importantly, what it is not responsible for.
- Write the top level's entity with the right port order and the right types.
- Explain why the architecture is empty now and which chapter fills which part of it.
- Run GHDL's first two steps on the empty top level, read what each reports, and say what the third step requires that does not exist yet.
- Say what positional port binding buys and what it gives up.

Questions to test yourself with

1. Why does `bit_timer` own only *when*, and not *what*?
2. Which block knows the CAN frame's format, and why only one?
3. Why does `can_controller` synchronise its own asynchronous inputs instead of requiring whoever instantiates it to do so?
4. What breaks if you swap two ports of the same type, and when would you notice?
5. What do `ghdl -a`, `ghdl -e` and `ghdl -r` each do?
6. Which register-mapped operation corresponds to `tx_req`, and which to `tx_done`?

Target

The top level's entity should exist and analyse cleanly. After that it is up to the group: someone may well start on `bit_timer` or `crc15` straight away. The four leaf modules, `bit_timer`, `crc15`, `tx_shift_reg` and `rx_shift_reg`, do not depend on each other and each has a test-bench of its own, so a group of 4–5 can build them in parallel. It is the one point in the project where the order genuinely matters to how fast you make progress; see appendix C.

Looking ahead

Chapter 12 builds the first two building blocks going into the empty architecture, both small: the synchroniser `meta_prev`, which knows nothing at all, and `bit_timer`, the first block that knows something about CAN.

11.8 Exercises

These exercises consolidate section 11.1 to 11.6: the block architecture, the interface it lands in, and the register map it maps onto. The `can_def` package they all rest on was written in chapter 10.

Exercise 11.1 and 11.2 are done on paper. Exercise 11.3 and 11.4 are done at the keyboard, against the controller/`can_controller.vhd` written during this session. `meta_prev` and its exercises belong to chapter 12, together with `bit_timer`.

EXERCISE SET 1

Reasoning about the architecture

Exercise 11.1 Who owns what

REASONING

- a) `tx_shift_reg` and `rx_shift_reg` do not know whether the bits they shift belong to the ID, the control field, a data byte or the CRC value. Explain, with reference to the block diagram, *which* block does know that, and how it can reuse the same shift register for all of it instead of needing five different ones.
- b) `crc15` is fed one bit at a time and told when to stop by whichever of `tx_shift_reg` and `rx_shift_reg` is active at the time (via their outputs `stuff` and `real_bit_valid` respectively, which gate `crc15.enable`). Why does `crc15` have to be told to *skip* stuff bits in particular, instead of simply being fed every bit that turns up on the wire?
- c) `can_controller` instantiates the four other blocks inside itself, so from outside the whole controller is a single component. Suppose someone proposes factoring the arbitration-loss check out into a block of its own *beside* `can_controller`. Explain what that block would need access to, and why that makes the solution worse than leaving the check where it is.

Exercise 11.2 Splitting a different frame into groups

REASONING

Section 11.2 follows a frame with DLC 2 through the blocks and arrives at it reaching the bus as eight loads of `tx_shift_reg`, 50 bits together, plus a 10-bit tail driven directly.

- Redo that split for a frame with ID `0x7A0` and DLC 5. State the list of groups (every group's contents and bit count, in transmission order), the total number of bits in the stuffed region, and the total number of bits in the frame before any stuff bit is inserted.
- How many groups would a frame with DLC 0 need, and which entries from your list in a) disappear? What does that tell you about how `can_controller` has to be structured around the data field, compared with the identifier or the CRC?
- Four of the groups in your list are narrower than 8 bits, and `tx_shift_reg`'s data input is always a whole byte. Name them, and say what `can_controller` has to pass along with the data for the register to send the right number of bits. (You do not need chapter 14's answer to *how*; the question is what information the register cannot work out on its own.)
- None of your group counts is the number of bits actually driven out on the wire. Explain what is missing, why it cannot be known from the DLC alone, and which block is responsible for it.

EXERCISE SET 2

The entity and the interface

Exercise 11.3 The entity, checked

SIMULATION

- Run `make build-project` from the root of the group's repository, exactly as in chapter 10. Two of your own files should now be reported, alongside the two `bridge/` files handed out, and every testbench should still be skipped:

```
--> controller/can_def.vhd analyzes cleanly
--> controller/can_controller.vhd analyzes cleanly
--> bridge/spi_def.vhd analyzes cleanly
--> bridge/spi_slave.vhd analyzes cleanly

==> meta_prev_tb (skipped: no meta_prev.vhd yet)
==> bit_timer_tb (skipped: no bit_timer.vhd yet)
==> crc15_tb (skipped: no crc15.vhd yet)
==> tx_shift_reg_tb (skipped: no tx_shift_reg.vhd yet)
==> rx_shift_reg_tb (skipped: no rx_shift_reg.vhd yet)
==> can_controller_tb (skipped: no meta_prev.vhd bit_timer.vhd crc15.vhd ↵
    ↵ tx_shift_reg.vhd rx_shift_reg.vhd yet)
```

```
==> register_bank_tb (skipped: no register_bank.vhd yet)
==> spi_reg_bridge_tb (skipped: no spi_reg_bridge.vhd yet)

0 testbench(es) run, 8 skipped.
```

The second line is the whole point of this exercise: an entity with an empty architecture is still a complete, valid design unit. Now go beyond what `make build-project` does and elaborate it by hand, which is the second of GHDL's three steps:

```
cd controller
ghdl -a --std=93 can_def.vhd can_controller.vhd
ghdl -e --std=93 can_controller
```

It elaborates cleanly, with no subblocks inside it and nothing to simulate. There is no third step to run here, since a design with no behaviour has nothing to check and no testbench of its own; `can_controller_tb` drives the *finished* controller and is waiting for chapter 17. It is in chapter 12 that the third step turns up, with the first module that has a testbench of its own.

b) `can_controller_tb` is already in `controller/`, and it needs seven modules, two of which you have written. Say which five are missing and in which chapter each one arrives. Why is it better for the build to skip that testbench than for it to try and fail?

c) *Optional, and only if you have Quartus Prime Lite installed.* Take the same file through Quartus, following appendix B: New Project Wizard, device 5CEBA4F23C7N, add `can_def.vhd` and `can_controller.vhd`, set `can_controller` as the top-level entity, and compile.

It passes. Read the warnings rather than skipping them, and say which of them are unavoidable for an architecture that drives nothing at all. Then find the pin count in the fitter's report and compare it with what a DE0-CV actually offers a human being: ten switches, four buttons, ten LEDs. Write down what you think has to sit between this entity and the board, and keep the note; chapter 18 and 19 build exactly that, and call it `register_bank` and `can_spi_node`.

Nothing later in the book depends on this, and chapter 12 to 19 need only GHDL. It is here because that pin count alone is the whole argument for the two layers above the controller, seven chapters in advance, and reading it off a real fitter report makes it yours rather than a number the book asserts.

d) Temporarily swap `tx_bus` and `bus_en` in the entity's port list, so that they stand in reverse order, and rerun `make build-project`. Both are outputs of type `std_logic`, so the file still analyses cleanly. Explain why nothing catches it now, in which chapter it would first do damage, and what that damage would look like on the bus (the clue is section 11.3's section on the bus side's five ports). Then say why a mix-up between two ports of the same type is

harder to find than a misspelled name would have been, given that everything in this book is bound positionally. Swap them back.

Exercise 11.4 Reading the interface

REASONING

- a) A caller wants to send a frame with ID 0x123, DLC 3 and the data bytes AA BB CC. List every port it has to drive, and the value of each. `tx_data` is `data_t`, 64 bits wide, and byte 0 sits in the most significant bits: give it in full.
- b) The same caller now wants to know whether the frame was sent. It has three status ports at its disposal (`tx_done`, `rx_valid`, `error`). Which does it poll, and what is the one thing it can *never* find out from them, however often it polls? (The clue is section 11.3's remark about what error means; the reason is section 10.4's ACK field.)
- c) `error` stays high until the next frame begins, while `tx_done` and `rx_valid` are one-cycle pulses. Explain what would go wrong for the parallel class's C++ driver if `error` were a pulse like the other two.

Synchronisation and the bit timer

The first two building blocks going into the empty architecture: the one that knows nothing at all, and the first that knows something about CAN.

IN THIS CHAPTER

- `meta_prev`: the two-flip-flop synchroniser, written once with a width generic, and instantiated twice inside `can_controller`, for `reset_n` and `rx_bus`.
- `bit_timer`: a free-running counter producing the pulses `sample` and `bit_done`.
- Running a testbench handed out against each module and reading its report. This is the chapter where the GHDL flow gets its third step.
- Instantiating both inside `can_controller`.

Two modules instead of one, since both are small and they are the design's only blocks holding no protocol state: `meta_prev` knows nothing at all, and `bit_timer` knows only *when*.

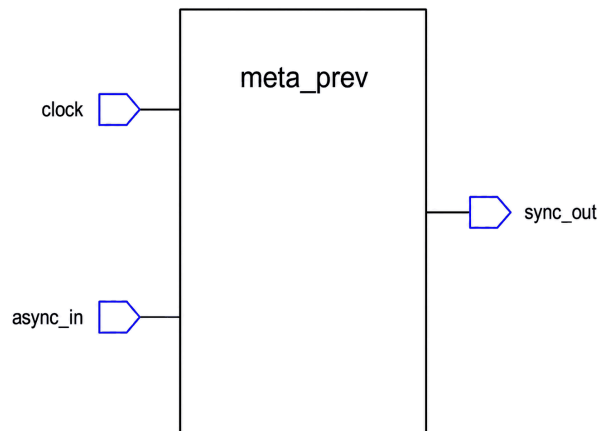
Design each module from the contract and the behaviour below, and then verify against the testbench handed out, without reaching for any reference. Appendix A is the project's GHDL reference.

12.1 Designing `meta_prev`

`rx_bus` arrives from a transceiver on another chip, and `reset_n` from a button or a supervisor, so nothing in this FPGA has ever controlled either. Sampling one of them straight into `clock`'s domain can leave a flip-flop metastable, which is the problem chapter 4 solved with two flip-flops in series. You have built this module before, as `sync` in Exercise 5.4; this is the same idea, written once, properly, to be kept in this design.

Section 11.5 already settled *that* the controller synchronises its own two asynchronous inputs, and why: they are asynchronous whoever instantiates `can_controller`, so the controller owns them and an integrator connects both straight to pins without thinking about it further. This section builds the module that does it. It is the only module in the whole book that contains nothing at all about CAN, which is exactly why it is reusable enough to end up with three instances by chapter 19.

12.1.1 Interface



Port generic	/ Dir.	Type	Meaning
WIDTH	generic	natural := 1	Number of independent bits to synchronise.
clock	in	std_logic	50 MHz system clock, the domain being crossed <i>into</i> .
async_in	in	std_logic_vector(WIDTH-1 downto 0)	The asynchronous input or inputs.
sync_out	out	std_logic_vector(WIDTH-1 downto 0)	The same bits, safe to use in this domain.

`meta_prev_tb` binds its ports positionally, as everything in this book does, so the order and the types above have to match exactly. **The generic map is positional too**, so `WIDTH` has to be the module's only generic (or at least its first). The testbench instantiates the module twice, once with the default width and once with width 4, so that both the default and the generic get exercised. It is also the only module here that reads no package at all, so it needs neither `can_def` nor anything else.

12.1.2 Behaviour

Two flip-flops per bit, in series, on `clock`'s rising edge: the first can go metastable, and the second gives it a whole clock period to settle before anything downstream sees it. `sync_out` therefore follows `async_in` after exactly two rising edges.

Three things differ from the `sync` module you wrote in Exercise 5.4, and all three are deliberate:

- **No reset port.** sync took a reset_s2_n. This one takes none: after two clock edges a synchroniser holds real data regardless of what it started from. Dropping the port is also what lets the *same* module synchronise a reset, without the circularity of a reset synchroniser needing a synchronised reset. The testbench clocks the initial level through before checking anything, for exactly that reason.
- **No PRESET generic.** Chapter 5 argued at length that the safe reset value depends on what the signal *means*, and that only the instantiating design knows that. That argument was about the two cycles after reset, and it does not survive dropping the reset: without a reset branch there is no value to preset. What replaces it is the caller's obligation to ignore sync_out during the first two edges, which is the same obligation expressed differently.
- **SIZE is called WIDTH.** The same generic, the same job, renamed to the name every subblock in this project uses.

This design instantiates it three times, all with WIDTH = 1: twice inside can_controller, for reset_n and for rx_bus, and once at can_spi_node's top level for its own reset (chapter 19). One module, three instances, no copies. The generic still earns its place — the testbench exercises WIDTH = 4, and a synchroniser written for one bit is a synchroniser you copy the next time you need four.

A caveat to mention now, since chapter 19 leans on it: a synchroniser handles metastability, not *skew*. The second flip-flop makes it overwhelmingly unlikely that a metastable value propagates (never impossible; the probability falls exponentially with settling time, which is why two flip-flops are the accepted trade-off), but every bit still crosses independently, so two bits changing together can land one clock apart. That is why a synchroniser is not a general answer for a bus whose bits have to stay together — and it is why can_spi_node synchronises its reset with this module but leaves the SPI lines to spi_slave, which frames them, rather than widening a meta_prev across them.

12.1.3 What the testbench pins down

- sync_out follows async_in after exactly two rising edges: not one (a single flip-flop is no synchroniser) and not three.
- The same holds from high to low as from low to high.
- With WIDTH = 4 all four bits arrive together, two edges after the input changed.
- A stable input leaves sync_out stable.

This is the first testbench in the project that actually *runs* rather than merely analyses, so it is also where the third GHDL step arrives. Chapter 11 analysed and elaborated; see appendix A for what ghdl -r adds and why --assert-level=error is not optional.

12.1.4 Putting it into `can_controller`

`meta_prev`'s ports are vectors even at width 1, so a scalar port needs a one-element signal on each side. In the architecture's declarative part (between `is` and `begin`):

```
-- meta_prev, synchronizing the two inputs that arrive from off-chip.
signal reset_n_v, reset_s2_n_v : std_logic_vector(0 downto 0);
signal rx_bus_v,  rx_bus_s2_v  : std_logic_vector(0 downto 0);
signal reset_s2_n, rx_bus_s2   : std_logic;
```

and after `begin`:

```
reset_n_v(0) <= reset_n;
rx_bus_v(0)  <= rx_bus;

reset_s2_n <= reset_s2_n_v(0);
rx_bus_s2  <= rx_bus_s2_v(0);

reset_sync: entity work.meta_prev
    port map(clock, reset_n_v, reset_s2_n_v);

rx_bus_sync: entity work.meta_prev
    port map(clock, rx_bus_v, rx_bus_s2_v);
```

Two instances rather than one with `WIDTH => 2`. Both bits would ride along together perfectly well, since they are independent and skew between them means nothing, but one named instance per concern reads better on the diagram, and later making the reset assert asynchronously would then tear nothing up.

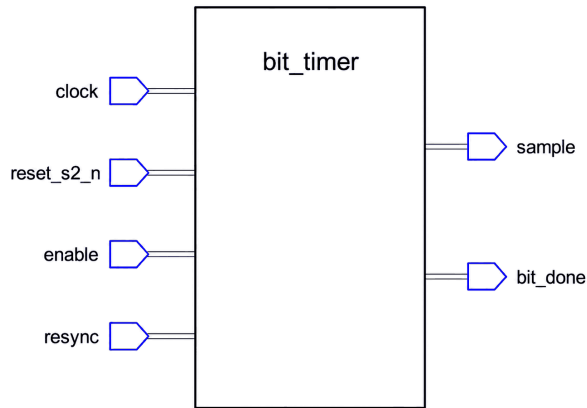
From now on, **every block takes `reset_s2_n`, never `reset_n`, and every block that reads the bus reads `rx_bus_s2`, never `rx_bus`**. That is the whole reason these two instances exist, it applies to `bit_timer` in the very next section, and it is easy to forget again when `rx_shift_reg` is wired in in chapter 15.

12.2 Designing `bit_timer`

`bit_timer` divides the 50 MHz system clock into CAN bit periods and produces two one-cycle pulses that the rest of the controller runs on: one marking where in the bit the bus is to be sampled, one marking the bit's end.

Why a bit period is divided up at all, and why the sample point sits late in it rather than in the middle, is in chapter 5 of *CAN: the bus, the frame and the controller*; this section builds the module that does what that chapter describes, with a fixed sample point instead of configurable segments.

12.2.1 Interface



Port	Dir.	Type	Meaning
clock	in	std_logic	50 MHz system clock.
reset_s2_n	in	std_logic	Active-low reset, synchronised with two flip-flops.
enable	in	std_logic	Runs the counter while '1'; holds it while '0'.
resync	in	std_logic	Forces the counter back to tick 0 at the next clock edge.
sample	out	std_logic	Flag indicating that the bus is to be sampled.
bit_done	out	std_logic	Flag indicating that this bit is over and the next one begins.

No generics. `bit_timer_tb` binds its ports **positionally**, so the order and the types above have to match exactly. The names are yours; keeping these makes the book's text and your code speak about the same signals.

From `can_def` (chapter 10) it needs `TICKS_PER_BIT` (= 50, the ticks in a bit period) and `SAMPLE_TICK` (= 35, that is 70 % in, late enough for the bus to have settled).

12.2.2 Behaviour

A single free-running counter over $0 \dots \text{TICKS_PER_BIT} - 1$ is enough. At every clock edge: default both pulses to '0' and then, in priority order:

1. **reset_s2_n = '0'**: clear the counter and both pulses.
2. **resync = '1'**: restart the bit period; put the counter back to 0. This wins over `enable`, and is used once per frame, at SOF, so that this node's bit timing starts afresh, aligned with the edge opening the frame rather than wherever an earlier count happened to stand.

- 3. **enable = '1'**: pulse sample during the single tick where counter = SAMPLE_TICK. At counter = TICKS_PER_BIT - 1, set bit_done and let the counter wrap to 0; otherwise increment it.
- 4. **Otherwise (disabled)**: hold the counter, both pulses low.

This is the same counter-with-priority-branches idiom as chapter 7’s timer, only with two pulse outputs instead of one.

A note about enable, since it is the one port whose eventual use is not obvious. The natural guess is that can_controller holds the timer while the bus is idle, and it turns out not to: chapter 16’s bus-idle rule counts recessive **bit periods**, so the timer has to keep running in STATE_IDLE too, and the finished design ties enable high in every state. Build the port anyway; the testbench checks it, and a timer that cannot be stopped is a worse building block than one that can.

12.2.3 One bit period, tick by tick

The rules above are easier to trust once seen on a time axis. With TICKS_PER_BIT = 50 and SAMPLE_TICK = 35, an enabled bit period looks like this. Every column is a rising clock edge; counter is the value the process sees at that edge, and the two rows below it are the pulses that edge produces.

counter	0	1	2	..	34	35	36	..	48	49		0	1	..
sample	0	0	0	..	0	1	0	..	0	0		0	0	..
bit_done	0	0	0	..	0	0	0	..	0	1		0	0	..

Three things to read out of that. Both outputs are **one tick wide**, not levels. sample comes at tick 35 and bit_done at tick 49, so **sample always fires first, 14 ticks earlier in the same period**; a receiver reads the bus 70 % in and only after that does the period end. And the period boundary is the wrap from 49 back to 0, which is the same edge bit_done marks.

Now the same picture with a resync. Suppose the timer is 20 ticks into a period when SOF arrives and can_controller pulses resync:

tick	19	20	21	22	..	55	56	..	69	70	71
resync	0	1	0	0	..	0	0	..	0	0	0
counter	19	20	0	1	..	34	35	..	48	49	0
sample	0	0	0	0	..	0	1	..	0	0	0
bit_done	0	0	0	0	..	0	0	..	0	1	0

The counter is thrown away mid-count and starts over, so counter and tick no longer agree. What matters is the sequence: bit_done arrives at tick 70, a whole TICKS_PER_BIT after the resynchronisation, rather than at tick 49 where the interrupted period would have ended. The node has adopted the transmitter’s bit boundary instead of its own, which is the whole point of resync and exactly what the testbench’s fourth case measures.

Note as well that resync restarts the period even though enable is high throughout; that is why it sits above enable in the priority order.

12.2.4 What the testbench pins down

- Held in reset, sample and bit_done stay at '0'.
- While enable = '0' the counter never advances; both pulses stay at '0'.
- While enabled, sample pulses exactly at SAMPLE_TICK and bit_done exactly at the last tick, once per period.
- After a resync mid-period, the next bit_done comes exactly TICKS_PER_BIT ticks later, not where the interrupted period would have ended.

12.2.5 Putting it into can_controller

bit_timer joins the two meta_prev instances from section 12.1 inside the top level written in chapter 11, and is the first block in there that knows anything about CAN. Instantiating it is the same two-part change to controller/can_controller.vhd as meta_prev was, and it takes reset_s2_n from the synchroniser you just wired in rather than the port reset_n.

First declare the four signals the instance connects to, in the architecture's declarative part:

```
-- bit_timer.
signal bt_enable, bt_resync, bt_sample, bt_bit_done : std_logic;
```

Then instantiate the entity itself, after begin:

```
bit_timer1: entity work.bit_timer
    port map(clock, reset_s2_n, bt_enable, bt_resync, bt_sample, bt_bit_done);
```

Nothing drives bt_enable or bt_resync yet, and nothing reads bt_sample or bt_bit_done; the state machine doing both is chapter 16's. That is expected at this point. What matters now is that can_controller.vhd still analyses cleanly, which make build checks as soon as the file exists, and that bit_timer itself passes bit_timer_tb.

12.3 Summary

What you should be able to do now

- Implement a width-generic two-flip-flop synchroniser, and explain why it needs no reset and what widening it does *not* solve.
- Implement a counter-based bit timer producing correctly timed `sample` and `bit_done` pulses, with a `resync` input.
- Explain what `resync` does and why `sample` comes before `bit_done`.
- Read a tick-level trace over one bit period and say for every tick what every output should be.
- Run a testbench handed out all the way through and trace a failure back to the tick in the design that caused it.

Questions to test yourself with

1. Why does a synchroniser need two flip-flops rather than one, and why does it need no reset?
2. A widened `meta_prev` removes metastability but not skew between the bits. What is the difference, and when does a synchroniser stop being the right tool?
3. What does `bit_timer.resync` do, and when is it used?
4. Why does `sample` come before `bit_done` rather than the other way round?
5. `bit_timer` holds its counter while `enable = '0'`. Which state in `can_controller` is the only one that would ever want that, and why does it turn out not to want it either?
6. When a testbench handed out fails, what does GHDL print, and how do you find which check failed?

Target

`meta_prev` and `bit_timer` should be finished and merged within about a week. Both are short, and they are the only modules no other module can substitute for. These are also the first real pull requests: use the review, and check the port order, that the testbench passes, and that no constants are hardcoded past `can_def`.

Looking ahead

Chapter 13 builds `crc15`, a bit-serial checksum engine that both generates and checks a CRC with the same logic and with no mode switch.

12.4 Exercises

Exercise 12.1 consolidates section 12.1; Exercise 12.2 to 12.4 consolidate section 12.2. Design each module from the specification in its own section; the testbenches handed out, run per appendix A, are the check that they behave correctly.

EXERCISE SET 1

The synchroniser

Exercise 12.1 `meta_prev`

VHDL

You have built this module before, so most of what is here is about the two things that are new.

a) Write your own `meta_prev.vhd` from section 12.1 and run its testbench:

```
cd controller
ghdl -a --std=93 meta_prev.vhd meta_prev_tb.vhd
ghdl -e --std=93 meta_prev_tb
ghdl -r --std=93 meta_prev_tb --assert-level=error
```

Note that no `can_def.vhd` appears on the first line. `meta_prev` is the only module in the book that reads no package at all, which is also why it is the one reused three times.

b) Remove one of the two flip-flops, leaving a single register, and rerun. Which check fails, and what does its message tell you? Put it back. (A one-flip-flop version is not merely weaker; it is exactly the bug the testbench exists to catch, and it would otherwise pass every functional simulation in the book, since nothing in a simulator is ever metastable.)

c) `meta_prev` has no reset. Say what `sync_out` reads at the first clock edge after power-up, why that is harmless here, and what would go wrong in `can_controller` itself if the module *had* had a `reset_s2_n` input: the `reset_sync` instance is the one producing `reset_s2_n`, so what would it take for its own reset?

d) `can_controller` instantiates it twice, both at the default width. Say which two of its fifteen ports that concerns, and why those two and no others. Then say what the design's third instance synchronises — the one `can_spi_node` owns at its top level in chapter 19 — and why that one in particular cannot sit inside `can_controller` instead.

e) Two switches are flipped at the same instant into an instance with `WIDTH = 10`. Explain why `sync_out` can briefly show one switch's new value and the other's old one, why that is not a bug in `meta_prev`, and why it would matter considerably more if the ten bits were a counter being read across a clock domain boundary instead of switches set by hand.

EXERCISE SET 2

The bit timer

Exercise 12.2 `bit_timer`

VHDL

Write your own `bit_timer.vhd` from section 12.2. Then verify it:

```
cd controller
ghdl -a --std=93 can_def.vhd bit_timer.vhd bit_timer_tb.vhd
ghdl -e --std=93 bit_timer_tb
ghdl -r --std=93 bit_timer_tb --assert-level=error
```

If a check fails, read the assertion message (appendix A); it names exactly which tick and which expected value it concerns.

Exercise 12.3 A configurable resynchronisation offset

SIMULATION

Real CAN hardware sometimes needs a bit timer that can resynchronise *and* immediately jump forward a fixed number of ticks (a simplified stand-in for the phase segment adjustments real CAN bit timing logic performs, mentioned in section 10.7).

- Add a new generic, `RESYNC_OFFSET`: `natural := 0`, to `bit_timer`. When `resync = '1'` the counter should be set to `RESYNC_OFFSET` (instead of always 0) at the next clock edge.
- With `RESYNC_OFFSET = 0` your modified entity has to behave exactly like the original; confirm that by running the *unchanged* `bit_timer_tb.vhd` against it; it should still pass with no changes to the testbench.
- Work out on paper where **both** outputs land when `RESYNC_OFFSET` is 10: how many ticks after a `resync` pulse `sample` fires, and how many ticks after it `bit_done` does. Both move, and for the same reason — the counter restarts at 10 instead of 0, so the whole period is ten ticks shorter.

Then let the testbench handed out measure it for you. Temporarily change your generic's default value to 10 and rerun `bit_timer_tb`. It binds no generic map, so it picks up whatever default you set, and case 4 is the one that notices:

```
bit_timer_tb.vhd:132:17:@6832ns:(assertion error): bit_timer: unexpected sample at
↳ tick 25 after resync!
```

Note **which** of the two checks fails first. Case 4 checks not only that `bit_done` comes at the right tick but also that the sample point moves with it, and the sample point comes first in the period — so it is the one that fires, fourteen ticks before the `bit_done` check would have. Under `--assert-level=error` the simulation stops there, so you never see the message about `bit_done`.

The message names the tick itself: sample came at tick 25, that is, `SAMPLE_TICK - RESYNC_OFFSET`. If you want to see `bit_done` too, lower the sample check's severity `error` to severity `note` in your own copy of the testbench and rerun; it then reports tick 39, that is, `TICKS_PER_BIT - 1 - RESYNC_OFFSET`. Confirm that both agree with your paper answer, then put the default back to 0 and check that the *unchanged* testbench passes again.

That is worth noticing in itself: a testbench that fails *usefully* says not only that a design is wrong but by how much, which is why the assertion messages in this project all name the tick or the value they expected.

Exercise 12.4 How long can a bit timer free-run?

REASONING

`bit_timer` resynchronises **once per frame**, at SOF, and free-runs from there (section 10.5's clock recovery argument). That works only if the node's own oscillator stays close enough to every other node's for the whole frame. This exercise puts a number on "close enough".

Suppose a node's 50 MHz oscillator in fact runs 1 % fast, so that its ticks are 1 % shorter than the transmitter's, and that everything else is as specified: `TICKS_PER_BIT = 50`, `SAMPLE_TICK = 35`.

- a) How many ticks has this node's counter gained on the transmitter's after one bit period? (Express it as a fraction of one tick.)
- b) The node samples at tick 35 of 50, so relative to the true bit it can drift 15 ticks *later* before it samples past the bit's end, or 35 ticks *earlier* before it samples before the bit begins. A fast clock drifts the sample point earlier. After how many bit periods has it drifted the full 35 ticks?
- c) A standard frame with 8 data bytes runs to about 110–130 bits on the wire once the stuff bits are counted. Compare that with your answer to b): is this node still sampling correctly at EOF? At roughly which field does it start sampling the wrong bit?
- d) Real CAN controllers resynchronise *during* a frame, not only at SOF, using the phase segments and the synchronisation jump width section 10.7 mentions. Explain in one or two sentences what that buys them, expressed in terms of your answer to b).

CHAPTER 13

The CRC-15 engine

One engine, two jobs.

IN THIS CHAPTER

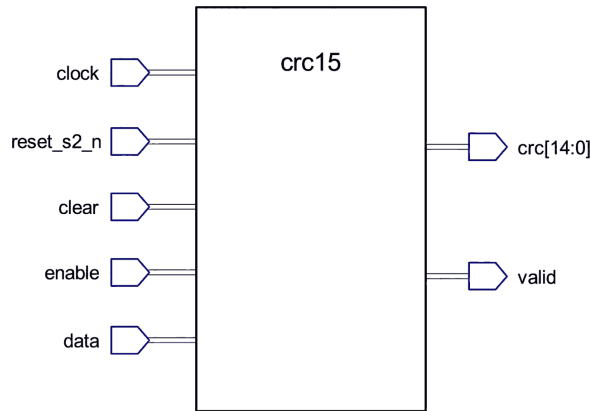
- `crc15`: a bit-serial CRC-15 engine.
- Why the same engine both generates and checks a CRC, with no mode switch.
- Why it nonetheless needs a `clear`, and what an aborted frame leaves behind without it.
- A short CRC worked through the recursion by hand.
- Instantiating `crc15` inside `can_controller`.

CAN's CRC-15 is computed **bit-serially**: one bit fed in per enabled clock edge, with no separate “compute the checksum now” step. The property that makes this book's design simple: **exactly the same engine, run in exactly the same way, both generates a CRC and checks one.**

Chapter 3 of *CAN: the bus, the frame and the controller* is the protocol side of this chapter: what the polynomial is, which fields are counted in, and why stuff bits are kept out. This chapter builds the engine; that book says what it is to compute over.

Design it from the contract and the recursion below, and then verify against the testbench handed out. `crc15` does not depend on `bit_timer` and may well have been built already.

13.1 Interface



Port	Dir.	Type	Meaning
clock	in	std_logic	50 MHz system clock.
reset_s2_n	in	std_logic	Active-low, synchronised reset.
clear	in	std_logic	Returns the register to all zeros at this edge, without a reset.
enable	in	std_logic	Integrate data into the register at this edge when '1'.
data	in	std_logic	The current bit, the bit being sent (TX) or just sampled from the bus (RX).
crc	out	crc_t	The register's current 15-bit CRC value.
valid	out	std_logic	'1' whenever the register currently reads all zeros.

No generics. `crc15_tb` binds its ports **positionally**, so the order and the types above are what has to match; the names are yours. `crc_t` (a 15-bit vector), its width `CRC_WIDTH` and the polynomial `CRC_POLY` come from `can_def` (chapter 10).

13.2 The recursion

Keep a 15-bit register (`crc_t`), cleared to all zeros by `reset_s2_n`. At every clock edge where `enable = '1'`, integrate one bit:

```

feedback = crc(CRC_WIDTH-1) xor data    -- Incoming bit XOR the register's top bit.
shifted  = crc shifted one step left, new lowest bit '0'
crc      = shifted xor CRC_POLY          -- if feedback = '1'
          = shifted                      -- otherwise

```

That is the whole engine, fifteen flip-flops and a shift XOR-gated by the polynomial's taps. When `enable` is '0', hold the register unchanged. Drive `crc` continuously from the register, and `valid` as “the register is all zeros”.

`clear` does to the register what `reset_s2_n` does, but it is a **synchronous command, not a reset**: it belongs inside the clocked branch, tested before `enable` so that it wins when both are high at the same edge. Do not fold it into the asynchronous reset condition; that would make `clear` asynchronous and level-sensitive, a different design that happens to pass the testbench (which always holds `clear` for a whole clock cycle). The interface table says “at this edge” and means it.

`clear` exists because the engine is reused for every frame and holds no per-frame state of its own: `can_controller` pulses it once at the start of every frame (chapter 16). A frame allowed to run to the end returns the register to zero by itself, through the generate-then-check property below, so on the happy path `clear` changes nothing. A frame that is *aborted* halfway does not, and that is the case it exists for.

13.3 One engine, two jobs

- **Generate:** enable the engine over SOF through the end of the data field. Then read `crc`; that is the value to send as the frame’s CRC field.
- **Check:** enable it over SOF through the data field *and then carry on*, feeding in the *received* CRC field’s own bits as well. If the frame arrived intact, that drives the register back to exactly zero, a property of the arithmetic, not something the engine was told to expect. `valid` reports precisely that. `can_controller` (chapter 17) checks `valid` only at that point; it needs no second, separate checking circuit.

Both jobs consume **real bits only**. Stuff bits (chapter 14) never enter the CRC on either side of the link; the caller’s gating of `enable` is what keeps them out (chapter 16 and 17 build exactly that), and the engine itself never knows they exist. That is why `crc15` needs no knowledge of frame fields or stuffing: whoever drives `enable` has already taken those decisions.

Both jobs assume the register starts a frame at zero, which is exactly what `clear` guarantees. Note how much rests on that: a node abandoning a frame halfway (it loses arbitration, or sees a stuffing violation, or fails the CRC check, all things chapter 17 does) stops feeding the engine with a *partial* value in it. Without a per-frame `clear`, the next frame gets that residue as its starting value, so the node computes a CRC nobody else agrees with and computes a wrong check on everything it receives, for every frame from then until reset. A single aborted frame would take the node off the bus permanently.

`valid` reads ‘1’ at reset as well, before any data at all; meaningless there, meaningful only at the check point above.

13.4 A worked example, by hand

Start with the simplest case. Feed a single 1 into a just-reset engine: feedback is 0 xor 1 = 1, shifted is still all zeros, so crc becomes 0 xor CRC_POLY, the polynomial itself. That is the first thing the testbench checks, and a good sanity check on your own code.

Now four bits, 1001, MSB first. top is the register's bit 14 *before* the edge, and CRC_POLY is 100010110011001:

	top	data	fb	shifted	xor poly	crc
reset	-	-	-	-	-	0000000000000000
bit 1	0	1	1	0000000000000000	yes	100010110011001
bit 2	1	0	1	000101100110010	yes	100111010101011
bit 3	1	0	1	001110101010110	yes	101100011001111
bit 4	1	1	0	011000110011110	no	011000110011110

Read one row at a time and the engine stops looking like arithmetic. Every edge does the same three things: XOR the incoming bit with the bit falling off the top, shift left, and XOR the polynomial back in only if that result was 1.

Two rows are worth stopping at. **Bit 2** feeds in a 0 and triggers the XOR all the same, because the 1 leaving the top is what sets feedback; the incoming bit is only half the decision. **Bit 4** feeds in a 1 and does *not* trigger it, because the top bit is also 1 and the two cancel. That is why a CRC responds to *where* a bit sits and not just to how many ones there are, and it is the property Exercise 13.3 uses to catch a corruption a simple sum cannot see.

Note as well that every row's shifted column is the previous row's crc shifted one step left, with a 0 coming in from below. Nothing is ever added at the low end; the register's contents come entirely from the polynomial being XORed in over and over at different offsets.

13.5 What the testbench pins down

- Just after reset, crc is all zeros and valid = '1'.
- A single '1' bit gives crc = CRC_POLY.
- clear returns a loaded register to all zeros without reset_s2_n being touched.
- A 19-bit sequence of SOF, ID, RTR and control field gives a CRC matching a value computed independently in Python (not derived from this VHDL).
- Feeding the just-computed CRC's own bits straight back in returns the register to all zeros (valid = '1'), the generate-then-check property.

13.6 Putting it into can_controller

The same two-part change as chapter 12's, one block further in. In controller/can_controller.vhd, add the signals under the bit_timer group:


```
-- crc15.
signal crc_clear, crc_enable, crc_data_in, crc_valid : std_logic;
signal crc_value                                     : crc_t;
```

and the instance under bit_timer1:

```
crc1: entity work.crc15
  port map(clock, reset_s2_n, crc_clear, crc_enable, crc_data_in, crc_value,
    ↪   crc_valid);
```

Note that `crc15` connects to nothing but the clock, the reset and its own signals: it takes its bits from whichever shift register is active, and both of those registers are as yet unwritten. The connection joining them is chapter 16's, and it is a single expression rather than a port map.

13.7 Summary

What you should be able to do now

- Implement a bit-serial CRC-15 engine matching CAN's polynomial.
- Explain why generating and checking are the same operation, and what a cleared CRC register at the end of a received frame proves.
- Work a few bits through the recursion by hand and compare against the simulation.
- Say why the engine has to be cleared between frames, and what failure arises if it is not.

Questions to test yourself with

1. Why does the engine need no mode for “generate” as against “check”?
2. What does it mean that the CRC register is zero after a received frame has been fed through it?
3. What happens if a frame is aborted and the next frame starts being fed in without a clear?
4. Why is a stuffed bit not fed into the CRC engine?
5. Which signal in `can_controller` decides when `crc15` may update, and why is it not `bit_done`?

Target

By this point two of the four leaf modules should be finished. Which two does not matter.

Looking ahead

Chapter 14 builds the transmit shift register: serialising out onto the bus, with the same bit stuffing rule as in chapter 10, now in VHDL. Chapter 14 and 15 together build the modules deciding which bits reach crc15 and when.

13.8 Exercises

These exercises consolidate the chapter. Design your own `crc15.vhd` from section 13.1 to 13.4; the testbench handed out, run per appendix A, is the check that it behaves correctly.

Exercise 13.1 `crc15`

VHDL

Write your own `crc15.vhd` from the specification in this chapter. Then verify it:

```
cd controller
ghdl -a --std=93 can_def.vhd crc15.vhd crc15_tb.vhd
ghdl -e --std=93 crc15_tb
ghdl -r --std=93 crc15_tb --assert-level=error
```

If a check fails, read the assertion message (appendix A).

Exercise 13.2 CRC by hand, then in simulation

SIMULATION

- a) Compute by hand, using section 13.2's recursion, the CRC-15 result of feeding the 4-bit sequence 1010 (MSB first) into a just-cleared engine. Show the working one bit at a time (feedback, shift, conditional XOR).
- b) Confirm your hand-computed answer with GHDL: write a short testbench (or adapt the structure of `crc15_tb.vhd`) that clears `crc15`, feeds in 1010 and reports the resulting `crc` value, either via an `assert` or via a `report ... severity note`; you read directly.
- c) `crc15`'s output `valid` reads '1' immediately after reset, before any real data at all has been fed in. Explain why that is not a bug, with reference to what `can_controller` (chapter 17) actually does with `valid`, and when.

Exercise 13.3 The error a summed checksum misses

SIMULATION

Section 10.1 claimed that a CRC “catches a much broader, mathematically well-understood class of errors” than the simple byte-wise sum its own frame format uses. This exercise makes that concrete by running a corruption the sum demonstrably cannot see through the engine you just built.

Take the payload 0x3C, 0x91 and the corruption 0x34, 0x99: bit 3 cleared in the first byte, bit 3 set in the second, so that one byte loses exactly what the other gains.

- a) Compute the byte-wise checksum of both payloads and confirm that they are **identical**: both come to 0xCD. A simple sum records only the total, so a receiver checking it would accept the corrupt frame as intact.
- b) Now feed all 16 bits of the *original* payload into a just-cleared crc15 (MSB first, first byte first) and note the resulting crc value. Then clear and do the same with the *corrupt* payload. Do the two CRC values agree? Use simulation; a report of both values is enough.
- c) The property that generating and checking are the same operation (section 13.3) says that a frame whose own CRC bits are fed back in returns the register to zero. Feed in the **corrupt** payload followed by the **original** payload’s CRC bits, that is, exactly what a receiver would see if this corruption happened in transit. Does the register reach zero? Does valid go high? Explain what the receiver concludes.
- d) Say in one sentence which property of the CRC recursion makes it notice a change a sum cannot, with reference to what every bit does to the register in section 13.2.

Transmit shift register and bit stuffing

Serialising out onto the bus, with the same bit stuffing rule as in chapter 10, now in VHDL.

IN THIS CHAPTER

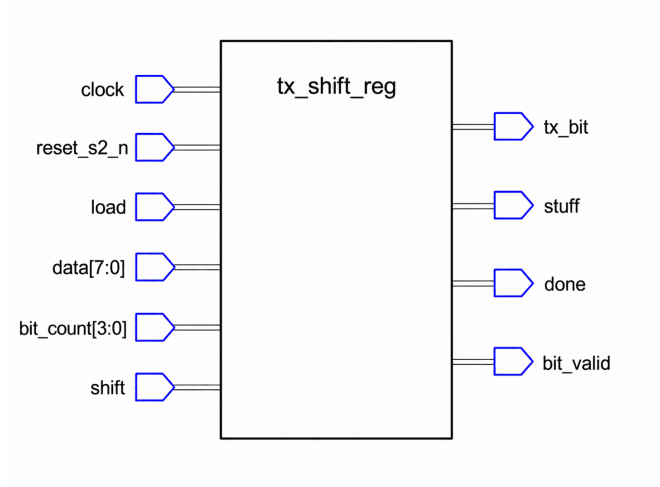
- `tx_shift_reg`: serialisation most significant bit first, and insertion of stuff bits.
- Why a loaded group's first bit has to be presented immediately on reload, and why done waits one extra shift.
- Why the stuffing state survives across reloads.
- `stuff` and `bit_valid`, and why the CRC engine is gated on them.
- Instantiating `tx_shift_reg` inside `can_controller`.

`tx_shift_reg` serialises a group of up to 8 bits MSB first onto the bus and **inserts** a stuff bit after five identical transmitted bits in a row. Its mirror image, `rx_shift_reg` (which *discards* those stuff bits), is chapter 15.

The bit stuffing rule itself is unchanged from section 10.5 and chapter 3 of *CAN: the bus, the frame and the controller*, where you applied it on paper to a whole bit sequence at a time, able to look back over the bits you had already written down. Here it runs continuously, one bit per clock, with no sequence to look back over and no “end of buffer” to stop at: whatever the rule needs to remember has to live in a register. `can_controller` (chapter 16–17) reuses **one single** `tx_shift_reg` for every stuffed field, reloading it and changing its bit count group by group. The unstuffed tail (CRC delimiter, ACK slot and ACK delimiter, EOF) is fixed bits that `can_controller` drives directly, past this register.

This is the first module where drawing the timing diagram before you write the code pays off. Take the time; most failures here are one-cycle timing errors, not logic errors.

14.1 Interface



Port	Dir.	Type	Meaning
clock, reset_s2_n	in	std_logic	50 MHz clock, synchronised active-low reset.
load	in	std_logic	Latches data/bit_count as the next group and presents its first bit this cycle.
data	in	byte_t	The group, with its bit_count valid bits packed into the most significant bits (the first bit sent is bit 7).
bit_count	in	std_logic_vector(3 downto 0)	Number of valid bits in data, 1–8.
shift	in	std_logic	Pulses once per CAN bit (from bit_timer.bit_done) to drive the stream forward.
tx_bit	out	std_logic	Registered bit to drive during the coming bit period.
stuff	out	std_logic	'1' on the very shift that puts out a stuff bit; masks that bit out of the CRC feed.
done	out	std_logic	Pulses when the group has been fully sent (see the timing).
bit_valid	out	std_logic	'1' the one cycle where a new bit was put onto tx_bit.

tx_bit is the only output that is a *level*, held until the next bit is presented. stuff, done and bit_valid are all **one-cycle pulses**: default them to '0' at the top of the clocked process and raise them only in the branch that earns them, the same idiom bit_timer uses for sample and bit_done.

That matters most for bit_valid and stuff, since can_controller builds crc15's enable out of the pair, as bit_valid and not stuff (chapter 16): bit_valid says a bit was put out,

and `stuff` masks out those put-out bits the CRC must never see, since the CRC covers real bits only. Hold `bit_valid` as a level instead and the engine integrates the same bit at every clock edge of the bit period, fifty times over, silently destroying the CRC.

`load` and `shift` are mutually exclusive within a cycle; `can_controller` never asserts both. `tx_shift_reg_tb` binds **positionally**, so the order and the types above are what has to match; the names are yours.

14.2 What the module has to remember

Start where the introduction pointed: the paper version of the bit stuffing rule looked *back over the sequence*, and the hardware has no sequence to look back over. So before any process is written, list what has to live in a register. Everything below follows from the module's two jobs — keeping a group moving, and keeping the bit stuffing rule honest — and every rule in the rest of this chapter speaks in terms of exactly these five signals:

```

signal shift_reg    : byte_t;                -- Remaining chunk bits, next ↵
    ↳ one in bit 7.
signal bits_left    : natural range 0 to 8;   -- Real bits of the chunk still ↵
    ↳ to present.
signal last_bit     : std_logic;             -- The bit most recently placed ↵
    ↳ on the wire.
signal consecutive  : natural range 0 to MAX_RUN; -- Current run length; 0 = none.
signal stuff_pending: std_logic;             -- A stuff bit is owed on the ↵
    ↳ next shift.

```

- **shift_reg** and **bits_left** are the group in flight. The register holds the unsent bits left-aligned so that “the next bit” is always `shift_reg(7)`; `bits_left` is how many real bits remain. There is no more group bookkeeping than that.
- **last_bit** and **consecutive** are the bit stuffing rule's memory, and together they are called the **run tracker** below: the bit most recently sent, and how many identical bits in a row it ended. `MAX_RUN` identical bits — five, a constant `can_def` supplies, since both ends of the link have to agree on it exactly — is a full run; the pair is everything the rule ever needed from “looking back”. Both are written in exactly one place, a procedure the process declares for the purpose (below).
- **stuff_pending** is the rule's one-bit debt. When a run reaches five, the stuff bit is *not* sent immediately — the fifth bit still needs its own full period on the bus. Reaching five therefore only *arms* an insertion, and this flag carries the debt forward to the next shift.

The outputs are registered in the same process: `tx_bit` is a level, held until the next bit replaces it; `stuff`, `done` and `bit_valid` are the one-cycle pulses section 14.1 described, defaulted low at the top of the process and raised only in the branch that earns them.

14.3 The process, top to bottom

One clocked process holds all of the above. The walkthrough below follows the code in the order it is written, so every line has a home before the next turns up; the code fragments are the process's declarative part followed by successive slices of a single `if/elsif` chain, and placed in sequence inside the skeleton below, whose reset branch you fill in from its one-line description, they are the whole process. One line uses `to_integer(unsigned(...))`, so the file needs use `ieee.numeric_std.all`; alongside `ieee.std_logic_1164` and `work.can_def`.

The skeleton. The process declares its own procedure before `begin`; then asynchronous reset first, everything else on the rising edge:

```
process(clock, reset_s2_n) is
    -- One procedure, the run tracker (below).
begin
    if (reset_s2_n = '0') then
        -- Clear everything.
    elsif (rising_edge(clock)) then
        -- Defaults, then the load branch or the shift branch.
    end if;
end process;
```

14.3.1 The run tracker, written once

Every branch putting out a bit needs the same few lines, so they live in a procedure declared in the process's own declarative part, between `process(...)` `is` and `begin`:

```
-- The run tracker: runs on the bit just placed on the wire.
procedure track_run(new_bit: in std_logic) is
begin
    if ((new_bit = last_bit) and (consecutive /= 0)) then
        consecutive <= consecutive + 1;
        if (consecutive = MAX_RUN - 1) then -- Run reaches MAX_RUN.
            stuff_pending <= '1';
        end if;
    else
        consecutive <= 1; -- Start a new run.
    end if;
    last_bit <= new_bit;
end procedure;
```

This is the book's first subprogram, so four things about it before anything calls it:

- **It reaches the signals directly.** `last_bit`, `consecutive` and `stuff_pending` are architecture signals, visible inside the procedure because the procedure is declared inside the

process, which sits inside the architecture. They are not parameters and should not be; only the *placed bit* differs between callers, and that is the only parameter there is.

- **Declaring it inside the process is what makes the signal assignments legal at all.** A subprogram may assign a signal only in two situations: the signal is a formal parameter of class `signal`, or the subprogram is declared inside a process, in which case its assignments use that process's drivers, exactly as if the lines had been written at every call site. This procedure rests on the second case. Move it up unchanged to the architecture's declarative part and it stops compiling, one error per assignment: `signal "consecutive"` is not a formal parameter. Threading `consecutive`, `last_bit` and `stuff_pending` through as formal parameters of class `signal` would satisfy the rule, at the cost of three extra arguments at every call and a real question about which process owns the drivers. Declared here, only this process can call it, so only this process drives them.
- **The timing is unchanged.** An in parameter is evaluated at the call, so `new_bit` carries the pre-edge value the caller passed in, and the signal reads inside the body are also from before the edge (the variables-versus-signals rule from chapter 5). The procedure behaves exactly like the same lines written inline.
- **Synthesis inlines it.** The result is the same logic replicated under every caller's branch condition; nothing is shared, multiplexed or called at run time. This is factoring at the source level, and the netlist comes out identical either way.

Two details inside the body, and one about where it is called:

- **`consecutive = MAX_RUN - 1` means "the run is now full".** The read gives the pre-edge value, so the comparison is against the run length *before* this bit extended it. That semantics is exactly right here, and it is why the code compares against `MAX_RUN - 1` and not `MAX_RUN`.
- **Keep track of which if owns which branch.** `consecutive <= 1` is the *outer* if statement's else (every placed bit breaking the run starts a new one), and `last_bit <= new_bit` stands after the whole if statement and updates at *every* placement, identical or not. Hang either of them off the inner `MAX_RUN - 1` check and a broken run never restarts, or `last_bit` freezes.
- **Call it from a branch that places a bit, never from the process's top level.** Nothing on the wire changes during the forty-nine-odd clock edges between presentations, so a call running at every edge counts phantom bits. It knows nothing about groups either way: the wire has no idea where one group ends and the next begins, so neither does the tracker.

14.3.2 Reset and defaults

Reset clears everything: the outputs, the group (`shift_reg`, `bits_left`) and the run tracker (`last_bit`, `consecutive`, `stuff_pending`). This is the **only** place the tracker is ever cleared. `load` must not touch it: the stuffing sees the transmitted stream as continuous across the whole frame, so a run ending one group and continuing into the next still forces a stuff bit

right at the boundary. (That is the chained-reload scenario, test 3 in the testbench: one group ends with four '1's, the next opens with a '1', and the stuff bit lands before that group's second real bit.)

The defaults next, and only these three. The pulse outputs fall at the start of every clock edge, the `bit_timer` idiom, so the branches below only *raise* them:

```
stuff      <= '0';
done       <= '0';
bit_valid <= '0';
```

`stuff_pending` does **not** belong here, however much it resembles them. It is state, not a pulse: armed at the shift completing a run, paid at the *next* shift, a whole bit period (fifty clock edges) later. Default it low here and the debt evaporates at the first idle edge in between, and the module then simply never stuffs anything. It is written in exactly three places: cleared at reset, armed inside `track_run`, and cleared by the branch that pays it.

14.3.3 The load branch

load = '1' presents the first bit immediately. A bit period begins the moment the previous group's last `bit_done` goes, which is also when the next group's `load` goes. If `load` merely latched the data and waited for the first shift pulse, the first period would still show the previous group's last bit, a whole bit late. So the branch puts out `data(7)` now, pre-shifts the register past it, sets `bits_left` to `bit_count - 1`, and then runs the placed bit through the run tracker:

```
if (load = '1') then
  tx_bit      <= data(7);                -- Output the first bit immediately.
  shift_reg <= data(6 downto 0) & '0'; -- Remove the output bit and prepare the-
  -- next one.
  bits_left <= to_integer(unsigned(bit_count)) - 1; -- bit_count is 1-8 by --
  -- contract:
  --
  --                                -- 0 would underflow --
  -- bits_left here.
  bit_valid <= '1';                    -- Indicate that a valid bit has been --
  -- output.

  track_run(data(7));                  -- data(7) was just placed on the wire.
```

The call is neither optional nor a detail: `load` places a bit on the wire, so the tracker has to see it like any other. Leave it out and the run count runs one bit behind for the rest of the frame; test 1 in the testbench catches it at shift 5, where the stuff bit the five '1's earned fails to appear.

14.3.4 The shift branch

shift = '1' does exactly one of three things (never in the same cycle as load; that is why the branch is an `elsif`), tested in this priority order. First the stuff bit that is owed, if there is one:

```
elsif (shift = '1') then
  if (stuff_pending = '1') then
    tx_bit      <= not last_bit; -- The stuff bit inverts the run.
    stuff       <= '1';
    bit_valid   <= '1';
    stuff_pending <= '0';        -- The debt is paid.

    track_run(not last_bit);      -- A stuffed bit is a placed bit like any ↵
    ↳ other.
```

A stuff bit goes through the tracker exactly like a real one, and passing in `not last_bit` is what makes that come out right with no special case: inside the body the comparison `new_bit = last_bit` is then false by construction, so the call takes the `else` path and does precisely what the rule wants, restarting the run at 1 with the stuff bit's own value in `last_bit`. Nothing arms a new debt here either, since the arming sits in the branch that cannot be taken. The group itself makes no progress: `shift_reg` and `bits_left` are left untouched.

Next the *closing* shift:

```
elsif (bits_left = 0) then
  done <= '1'; -- The closing shift changes nothing else.
```

Every bit has been put out and the last has now had its full period on the bus; `tx_bit` in particular keeps its value. Otherwise: send the next real bit, the same shape as the load branch with `shift_reg(7)` in `data(7)`'s place:

```
else
  tx_bit  <= shift_reg(7); -- Output the next bit.
  shift_reg <= shift_reg(6 downto 0) & '0';
  bits_left <= bits_left - 1;
  bit_valid <= '1';

  track_run(shift_reg(7)); -- May arm a stuff bit for the next ↵
  ↳ shift.
end if;
end if;
```

The priority order is load-bearing: a pending stuff bit comes before the closing shift, and that is what makes the special case below work with no extra code; a group whose *last* real bit

completes a run still gets its stuff bit sent before done goes. (load never has an outstanding stuff bit to worry about: can_controller loads only after done, and done comes after every pending stuff bit has been sent.)

14.4 The outputs' timing, the part the testbench pins down exactly

- tx_bit is set by **both** load and shift; the closing shift leaves it unchanged.
- stuff pulses only at the shift that actually puts out a stuff bit, never at load (a group's first bit may *trigger* a stuff bit, but that stuff bit is sent at a later shift).
- done does **not** pulse at the shift presenting the group's last real bit or stuff bit; that bit first needs its own full period on the bus. It pulses one shift later, at the closing shift, which presents nothing new. Letting done go early would let can_controller load the next group a whole bit period too soon.
- bit_valid is '1' at load and at every shift putting out a new bit (real or stuff), and '0' at the closing shift. Consumers passing tx_bit on bit by bit gate on bit_valid, not on "load or shift", or they would sample the closing shift's unchanged tx_bit as a duplicate. crc15 is the consumer that matters, and it needs stuff too: chapter 16 gates its enable on bit_valid and not stuff, since a stuff bit is put out on the wire but must never enter the CRC.
- Special case: if bit_count = 1 and that single bit triggers a stuff bit, done still waits for the stuff bit to be sent first.

14.5 One group, presentation by presentation

Four timing rules interact here, and prose is a poor way to see them. Below is a complete group: data = "11111000", bit_count = 8, loaded into a just-reset register. Every row is a *presentation*, either load or a shift pulse, and shows the outputs after it. This is the same sequence tx_shift_reg_tb checks in its first test, so your own module has to reproduce it row for row.

	data bit	tx_bit	stuff	bit_valid	done	consecutive
load	1	1	0	1	0	1
shift 1	2	1	0	1	0	2
shift 2	3	1	0	1	0	3
shift 3	4	1	0	1	0	4
shift 4	5	1	0	1	0	5
shift 5	-	0	1	1	0	1
shift 6	6	0	0	1	0	2
shift 7	7	0	0	1	0	3

shift 8	8	0	0	1	0	4
shift 9	-	0	0	0	1	4

Ten presentations to send eight bits. Each of the four rules shows up in it:

- **load presents immediately.** The first row already has data bit 1 on `tx_bit`, with `bit_valid` high. No shift has happened yet. Had load merely latched the group, this row would still show the *previous* group's last bit, and the whole frame would run one bit period late.
- **The run count drives everything.** It climbs from 1 to 5 over the five '1's, and at shift 4 it reaches five. That puts out no stuff bit; it *arms* one for the next presentation.
- **stuff is one presentation wide and carries no data.** Shift 5 drives not `last_bit`, marks `stuff = '1'` and resets the run to 1. The data-bit column shows -: the group has made no progress, which is why eight data bits need nine bit-placing presentations here. `bit_valid` is still '1', since a bit really was placed on the wire; it just was not one of yours.
- **done lags by one, and bit_valid marks the difference.** Shift 8 puts out the last real bit, and `done` stays low, since that bit has not yet had its bit period on the bus. Shift 9 is the closing shift: `tx_bit` holds its previous value, nothing new is put out, `bit_valid` falls to '0', and only now does `done` pulse.

The last row is the one worth staring at. `tx_bit` still reads '0', exactly as at shift 8, so anything gating on shift alone would feed `crc15` the same bit a second time and silently corrupt the checksum. `bit_valid` is the only output distinguishing "a bit was placed" from "a period elapsed", which is why it exists and why chapter 16 gates the CRC on it.

Finally, note that consecutive ends at 4 **on a run of '0's**, and is **not** cleared. The next load inherits both the count and `last_bit`, so if that group opens with a '0' the run reaches five at its very first bit and a stuff bit is forced before its second. If it instead opens with a '1' the run is broken and the count starts over at 1. That is the same mechanism as the chained reload in the testbench's test 3, where a group instead inherits a run of four '1's and opens with a '1'.

14.6 What the testbench pins down, and putting the module in

- "11111000" gives a stuff bit ('0') immediately after the fifth '1'; a run of exactly four is **not** stuffed.
- Loading "10101010" straight after a group that ended with four '1's (with no reset in between) forces a stuff bit before this group's second real bit, which proves that the run state survives load.
- `bit_valid` is '1' at load and at every real-bit or stuff-bit shift, '0' at the closing shift; `done` pulses only at that closing shift.

- `bit_valid` is back at '0' the cycle *after* every presentation, so a version holding the level fails even if it puts out every bit correctly.

14.6.1 Putting it into `can_controller`

The third block, the same edit. The signal group in `controller/can_controller.vhd` is larger here, since `tx_shift_reg` has more to say:

```
-- tx_shift_reg.
signal txsr_load           : std_logic;
signal txsr_data           : byte_t;
signal txsr_bit_count      : std_logic_vector(3--
  ↳ downto 0);
signal txsr_shift          : std_logic;
signal txsr_tx_bit, txsr_stuff, txsr_done, txsr_bit_valid : std_logic;
```

```
tx_shift_reg1: entity work.tx_shift_reg
  port map(clock, reset_s2_n, txsr_load, txsr_data, txsr_bit_count, --
  ↳ txsr_shift,
          txsr_tx_bit, txsr_stuff, txsr_done, txsr_bit_valid);
```

One single instance, reused for every stuffed field in every frame. If that still looks surprising, reread the chapter's introduction: the register is reloaded with a new byte and a new `bit_count` per field, which is exactly what makes one copy enough. (Take the `reset_s2_n` connection as it is for now; chapter 17 swaps it for a per-frame reset, for a reason only the arbitration can show.)

14.7 Summary

What you should be able to do now

- Implement a shift register serialising MSB first and inserting stuff bits per the five rule.
- Explain why the stuffing counter must not be cleared when a new group is loaded.
- Distinguish `bit_valid` from `bit_done`, and say which of them the CRC engine should be gated on.
- Say what stuff is for, and why it is needed outside the module.

Questions to test yourself with

1. Why must the first bit of a newly loaded group be out straight away, instead of at the next shift?
2. Why is the stuffing counter not cleared on reload?
3. What would go wrong if `can_controller` fed the CRC engine on `bit_done` instead of on `bit_valid` and not stuff?
4. A stuff bit always has the opposite value to the five preceding ones. Why is “insert a zero” not enough?
5. Why does `done` wait one extra shift instead of going high straight away when the last data bit has been shifted out?

Target

About halfway through the project. Three of the four leaf modules are usually finished here, and it is also about now that it becomes clear which group actually reviews each other’s code and which one just clicks “Approve”.

Looking ahead

Chapter 15 builds `rx_shift_reg`, the receive side’s mirror image: deserialisation, destuffing and flagging violations of the bit stuffing rule. Chapter 16 then puts both registers to work inside `can_controller`.

14.8 Exercises

These exercises consolidate the chapter. Design your own `tx_shift_reg.vhd` from the specification in section 14.1 to 14.5; the testbench handed out, run per appendix A, is the check that it behaves correctly.

Exercise 14.1 `tx_shift_reg`

VHDL

Write your own `tx_shift_reg.vhd` from the chapter. Verify it:

```
cd controller
ghdl -a --std=93 can_def.vhd tx_shift_reg.vhd tx_shift_reg_tb.vhd
ghdl -e --std=93 tx_shift_reg_tb
ghdl -r --std=93 tx_shift_reg_tb --assert-level=error
```

If a check fails, read the assertion message; it names exactly which shift it concerns and what was expected.

Exercise 14.2 Trace a stuffed sequence by hand, then in simulation**SIMULATION**

a) Trace by hand `tx_shift_reg` loaded with `data = "11100000"`, `bit_count = 8`, starting at a group boundary where the previous group's last bit was 0 with a run length of 3 (that is, `consecutive = 3` and `last_bit = '0'` going into the group). Write out, bit by bit: what is presented at load, and then at every subsequent shift, with every inserted stuff bit marked and stating which shift raises done.

b) Confirm your hand trace in simulation: write a short testbench (or reuse the structure of `tx_shift_reg_tb.vhd`) that preloads the same starting conditions with run length 3, then loads "11100000" and at every shift asserts `tx_bit`, `stuff` and `done` against the values you predicted (or reports them so that you can read them). Does its pass-or-fail outcome agree with your hand trace?

Exercise 14.3 The one-bit group that still owes a stuff bit**SIMULATION**

Section 14.4's last point names an edge case without working it through: *if `bit_count = 1` and that single bit triggers a stuff bit, `done` still waits for the stuff bit to be sent first*. That is not hypothetical; `can_controller` loads one-bit groups (SOF) and four-bit groups (the ID's lower half plus RTR), so a group really can end mid-run.

Set it up exactly as tests 2 and 3 in `tx_shift_reg_tb` do, with no reset in between: load "00001111" with `bit_count = 8` and shift it out completely, so that the run counter stands at four '1's in a row. Then load a **one-bit group** whose single bit is also '1' (`data = "1-----"`, `bit_count = 1`).

a) Trace what happens, presentation by presentation: what load puts on `tx_bit`, what the run counter reaches, and then what every following shift does. Mark which presentation carries the stuff bit and which raises done.

b) How many shift pulses does this one-bit group consume in total, and how many would it have consumed if the run counter had instead stood at zero? Say where the difference comes from.

c) Suppose `done` had been allowed to pulse as soon as the group's single real bit was presented, with no regard for the pending stuff bit. `can_controller` would then load the next group during the next bit period. Describe exactly what the receiving node would then see on the wire, and which of `rx_shift_reg`'s outputs would eventually report it (chapter 15).

d) Confirm your walkthrough in simulation, by extending the pattern from test 2 and test 3 in `tx_shift_reg_tb.vhd` with this one-bit group as a fourth case.

Exercise 14.4 Where `track_run` is called from

SIMULATION

Section 14.3.1 puts the run counter in a procedure and is explicit about where that procedure may be called from: from a branch placing a bit, never from the process's top level. Both halves of that rule are worth breaking deliberately, in a scratch copy of your module.

a) Move the call `track_run(data(7));` out of the `load` branch, so that `load` latches the group and presents its first bit but the counter never sees that bit. Predict what the run counter then shows for the rest of the frame, and which of the testbench's three tests fails first. Run it.

b) Now put a single `track_run(tx_bit);` at the top of the `elsif (rising_edge(clock))` branch, above the defaults, and remove all three calls inside the branches. `tx_bit` is an out port, so start by saying why that does not even compile, and then repeat the experiment with an internal copy of the transmitted bit in its place. With `TICKS_PER_BIT` set to 50, how many times does the counter now run per CAN bit, and what does consecutive do on the second clock edge of the very first bit period?

c) Section 14.3.1 also says that the procedure has to be declared *inside* the process rather than in the architecture. Move it out to the architecture's declarative part, unchanged, and analyse the file. It does not compile; read the four errors before you go on. Then make an architecture-level version that *does* compile, with identical behaviour and a passing testbench. Compare the two call sites and say what the second version costs, and which new question it raises that the inside-the-process version cannot raise.

Receive shift register and destuffing

The receive side's mirror image: deserialisation, removal of stuff bits, and flagging of stuffing errors.

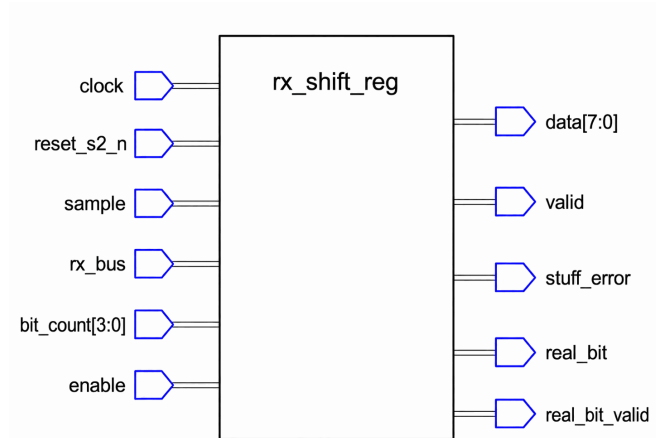
IN THIS CHAPTER

- `rx_shift_reg`: deserialisation, destuffing and detection of stuffing errors.
- Sampling mid bit period (`bit_timer.sample`) rather than at `bit_done`.
- `real_bit` and `real_bit_valid`: feeding `crc15` one destuffed bit at a time.
- Instantiating `rx_shift_reg` inside `can_controller`.

`rx_shift_reg` is the receive side's mirror image of chapter 14's `tx_shift_reg`: it deserialises a group MSB first from the bus and **discards** the stuff bits `tx_shift_reg` inserted, and flags a violation if an expected stuff bit fails to appear. The run counting and the rule about five identical bits in a row are identical, only reversed — chapter 3 of *CAN: the bus, the frame and the controller* is the same rule seen from the protocol, and it is worth reading once more from here, since it is the one rule in the book that two modules have to agree on independently. `can_controller` (chapter 17) reuses **one single** `rx_shift_reg` for every stuffed field of an incoming frame, changing its `bit_count` between groups; the unstuffed tail (CRC delimiter, ACK, ACK delimiter, EOF) it reads directly, past this register.

This chapter also contains the project's **compulsory code review seminar**; see appendix C.

15.1 Interface



Port	Dir.	Type	Meaning
clock, reset_s2_n	in	std_logic	50 MHz clock, synchronised active-low reset.
sample	in	std_logic	One pulse per CAN bit (bit_timer.sample) marking when rx_bus is to be read.
rx_bus	in	std_logic	The bus level, valid at the sample point.
bit_count	in	std_logic_vector(3 downto 0)	Number of real bits to collect, 1–8; read live, may change between groups.
enable	in	std_logic	Decides whether sample pulses are acted on.
data	out	byte_t	The collected group, right-justified in its lowest bit_count bits.
valid	out	std_logic	Pulses when bit_count real bits have been collected.
stuff_error	out	std_logic	Pulses if an expected stuff bit does not invert.
real_bit	out	std_logic	The single destuffed bit accepted at this sample.
real_bit_valid	out	std_logic	'1' at a sample that accepted a real bit; '0' at a discarded stuff bit.

data and real_bit are levels; valid, stuff_error and real_bit_valid are **one-cycle pulses**, defaulted to '0' at the top of the clocked process, exactly like tx_shift_reg's. real_bit_valid is the one that bites: can_controller gates crc15's enable directly on it (chapter 17), so a level lingering until the next sample feeds the same bit in for a whole bit period and silently destroys the CRC. The testbench checks that it has fallen again the cycle after every sample, for exactly that reason.

There is no load: the sampling happens mid-period (sample, not bit_done), once every node's signals have had time to settle. A transmitter changes the bus at a period boundary,

so a receiver also acting on `bit_done` would read at the very moment the level changes; section 12.2.3's tick trace shows the two pulses and the 14 ticks between them. `can_controller` only changes `bit_count` between groups; `valid` itself reports every group's completion. `rx_shift_reg_tb` binds **positionally**, so the order and the types above are what has to match; the names are yours.

15.2 What the module has to remember

As in chapter 14: list the state before any process is written. The receive side has the same two jobs, mirrored — keeping a group *filling up*, and keeping the bit stuffing rule honest — plus one it did not have: *predicting* where the transmitter must have inserted a stuff bit. Every rule below speaks in terms of exactly these five signals:

```

signal data_reg      : byte_t;                -- Collected real bits, filling-
    ↳ from the right.
signal collected     : natural range 0 to 8;   -- Real bits accepted toward ↳
    ↳ this chunk so far.
signal last_bit      : std_logic;             -- The bit most recently ↳
    ↳ sampled off the bus.
signal consecutive   : natural range 0 to MAX_RUN; -- Current run length; 0 = none-
    ↳ .
signal stuff_expected: std_logic;             -- The next sampled bit must be-
    ↳ a stuff bit.

```

- **data_reg** and **collected** are the **group filling up**. Every accepted bit is shifted in from the right (`data_reg(6 downto 0) & rx_bus`), which is where the right-justification comes from, and `collected` counts accepted bits towards `bit_count`. The register behind `data` has to be this internal signal, with `data <= data_reg`; as a concurrent assignment outside the process: an out port cannot be *read*, and a shift is a read-modify-write.
- **last_bit** and **consecutive** are the **same run tracker** as chapter 14's, character for character: the bit most recently on the wire, and how long the identical run it ended is. Both sides of the link run the same tracker over the same bits, against the same `MAX_RUN` from `can_def`; that is what keeps them agreeing on where the stuff bits belong. As in chapter 14, the pair is written from a single procedure the process declares.
- **stuff_expected** is the **mirror image** of chapter 14's **stuff_pending** — the same flag with the opposite job. The transmitter's flag says "I owe a stuff bit next"; the receiver's says "the next bit had better *be* one". The same arming condition (a run reaching five), the same timing (checked at the next event), the opposite direction of obligation — and the receiver's version can be *disappointed*, and that is where `stuff_error` comes from.

The outputs are registered in the same process: `data` and `real_bit` are levels; `valid`, `stuff_error` and `real_bit_valid` are the one-cycle pulses section 15.1 described.

15.3 The process, top to bottom

One clocked process holds all of the above, plus one concurrent assignment outside it. As in chapter 14, the walkthrough follows the code in the order it is written. **The run tracker** is line for line chapter 14's, with only the flag renamed after what it now means: the same procedure, declared inside this module's process and called from every branch taking a bit off the wire but one, and the exception is noted where it arises. As in chapter 14, collected is compared against bit_count via `to_integer(unsigned(...))`, so the file needs use `ieee.numeric_std.all`;

The skeleton. The concurrent line first: data is an out port and cannot be *read*, while a shift is a read-modify-write, so the register is the internal data_reg and the port merely mirrors it. Then the process, with its procedure declared before begin, asynchronous reset first and everything else on the rising edge:

```
data <= data_reg; -- Concurrent, outside the process.

process(clock, reset_s2_n) is
    -- One procedure, the run tracker (below).
begin
    if (reset_s2_n = '0') then
        -- Clear everything.
    elsif (rising_edge(clock)) then
        -- Defaults, then the sample branch.
    end if;
end process;
```

15.3.1 The run tracker, written once

Chapter 14's procedure with `stuff_expected` in place of `stuff_pending`; that single rename is the whole difference on the receive side. Everything section 14.3.1 says about the idiom (why it reads the signals directly, why it is declared inside the process rather than in the architecture, why an `in` parameter preserves the pre-edge timing, and why synthesis inlines it) applies here unchanged:

```
-- The run tracker: runs on the bit just taken off the wire.
procedure track_run(new_bit: in std_logic) is
begin
    if ((new_bit = last_bit) and (consecutive /= 0)) then
        consecutive <= consecutive + 1;
        if (consecutive = MAX_RUN - 1) then -- Run reaches MAX_RUN.
            stuff_expected <= '1';
        end if;
```

```

else
    consecutive <= 1;           -- Start a new run.
end if;
last_bit <= new_bit;
end procedure;

```

15.3.2 Reset and defaults

Reset clears everything: the group (data_reg, collected), the run tracker (last_bit, consecutive, stuff_expected) and the outputs. This is the **only** place the tracker is ever cleared. The groups do not clear it: the transmitter's stream is continuous over the whole frame, so the receiver's picture of it has to be as well.

The defaults next, and only these three, so that the branches below only *raise* them:

```

valid          <= '0';
stuff_error    <= '0';
real_bit_valid <= '0';

```

stuff_expected does **not** belong here, for exactly the same reason stuff_pending did not belong in chapter 14's defaults: it is state, armed at the sample completing a run and settled at the *next* sample, a whole bit period (fifty clock edges) later. Default it low and the prediction evaporates before it can fall due, and then stuff bits are neither discarded nor checked. It is written in exactly three places: cleared at reset, armed inside track_run, and cleared when the prediction falls due (both outcomes below).

15.3.3 The sample branch

(sample = '1') and (enable = '1') is the **only working branch**; there is no load on the receive side. It does exactly one of three things, tested in this priority order. First: an outstanding prediction falls due, and the sampled bit settles it one way or the other:

```

if ((sample = '1') and (enable = '1')) then
    if (stuff_expected = '1') then
        if (rx_bus /= last_bit) then
            stuff_expected <= '0';    -- The stuff bit arrived; prediction settled.
        else
            track_run(rx_bus);        -- A stuff bit is a bit off the wire like
            -- any other.
        end if;
    else
        stuff_error    <= '1';    -- Six identical bits: a stuffing violation.
    end if;
end if;

```

```

stuff_expected <= '0';
consecutive    <= 1;      -- Restart the run at the violating bit; the
                           -- tracker must never count past MAX_RUN.

end if;

```

The arrived stuff bit is *discarded*: no shift, no change to collected, and `real_bit_valid` stays '0', since a stuff bit is never a real bit (this is the bit `crc15` must not see). The call to the tracker is the mirror image of chapter 14's in its stuff branch: a discarded stuff bit resyncs the tracker exactly as a transmitted one does, since the wire does not distinguish real bits from stuff bits and the tracker must not either. Here it needs no special argument, since `rx_bus /= last_bit` is this branch's own condition, so `track_run(rx_bus)` takes the new-run path by construction and restarts the count at 1.

In the else branch, six identical bits in a row are impossible on a correctly stuffed bus, so `stuff_error` pulses. The cleanest thing for the rest is to clear the flag, accept nothing, and restart the run count at the violating bit (`last_bit` already holds its value), since it is not this module's job to decide what a violation *means*. The restart is not optional tidiness: without it `consecutive` stays at `MAX_RUN`, the next identical bit runs the tracker's + 1, and the signal's declared range is exceeded; the testbench feeds in a bit past the violation to pin down exactly that. This is also the one placement `track_run` cannot serve, and for the same reason: `rx_bus = last_bit` here, so the call would take the + 1 path with `consecutive` already at `MAX_RUN`. The violating bit is the exception to the rule the procedure encodes, so its two lines stay written out. `can_controller` treats it as fatal and aborts the reception (chapter 17), exactly as a real controller would.

Otherwise: accept a real bit, shift it in from the right, expose it bit-wise to `crc15`, complete the group if this bit completes it, and run the tracker on the bit just accepted:

```

else
  data_reg      <= data_reg(6 downto 0) & rx_bus; -- Shift in from the
-- right.
  real_bit      <= rx_bus;
  real_bit_valid <= '1';

  if (collected = to_integer(unsigned(bit_count)) - 1) then
    valid      <= '1';      -- This bit completes the chunk.
    collected <= 0;
  else
    collected <= collected + 1;
  end if;

  track_run(rx_bus); -- rx_bus was just accepted off the wire.
end if;
end if;

```


The comparison here is chapter 14's pre-edge signal read: collected against `bit_count - 1` is the "run-is-now-full" idiom applied to the group's completion, the same shape as `consecutive = MAX_RUN - 1` inside `track_run`. Nothing else happens at a group boundary: `bit_count` is read live, so the next group simply begins with the next accepted bit at whatever width `can_controller` has set by then, and `data_reg` is *not* cleared; the new group's bits are shifted in beneath whatever the old group left above them (Exercise 15.3 is about exactly that).

Two mirror notes worth stopping at:

- **There is no closing sample.** Chapter 14's done had to lag by one shift, since a sent bit still needed its full period on the bus. A receiver samples mid-period, once the bit has already had its settling time, so the moment the last real bit is accepted the group is simply *there*: valid goes at that very sample, with nothing to wait for.
- **data/valid are per group, real_bit/real_bit_valid per bit.** A whole group is too coarse for `crc15`, which needs every real bit as it arrives; `real_bit` and `real_bit_valid` expose exactly the single accepted bit, the receive side's counterpart to `tx_shift_reg`'s pair `bit_valid/stuff`. Counterpart, not mirror image: on the transmit side stuff bits still carry `bit_valid = '1'` and the caller masks them out with `stuff`, while the discarded stuff bit here never raises `real_bit_valid` at all, so `real_bit_valid` alone is already the CRC gating (chapter 17).

15.4 One group, sample by sample

Receiving is not simply chapter 14 backwards. A transmitter *decides* to insert a stuff bit; a receiver *predicts* one and then has to check whether it actually came. That difference is where both of this module's interesting outputs come from.

Here is the wire sequence `tx_shift_reg` produced for "11111000" in section 14.5, arriving at a just-reset `rx_shift_reg` with `bit_count = 8`. Every row is a sample pulse. The run column is the number of identical bits in a row after the sample, and data shows the register filling from below.

	rx_bus	action	real_bit	real_bit_valid	run	data	valid
sample 1	1	accept	1	1	1	00000001	0
sample 2	1	accept	1	1	2	00000011	0
sample 3	1	accept	1	1	3	00000111	0
sample 4	1	accept	1	1	4	00001111	0
sample 5	1	accept	1	1	5	00011111	0
sample 6	0	discard	-	0	1	00011111	0
sample 7	0	accept	0	1	2	00111110	0
sample 8	0	accept	0	1	3	01111100	0
sample 9	0	accept	0	1	4	11111000	1

Nine samples to recover eight bits, exactly the nine presentations chapter 14's transmitter needed to send them. Three rows carry the lesson:

- **Sample 5** takes the fifth '1' in a row. Nothing is discarded here. Reaching five only means that the *next* bit is expected to be a stuff bit; this bit is real and is collected as usual.
- **Sample 6** is the discarded stuff bit. `real_bit_valid` falls to '0' and data does not move: the group makes no progress. This is the row that matters to chapter 13's engine, since `crc15` must not see this bit. The run resyncs to 1 and takes the stuff bit's own value as the start of the new run.
- **Sample 9** completes the eighth *real* bit, so valid pulses and data reads 11111000, the original group. The stuff bit has vanished without `can_controller` ever finding out it existed.

Note that data fills from the right, one position per accepted bit. After nine samples the group happens to occupy all eight bits, but a 6-bit group would end up with its bits in `data(5 downto 0)` and leave whatever sits above alone. That is the right-justification, and Exercise 15.3 is about what sits above it.

When the stuff bit does not come. The same mechanism, one bit different. Six '0's in a row where the sixth should have inverted:

	<code>rx_bus</code>	expected	action	<code>real_bit_valid</code>	run	<code>stuff_error</code>
sample 1	0	real bit	accept	1	1	0
sample 2	0	real bit	accept	1	2	0
sample 3	0	real bit	accept	1	3	0
sample 4	0	real bit	accept	1	4	0
sample 5	0	real bit	accept	1	5	0
sample 6	0	stuff '1'	violation	0	1	1

The violation is detected at sample 6, not sample 5. Five identical bits are perfectly legal and are even required before any stuff bit exists at all; it is the sixth, arriving where an inverted bit was owed, that is impossible on a correctly stuffed bus. `stuff_error` pulses for one cycle and the module goes on collecting; deciding what a violation *means* belongs to `can_controller`, which aborts the frame (chapter 17).

15.5 What the testbench pins down

- The wire sequence for "11111000" (stuff bit after the fifth '1') is destuffed back to "11111000", valid pulses at the last sample, and `real_bit_valid` is '0' at the discarded stuff bit and '1' (with `real_bit = rx_bus`) at every real bit.
- `real_bit_valid` is back at '0' the cycle *after* every sample, so a version holding the level fails even if it destuffs perfectly.

- Six '1's in a row raise `stuff_error` at the sixth sample, and a seventh identical bit is accepted as usual afterwards: the run count restarted at the violating bit instead of counting past `MAX_RUN`.
- A 6-bit group pulses valid at its sixth sample, not its eighth, and lands right-justified in `data(5 downto 0)`.
- A 3-bit group fed in immediately afterwards, **with no reset in between**, picks up the new `bit_count` and lands in `data(2 downto 0)`. This is the group-to-group resizing `can_controller` performs between every field.

15.6 Putting it into can_controller

The last of the four CAN blocks, and the first whose port map connects to something other than the clock and its own signals:

```
-- rx_shift_reg.
signal rxsr_bit_count      : std_logic_vector(3 downto 0);
signal rxsr_enable        : std_logic;
signal rxsr_data          : byte_t;
signal rxsr_valid, rxsr_stuff_error : std_logic;
signal rxsr_real_bit, rxsr_real_bit_valid : std_logic;
```

```
rx_shift_reg1: entity work.rx_shift_reg
  port map(clock, reset_s2_n, bt_sample, rx_bus_s2, rxsr_bit_count,
    rxsr_enable,
      rxsr_data, rxsr_valid, rxsr_stuff_error, rxsr_real_bit,
    rxsr_real_bit_valid);
```

Two of the connections there are worth stopping at. `sample` comes straight from `bit_timer1`'s output, so the receive path samples on the same 70 % tick the rest of the design times against; nothing in between decides when the bus is read. And the bus level is `rx_bus_s2`, the output of chapter 12's `rx_bus_sync` instance, **not** the `rx_bus` port itself. `rx_shift_reg` is the only block that reads the bus, and it reads the synchronised copy; nothing in the design reads the raw pin. (As with `tx_shift_reg`: take the `reset_s2_n` connection as it is for now; chapter 17 moves both shift registers to a per-frame reset.)

With this, `can_controller`'s architecture holds all four CAN subblocks and their signals, alongside chapter 12's `meta_prev`, and no frame behaviour at all. Every remaining connection between them is chapter 16's state machine.

15.7 Summary

What you should be able to do now

- Implement a shift register deserialising from the bus and discarding stuff bits.
- Detect a stuffing error and say exactly what condition constitutes one.
- Explain why the receiver samples at `sample` and the transmitter shifts at `bit_done`.
- Feed the CRC engine from the receive side without counting stuff bits.
- Read somebody else's VHDL and write a review that is useful rather than polite.

Questions to test yourself with

1. Why does the receiver sample at `sample` rather than at `bit_done`?
2. What condition exactly constitutes a stuffing error, and why is it six identical bits that is the limit on the receive side when the transmitter counts five?
3. What happens to the stuff bit when it is correct?
4. Why may `crc15` only see `real_bit`, and what would a wrongly fed stuff bit do?
5. Why is counting received bits not enough to know when a byte is complete?

The code review seminar

About an hour, with the whole class. Every group gets ten minutes and shows a merged pull request (what it changed, how it was split up, and what the commit messages say), a review the group wrote (what the reviewer looked for, and what actually changed as a result of the comments), and one thing that has not worked in the way of working, and what you have done about it.

Pick out, within the group, one merged pull request and one review somebody in the group wrote to show. It does not have to be the prettiest one; a PR that got justified criticism is worth more showing than one nobody read.

The seminar is compulsory and is one of the project's two fixed checkpoints. It is not graded separately, but it feeds into the overall assessment of the way of working described in appendix C.

Target

With `rx_shift_reg` finished, all four leaf modules exist, and `can_controller` is the only one left on the CAN side. Groups that are behind should spend the time here rather than starting on the state machine: it needs all four.

Looking ahead

Chapter 16 takes the frame as a state machine, first explained and then built on the transmit side.

15.8 Exercises

These exercises consolidate the chapter. Design your own `rx_shift_reg.vhd` from the specification in section 15.1 to 15.4; the testbench handed out, run per appendix A, is the check that it behaves correctly.

Exercise 15.1 `rx_shift_reg`

VHDL

Write your own `rx_shift_reg.vhd` from the chapter. Verify it:

```
cd controller
ghdl -a --std=93 can_def.vhd rx_shift_reg.vhd rx_shift_reg_tb.vhd
ghdl -e --std=93 rx_shift_reg_tb
ghdl -r --std=93 rx_shift_reg_tb --assert-level=error
```

If a check fails, read the assertion message; it names exactly which sample it concerns and what was expected.

Exercise 15.2 A deliberate stuffing violation

SIMULATION

- a) Construct (on paper) a 6-bit sequence on the wire that constitutes a genuine violation of the bit stuffing rule: five identical real bits in a row, followed by a sixth identical bit where a stuff bit should have broken the run.
- b) Feed your sequence from a) into `rx_shift_reg` (either by extending `rx_shift_reg_tb.vhd` or by writing a short new testbench) and confirm that `stuff_error` pulses on exactly the sixth bit, not earlier.
- c) `rx_shift_reg` makes no attempt to recover or resynchronise after a `stuff_error`; it merely reports it for one cycle and goes on accumulating as if nothing had happened. Explain, in one or two sentences, why that is an acceptable design here, given how `can_controller` (chapter 17) will react to it.

Exercise 15.3 The right-justification, and what sits above it SIMULATION

Section 15.1 says that a completed group ends up **right-justified in its lowest bit_count bits**, and that `can_controller` relies on that when it slices up reconstructed fields. Tests 3 and 4 in the testbench handed out exercise it: a 6-bit group, then a 3-bit group, with no reset in between, with `bit_count` changed between the groups in exactly the way `can_controller` does between one field and the next.

Assume a just-cleared `rx_shift_reg`, `enable = '1'` and no stuff bits anywhere (no run ever reaches five). The process in section 15.3 shifts every accepted bit into the register's least significant end and never clears the register between groups; assume yours does the same. The testbench handed out does not actually require that, and part d) returns to why.

- a) With `bit_count = 6`, feed in the six bits 0 1 0 1 0 1 (in that order, one per sample). Write out all eight bits of data when valid pulses. What is in `data(7 downto 6)`, and why?
- b) Set `bit_count = 3` without clearing, and feed in 0 1 1. Write out all eight bits of data when valid pulses again. What is now in `data(7 downto 3)`?
- c) Your answer to b) shows that the bits above `bit_count` hold residue from the *previous* group. Explain why that is harmless here, and what `can_controller` has to do (and must never do) when it reads `rxsr_data` for a field narrower than 8 bits.
- d) `rx_shift_reg` holds two rather different kinds of state: the shift register accumulating the current group, and the run counter (consecutive, `last_bit`) deciding where stuff bits belong. Clearing them is not equally safe in both cases.
 - Suppose the module cleared its **shift register** at every valid. Which of your answers above would change, and would any field ever be *misread* as a result? Note that the testbench handed out checks only the lowest `bit_count` bits of data, so both designs pass it.
 - Suppose it cleared the **run counter** at the same moment. Name what breaks, and say which rule from chapter 14 that mirrors.

The difference between the two answers is why section 15.3.2 says the run counter is cleared only by reset and says nothing at all about the shift register.

The CAN controller I: the state machine and the transmit path

The frame as a state machine, first explained and then built on the transmit side.

IN THIS CHAPTER

- The frame-sequencing state machine as a concept: the state diagram, the field table and the STATE_LOAD_*/shift pattern, before any VHDL is written.
- Adding the state type and the machine's signals to `can_controller`, and then writing its transmit path.
- Driving the open-drain bus from the shift register's current bit.
- Feeding `crc15` correctly: gate on `bit_valid`, not on `bit_done`.

`can_controller` is the only block that knows the actual CAN frame format (chapter 10). It instantiates `bit_timer`, `crc15`, `tx_shift_reg` and `rx_shift_reg` internally and sequences a whole frame field by field. This chapter builds the **transmit path** (a node sending a complete frame); chapter 17 adds reception and arbitration.

The entity and its fifteen ports were written in chapter 11 and are unchanged; see section 11.3 for what each is for. Nothing in this chapter or in chapter 17 changes them: the whole of `can_controller`'s transmit and receive behaviour is built *inside* an interface that has stood still since chapter 11.

Two reminders that matter to the state machine below:

- `can_controller_tb` binds **positionally**, so the port order settled in chapter 11 is what has to match; the names are yours.
- `tx_bus` and `bus_en` go together. Driving the bus dominant means `bus_en = '1'` with `tx_bus = '0'`; releasing it means `bus_en = '0'`. A node never drives recessive actively.

This is the project's largest single piece of coding and the one part that is hard to split between several people. A reasonable arrangement is two writing and the rest reviewing closely, rather than four writing one part each of the same process.

16.1 One node, two roles

A node either **transmits** its own requested frame or **receives** somebody else's, never both for the same frame. A registered role ('1' = transmitting, '0' = receiving), latched once per frame at the edge leaving STATE_IDLE for STATE_START and held until the frame is over, decides which of the subblocks' ports matter. Both ways out of STATE_IDLE require **a bus that has been idle**: with that established, tx_req = '1' starts a transmission (role = '1'), and otherwise a dominant rx_bus_s2 (somebody else's SOF) starts a reception (role = '0'). Follow the role = '1' path in this chapter.

Why “a bus that has been idle” does real work

The condition earns its place twice over, for different reasons at each of the two entrances.

At the receive entrance. A dominant bit by itself does not mean a frame is starting; most bits in most frames are dominant. STATE_IDLE has to tell the dominant bit that *opens* a frame from the many dominant bits inside one, and a node in STATE_IDLE has no frame context to tell them apart with.

Two tempting rules both fail:

- **A dominant level.** Fires on any dominant bit at all, so a node sitting in STATE_IDLE while another node transmits immediately tries to “receive” a frame that is already half over.
- **A recessive-to-dominant edge.** Sounds like section 10.4's description of SOF, but bit stuffing forces a transition after every fifth identical bit, so edges in both directions, the recessive-to-dominant one included, occur constantly in the middle of a frame. The edge test fires within a few bits of the level test.

The rule that actually works is a **count of recessive bit periods**, and its threshold takes one more step of care than the bit stuffing rule alone. Inside the stuffed region, section 10.5's rule guarantees no run longer than five identical bits. But the tail is unstuffed and almost entirely recessive: an acknowledged frame ends with the ACK delimiter plus EOF, eight recessive bit periods in a row, and a frame nobody acknowledges shows ten (CRC delimiter, recessive ACK slot, ACK delimiter, EOF). A threshold of six to ten would declare the bus idle while a frame's own tail was still passing over it, and a pending tx_req would then drive SOF out over somebody's last EOF bits.

Count recessive bit periods in STATE_IDLE and require **eleven**, the same number real CAN gives a node that has to judge whether the bus is idle with no frame context: the acknowledged tail of eight plus the standard's three-bit intermission, and past even the ten-bit tail of an unacknowledged frame. Only once the count is full does a dominant rx_bus_s2 mean SOF. Two consequences for the rest of the design:

- bit_timer has to keep running in STATE_IDLE, since the count is in bit periods.

- Reset has to start the count **full**. A node coming out of reset has no reason to believe a frame is in progress, and starting it empty would make it ignore the first frame on the bus.

At the transmit entrance. The same count answers a different question: may this node start driving the bus at all? Real CAN says a node may only start transmitting on an idle bus, and the reason is not politeness. Arbitration settles a dispute only between nodes that started at the *same* SOF, by comparing what each sent against what the bus reads, bit by bit, down through the identifier. A node starting in the middle of a frame never takes part in that contest; it merely drives dominant bits over somebody else's fields. The frame in progress is corrupted, and the damage turns up at the other end as a stuffing violation or a CRC error, with nothing pointing back at the node that caused it. So `tx_req` waits on the count too.

Together these conditions keep error observable. A node losing arbitration (chapter 17) falls back to `STATE_IDLE` while the winner is still driving the bus, and `STATE_START` clears error. Under either of the two failed receive rules above it would enter `STATE_START` again within a few bit periods and wipe the error it had just raised; with the transmit entrance ungated, a caller still holding `tx_req` high would wipe it the very next cycle. Gating both means the loser waits, correctly, for the next frame, and that a caller polling error now and then still sees the error.

16.2 The frame, field by field

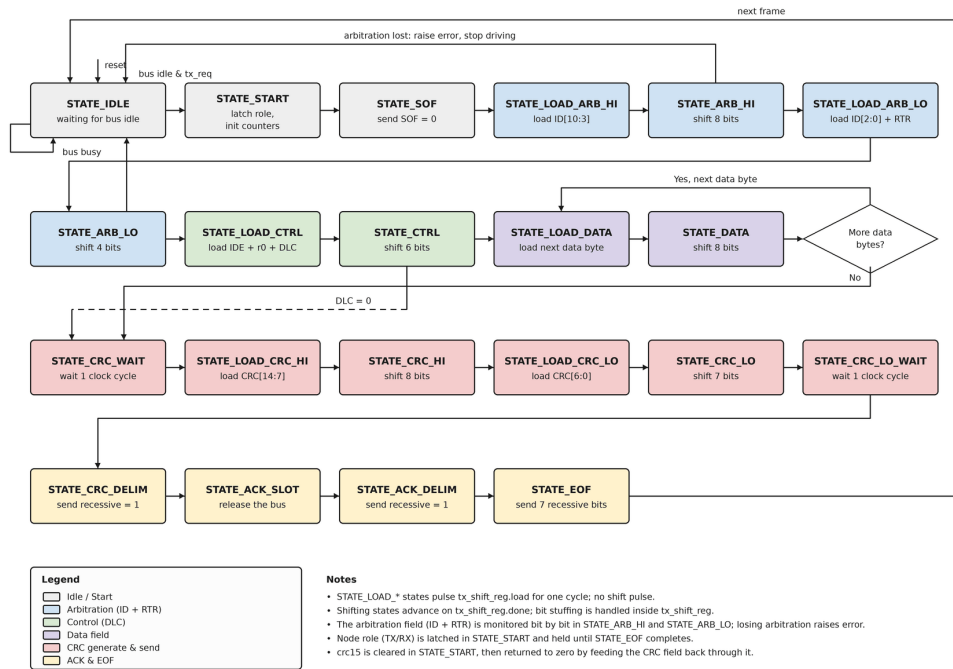


Figure 16.1 can_controller's transmit state machine.

Every **stuffed** field goes through tx_shift_reg as one group, sent MSB first. Every field gets **two** states: a one-cycle STATE_LOAD_* pulsing tx_shift_reg.load, and then the shift state waiting for tx_shift_reg.done.

State	Field	Bits	Value sent	Stuffed
STATE_START / STATE_SOF	SOF	1	dominant '0'	yes
STATE_LOAD_ARB_HI STATE_ARB_HI	/ ID high	8	tx_id(10 downto 3)	yes
STATE_LOAD_ARB_LO STATE_ARB_LO	/ ID low + RTR	4	tx_id(2 downto 0), then RTR='0'	yes
STATE_LOAD_CTRL STATE_CTRL	/ Control	6	IDE='0', r0='0', then 4-bit tx_dlc	yes
STATE_LOAD_DATA STATE_DATA	/ Data byte	8	current byte, repeated DLC times	yes
STATE_CRC_WAIT	(settling)	0	let crc15 settle, then latch it into crc_tx	no
STATE_LOAD_CRC_HI STATE_CRC_HI	/ CRC high	8	crc_tx(14 downto 7)	yes
STATE_LOAD_CRC_LO STATE_CRC_LO	/ CRC low	7	crc_tx(6 downto 0)	yes
STATE_CRC_LO_WAIT	(settling)	0	let crc_valid settle	no
STATE_CRC_DELIM	CRC delimiter	1	recessive	no
STATE_ACK_SLOT	ACK slot	1	recessive (the receiver drives dominant)	no
STATE_ACK_DELIM	ACK delimiter	1	recessive	no
STATE_EOF	EOF	7	recessive	no

Notes on the shift register's loads:

- tx_shift_reg.data is a byte_t, always 8 bits, and sends the top bit_count of them (chapter 14). A group narrower than a byte is therefore **left-justified and zero-padded** on load, and the padding is never sent. The table above names every group after its real bits; the padding is mechanical.
- The control byte is loaded as "00" & tx_dlc & "00" with bit_count = 6, so the six bits sent are the top six: 0, 0, DLC[3:0].
- The low CRC group is loaded as crc_tx(6 downto 0) & '0' with bit_count = 7.
- The low ID group is loaded as tx_id(2 downto 0) & '0' & "0000" with bit_count = 4: the identifier's three remaining bits, and then the always dominant RTR that ends the arbitration field.
- Data selects one byte out of tx_data per data_byte_idx; loop STATE_LOAD_DATA/STATE_DATA once per byte, DLC times (jump straight to STATE_CRC_WAIT when DLC = 0).
- tx_dlc above 8 is outside the contract: section 10.4's DLC encodes 0 to 8, and the byte loop runs outside data_byte_idx's range for anything larger. Real controllers clamp the encodings 9 to 15 to eight bytes; this design simply requires the caller not to send one.

The **unstuffed** closing fields (CRC delimiter, ACK slot, ACK delimiter, EOF) do not go through `tx_shift_reg`; they are fixed bit counts driven directly, one bit per `bit_timer.bit_done`, identically for both roles.

16.3 What you add to the architecture

`can_controller.vhd` already holds the entity (chapter 11), the `meta_prev` synchroniser (chapter 12) and all four CAN subblock instances with the signals they connect to (chapter 12 to 15). It holds no frame behaviour at all. Everything below is new in this chapter.

The state type and the state machine's own signals belong in the architecture's declarative part, above the four subblocks' signal groups:

```
-- One state per frame field. Every stuffed field is a LOAD/shift pair; the
-- trailer fields (CRC delimiter, ACK slot/delim, EOF) are driven directly.
type state_t is (
    STATE_IDLE,
    STATE_START,
    STATE_SOF,
    STATE_LOAD_ARB_HI, STATE_ARB_HI,
    STATE_LOAD_ARB_LO, STATE_ARB_LO,
    STATE_LOAD_CTRL,   STATE_CTRL,
    STATE_LOAD_DATA,   STATE_DATA,
    STATE_CRC_WAIT,
    STATE_LOAD_CRC_HI, STATE_CRC_HI,
    STATE_LOAD_CRC_LO, STATE_CRC_LO,
    STATE_CRC_LO_WAIT,
    STATE_CRC_DELIM,
    STATE_ACK_SLOT,
    STATE_ACK_DELIM,
    STATE_EOF);

signal state : state_t;

-- '1' while transmitting this frame, '0' while only receiving. Latched per ↵
-- frame.
signal role : std_logic;

-- Consecutive recessive bit periods seen in STATE_IDLE, saturating at 11. ↵
-- Only once this is
-- full may the node leave STATE_IDLE at all. Resets to 11, not 0.
signal idle_bits : natural range 0 to 11;

-- Data-byte index, and a countdown for the fixed-width trailer fields.
```

```

signal data_byte_idx    : natural range 0 to 8;
signal field_bit_count : natural range 0 to 7;

-- Accumulators for the frame being received.
signal rx_id_acc    : id_t;
signal rx_dlc_acc   : dlc_t;
signal rx_data_acc  : data_t;

-- This frame's own CRC, captured at STATE_CRC_WAIT before the engine is fed
-- the CRC field's own bits and moves on.
signal crc_tx : crc_t;

-- The data byte selected by data_byte_idx, MSB-first (byte 0 first).
signal tx_data_shifted : data_t;
signal tx_current_byte : byte_t;

```

The three rx*_acc accumulators are not used until chapter 17; declaring them with the rest keeps the group together instead of splitting it across two chapters.

Two concurrent assignments pick out the data byte currently being sent. They belong after begin, alongside the subblock instances:

```

tx_data_shifted <= std_logic_vector(shift_left(unsigned(tx_data), 8 * ↵
↵ data_byte_idx));
tx_current_byte <= tx_data_shifted(63 downto 56);

```

shift_left and unsigned come from numeric_std, so add use ieee.numeric_std.all; to the context clause if it is not already there.

Everything else in this chapter is the two processes you write: a **combinational** one driving the subblocks per state, and a **sequential** one advancing the state machine and updating the bookkeeping.

Which process drives what. The split is worth settling before the walkthrough, since every line in it lands in one of the two processes, and which one should be no mystery:

- The **combinational** process drives every input on the subblocks (bt_enable, bt_resync, the four txsr_* inputs, the two rxsr_* inputs in chapter 17) and the bus pair tx_bus/bus_en. Give them all a default at the top and override per state, in the same style as bit_timer's own branches.
- bt_enable is '1' in **every** state, STATE_IDLE included, since the count in STATE_IDLE is measured in bit periods. The port exists so that bit_timer can be held, and this design never wants to.
- txsr_shift is bt_bit_done, but only while transmitting and only in the **shift states**, never in a STATE_LOAD_* state. That is what keeps chapter 14's promise that "load and shift are mutually exclusive": a load state issues a load and nothing else.

- The **sequential** process owns state and the bookkeeping (role, idle_bits, data_byte_idx, field_bit_count, crc_tx, the rx*_acc accumulators) and every register-side output port: tx_done, rx_valid, error and the rx_* registers. The bus pair stays combinational, driven per state as above. tx_done and rx_valid are one-cycle pulses, so default them to '0' at every edge and raise them where they belong.
- error is cleared by reset_s2_n as well as by STATE_START. It is a level a caller polls, so it has to read '0' out of reset rather than whatever a flip-flop happened to power up as.
- tx_id, tx_dlc and tx_data are **not** latched at STATE_START; the load states read them live, each at the moment its group is loaded. That is a deliberate simplification and it puts the obligation on the caller: hold all three stable from tx_req to tx_done. A register block in the same clock domain does that for free, and that is precisely the caller section 11.3 assumes.

16.4 The combinational process

It drives the subblocks' inputs and the bus pair from state, with the defaults first and the per-state overrides after, and it is compact enough to show whole. Three things to see in it before the code. Every STATE_LOAD_* arm loads its group and shares an arm with the paired shift state, since the pair also shares a bit_count. txsr_shift is bt_bit_done only in the shift states, never in a STATE_LOAD_* state: load already presents the new group's first bit during the load cycle (chapter 14), so a dedicated one-cycle load state is what keeps that promise; getting this the wrong way round is the frame-level version of the one-bit timing error chapter 14 warned about. And the bus is open-drain: a node drives *actively* only when the bit is dominant, and otherwise releases, so that another node can pull the same wire dominant during the same period.

```
COMBINATIONAL_PROCESS: process(state, role, bt_bit_done, tx_id, tx_dlc,
                               tx_current_byte, crc_tx, txsr_tx_bit) is
begin
    -- Defaults: every sub-block input, every cycle, then per-state overrides.
    bt_enable      <= '1';                -- Never held; the idle count needs it.
    bt_resync      <= '0';
    txsr_load      <= '0';
    txsr_data      <= (others => '0');
    txsr_bit_count <= "0000";
    txsr_shift     <= '0';
    rxsr_enable    <= '0';                -- The receive side arrives in L17.
    rxsr_bit_count <= "0000";
    tx_bus         <= '1';                -- Release the bus unless driving -
    -- dominant.
    bus_en         <= '0';
```

```

-- Each STATE_LOAD_* arm loads its chunk. The paired shifting state keeps
-- the same bit_count applied, so grouping the pair in one arm is natural.
case state is
  when STATE_START =>
    bt_resync <= '1';          -- Adopt S0F as this node's bit boundary.

    if (role = '1') then
      txsr_load      <= '1';
      txsr_data      <= "00000000";          -- S0F: one
-- dominant bit.
      txsr_bit_count <= "0001";
    end if;
  when STATE_LOAD_ARB_HI | STATE_ARB_HI =>
    if ((role = '1') and (state = STATE_LOAD_ARB_HI)) then
      txsr_load <= '1';
      txsr_data <= tx_id(10 downto 3);
    end if;
    txsr_bit_count <= "1000";
  when STATE_LOAD_ARB_LO | STATE_ARB_LO =>
    if ((role = '1') and (state = STATE_LOAD_ARB_LO)) then
      txsr_load <= '1';
      txsr_data <= tx_id(2 downto 0) & '0' & "0000"; -- ID low,
-- then RTR.
    end if;
    txsr_bit_count <= "0100";
  when STATE_LOAD_CTRL | STATE_CTRL =>
    if ((role = '1') and (state = STATE_LOAD_CTRL)) then
      txsr_load <= '1';
      txsr_data <= "00" & tx_dlc & "00";          -- IDE, r0,
-- DLC.
    end if;
    txsr_bit_count <= "0110";
  when STATE_LOAD_DATA | STATE_DATA =>
    if ((role = '1') and (state = STATE_LOAD_DATA)) then
      txsr_load <= '1';
      txsr_data <= tx_current_byte;
    end if;
    txsr_bit_count <= "1000";
  when STATE_LOAD_CRC_HI | STATE_CRC_HI =>
    if ((role = '1') and (state = STATE_LOAD_CRC_HI)) then
      txsr_load <= '1';
      txsr_data <= crc_tx(14 downto 7);
    end if;

```

```

        txsr_bit_count <= "1000";
    when STATE_LOAD_CRC_LO | STATE_CRC_LO =>
        if ((role = '1') and (state = STATE_LOAD_CRC_LO)) then
            txsr_load <= '1';
            txsr_data <= crc_tx(6 downto 0) & '0';
        end if;
        txsr_bit_count <= "0111";
    when others =>
        null;
end case;

-- Shifting states advance the register once per bit period; a LOAD
-- state never does, which keeps load and shift mutually exclusive.
if (role = '1') then
    case state is
        when STATE_SOF | STATE_ARB_HI | STATE_ARB_LO | STATE_CTRL
            | STATE_DATA | STATE_CRC_HI | STATE_CRC_LO =>
            txsr_shift <= bt_bit_done;
        when others =>
            null;
        end case;
end if;

-- Driving the bus, open-drain: only the owning states, only dominant
-- bits. A receiver's one dominant bit is the ACK slot (L17).
case state is
    when STATE_SOF
        | STATE_LOAD_ARB_HI | STATE_ARB_HI
        | STATE_LOAD_ARB_LO | STATE_ARB_LO
        | STATE_LOAD_CTRL   | STATE_CTRL
        | STATE_LOAD_DATA   | STATE_DATA
        | STATE_LOAD_CRC_HI | STATE_CRC_HI
        | STATE_LOAD_CRC_LO | STATE_CRC_LO =>
        if ((role = '1') and (txsr_tx_bit = '0')) then
            bus_en <= '1';
            tx_bus <= '0';
        end if;
    when others =>
        null;
    end case;
end process;

```


The bounding of the bus driving in that last case statement deserves a sentence of its own: `txsr_tx_bit` is a level and holds its most recent value forever, so an unbounded version goes on driving after the group that produced it. A frame whose CRC ends on a dominant bit would still pull the bus down through the whole tail, and a node sitting in `STATE_IDLE` with role still '1' from its last frame would hold the entire bus dominant forever. The tail's fields are recessive for both roles and drive nothing, with the single exception of a receiver's ACK slot (chapter 17).

16.5 The sequential process

It owns state, the bookkeeping and the output ports. Skeleton, reset and defaults first:

```
SEQUENTIAL_PROCESS: process(clock, reset_s2_n) is
begin
    if (reset_s2_n = '0') then
        state          <= STATE_IDLE;
        role           <= '0';
        idle_bits      <= 11;          -- Full: an idle bus is assumed at power-
        -- up.
        data_byte_idx  <= 0;
        field_bit_count <= 0;
        rx_id_acc       <= (others => '0');
        rx_dlc_acc      <= (others => '0');
        rx_data_acc     <= (others => '0');
        crc_tx          <= (others => '0');
        tx_done         <= '0';
        rx_valid        <= '0';
        error           <= '0';
        rx_id           <= (others => '0');
        rx_dlc          <= (others => '0');
        rx_data         <= (others => '0');
    elsif (rising_edge(clock)) then
        tx_done <= '0';          -- The two pulse outputs default low;
        rx_valid <= '0';        -- error is a level and holds.

        case state is
```

Everything below is arms of this single case statement. First the two states every frame passes through once:

```
when STATE_IDLE =>
    if (bt_sample = '1') then
        if (rx_bus_s2 = '1') then
```

```

        if (idle_bits /= 11) then
            idle_bits <= idle_bits + 1;
        end if;
    else
        idle_bits <= 0;
    end if;
end if;
if (idle_bits = 11) then
    if (tx_req = '1') then
        role      <= '1';
        idle_bits <= 0;
        state     <= STATE_START;
    elsif (rx_bus_s2 = '0') then
        role      <= '0';    -- Someone else's SOF: receive (→
-- L17).

        idle_bits <= 0;
        state     <= STATE_START;
    end if;
end if;
when STATE_START =>
    error          <= '0';
    data_byte_idx  <= 0;
    rx_data_acc    <= (others => '0');
    state          <= STATE_SOF;

```

STATE_IDLE counts recessive bit periods at the sample point and saturates at 11; a dominant sample clears the count. Nothing leaves the state until the count is **full**, whatever tx_req and rx_bus_s2 do, so a tx_req pulse arriving while the bus is busy is **discarded**, not queued; the caller asks again. STATE_START needs no code for the SOF load or the resync, since the combinational process issues both from the state itself; setup that has to be *remembered*, clearing error, the byte index and the received-data accumulator, is what lands here. rx_data_acc needs the clear because a received frame writes only the bytes its DLC covers (chapter 17): a five-byte frame arriving after an eight-byte frame has to report zeros in bytes 5 to 7, not the previous frame's residue. rx_id_acc and rx_dlc_acc are always written in full, so they need none. (crc15 is cleared by this state too; see section 16.8.)

Then the LOAD/shift pairs. This is the shape every stuffed field follows, shown once for the identifier's high group; the transmit half of a LOAD state moves on after its single cycle, and the shift state waits for tx_shift_reg.done.

A timing note before the waveforms surprise you: chapter 14 described load as issued at the very moment of the previous group's last bit_done, and here it is issued two clock edges later, one for the state machine to see txsr_done and one for the LOAD state itself. Every group's first


```

when STATE_CTRL =>
  if (role = '1') then
    if (txsr_done = '1') then
      if (unsigned(tx_dlc) = 0) then
        state <= STATE_CRC_WAIT;    -- No data field at ↪
↪ all.

      else
        state <= STATE_LOAD_DATA;
      end if;
    end if;
  end if;
  -- role = '0': L17.
when STATE_LOAD_DATA =>
  if (role = '1') then
    state <= STATE_DATA;
  end if;
  -- role = '0': L17.
when STATE_DATA =>
  if (role = '1') then
    if (txsr_done = '1') then
      if (data_byte_idx + 1 = to_integer(unsigned(tx_dlc))) ↪
↪ then

        state <= STATE_CRC_WAIT;
      else
        data_byte_idx <= data_byte_idx + 1;
        state <= STATE_LOAD_DATA;
      end if;
    end if;
  end if;
  -- role = '0': L17.

```

Then the CRC. STATE_CRC_WAIT is an empty cycle so that the last data bit has time to finish being integrated before `crc_value` is read, and its real job is the latching: both CRC groups are loaded from `crc_tx`, never from the live engine (section 16.6 explains what goes wrong otherwise). The two CRC pairs have the arbitration pairs' shape again:

```

when STATE_CRC_WAIT =>
  crc_tx <= crc_value;    -- Capture before the engine ↪
↪ moves on.

  state <= STATE_LOAD_CRC_HI;
when STATE_LOAD_CRC_HI =>
  if (role = '1') then
    state <= STATE_CRC_HI;
  end if;
  -- role = '0': L17.
when STATE_CRC_HI =>
  if (role = '1') then

```

```

        if (txsr_done = '1') then
            state <= STATE_LOAD_CRC_L0;
        end if;
    end if; -- role = '0': L17.
when STATE_LOAD_CRC_L0 =>
    if (role = '1') then
        state <= STATE_CRC_L0;
    end if; -- role = '0': L17.
when STATE_CRC_L0 =>
    if (role = '1') then
        if (txsr_done = '1') then
            state <= STATE_CRC_L0_WAIT;
        end if;
    end if; -- role = '0': L17.

```

Finally the tail, where the shift register bows out and the state machine counts `bt_bit_done` pulses itself. `STATE_CRC_L0_WAIT` is one clock cycle for a transmitter, **not** a bit period: `tx_shift_reg.done` pulses at the closing shift, which is itself a `bit_done`, so the transmitter is already on a bit boundary and the tail can begin immediately; waiting a whole period would push an extra recessive bit out onto the wire. (A receiver, *by contrast*, waits a whole `bit_done` here, and checks `crc_valid` while it waits; chapter 17.)

```

        when STATE_CRC_L0_WAIT =>
            if (role = '1') then
                state <= STATE_CRC_DELIM; -- One clock cycle, not a bit ↵
            end if; -- role = '0': L17 checks ↵
        ↵ period.
        ↵ crc_valid here.
        when STATE_CRC_DELIM =>
            if (bt_bit_done = '1') then
                state <= STATE_ACK_SLOT;
            end if;
        when STATE_ACK_SLOT =>
            if (bt_bit_done = '1') then -- Transmitter releases; a ↵
        ↵ receiver
                state <= STATE_ACK_DELIM; -- acknowledges here (L17).
            end if;
        when STATE_ACK_DELIM =>
            if (bt_bit_done = '1') then
                field_bit_count <= 6; -- Arm the 7-bit EOF countdown.
                state <= STATE_EOF;
            end if;
        when STATE_EOF =>

```

```

        if (bt_bit_done = '1') then
            if (field_bit_count = 0) then
                if (role = '1') then
                    tx_done <= '1';
                end if;
                -- role = '0': L17 latches rx_*
            here.

            state <= STATE_IDLE;
        else
            field_bit_count <= field_bit_count - 1;
        end if;
    end if;
end case;
end if;
end process;

```

The tail's states need no role test to move on, since both roles count the same whole bit periods through the same fields; only what happens *inside* them differs, and those differences are all chapter 17's.

16.6 Capturing the CRC

STATE_CRC_WAIT latches `crc_value` into `crc_tx`, and both CRC groups are loaded from `crc_tx` rather than from the engine. That detour looks redundant and is not, for a reason worth following closely, since it is the one place in this design where two individually correct rules together make a bug.

Rule one: the engine keeps running through the CRC field, since that is what returns it to zero (section 16.7). Rule two: every group is loaded when its STATE_LOAD_* state runs.

Now put them together. STATE_LOAD_CRC_HI runs first and reads the right value. Then STATE_CRC_HI shifts out eight bits, and every one of them goes back into the engine, which dutifully advances the register. By the time STATE_LOAD_CRC_LO runs, `crc_value` is **no longer the frame's CRC**: it is the register eight bits further on, which for a correct frame means the remaining seven CRC bits shifted up with zeros shifted in beneath them. Load `crc_value` (6 downto 0) there, and the frame's last seven CRC bits go out as seven zeros, in every frame, always.

The failure is silent in the worst possible way. The transmitter is perfectly content; nothing it can see is wrong. The receiver's engine does not reach zero, so it reports a CRC error, and the obvious place to look is the receive path, which is entirely reasonable. Latching the value once, at the moment it is finished and before anything else touches the engine, removes the interaction entirely.

16.7 Feeding crc15

crc15 has to see every real bit from SOF through the end of the **CRC field**, and no others. Gate its enable on `tx_shift_reg.bit_valid` ('0' at every group's closing shift, so that the unchanged bit is not fed in twice), and shut out the stuff bits and the receive path:

```

crc_enable <= txsr_bit_valid and not txsr_stuff and role;
crc_data_in <= txsr_tx_bit;

```

Three things follow from this being a simple concurrent assignment with no state term in it.

The CRC field is fed back into the engine, and that is deliberate. The value being sent sits safely in `crc_tx`, so nothing has to read the engine again once `STATE_CRC_WAIT` has captured it, and the expression has no state term that would stop the feed at the data field. On the transmit side the feedback is merely harmless: chapter 13's generate-then-check property means that a message followed by its own CRC returns the register to zero, and `STATE_START`'s clear (section 16.8) hands the next frame a fresh engine anyway. Where the feedback earns its place is on the **receive side**: chapter 17's CRC check *is* the observation that the register has returned to zero once the received CRC field has gone in, and that observation exists only because the CRC field is fed back. One rule for both roles, every real bit from SOF through the end of the CRC field, is what lets a single engine generate on one node while it checks on another.

The tail excludes itself. CRC delimiter, ACK slot, ACK delimiter and EOF never go through `tx_shift_reg` at all, so `txsr_bit_valid` is already low across all of them. No state term is needed to keep them out of the checksum.

role is the placeholder for the receive side. With `role = '0'` this expression holds `crc15` disabled, which is correct for this chapter (transmission only) but not for the finished controller. Chapter 17 replaces the whole assignment with one feeding `rx_shift_reg.real_bit` on `rx_shift_reg.real_bit_valid` during reception; the transmit half above is unchanged by that.

16.8 Clearing crc15

One more concurrent assignment, and it is the one that makes the engine safe to reuse frame after frame:

```

crc_clear <= '1' when state = STATE_START else '0';

```

On the happy path it does nothing at all, since a frame that runs all the way has already returned the register to zero. It is there for the frames that do *not* finish. Chapter 17 gives this node three ways to abandon a frame halfway (it loses arbitration, it sees a stuffing violation, its CRC check fails), and each leaves the engine holding a half-finished value with

no prospect of ever reaching zero by itself. Every frame the node sent after that would carry a CRC computed from that residue, and every frame it received would fail the check, until somebody pressed reset.

Two things are worth noticing. `STATE_START` is the right home for it since it is already the “set this frame up” state, alongside clearing error and the byte index; the clear belongs there rather than among the CRC states. And `can_controller_tb` cannot catch a missing clear, since neither of its two scenarios lets a node transmit again after an abort. That makes this a rule to get right by reasoning rather than by waiting for a red testbench, the same lesson `crc_tx` gave in section 16.6: the silent failures are the ones to reason out in advance.

16.9 Summary

What you should be able to do now

- Draw the CAN frame’s state diagram and say which field every state corresponds to.
- Explain why every field needs a `LOAD` state before its shift state.
- Implement the transmit path: load the shift register, shift out, and move on at the right pulse.
- Drive the open-drain bus correctly: `bus_en` controls, `tx_bus` decides what.
- Gate the CRC engine correctly, and say why the wrong gating gives a CRC that looks plausible and is wrong.

Questions to test yourself with

1. Why does every field need a `LOAD` state of its own?
2. A transmitting node releases the bus during the ACK slot. Why, and what does it expect to see there?
3. `bus_en` and `tx_bus` are two separate signals. What would have been lost with just one?
4. Why is `crc15` gated on `bit_valid` and not `stuff` and not on `bit_done`?
5. Which fields does the CRC cover, and where exactly does the coverage end?
6. Why is the state machine a Moore machine here?

Target

The transmit path should be in place when chapter 17 begins, since the receive path is built into the same machine. If you do not get all the way: finish the transmission through the CRC field, and leave ACK and EOF to the next session.

Looking ahead

Chapter 17 completes `can_controller` with reception and arbitration, and runs the system testbench all the way through.

16.10 Exercises

You are building the transmit path, and `can_controller_tb` drives two nodes through a complete exchange, so it cannot pass until chapter 17 adds the receive half. make `build-project` knows that and reports it as *skipped* instead of running it and failing:

```
==> can_controller_tb (skipped: can_controller.vhd has no receive path yet; ↵
    ↵ ...)
```

So the build stays green this chapter, and these exercises reason their way through the transmit path and analyse it in GHDL instead. The testbench becomes the verdict one chapter from now.

Exercise 16.1 Trace a short frame by hand

REASONING

From the frame format in section 10.4 and this chapter's field table, write out the whole sequence of states `can_controller` passes through (in the TX role) to send a frame with ID `0x001`, DLC `0` (no data field), from `STATE_IDLE` back to `STATE_IDLE`. Note for every state roughly how many bit periods it occupies.

Exercise 16.2 Why the `STATE_LOAD_*` state exists

REASONING

- Explain in your own words what would go wrong if a stuffed field were sequenced with a single state that asserted `tx_shift_reg`'s load *and* waited for done, instead of a separate one-cycle `STATE_LOAD_*` followed by the shift state.
- Point out the specific timing rule from chapter 14 (that `tx_shift_reg`'s load presents immediately, and its done comes one extra shift later) that makes the split into two states necessary.

Exercise 16.3 What reaches crc15, and what does not

REASONING

- a) For a transmitted frame, sort these categories of bits into “fed into crc15” and “excluded”: the SOF bit, real bits from the ID, control field and data, inserted stuff bits, the CRC field’s own bits, and the unstuffed tail (CRC delimiter, ACK slot, ACK delimiter, EOF). For every excluded category, say what excludes it: either a term in `crc_enable <= txsr_bit_valid` and not `txsr_stuff` and `role;`, or the fact that the field never passes through `tx_shift_reg` at all.
- b) One of those categories may surprise you: the CRC field’s own bits go back into the engine. Using chapter 13’s property that generating and checking are the same operation, say what value the register holds after the last CRC bit has been fed in, and name the check in chapter 17 that depends on precisely that value. Then explain why the transmit path, taken on its own, does not depend on it: what does `STATE_START`’s `clear` already guarantee about the second frame a node sends?
- c) Suppose you had instead gated `crc_enable` off during `STATE_CRC_HI` and `STATE_CRC_LO`. Would the first frame a node sends carry a correct CRC? Would the second? Be precise: that `crc15` is cleared is part of the answer. Then say whether any simulation in this book would catch the change, and what that tells you about where the fed-back CRC field actually earns its keep.

Exercise 16.4 How long is a frame, exactly?

REASONING

Answering that requires holding the whole field table and the bit stuffing rule in your head at once, which is the point. Take the frame `can_controller` sends for ID `0x123`, DLC 3, data bytes `FF 00 00`.

- a) Write out the stuffed region bit by bit, SOF through the end of the data field, using this chapter’s field table. (It should come to 43 bits: $1 + 11 + 1 + 2 + 4 + 24$.) The identifier is `0x123`; remember that `RTR`, `IDE` and `r0` are all 0 and that DLC is `0011`.
- b) The CRC-15 over those 43 bits is `0x0FEE`, that is, `000111111101110` (you can confirm that against your `crc15` from chapter 13 if you like). Add it, which gives the whole stuffed region of 58 bits, SOF through CRC.
- c) Apply the bit stuffing rule over all 58 bits and count the inserted stuff bits. Watch the boundary between the data field’s closing zeros and the CRC’s leading zeros: the run counter is not cleared at field boundaries.
- d) Add the unstuffed tail (CRC delimiter, ACK slot, ACK delimiter, EOF) and state the total number of bits actually driven out on the wire. How many microseconds is that at 1 Mbit/s?
- e) The same frame with the data `11 22 33` needs only one stuff bit instead of the number you got in c). Explain in one sentence why a frame’s duration on a CAN bus depends on the

payload's value and not only on its length, and what that means for anyone trying to work out worst-case bus timing.

The CAN controller II: reception and arbitration

Completing can_controller. After this chapter the CAN half of the project is finished.

IN THIS CHAPTER

- The receive role: the timing of rx_shift_reg's enable, and the single cycle of pre-emption.
- Reassembling a received frame, and feeding crc15 on the receive side.
- A clean slate between frames, and why both shift registers are held in reset while the bus is idle.
- Detecting lost arbitration, checked bit by bit across the arbitration field.
- Running can_controller_tb all the way through, transmission and reception together.

Chapter 16 built the transmit path. This chapter completes can_controller with the role = '0' path (reception) and the arbitration logic deciding which of two competing transmitters gets to carry on. The frame layout, the state sequence and the per-field group sizes are the same as in chapter 16; the receive side merely reads every field back instead of driving it.

17.1 Reading the bus back

A receiving node collects real (destuffed) bits through rx_shift_reg, gated at the bit_timer.sample point (mid-period, once the bus has settled), not at bit_done. It sets rx_shift_reg.bit_count to the same group sizes the transmit side uses, and reads every completed group out of rx_shift_reg.data, which is **right-justified** in its lowest bit_count bits (chapter 15). It reassembles the frame in accumulators:

- **ID**: the high 8 bits from ARB_HI (rxsr_data(7 downto 0)), the low 3 from ARB_L0 (rxsr_data(3 downto 1)). The group's last bit, rxsr_data(0), is RTR: dominant marks

the data frames this book supports, so a recessive RTR raises error and aborts the frame (a remote frame, unsupported).

- **DLC:** from CTRL (rxsr_data(3 downto 0)). The group's top two bits, IDE and r0, are **not** checked: both are always dominant in this book's frames. A receiver meeting the recessive IDE of a real extended-identifier frame would misread it as a standard frame right up until the CRC check failed; checking IDE and handling what it announces belongs in a production controller (section 19.7 lists it among the gaps).
- **Data:** every received byte is placed at its **fixed** position by byte index, byte 0 in bits 63 downto 56, byte 1 in 55 downto 48, and so on. This is a position-driven case, **not** a running left shift (which would only land byte 0 at the top for a full 8-byte frame).

At the end of STATE_EOF, latch rx_id, rx_dlc and rx_data from the accumulators and pulse rx_valid.

17.2 The receive path, state by state

The same state sequence as chapter 16's transmit path, walked as a receiver. role = '0' throughout. Read this alongside section 16.2's field table: the group sizes are identical, only what happens to each group differs.

"Pre-empts" is the rxsr_bit_count/rxsr_enable the **combinational** block drives while state still reads the STATE_LOAD_* value, one cycle before the shift state becomes visible; section 17.5 explains why that is necessary.

State	Pre-empts	Advances on	Accumulator / action
STATE_IDLE	nothing	dominant rx_bus_s2, idle_bits full	latch role = '0', clear idle_bits
STATE_START	nothing	immediately	clear error, reset the byte index, bt_resync
STATE_SOF	1	rxsr_valid	collect SOF as a one-bit group; the bit itself is discarded
STATE_LOAD_ARB_HI	8	bt_sample	(waiting: the previous group's last bit is still on the wire)
STATE_ARB_HI	8	rxsr_valid	rx_id_acc(10 downto 3) <= rxsr_data(7 downto 0)
STATE_LOAD_ARB_LO	4	bt_sample	
STATE_ARB_LO	4	rxsr_valid	rx_id_acc(2 downto 0) <= rxsr_data(3 downto 1); rxsr_data(0) is RTR: if recessive, error <= '1' and abort
STATE_LOAD_CTRL	6	bt_sample	
STATE_CTRL	6	rxsr_valid	rx_dlc_acc <= rxsr_data(3 downto 0); DLC = 0 jumps to STATE_CRC_WAIT
STATE_LOAD_DATA	8	bt_sample	
STATE_DATA	8	rxsr_valid	place rxsr_data at the fixed position for byte data_byte_idx
STATE_CRC_WAIT	nothing	immediately	one settling cycle; no wait for bt_sample
STATE_LOAD_CRC_HI	8	bt_sample	
STATE_CRC_HI	8	rxsr_valid	keep feeding crc15; the value itself is not kept
STATE_LOAD_CRC_LO	7	bt_sample	
STATE_CRC_LO	7	rxsr_valid	the last bits into crc15
STATE_CRC_LO_WAIT	nothing	bt_bit_done	check crc_valid: if low, error <= '1' and abort
STATE_CRC_DELIM	nothing	bt_bit_done	
STATE_ACK_SLOT	nothing	bt_bit_done	drive the bus dominant to acknowledge
STATE_ACK_DELIM	nothing	bt_bit_done	arm the 7-bit EOF countdown
STATE_EOF	nothing	bt_bit_done x7	at the last bit, latch rx_id, rx_dlc, rx_data, pulse rx_valid

Two patterns run through the whole table. Every STATE_LOAD_* row exists purely to consume the previous group's closing bit period: the state machine harvests nothing there, even though the bt_sample ending the state is exactly the one rx_shift_reg collects the next group's first bit on. Every closing (unstuffed) row advances on bt_bit_done instead of bt_sample, since those fields bypass rx_shift_reg entirely and are counted as whole bit periods.

Why STATE_SOF enables rx_shift_reg for a bit nobody keeps

The collected value is discarded; SOF is always dominant and STATE_IDLE already knew that when it started this frame. What matters is that the bit passes through the destuffing's run tracker. That tracker (consecutive, last_bit) survives group boundaries and is cleared only by reset (chapter 15), and the transmitter's copy inside tx_shift_reg counted SOF, since chapter 16 sends it through tx_shift_reg like any other stuffed field. Skip it here, and the receiver runs the whole frame one dominant bit behind the transmitter.

That is no small window. Every identifier whose top four bits are zero, that is, **every ID below 0x080**, gives SOF plus four dominant identifier bits: the transmitter's run reaches five and it inserts a stuff bit, while a receiver that missed SOF stands at only four, expects none, and takes it for a real bit. Everything after it is shifted. The failure turns up far later as a CRC error or a stuffing violation, and it strikes precisely the high-priority identifiers a real design reserves for its most important messages.

The rule underneath is worth stating on its own: **both run trackers have to see exactly the same bits, SOF through the end of the CRC field, and nothing else.** The tail is excluded on both sides since neither shift register is enabled for it. Across *frame boundaries*, on the other hand, "the same bits" is no guarantee the design can lean on at all, which is section 17.3.

STATE_SOF needs no pre-emption for this. STATE_START pulses bt_resync, so the SOF bit's sample point is a whole SAMPLE_TICK away (chapter 12); the state machine is well inside STATE_SOF by then, and an ordinary combinational enable is enough.

17.3 A clean slate between frames

The run trackers are cleared only by their reset (chapter 14, 15), and within a single frame that persistence is exactly right. Let it stretch *across* frame boundaries, and a subtle asymmetry appears: a tracker's state at SOF now depends on which bits that particular node happened to push through it earlier, and different nodes have different histories. A node that transmitted the previous frame ends it with its tx_shift_reg tracker holding the CRC field's tail; a node that only received holds that history in its rx_shift_reg tracker instead, and its transmit tracker still shows whatever its *own* most recent transmission left, or nothing at all after reset.

Now let two such nodes start transmitting at the same SOF, which is exactly what arbitration exists for. Identical identifier bits come out of both, but the node carrying a leftover run reaches five identical bits *earlier* and inserts its stuff bit one position before the other node. At that position it sends a recessive stuff bit while the other node is still sending a real dominant bit; the wired-AND reads dominant, the stuffing node is sending recessive inside the arbitration field, and it concludes that it has lost arbitration. No contest took place: the *would-be winner* walks away from its own frame with error raised. This is not hypothetical; it is exactly how can_controller_tb's case 2 fails if the trackers persist, with node A entering it still holding the tail of case 1's CRC.

Real CAN has no such failure, since the stuff bit count starts afresh at every SOF, and this design gets the same guarantee with a single signal: both shift registers are **held in reset**

while the bus is idle, so every frame starts with clean trackers on every node, whatever each of them did before. In the declarative part:

```
-- Shift-register reset: asserted while the bus is idle, so every frame
-- starts with clean run trackers on every node.
signal srs_reset_n : std_logic;
```

after begin:

```
srs_reset_n <= '0' when ((reset_s2_n = '0') or (state = STATE_IDLE)) else '1';
```

and both shift register instances change their second association from `reset_s2_n` to `srs_reset_n`, so that the port maps from chapter 14 and 15 become:

```
tx_shift_reg1: entity work.tx_shift_reg
  port map(clock, srs_reset_n, txsr_load, txsr_data, txsr_bit_count, ↵
↵ txsr_shift,
           txsr_tx_bit, txsr_stuff, txsr_done, txsr_bit_valid);

rx_shift_reg1: entity work.rx_shift_reg
  port map(clock, srs_reset_n, bt_sample, rx_bus_s2, rxsr_bit_count, ↵
↵ rxsr_enable,
           rxsr_data, rxsr_valid, rxsr_stuff_error, rxsr_real_bit, ↵
↵ rxsr_real_bit_valid);
```

The reset releases at the edge leaving `STATE_IDLE`, a whole cycle before the SOF load is clocked into `tx_shift_reg`, so the registers are awake in time for every frame. Note what this does *not* change: within a frame nothing is ever cleared, so the chained reload behaviour chapter 14's test 3 pins down (a run continuing across a group boundary) is untouched, and the SOF-skip argument above holds bit for bit. The clean slate begins at `STATE_IDLE`, and `STATE_IDLE` is reachable only between frames.

17.4 Feeding crc15 on the receive side

The mirror image of chapter 16's transmit-side gating: enable `crc15` on `rx_shift_reg.real_bit_valid` (which already excludes both "no new bit yet" and every discarded stuff bit), fed with `rx_shift_reg.real_bit`, over SOF through the CRC field. Feeding the received CRC field's own bits back returns the register to exactly zero if the frame arrived intact (chapter 13's generate-then-check), which `crc_valid` reports.

There is one `crc15` and one enable, so this replaces chapter 16's assignment rather than standing beside it. `role` now chooses the source instead of switching the engine off:

```

crc_enable <= (txsr_bit_valid and not txsr_stuff and role) or
              (rxsr_real_bit_valid and not role);
crc_data_in <= txsr_tx_bit when role = '1' else rxsr_real_bit;

```

Leave chapter 16's version in place and a receiving node never accumulates anything, so `crc_valid` reads '1' (the reset value) throughout and the check in `STATE_CRC_LO_WAIT` passes for every frame, corrupted or not.

`crc_clear` is untouched by any of this and stands exactly as chapter 16 wrote it, a pulse in `STATE_START` whatever the role. It matters more now than it did then: this is the chapter where the node gets three ways to leave a frame early, and each leaves the engine in the middle of a message it will never finish. None of the abort paths below has to clear it on the way out, since the next frame clears it on the way in.

17.5 Waiting for one extra sample: the decisive timing lesson

On the transmit side, `tx_shift_reg.done` already spends an extra bit period letting a group's last bit run out before the next group is loaded. A receiver reacts to `rx_shift_reg.valid` with no such built-in wait, so `can_controller` has to add it explicitly at each of the RX side's `STATE_LOAD_*` states: **wait for one more `bt_sample`** before moving on, since the wire still shows the previous group's last bit for the whole of its closing period.

There is a second catch: state becomes the shift state only **one cycle after** the `bt_sample` that triggered the transition, one cycle too late to enable `rx_shift_reg` for that very sample. That is why the combinational control block **pre-empts** `rxsr_enable` (and sets the coming group's `bit_count`) for exactly that sample while it is still in the `STATE_LOAD_*` state.

What it looks like cycle by cycle. Here is a group boundary on the receive side, where the 8-bit-wide high ID group hands over to the 4-bit-wide low ID plus RTR group. The cycles are the 50 MHz clock's, so the two `bt_sample` pulses are 50 cycles apart; the run of identical cycles between them is elided.

clock cycle	bt_sample	state	rxsr_bit_count	rxsr_enable	rxsr_valid
n	0	STATE_ARB_HI	8	1	0
n + 1	1	STATE_ARB_HI	8	1	0
n + 2	0	STATE_ARB_HI	8	1	1
n + 3	0	STATE_LOAD_ARB_LO	4	1	0
...	0	STATE_LOAD_ARB_LO	4	1	0
n + 51	1	STATE_LOAD_ARB_LO	4	1	0
n + 52	0	STATE_ARB_LO	4	1	0

Read the two sample cycles against each other. At `n + 1` the eighth bit of the high ID group is taken, and `rxsr_valid` appears one cycle later at `n + 2`, which is when the state machine decides to move on. state does not read `STATE_LOAD_ARB_LO` until `n + 3`.

Now look at $n + 51$, the sample that has to catch the *first* bit of the low ID group. `state` still reads `STATE_LOAD_ARB_L0` at that moment. It does not become `STATE_ARB_L0` until $n + 52$, one cycle after the bit has already come and gone. That is the whole problem in one line: the state that names the group always arrives one cycle later than the sample that starts it.

The pre-emption is what makes `rxsr_bit_count = 4` and `rxsr_enable = '1'` true at $n + 51$ all the same, by driving them from the `STATE_LOAD_ARB_L0` arm instead of the `STATE_ARB_L0` arm. Drive them from `STATE_ARB_L0` instead, and the same boundary looks like this:

clock cycle	bt_sample	state	rxsr_bit_count	rxsr_enable	rxsr_valid
$n + 3$	0	STATE_LOAD_ARB_L0	0	0	0
...	0	STATE_LOAD_ARB_L0	0	0	0
$n + 51$	1	STATE_LOAD_ARB_L0	0	0	0
$n + 52$	0	STATE_ARB_L0	4	1	0

`rx_shift_reg` is disabled across the one sample that mattered. The first bit of the identifier's low group is never collected, so the group completes one bit short against the wire, every later group is shifted by one, and `crc15` is fed a stream that never matches. Nothing reports any error at $n + 51$; the failure turns up at `STATE_CRC_L0_WAIT`, some forty bit periods later, as a CRC that does not go to zero.

Note that this costs nothing extra on the wire. `STATE_LOAD_ARB_L0` already consumes the previous group's closing bit period, which is why it advances on `bt_sample` and collects nothing of its own; the pre-emption merely puts the register in the right configuration before that period is over.

This pattern repeats at every group boundary and is the design's single most important lesson: **a signal that looks "done" one cycle too early or too late does not cancel itself out across group boundaries; it accumulates into a silent misalignment between transmitter and receiver that only much later turns up as a CRC error or a stuffing violation, far from its cause.**

17.6 Detecting lost arbitration

Exactly section 10.6's rule — chapter 4 of *CAN: the bus, the frame and the controller* is the fuller version — checked once per bit at the sample point, during `STATE_ARB_HI` and `STATE_ARB_L0` alone, which is precisely the arbitration field, ID plus RTR. Unique IDs guarantee that arbitration is settled within the 11-bit identifier; the check harmlessly covers RTR as well, where every node here sends dominant, a bit that can never lose:

```
if ((bt_sample = '1') and (txsr_tx_bit = '1') and (rx_bus_s2 = '0')) then
    error    <= '1';    -- Sent recessive, read dominant: lost.
    tx_done  <= '1';    -- The attempt is over; say so, win or lose.
```

```

    role      <= '0';
    idle_bits <= 0;
    state     <= STATE_IDLE;
end if;

```

The check lives inside STATE_ARB_HI's and STATE_ARB_LO's if (role = '1') branch, and that is what bounds it to a transmitting node; the three terms above are the sensing condition itself.

Why tx_done pulses here too, on a frame that was never sent. It reads wrongly at first: tx_done means “the frame completed”, and this one did not. Read it instead as “*the transmission attempt is over, and the port is yours again*” — which is the only question a caller can act on. error is what says whether the attempt succeeded, and together the two carry the whole outcome: tx_done alone means sent, tx_done with error means lost.

Leave it out, and the caller has no edge at all to wait for. A node losing arbitration returns to STATE_IDLE and is perfectly able to transmit again, but nothing ever announces that, so any caller waiting for tx_done before asking again waits forever. That is not hypothetical: it is exactly how register_bank (chapter 18) sets its TX-ready bit, and without this pulse a single lost arbitration — the normal case on a two-node bus — would leave the whole node unable to transmit until it was reset.

Note that it is rx_bus_s2, the synchronised copy, and not the rx_bus port. Section 11.5's rule applies everywhere, this comparison included: nothing in the design reads the raw pin. It would be easy to talk your way out of this one, since the sample point is 35 ticks into a bit that settled near tick 0, so the level is certainly stable by then. That argument is about *when* the value is stable on the wire, and metastability is about what a flip-flop does when it samples a signal no clock in this chip controls. The two extra flip-flops cost two clock ticks out of fifty and buy the one comparison the whole arbitration scheme rests on.

Clearing idle_bits is not bookkeeping tidiness. Without it the node lands back in STATE_IDLE with idle_bits full while the winner is in the middle of its frame, so the winner's next dominant bit is read as a new SOF, the node enters STATE_START, and STATE_START wipes the error it has just raised. The same clear is required at every other abort below, for the same reason.

17.7 Completing the processes

The same rules as in chapter 16's walkthrough: the code fragments are successive slices in the order the code reads, and nothing from chapter 16 is rewritten beyond its marked placeholders; every slice below fills one of them in.

The combinational process

It gets two new blocks, and `crc_valid` joins its sensitivity list (the ACK driving below reads it). First, between the transmit side's case for the shift gating and the case statement for the bus driving, the receive side's enable and group widths, which is the pre-emption made concrete:

```
-- The receive-side enable and chunk widths: the pre-empt. Active
-- through every stuffed field's LOAD and shifting state, so the sample
-- that falls while state still reads STATE_LOAD_* is collected at the
-- new width.
if (role = '0') then
  case state is
    when STATE_SOF =>
      rxsr_enable    <= '1';
      rxsr_bit_count <= "0001";
    when STATE_LOAD_ARB_HI | STATE_ARB_HI
      | STATE_LOAD_DATA   | STATE_DATA
      | STATE_LOAD_CRC_HI | STATE_CRC_HI =>
      rxsr_enable    <= '1';
      rxsr_bit_count <= "1000";
    when STATE_LOAD_ARB_LO | STATE_ARB_LO =>
      rxsr_enable    <= '1';
      rxsr_bit_count <= "0100";
    when STATE_LOAD_CTRL | STATE_CTRL =>
      rxsr_enable    <= '1';
      rxsr_bit_count <= "0110";
    when STATE_LOAD_CRC_LO | STATE_CRC_LO =>
      rxsr_enable    <= '1';
      rxsr_bit_count <= "0111";
    when others =>
      null;
  end case;
end if;
```

Every `STATE_LOAD_*` state pre-empts the *coming* group's width, one `bt_sample` before state visibly changes, exactly as section 17.5 derived; grouping it with its shift state in one and the same arm is what makes that automatic. Second, the receiver's single dominant bit: a new arm in the case statement for the bus driving, above its `when others`:

```
when STATE_ACK_SLOT =>
  if ((role = '0') and (crc_valid = '1')) then
    bus_en <= '1';
    tx_bus <= '0';
```

```
end if;
```

The sequential process

Every -- role = '0': L17. marker becomes the arm's second half. There are only two receive shapes. A receiving STATE_LOAD_* state waits for one extra bt_sample (the same elsif in all six), and a receiving shift state advances on rxsr_valid and harvests its group if the frame keeps any of it. Here are both shapes filled in, on STATE_SOF and the identifier's high pair; note that STATE_ARB_HI's transmit half grew too, taking up section 17.6's arbitration check before the done test. (The two can never fire in the same cycle, since bt_sample is mid-period and done follows a bit_done, so the order is about readability rather than priority; a loss on a group's last bit still simply wins by coming first, at that bit's sample point.)

```
when STATE_SOF =>
  if (role = '1') then
    if (txsr_done = '1') then
      state <= STATE_LOAD_ARB_HI;
    end if;
  else
    if (rxsr_valid = '1') then
      state <= STATE_LOAD_ARB_HI;
    end if;
  end if;
when STATE_LOAD_ARB_HI =>
  if (role = '1') then
    state <= STATE_ARB_HI;
  elsif (bt_sample = '1') then
    state <= STATE_ARB_HI;
  end if;
when STATE_ARB_HI =>
  if (role = '1') then
    if ((bt_sample = '1') and (txsr_tx_bit = '1') and (↵
↵ rx_bus_s2 = '0')) then
      error      <= '1';      -- Sent recessive, read dominant:↵
↵ lost.

      tx_done    <= '1';
      role       <= '0';
      idle_bits  <= 0;
      state      <= STATE_IDLE;
    elsif (txsr_done = '1') then
      state <= STATE_LOAD_ARB_LO;
    end if;
  else
```

```

        if (rxsr_valid = '1') then
            rx_id_acc(10 downto 3) <= rxsr_data(7 downto 0);
            state <= STATE_LOAD_ARB_L0;
        end if;
    end if;

```

STATE_LOAD_ARB_L0, STATE_LOAD_CTRL, STATE_LOAD_DATA, STATE_LOAD_CRC_HI and STATE_LOAD_CRC_L0 all take the identical `elsif (bt_sample = '1')` wait, and STATE_CRC_HI/STATE_CRC_L0 take the identical `rxsr_valid` advance without harvesting anything: their bits reach `crc15` bit by bit through `real_bit`, and the assembled groups mean nothing. The states with real receive work in them:

```

when STATE_ARB_L0 =>
    if (role = '1') then
        if ((bt_sample = '1') and (txsr_tx_bit = '1') and (
-- rx_bus_s2 = '0')) then
            error      <= '1';
            tx_done    <= '1';
            role       <= '0';
            idle_bits  <= 0;
            state      <= STATE_IDLE;
        elsif (txsr_done = '1') then
            state <= STATE_LOAD_CTRL;
        end if;
    else
        if (rxsr_valid = '1') then
            rx_id_acc(2 downto 0) <= rxsr_data(3 downto 1);
            if (rxsr_data(0) = '1') then
                error      <= '1';    -- Recessive RTR: a remote --
-- frame.

                idle_bits <= 0;
                state      <= STATE_IDLE;
            else
                state <= STATE_LOAD_CTRL;
            end if;
        end if;
    end if;
when STATE_CTRL =>
    if (role = '1') then
        if (txsr_done = '1') then
            if (unsigned(tx_dlc) = 0) then
                state <= STATE_CRC_WAIT;
            else

```

```

        state <= STATE_LOAD_DATA;
    end if;
end if;
else
    if (rxsr_valid = '1') then
        rx_dlc_acc <= rxsr_data(3 downto 0);
        if (unsigned(rxsr_data(3 downto 0)) = 0) then
            state <= STATE_CRC_WAIT;
        else
            state <= STATE_LOAD_DATA;
        end if;
    end if;
end if;
when STATE_DATA =>
    if (role = '1') then
        if (txsr_done = '1') then
            if (data_byte_idx + 1 = to_integer(unsigned(tx_dlc))) ↪
↪ then
                state <= STATE_CRC_WAIT;
            else
                data_byte_idx <= data_byte_idx + 1;
                state <= STATE_LOAD_DATA;
            end if;
        end if;
    else
        if (rxsr_valid = '1') then
            case data_byte_idx is -- Fixed position per byte ↪
↪ index.
                when 0 => rx_data_acc(63 downto 56) <= rxsr_data;
                when 1 => rx_data_acc(55 downto 48) <= rxsr_data;
                when 2 => rx_data_acc(47 downto 40) <= rxsr_data;
                when 3 => rx_data_acc(39 downto 32) <= rxsr_data;
                when 4 => rx_data_acc(31 downto 24) <= rxsr_data;
                when 5 => rx_data_acc(23 downto 16) <= rxsr_data;
                when 6 => rx_data_acc(15 downto 8) <= rxsr_data;
                when others => rx_data_acc(7 downto 0) <= ↪
↪ rxsr_data;
            end case;
            if (data_byte_idx + 1 = to_integer(unsigned(rx_dlc_acc↪
↪ ))) then
                state <= STATE_CRC_WAIT;
            else
                data_byte_idx <= data_byte_idx + 1;
                state <= STATE_LOAD_DATA;
            end if;
        end if;
    end if;
end if;

```



```

        end if;
    end if;
end if;

```

Two pre-edge reads to notice, both the same rule as chapter 14's MAX_RUN - 1. STATE_CTRL decides the DLC branch directly from rxsr_data(3 downto 0), since rx_dlc_acc is being assigned at this very edge and still reads the previous frame's value. And STATE_DATA compares against rx_dlc_acc, which by then *is* stable, latched a whole field earlier.

The tail differs from chapter 16's in exactly two arms. STATE_CRC_LO_WAIT is where the receiver consumes the last CRC bit's closing period and passes its verdict, and STATE_EOF's last bit is where a receiver publishes the frame:

```

when STATE_CRC_LO_WAIT =>
    if (role = '1') then
        state <= STATE_CRC_DELIM;    -- One clock cycle, not a bit-
        period.
    elsif (bt_bit_done = '1') then
        if (crc_valid = '1') then
            state <= STATE_CRC_DELIM;
        else
            error    <= '1';        -- The engine did not return to
            to zero.
            idle_bits <= 0;
            state    <= STATE_IDLE;
        end if;
    end if;
end if;

```

```

when STATE_EOF =>
    if (bt_bit_done = '1') then
        if (field_bit_count = 0) then
            if (role = '1') then
                tx_done <= '1';
            else
                rx_id    <= rx_id_acc;
                rx_dlc   <= rx_dlc_acc;
                rx_data  <= rx_data_acc;
                rx_valid <= '1';
            end if;
            state <= STATE_IDLE;
        else
            field_bit_count <= field_bit_count - 1;
        end if;
    end if;

```

```
end if;
```

Finally an overlay that belongs to no single state. After the whole case statement, still inside the clocked branch:

```
-- A stuffing violation aborts a reception from any stuffed state.
if ((role = '0') and (rxsr_stuff_error = '1')) then
    error      <= '1';
    idle_bits <= 0;
    state      <= STATE_IDLE;
end if;
```

Placed after the case statement it wins over whatever the state arm decided at the same edge, which is the whole point: a violation is fatal wherever it fires, and a sample can fire in any of the stuffed fields' arms, the LOAD states included (their pre-empted sample is treated like any other). Writing it once is better than repeating three lines in a dozen arms.

Acknowledging, checking and aborting

- **The ACK slot:** a receiver whose `crc_valid` is set drives the bus dominant for that single bit to acknowledge the frame.
- **The CRC check:** in `STATE_CRC_L0_WAIT` a receiver reads `crc_valid`. If it is set it waits for `bt_bit_done` and then moves on to `STATE_CRC_DELIM`; if it is not, it aborts.
- **Stuffing violation:** `rx_shift_reg.stuff_error` aborts a reception from any state. A transmitter cannot violate the bit stuffing rule, so this applies to RX only.
- **Arbitration loss** is the only abort that happens while *transmitting*, and therefore the only one that also pulses `tx_done`. The receive-side aborts end a reception, and a reception has no caller waiting for a completion pulse: `rx_valid` simply fails to appear, which is exactly right, since no frame arrived.
- **error holds:** cleared in `STATE_START`, held otherwise, so that a caller sees it right up until the next frame begins. That is precisely why `STATE_IDLE` requires an **idle bus** before the node leaves that state at all, and why every abort here has to clear `idle_bits` on the way out. Every abort lands back in `STATE_IDLE` in the middle of a frame, while the bus is still busy. Without the idle-bus requirement the node would enter `STATE_START` again within a few bit periods and wipe the error it raised, so that a caller polling error would never see it. Requiring eleven recessive bit periods first is exactly the guarantee “right up until the next frame begins” issues.

17.8 Verifying it: `can_controller_tb`

Now that the receive path is complete, `controller/can_controller_tb.vhd` finally runs all the way. It connects **two** `can_controller` nodes to a single open-drain bus, combining every

node's pair (`tx_bus`, `bus_en`) the way section 10.2 described a wired-AND, and runs two cases.

Case 1, reception. Node A sends ID 0x123, DLC 5, data 11 22 F8 44 55 while node B receives. It checks `tx_done` and `rx_valid`, and that B recovers the ID, the DLC and **every** data byte at its fixed position. The byte 0xF8 is chosen to force a stuff bit inside the data field, so this exercises the transmit path, the receive path, the destuffing and the multi-byte placement in one and the same frame. A single node cannot test any of it: `rx_shift_reg` is active only while `role = '0'`, so a transmitter never receives its own frame.

Case 2, arbitration. Both nodes request during the same cycle, A with ID 0x000 and B with 0x001, separated only in the last identifier bit. B has to detect the loss at exactly that bit and stop driving the bus; A has to complete as usual, unaware that a contest took place. The last assertion is the one to notice: it reads B's error *after* A's frame has completed, so it fails unless both the lingering error and the `idle_bits` discipline above hold.

17.9 Summary

What you should be able to do now

- Implement the receive path in the same state machine that transmits.
- Explain how a node decides that it has lost arbitration, and what it does then.
- Reassemble `rx_id`, `rx_dlc` and `rx_data` from the received bit stream and raise `rx_valid` at the right moment.
- Run the system testbench with two nodes on a bus and read what it reports.
- Say what error means, which four causes set it, and when it is cleared.

Questions to test yourself with

1. How does a node detect that it has lost arbitration, and exactly which bits does it compare?
2. What does a node that has lost arbitration do, and why is that not “try again immediately”?
3. Why must `rx_shift_reg`'s enable be pre-empted by one cycle?
4. A node cannot transmit and receive at the same time. Where in the state machine is that visible?
5. What sets error, and what clears it?
6. What does `can_controller_tb` prove that the four module testbenches together do not?

Target

This is where the project moves from CAN to SPI. A group that has not got `can_controller_tb` passing should prioritise that over the register bank: the SPI half can be built and tested against a controller that does not work yet, but the bring-up in chapter 19 cannot.

The system testbench is skipped by the build until the controller has a receive path. If you still see “skipped” when you think it should run, run with `CI_BUILD_ALL=1`; appendix A explains why that check works the way it does.

Looking ahead

Chapter 18 takes the register bank, and SPI from the waveform upwards. This is where the half begins that makes the controller reachable from a microcontroller.

17.10 Exercises

Once your receive logic is finished, `controller/can_controller_tb.vhd` verifies transmission and reception together, and make build-project stops skipping it: all six testbenches in `controller/` run from here on. The two in `bridge/` go on being skipped until chapter 18 and 19 write the modules they exercise.

If it is still reported as skipped after you believe the receive path is finished, read the message before your design. The build settles the matter by looking for chapter 17’s receive-side CRC gating inside `can_controller.vhd`, so a correct controller spelling the two expressions differently is skipped rather than run. `CI_BUILD_ALL=1` make build-project forces it, and appendix A explains why the check works that way.

Exercise 17.1 Reasoning about the RX-side timing fix

REASONING

- a) Section 17.5 shows that `rx_shift_reg` has to be enabled *combinationally*, during the `STATE_LOAD_*` state itself, instead of waiting until state visibly becomes the shift state. Explain in your own words why first waiting for the state transition to complete would make `rx_shift_reg` miss the very sample that triggered the wait in the first place.
- b) `STATE_CRC_LO_WAIT` waits for `bt_bit_done`, not for `bt_sample`, unlike every wait at a group boundary in the design (the `STATE_LOAD_*` states). Explain in your own words why that transition in particular has to align with a *bit period boundary* and not with a sample point. Which other states share the `bt_bit_done` trigger, and what do they have in common with it?
- c) Suppose (hypothetically) that `STATE_CRC_WAIT` also waited for `bt_sample` in the RX role, on top of `STATE_LOAD_CRC_HI` doing the same immediately afterwards. Explain what would

go wrong, and why it would show up not so much as an outright failure as a receiving node ending up one bit period further behind a transmitting node than it should be.

Exercise 17.2 Arbitration by hand

REASONING

Two nodes request transmission on the same cycle: node A with ID 0x2A0, node B with ID 0x2C0.

- a) Write both IDs in binary (11 bits, MSB first) and mark the first bit position where they differ.
- b) Using section 10.6's arbitration rule (dominant 0 wins), say which node wins and at which bit the loser drops out.
- c) The losing node detects its loss with `bt_sample = '1'` and `txsr_tx_bit = '1'` and `rx_bus_s2 = '0'`. For the loser at that bit, say what each of the three terms evaluates to and why the condition fires.
- d) The loser sets `error <= '1'` and drops back to `STATE_IDLE` while the winner is still driving the bus. `error` is cleared in `STATE_START`. Explain why the loser's error would become unobservable to a caller if `STATE_IDLE` entered a reception on a dominant *level*.
- e) A tempting fix is to look for the *edge* from recessive to dominant instead, in line with how section 10.4 describes SOF. Show that that does not really work either, using the bit stuffing rule: what does the transmitter insert after five dominant bits in a row, and what does the bus do on the bit after that? Roughly how far into the winner's frame does the loser get before the edge test fires anyway?
- f) State the rule the design actually uses (section 16.1). Two facts make it hold up; give both: what the bit stuffing rule bounds inside the stuffed region, and what the unstuffed tail adds on top of that which pushes the threshold to eleven. Then say why the idle counter has to be cleared to **full** and not to empty when the node comes out of reset.

Exercise 17.3 The SOF bit nobody keeps

REASONING

STATE_SOF enables rx_shift_reg for one bit and then throws the value away. Section 17.2 explains why; this exercise is about seeing the failure with your own eyes.

- a) Take ID 0x000, DLC 0. Write out the bits a transmitter puts on the wire from SOF through the end of the control field, using section 16.2's field table, and mark where tx_shift_reg inserts every stuff bit. Remember that the run counter counts SOF.
- b) Now do the same as seen by a receiver that skipped SOF, so that its run counter is one dominant bit behind. Mark where *it* expects stuff bits. Line the two up against each other and state the first bit position where they disagree.
- c) At that position the transmitter sends a stuff bit and the receiver expects none. What does the receiver do with it, and what does that make of every bit after it?
- d) Repeat a) and b) for ID 0x123, the ID can_controller_tb uses in case 1, and confirm that the two counters agree all the way. State the general rule for which identifiers break: what has to be true of the first identifier bits, and how many of the 2048 standard IDs satisfy it?
- e) Case 2 in can_controller_tb *does* send ID 0x000. Explain why the bug would still not show up there, and say what you would add to the testbench to catch it.

Exercise 17.4 Receiving a frame with no data

REASONING

Walk through the **receive role** over a complete frame with DLC = 0, using the per-state table in section 17.2. A receiver does not know the DLC until it has decoded the control field, so this is also the path that tests whether your transition in STATE_CTRL is right.

- a) List every state the node passes through, from STATE_IDLE back to STATE_IDLE. Mark the one transition that differs from a frame carrying data, and say what makes it fire.
- b) For every STATE_LOAD_* state in your list, state the rxsr_bit_count the combinational network pre-empts, and say which bt_sample that pre-emption has to be in place for: the one triggering the transition out of STATE_LOAD_*, or the next one.
- c) Suppose the pre-emption were one cycle too late everywhere, that is, that rxsr_enable went high only once state visibly read the shift state. How many real bits would rx_shift_reg have missed for a frame with DLC = 0 by the time the node reaches STATE_CRC_L0_WAIT, and what would crc_valid read there?
- d) Your answer to c) is a CRC error reported many bit periods after the actual mistake. Relate that to section 17.5's claim that a one-cycle misalignment accumulates into a silent misalignment that only shows up later, far from its cause, and say why that makes this kind of bug expensive to debug from the testbench's output alone.

The register bank and SPI from the waveform

This is where the half begins that makes the controller reachable from a microcontroller.

IN THIS CHAPTER

- Why `tx_done` and `rx_valid` as one-cycle pulses are unusable for a polling driver, and what the register map promises instead: sticky, pollable levels with explicit clears.
- `register_bank`: the STATUS latching, `TX_SEND` as a write-triggered event, capturing a received frame, and masking on write.
- SPI from the waveform upwards: modes, MSB first, SS as framing.
- Why the slave does **not** clock on SCK but oversamples it in the 50 MHz domain.
- The `spi_slave` handed out: the byte engine getting SCK, MOSI and SS safely into the system clock domain and handing over one clean byte at a time.

`can_controller` is finished as of chapter 17. This chapter builds the first of the two layers that make it reachable from outside. The module can be written long before the controller is finished — it needs only `can_def` — so it is a full work stream that can start in parallel.

The authoritative source for every register number in this chapter is appendix D. Where the chapter and the specification disagree, the specification wins.

18.1 Why this layer exists

`can_controller`'s status outputs are made for a caller in the same clock domain that sees every cycle: `tx_done` and `rx_valid` are **one-cycle pulses**, 20 ns at 50 MHz (chapter 11). A driver polling over SPI samples the system a few *microseconds* at a time at best — a pulse is gone thousands of cycles before the next poll arrives. The register map (appendix D) therefore promises something else: **sticky, pollable levels** with explicit clears (`RX_ACK`, `ERROR_FLAGS`), and write-triggered events (`TX_SEND`).

Converting between the two worlds is this module's entire job. It contains no protocol knowledge in either direction: nothing about CAN frames (that stays in `can_controller`)

and nothing about SPI (that comes later in this chapter, and in chapter 19). It is registers, latches and a pulse generator — which is exactly why it can be verified completely during a single session, before any SPI exists at all.

You have seen both of its core tricks before, in chapter 3 and 4. A sticky status bit is chapter 3’s flip-flop with the two questions answered explicitly: what sets it, and what clears it. And turning a held level into a one-cycle pulse is exactly what `button_sync`’s edge detection did in chapter 4 — here it runs the same way for `TX_SEND` (a write becomes a pulse) and the opposite way for `tx_done` (a pulse becomes a level that stays). This chapter promotes both from an exercise to a real architectural layer with a documented contract.

18.2 The interface

Sixteen ports, **every input first, then every output**, the same convention as every module in the project — and, as everywhere else in it, the binding is *positional*, so the order below is the contract `register_bank_tb` holds you to:

1 clock	5 reg_wdata	9 rx_data	13 tx_id
2 reset_s2_n	6 tx_done	10 rx_valid	14 tx_dlc
3 reg_index	7 rx_id	11 error	15 tx_data
4 reg_write	8 rx_dlc	12 reg_rdata	16 tx_req

The types come from `can_def` (use `work.can_def.all;`), so the ports facing the controller have exactly the same types as `can_controller`’s own. Besides `clock` and `reset_s2_n` (ports 1 and 2), the other fourteen fall into two groups.

The side facing the bridge (ports 3–5 and 12). A deliberately minimal register access port — what chapter 19’s bridge will drive, and what the testbench drives directly today:

- `reg_index` : `std_logic_vector(3 downto 0)` — which register, the protocol’s index (offset divided by 4).
- `reg_write` : `std_logic` — a **one-cycle strobe**; at its rising clock edge `reg_wdata` is committed to the addressed register.
- `reg_wdata` : `std_logic_vector(31 downto 0)` — the value to write.
- `reg_rdata` : `std_logic_vector(31 downto 0)` — the addressed register’s current value, **combinationally**: it follows `reg_index` with no clock edge involved. The latch-once-on-read rule from the protocol specification is the *bridge’s* job (chapter 19), not this module’s — the bank always tells the current truth, and the bridge decides when it takes its single atomic sample.

The side facing the controller (ports 6–11 and 13–16). Mirrors `can_controller`’s register-side ports one for one: `tx_id`, `tx_dlc`, `tx_data` and `tx_req` out to the controller, and `tx_done`, `rx_id`, `rx_dlc`, `rx_data`, `rx_valid` and `error` back from it. In chapter 19’s top level they are wired straight across, port to port, with nothing in between.

reset_s2_n is the *synchronised* reset, as for every subblock in the project. Chapter 19’s top level owns a meta_prev for it, exactly as can_controller owns its own internally.

18.3 The three STATUS bits: one latch each

Every STATUS bit is a flip-flop with a named set condition and a named clear condition, and nothing more:

Bit	Meaning	Set by	Cleared by
0	TX ready	reset; the controller’s tx_done pulse; a rising edge on error while this bit is low; an accepted write to TX_ABORT	an accepted write to TX_SEND
1	RX valid	the controller’s rx_valid pulse	a write of 0x1 to RX_ACK
2	Error	a rising edge on the controller’s error level	a write of 0x0 to ERROR_FLAGS

Three of them deserve a second look:

- **Bit 0 starts at '1'.** Out of reset the controller is idle, so the bank is ready to take a frame — a driver polling before it has ever transmitted has to see “ready”, or send() would fail forever on a freshly powered system.
- **Bit 2 latches an edge, not a level.** The controller holds error high until the next frame begins (chapter 11), on its own schedule. Latching the rising edge gives the *driver* a life cycle of its own for the flag: a clear removes it even while the controller’s level is still high, and the latch is not set again until a genuinely *new* error produces a new rising edge. Latching the level instead would make the clear appear broken — cleared and immediately set again — for as long as the controller holds the line.
- **Bit 0 has three ways back up, and that is deliberate.** Read the two extra ones as belt and braces around the one failure that can take the whole node out.

A transmission losing arbitration ends without ever reaching EOF. Chapter 17 has the controller pulse tx_done on that path too, precisely so that this bit comes back — but the controller is *yours*, and a version that forgets leaves the bit low forever, so every later TX_SEND is discarded and the node cannot transmit again until it is reset. The bank therefore does not rely on the controller getting it right. **A rising edge on error while bit 0 is already low can only mean that a transmission ended badly**, since a node is either transmitting or receiving, and a reception error cannot occur while a transmission is in progress. So that edge sets the bit too, and no role port is needed to know it.

TX_ABORT is the third path: an explicit escape hatch the driver can pull if it judges that it has waited long enough. Nothing in the design should ever need it. Build it in anyway — the cost is a single write-only register, and the alternative is a node a driver has no way of freeing.

A single ordering rule ties the latches together: in the clocked process, **the controller's events are applied after the bridge's writes**, so that if a TX_SEND write and a tx_done pulse ever landed on the same edge the hardware wins and the bank stays ready. A conflict that rare should be resolved in the direction that cannot deadlock the system, and that is the same principle the paragraph above applies three times over.

18.4 TX_SEND: a write becomes a pulse

A TX_SEND write with `reg_wdata(0) = '1'`, arriving while TX ready is set, does exactly two things at that clock edge: raises `tx_req` **for one cycle**, and clears TX ready. More precisely: `tx_req` is high for the single cycle *after* the write's clock edge, and low again at the next one — the testbench checks both ends of that claim.

Everything else about this register is what it *ignores*:

- A write while TX ready is low (a transmission is in progress) is discarded entirely — no pulse, no state change. Overwriting `tx_id`, `tx_dlc` or `tx_data` mid-transmission is the corruption the driver's own ready check guards against from the software side; this is the same guard on the hardware side, and neither side may assume the other is there.
- A write with `reg_wdata(0) = '0'` does nothing: the register map says “write 0x1 to trigger”, and the bank holds it to that.
- Reads give zero — TX_SEND stores nothing, it *does* something.

Why the pulse discipline matters shows most clearly in the failure it prevents. `can_controller` acts on `tx_req` as soon as it is high and the bus is idle (chapter 16), so a `tx_req` left *held* requests afresh the instant a frame ends, forever, wiping error at every pass back through the start state. The request has to be an event, and something has to make it one. At this layer that is easier — a write strobe is already an event, not a level — but it still has to be *enforced*: one write, one pulse, whatever the master does next.

18.5 TX_ABORT: the escape hatch

Index 12, write-only, and the mirror image of TX_SEND: a write of 0x1 sets TX ready back to '1'. A write of anything else does nothing, and a read gives 0x00000000.

It touches nothing else. It does not reach `can_controller` — the controller has no abort input, its fifteen ports were fixed in chapter 11 and `can_controller_tb` binds them positionally — and it needs none. A controller that has aborted a transmission is already back in STATE_IDLE and perfectly able to transmit again; the only thing stuck is this bank's sticky bit. TX_ABORT frees it, and that is the whole module.

It is worth a moment's pause, since it is the bug's general form. Nothing in the *hardware* had deadlocked. What had deadlocked was a **status bit whose only set condition hung on an event that no longer occurred**. Every sticky flag with one way up and one way down is one missed event away from the same failure, and the fix is always the same: make sure

every exit from the busy state also raises the flag, and give the caller a way to force it in case you are wrong about “every”.

18.6 The RX side: capturing at the pulse

When `rx_valid` pulses, the bank does two things at the same edge: sets STATUS bit 1, and **captures** `rx_id`, `rx_dlc` and `rx_data` into the four `RX_*` registers — `rx_data`’s upper word (bits 63–32, data bytes 0–3, byte 0 in the most significant position) into `RX_DATA_L0`, its lower word into `RX_DATA_HI`.

Capturing, instead of wiring the controller’s ports straight to the read multiplexer, is what gives the driver a stable frame to read: the `RX_*` registers hold their values while the driver works its way through four separate SPI read transactions, whatever the controller’s ports do meanwhile. A write to `RX_ACK` clears *only the flag* — the data registers keep their contents until the next capture, which is harmless: the register map’s contract is that `RX_*` is meaningful *while RX valid is set*.

What the capture does **not** give the driver is buffering. A new frame completing before the previous one has been acknowledged is simply captured over it — one frame is held, the newest wins. That is not this module being lazy; it is a documented gap inherited from the controller itself, which holds exactly one received frame and overwrites it at the next (chapter 11). The bank makes that gap visible at the register level instead of hiding it, and chapter 19 returns to what it costs a driver.

18.7 Masking on write

Writable registers store only the bits that exist in hardware, and read back as zeros above them:

Register	Bits stored	A write of 0xFFFFFFFF reads back as
<code>TX_ID</code>	10–0 (<code>id_t</code>)	0x000007FF
<code>TX_DLC</code>	3–0 (<code>dlc_t</code>)	0x0000000F
<code>TX_DATA_L0/HI</code>	all 32	0xFFFFFFFF

Note what the bank does *not* do: validate. A `TX_DLC` of 0xF is stored and passed on to the controller as it is — rejecting `dlc > 8` is the driver’s responsibility, in the parallel software class, and doing it twice in two languages is exactly how the two copies drift apart. The bank masks to the bits that physically exist; the driver enforces the meaning. STATUS and the `RX_*` registers ignore writes entirely, and the reserved indices 13–15 read as zero and ignore writes, exactly as the protocol specification requires.

The outputs to the controller come directly from the TX registers: `tx_data <= TX_DATA_L0 & TX_DATA_HI` — 64 bits with data byte 0 in the most significant position, the layout `can_controller` expects (chapter 16) and the same packing the driver performs on the other side of the map (see the data byte order diagram in appendix D).

18.8 Writing `register_bank.vhd`

The file lands in `bridge/`, beside your copied controller modules. The shape is two processes and a handful of concurrent assignments:

- **One clocked process** owning every register and latch: the reset values (TX ready '1', everything else zero), the write case over `reg_index`, and then the three updates from the controller's events, in that order.
- **One combinational process** for the read multiplexer: default `reg_rdata` to all zeros, and then overwrite the bits every register actually has — the same “assign a default, override in specific branches” style as every counter and state machine from chapter 6 onwards.
- Concurrent assignments wiring out `tx_id`, `tx_dlc`, `tx_data` and `tx_req` from the TX registers and the pulse flip-flop.

Analyse it and then run the testbench handed out. `can_def.vhd` is a directory away, in `controller/`: the bank reads `can_def` for its controller-facing types but belongs on the SPI side of the protocol boundary.

```
cd bridge
ghdl -a --std=93 ../controller/can_def.vhd register_bank.vhd register_bank_tb.vhd
ghdl -e --std=93 register_bank_tb
ghdl -r --std=93 register_bank_tb --assert-level=error
```

What the testbench checks. `register_bank_tb.vhd` works through the contract in eleven cases, in order: the reset state (bit 0 set, everything else zero); write and read back with masking on every writable register, and the controller-facing outputs following them; `TX_SEND`'s one-cycle pulse and the clearing of ready; the write discarded while a transmission is in progress; `tx_done` restoring ready; a `TX_SEND` write of `0x0` doing nothing; capturing a received frame, that it holds even as the inputs change, that it is read-only, and `RX_ACK`; the error latch's edge semantics, including the write-zero rule and a clear while the level is still high; the reserved indices; and last the two recovery paths returning TX ready after a lost arbitration, one automatic and one driven by `TX_ABORT`.

Every case comment in the file says what a failed case means, so read the testbench before you write the module — it is the executable form of this chapter. The last two cases are particularly worth reading in advance: without them the node sticks after its first lost arbitration, and since a two-node bus provokes that in normal operation it is no corner case.

18.9 SPI from the waveform upwards

The register bank presents registers. Something has to reach them from outside, and that is SPI. This section takes the transport from the waveform upwards, and ends in the `spi_slave` handed out, which you read rather than write.

Mode 0, MSB first, SS as framing

The project runs **SPI mode 0** (CPOL = 0, CPHA = 0) throughout:

- SCK idles low. The master generates it; the slave never produces a clock.
- Both sides **sample on the rising edge**. MISO therefore changes on the **falling** edge, so that the value has settled by the next rising one.
- **MSB first**, in every byte.
- SS is active low and is the **framing signal**: one transaction per low period. The master has to raise SS between transactions.
- SCK runs at at most 1 MHz. The reason is in section 18.9.1.

Draw the waveform by hand before you read on: SS falling, eight rising SCK edges with the byte 0xA5 on MOSI, most significant first, and SS rising. That is the whole picture, and Exercise 18.4 builds a five-transaction byte out of it.

18.9.1 Why the slave does not clock on SCK

The tempting reading is that SCK is a clock, so make it *your* clock: clock the shift register on it and be done. Do not, for two separate reasons.

The first is metastability, and it is chapter 4's rule unchanged. SCK, MOSI and SS all come in on pins driven by a microcontroller with its own crystal. They are asynchronous to clock in exactly the sense chapter 4 means, and every one of them has to go through two flip-flops before any logic gets to look at it.

The second is what a design with two unrelated clocks costs in an FPGA. A second clock domain means a second set of timing constraints, a genuine clock domain crossing on every signal passing between them, and a synthesis tool that can no longer analyse the design as a whole. For an interface running at 1 MHz against a fabric running at 50, that price is unnecessary.

The answer is **oversampling**: synchronise SCK into the 50 MHz domain and *detect its edges* instead of clocking on it. At 1 MHz that is at least 50 system cycles per SPI bit, an order of magnitude of margin. The protocol therefore guarantees correct operation up to 1 MHz; faster may work, and is deliberately not promised.

It costs a few 50 MHz cycles per edge, and that is where the protocol's timing requirements around SS come from: at least 60 ns (three cycles) before the first SCK edge, between transactions, and after the last edge. Appendix D has the whole table.

18.9.2 Reading spi_slave.vhd

spi_slave is handed out rather than written: it is transport, not the book's subject. But you have to be able to read it, and chapter 20 expects that you can.

```

bridge/spi_slave.vhd
-----
-- Byte-level SPI slave.
--
-- Inputs:
--   - clock      : 50 MHz system clock.
--   - reset_s2_n: Asynchronous active-low reset.
--   - sclk       : SPI clock.
--   - mosi       : Master Out Slave In.
--   - ss        : SPI Chip Select.
--   - tx_data    : Next byte to shift output.
-- Outputs:
--   - miso       : Master In Slave Out.
--   - rx_data    : Incoming byte.
--   - rx_valid   : '1' if incoming byte is fresh.
--   - ss_active  : '1' while selected (ss low).
-----

library ieee;
use ieee.std_logic_1164.all;

entity spi_slave is
    port(clock, reset_s2_n : in std_logic;
          sclk, mosi, ss    : in std_logic;
          tx_data           : in std_logic_vector(7 downto 0);
          miso              : out std_logic;
          rx_data           : out std_logic_vector(7 downto 0);
          rx_valid, ss_active: out std_logic);
end entity;

architecture behaviour of spi_slave is

    -- Two synchronizer stages, plus a third flop holding last cycle's s2 so an edge ↵
    --   ↵ can be detected
    -- by comparing the two. sclk and ss are used for their edges and need all three; ↵
    --   ↵ mosi is only
    -- ever read at an edge detected on sclk, so it needs the two synchronizer stages ↵
    --   ↵ and no history.
    type sync_t is record
        s1: std_logic; -- First synchronizer stage; may go metastable.
        s2: std_logic; -- Second synchronizer stage: the current, safe-to-use value.
        s3: std_logic; -- Previous cycle's s2, for edge detection.
    end record;

    -- Synchronized SPI signals.

```

```

signal sclk_s, ss_s      : sync_t;
signal mosi_s1, mosi_s2 : std_logic;

-- Bit counter.
signal bit_counter: natural range 0 to 7;

-- Shift registers.
signal rx_sh, tx_sh: std_logic_vector(7 downto 0);

begin
    ss_active <= not ss_s.s2;
    miso      <= tx_sh(7) when ss_s.s2 = '0' else '0';

    -- Synchronize SPI inputs.
    process(clock, reset_s2_n) is
    begin
        if (reset_s2_n = '0') then
            -- Each chain resets to the level its line idles at, so the module ↵
            ↵ comes out of reset
            -- believing what is actually true of an idle bus. sclk idles low in ↵
            ↵ SPI mode 0; ss
            -- is active low and idles HIGH, so resetting its chain to '0' would ↵
            ↵ make ss_active
            -- read '1' out of reset and the bridge see a frame nobody started. ↵
            ↵ This is the same
            -- rule L04's button_sync exercise exists to enforce.
            sclk_s <= ('0', '0', '0');
            ss_s   <= ('1', '1', '1');
            mosi_s1 <= '0';
            mosi_s2 <= '0';
            elsif (rising_edge(clock)) then
                sclk_s.s1 <= sclk;
                sclk_s.s2 <= sclk_s.s1;
                sclk_s.s3 <= sclk_s.s2;
                ss_s.s1   <= ss;
                ss_s.s2   <= ss_s.s1;
                ss_s.s3   <= ss_s.s2;
                mosi_s1   <= mosi;
                mosi_s2   <= mosi_s1;
            end if;
        end process;

    -- Run SPI slave logic.
    process(clock, reset_s2_n) is

```

```

variable sclk_rising, sclk_falling, ss_falling: std_logic := '0';
begin
    if (reset_s2_n = '0') then
        bit_counter <= 0;
        rx_sh       <= (others => '0');
        tx_sh       <= (others => '0');
        rx_data     <= (others => '0');
        rx_valid    <= '0';
    elsif (rising_edge(clock)) then
        rx_valid <= '0';

        -- Detect edges on sclk and ss.
        sclk_rising := sclk_s.s2 and (not sclk_s.s3);
        sclk_falling := (not sclk_s.s2) and (sclk_s.s3);
        ss_falling  := (not ss_s.s2) and ss_s.s3;

        -- Deselected: hold the bit counter at zero and ignore the SPI lines
        -- entirely. SCK
        -- belongs to whichever slave the master is addressing, and a line
        -- that is merely
        -- ringing is not data. Without this gate the branches below would
        -- shift MOSI and
        -- pulse rx_valid on edges arriving while ss is high, and
        -- spi_reg_bridge would lose
        -- byte alignment for every transaction afterwards.
        if (ss_s.s2 = '1') then
            bit_counter <= 0;

            -- ss just fell: a transaction is starting. Load the first byte to
            -- send and start the
            -- frame at bit 0. This arm comes before the SCK arms rather than
            -- beside them, so a
            -- rising SCK edge landing on this same system-clock cycle cannot
            -- advance the counter
            -- past the start of the frame and misalign every byte in it.
            elsif (ss_falling = '1') then
                bit_counter <= 0;
                tx_sh      <= tx_data;

            else
                -- Sample MOSI, MSB first on rising SCLK edge.
                if (sclk_rising = '1') then
                    if (bit_counter = 7) then
                        rx_data <= rx_sh(6 downto 0) & mosi_s2;

```



```

        rx_valid    <= '1';
        bit_counter <= 0;
    else
        rx_sh       <= rx_sh(6 downto 0) & mosi_s2;
        bit_counter <= bit_counter + 1;
    end if;
end if;

-- Present the next MISO bit on falling edge.
if (sclk_falling = '1') then
    if (bit_counter = 0) then
        tx_sh <= tx_data;
    else
        tx_sh <= tx_sh(6 downto 0) & '0';
    end if;
end if;
end if;
end process;
end architecture;

```

Four things are worth reading closely:

The record `sync_t` has three fields, but only two are synchronisation stages. `s1` and `s2` are the two-flip-flop synchroniser; `s3` is last cycle's `s2`, and exists only so that an edge can be detected by comparing the two. `sclk` and `ss` are used for their *edges* and need all three; `mosi` is read only at an edge already detected on `sclk`, so it needs the two synchronisation stages and no history.

The two chains reset to different values. `sclk_s` is cleared to all zeros and `ss_s` to all ones. Every chain resets to the level its wire idles at, so that the module comes out of reset believing what is actually true of an idle bus: `sclk` idles low in mode 0, and `ss` is active low and idles **high**. Clearing the `ss` chain to '0' would make `ss_active` read '1' out of reset and the bridge see a frame nobody started. It is exactly the same rule `button_sync` in Exercise 4.9 exists to uphold.

A deselected slave ignores the SPI lines entirely. The handling of `SCK` edges sits inside the `else` arm of a test on `ss_s.s2`. `SCK` belongs to the slave the master is addressing; for this slave it is an asynchronous input arriving over a wire on a breadboard, and a disturbance on that wire looks just like an edge. Without that gating the branches below would shift in `MOSI` and pulse `rx_valid` on edges arriving while `ss` is high, and `spi_reg_bridge` would lose byte alignment for every transaction thereafter.

`ss_falling` is an `elsif` before the `SCK` branches. The order matters at exactly one clock edge: the one where `SS` has just fallen *and* a rising `SCK` edge is detected at the same time.

Because the `ss_falling` arm comes first, that SCK edge cannot step the bit counter past the frame's beginning and scramble every byte in it.

`spi_slave` hands over one complete byte at a time on `rx_data`, with `rx_valid` as a pulse, and takes the next byte to send on `tx_data`. It knows nothing about registers. Chapter 19 builds `spi_reg_bridge`, the layer turning those byte streams into five-transaction bytes and into register accesses against the bank you have just written.

18.10 Summary

What you should be able to do now

- Explain the difference between a pulse and a sticky level, and why the register layer has to convert between them.
- Implement `register_bank.vhd` per the register map: the STATUS bits' setting and clearing, `TX_SEND` as a one-cycle pulse, and the masking on write.
- Read an SPI timing diagram in mode 0 and say which bit is on MOSI at every edge.
- Explain why an SPI slave in an FPGA oversamples SCK instead of using it as a clock.
- Read and use `spi_slave.vhd` without writing it.

Questions to test yourself with

1. Why can a driver polling every five microseconds not see a 20 ns pulse?
2. What sets STATUS bit 0, and what clears it?
3. Why does the error bit latch on a *rising edge* of error and not on the level?
4. A second frame is received before the first has been acknowledged. What happens, and why is that a deliberate choice rather than a bug?
5. STATUS bit 0 is set by three things during operation, besides reset. Which, and why is `tx_done` alone not enough?
6. What does `TX_ABORT` do, and what does it *not* reach?
7. A write of `0xFFFFFFFF` to `TX_DLC` reads back as `0x0000000F`. Where does the masking happen, and why does the bank not validate that the value is at most 8?
8. Why does `spi_slave` not use SCK as a clock? What would go wrong if it did?
9. Why does MISO change value on SCK's falling edge?

Target

`register_bank` is a self-contained module with a complete testbench and should be finishable during the session and the week after. It does not depend on SPI at all, so it can be written in parallel with somebody else in the group reading up on the transaction format. Put the `spi_slave.vhd` handed out into the repository unchanged, and read through `spi_reg_bridge_tb.vhd` in preparation for the next chapter: it is the contract for the module you write then.

Looking ahead

Chapter 19 builds the SPI bridge, the top level `can_spi_node`, and takes the whole chain out onto real hardware. It is the project's last session.

18.11 Exercises

Write your own `register_bank.vhd` from section 18.2 to 18.8; the testbench handed out, run per appendix A, is the check that it behaves correctly. Exercise 18.4 and 18.5 are about `spi_slave.vhd`, which is handed out rather than written — that one you read, and chapter 20 expects you to know it.

The authoritative source for every register number below is appendix D. Where an exercise and a specification disagree, the specification wins, and the exercise is the bug.

Exercise 18.1 `register_bank`

VHDL

Write your own `register_bank.vhd` from the chapter. Verify it:

```
cd bridge
ghdl -a --std=93 ../controller/can_def.vhd register_bank.vhd register_bank_tb.vhd
ghdl -e --std=93 register_bank_tb
ghdl -r --std=93 register_bank_tb --assert-level=error
```

If a check fails, read the assertion message; each of the eleven cases says what a failure in that particular case means.

Exercise 18.2 The three STATUS bits

REASONING

- a) Out of reset, STATUS reads 0x00000001. Say which bit that is and why the reset value is '1' and not '0'. Then describe what a driver's `send()` would do, forever, if it were '0'.
- b) Bit 2 latches a **rising edge** of the controller's error, not its level. Suppose it latched the level instead. Write down the sequence of driver operations — poll, clear, poll — that would then behave differently, and say what the driver's author would conclude was broken.
- c) Bit 1 is set by `rx_valid` and cleared by a write to `RX_ACK`. Bit 0's main set condition is `tx_done`, and it is cleared by a write to `TX_SEND`. One of those pairs has its set and clear conditions in the "natural" order, and the other is reversed. Say which is which, and why the reversed one is right all the same.
- d) Section 18.3 gives an ordering rule: controller events are applied **after** the bridge's writes in the clocked process. Construct the one clock edge where that rule changes the outcome, name both events, and say which of the two possible resolutions can lock the system up and which cannot.
- e) Bit 0 has *three* set conditions, and one of them is a rising edge of error while bit 0 is already low. Explain why that condition needs to know nothing about whether the controller was transmitting or receiving — what is it about a node's behaviour that makes "error rose while we were busy" mean "a transmission has just ended"?
- f) Remove that condition from your bank and rerun the testbench. Which case fails, and what consequence does its message say it would have had? Then explain why this is defence in depth rather than duplicated work, given that chapter 17 already has the controller pulse `tx_done` at an aborted transmission. (Who writes `can_controller`? Who wrote the testbench that would catch it?)
- g) `TX_ABORT` is the third path, and unlike the other two it does not correspond to anything the hardware did — it is the driver going around the bank. Say what it does *not* do: name at least two things you might expect a register called `TX_ABORT` to reach, and explain why it reaches neither. (The clue is `can_controller`'s port list, settled in chapter 11.)

Exercise 18.3 TX_SEND and the pulse discipline

SIMULATION

- a) A write of 0x1 to TX_SEND raises tx_req for exactly one cycle. Say exactly which cycle, relative to the clock edge that carried reg_write, and which two things register_bank_tb checks about it.
- b) Now break it deliberately. Change your TX_SEND branch so that tx_req becomes a **level**: set it at an accepted write and clear it only when tx_done arrives. Rerun register_bank_tb. Which check fails first, and what does its message say?
- c) Your broken version from b) would still *look* correct on an oscilloscope watching a single frame. Describe, using section 16.1's rule for when can_controller acts on tx_req, what it does at the second frame and every frame after it. Then say why this is a bug simulation catches easily and a test on the bench does not.
- d) A write to TX_SEND while STATUS bit 0 is low is discarded entirely — no pulse, no state change. Another design could queue it instead and fire when the controller reports ready. Give a concrete argument for the discard rule, given that the driver can poll STATUS. Put the module back as it was.

Exercise 18.4 Reading an SPI transaction off the waveform

REASONING

This chapter is called “from the waveform” for a reason: chapter 20 may well hand you one and ask what it means. Mode 0 throughout — SCK idles low, MOSI is sampled on the **rising** edge, MISO changes on the **falling** edge, MSB first.

- a) Draw by hand the whole five-byte transaction writing 0x00000001 to TX_SEND. Work out the command byte from the format in appendix D rather than copying it: bit 7 is the write bit, bits 6–4 are reserved, bits 3–0 are the register index, and TX_SEND is index 5. Label SS, SCK and MOSI, and mark the 40 rising edges in groups of eight.
- b) Mark on the same drawing where the master must not release SS, and what the slave has to do if it does anyway. Restate the rule from the protocol specification's section on aborts in your own words.
- c) A read of STATUS is the mirror image: command byte 0x00, four dummy bytes out, four bytes of register value back on MISO. The protocol specification says the slave latches the register value **once**, at the end of the command byte. Explain what a driver could observe if the slave instead reread the register for each of the four bytes, and why STATUS in particular is the register where that would hurt.
- d) Count the SCK edges in a whole transaction and, at the maximum of 1 MHz the protocol specification allows, work out how long a transaction takes. Compare that with the 20 ns pulse the register bank exists to translate. That ratio is the whole argument for this layer, in a single number.

Exercise 18.5 Reading `spi_slave.vhd`

REASONING

You do not write this module, but you have to be able to read it.

- a) `spi_slave` does not use SCK as a clock. It synchronises it into the 50 MHz domain and detects its edges instead. Give two separate reasons, one about metastability (chapter 4) and one about what a design with two unrelated clocks costs in an FPGA.
- b) The record `sync_t` has three fields, but only two of them are synchronisation stages. Say what the third is for, and why `mosi` needs two fields where `sclk` and `ss` need three.
- c) In the reset branch, `sclk_s` is cleared to all zeros and `ss_s` to all **ones**. Explain why they differ, and then trace what `ss_active` would read during the first two clock edges after reset if `ss_s` were cleared to zeros with no master connected. This is the same rule Exercise 4.9 was built around; say which check in that exercise's testbench corresponds to it.
- d) The SCK edge handling sits inside the `else` arm of a test on `ss_s.s2`, so a deselected slave ignores the SPI lines entirely. Construct the failure that would follow if it did not: start from a stray SCK edge while `SS` is high, and follow it through to what the *next* register write lands on.
- e) The `ss_falling` branch is an `elsif` before the SCK branches rather than a separate `if` beside them. Say at which single clock edge that order matters, and what would be wrong with the received bytes if the two branches ran in the reverse order.

Exercise 18.6 The capture, and the gap it does not close

REASONING

- a) When `rx_valid` pulses, the bank captures `rx_id`, `rx_dlc` and `rx_data` into the `RX_*` registers instead of wiring the controller's ports straight into the read multiplexer. Explain what a driver reading four separate SPI transactions would see without that capture.
- b) A write to `RX_ACK` clears `STATUS` bit 1 but leaves the `RX_*` registers holding their contents. Say why that is harmless, and quote the sentence in the register map that makes it so.
- c) A second frame arrives before the first has been acknowledged. Say exactly what the driver observes, and why this is a documented gap inherited from `can_controller` (chapter 11) rather than a shortcoming of the register bank.
- d) Design, on paper alone, the smallest change to the *register map* that would let a driver detect that it had missed a frame. You are not adding a receive queue; you are adding the ability to know that one was needed. Say what it costs in hardware, and in which existing register it could live without breaking the contract the parallel class is already writing to.

The SPI bridge, `can_spi_node` and bring-up

The project's last session, and its capstone: the transaction layer, the top level tying everything together, and the whole chain verified first in simulation and then on real hardware.

IN THIS CHAPTER

- System verification: two nodes on a wired-AND bus, and what that testbench does not cover.
- The transaction FSM: command byte, four data bytes, latch once on read, commit on the fifth byte, and abort when SS goes high early.
- `can_spi_node`: composing `spi_slave`, `spi_reg_bridge`, `register_bank` and `can_controller` into one node.
- Bring-up in two halves: two boards on a shared bus line, then a microcontroller as SPI master against them, in eight steps.
- What real CAN controllers do that this design does not.

`can_controller` is finished as of chapter 17: transmission, reception and arbitration. It instantiates `bit_timer`, `crc15` and both shift registers inside itself, so from outside the whole controller is a single component with the 15-port register-level interface chapter 11 previewed.

Then two layers go in front of it: `register_bank` (chapter 18) turns that interface into the register map, and `spi_reg_bridge` plus the `spi_slave` handed out turn the register map into a five-byte SPI transaction. What the parallel class writes its driver against is the far end of that chain — the register map and the protocol specification in appendix D, never `can_controller`'s ports.

This is the book's tightest chapter, and it is worth saying outright: two modules get written, the whole chain has to be simulated, the hardware has to come up, and every group presents.

So run ahead. `register_bank` and `spi_reg_bridge` do not depend on `can_controller` at all — the former needs only `can_def`, the latter only `spi_def` — so both can be written in parallel with the controller, from chapter 10 onwards. A group arriving here with the register bank finished and the transaction format read has the session for wiring up and debugging. A group starting here will not get through it all.

19.1 Verifying the whole system

Before any part of the transport can be trusted, the controller has to be shown working as a *system*. That does not mean one module against a stub, but two independent nodes on one wire fighting over it the way real nodes do.

Modelling the bus

Nothing in the design models the shared bus, and that is deliberate. `can_controller` drives a pair (`tx_bus`, `bus_en`) and reads `rx_bus`, which is exactly what a real CAN transceiver presents on its digital side (chapter 10). The wired-AND lives in the testbench, and on hardware it lives in the transceiver.

`can_controller_tb.vhd` instantiates two nodes on one wire and combines them directly. Every node's effective output is its driven value when `bus_en = '1'`, otherwise released (`'1'`), and the bus is their AND, so it reads dominant the moment *any* of the nodes pulls low:

```
bus_line <= (a_tx_bus or not a_bus_en) and (b_tx_bus or not b_bus_en);
```

That single line is the whole of section 10.2's “any dominant beats all recessives”, and it is what makes arbitration testable in simulation.

Worth working through rather than reading. Every node contributes `tx_bus` or not `bus_en`, which is 1 as soon as it has released the bus, and the wire is the two contributions ANDed:

Node A	Node B	a_tx_bus	a_bus_en	b_tx_bus	b_bus_en	bus_line
released	released	-	0	-	0	1 recessive
driving dominant	released	0	1	-	0	0 dominant
released	driving dominant	-	0	0	1	0 dominant
driving dominant	driving dominant	0	1	0	1	0 dominant

A released node's `tx_bus` is - because it genuinely does not matter: not `bus_en` alone makes its contribution 1, so a node that has let go cannot affect the wire whatever its output happens to sit at. Across all sixteen input combinations the wire reads dominant exactly

when at least one node has `bus_en = '1'` with `tx_bus = '0'`, which is the rule expressed as hardware.

Two rows carry the arbitration argument. **Row two is the contest itself:** A drives dominant, B sends recessive and has therefore released, and the wire reads dominant. B, monitoring while it transmits, sees that it sent recessive and that the bus reads dominant, and that is exactly the loss condition chapter 17 checks. Meanwhile A reads back precisely the bit it sent and notices nothing.

Row one is why the loser can leave. Once B has stopped driving it contributes 1 forever, so the wire is A's alone. The loser's retreat is invisible to the winner, and that is what makes a collision a non-event rather than something both nodes have to recover from.

Row four is worth a look too: when both nodes drive dominant the wire is dominant and each reads back what it sent, so neither can tell the other exists. That is the normal state for every dominant bit of the identifier before the two IDs differ.

The two scenarios

1. **Reception.** Node A sends a 5-byte frame (ID 0x123, data 11 22 F8 44 55) while node B receives it. 0xF8 forces a stuff bit inside the data field, so both sides' CRC engines have to skip it. The testbench checks A's `tx_done` and B's `rx_valid`, that B reconstructed the ID, the DLC and **every** data byte at its fixed position, and that neither node raised error.
2. **Arbitration.** Nodes A (ID 0x000) and B (ID 0x001) both request during the same cycle, differing only in the identifier's last bit. A has to win and complete normally, unaware that any contest took place; B has to detect the loss at that bit, raise error and stop driving. The check runs after A's frame has completed, so it also confirms that B's error is still readable then, rather than having been wiped by a re-entry into `STATE_START`.

A transmitting node never receives its own frame (`rx_shift_reg` runs only while `role = '0'`), so both scenarios genuinely need two nodes; neither could be tested with one node alone.

The two one-cycle pulses the first scenario waits for, A's `tx_done` and B's `rx_valid`, are **latched** in the testbench, since two independent state machines need not pulse at the same edge, and a plain wait until (`a = '1'` or `b = '1'`) would resume on the first and miss the second.

What the testbench does not cover

Two passing scenarios are not the same thing as a proved design, and knowing where a testbench ends is part of reading it. Five gaps are worth naming, since each is a place where this design is either deliberately simplified or simply untested:

- **Both nodes share one clock and one reset_n.** Real nodes run on independent oscillators and drift apart. Be precise about what that leaves untested, because it is not resync itself: both nodes pulse resync at every frame's start (chapter 16), and that pulse is what

phases B's sample points against the arbitrary moment A began transmitting, so case 1 fails without it. What a shared clock can never exercise is **drift**, two timers starting in phase and wandering apart mid-frame, the case Exercise 12.4 quantifies and the reason real CAN resynchronises at every recessive-to-dominant edge instead of once per frame. This is the largest gap, and closing it in simulation takes a second clock generator with a deliberate frequency deviation. Section 19.6 gets a frame across **two boards** with two separate crystals, finally genuinely independent clocks, with an honest caveat noted there.

- **No frame is ever corrupted.** Nothing injects a stuffing error, flips a bit on the wire or damages a CRC, so the receive side's error paths are exercised only at module level by `rx_shift_reg_tb` and `crc15_tb`, never end to end.
- **The ACK slot is driven but never read back.** A receiver with a valid CRC pulls it dominant (chapter 17), but the transmitter never checks it, so a frame nobody acknowledged completes exactly like one that was. That is a deliberate simplification (section 19.7), and the testbench inherits it instead of catching it.
- **Arbitration is always two-way, and always lost at the same bit.** `0x000` against `0x001` differ in the identifier's last bit, so the loss is detected in the same state every run. Exercise 19.1 adds a third node and moves the losing bit earlier.
- **No node ever transmits again after an abort.** Node B loses arbitration in case 2 and is never asked to send another frame, so nothing here checks that a node recovers from an abort at all. That matters more than it sounds: `can_controller` clears `crc15` in `STATE_START` (chapter 16) precisely so that an abandoned frame cannot leave its residue behind, and this testbench would pass just as happily with that clear removed. Exercise 19.7 is where you find out what it did.

19.2 The transaction bridge `spi_reg_bridge`

`spi_slave` (section 18.9.2) speaks only *bytes*. `register_bank` (chapter 18) speaks only *register semantics*. Between them sits a layer that speaks *transactions*, and that is `spi_reg_bridge`. It knows nothing about which registers exist, and nothing about what any of them means.

Interface

Ten ports, in this order: `clock`, `reset_s2_n`, `ss_active`, `rx_data`, `rx_valid`, `reg_rdata`, `tx_data`, `reg_addr`, `reg_wdata`, `reg_write`. The first five plus `reg_rdata` are inputs; the last four are outputs. The binding is positional, as everywhere else.

The first three come from `spi_slave`: `ss_active` is high while the slave is selected, `rx_data` carries the most recently received byte, and `rx_valid` pulses once per byte. `tx_data` is the next byte for the slave to shift out. The four register signals connect straight to `register_bank`'s bridge-facing side.

The transaction

Every transaction is exactly **5 bytes** long: one command byte, then four data bytes. `SS` falls before the command byte and rises after the fifth byte; it has to stay low for the whole transaction.

Bit:	7	6	5	4	3	2	1	0
	+---+---+---+---+---+---+---+---+							
	W	0	0	0	register index			
	+---+---+---+---+---+---+---+---+							

- **Bit 7 (W)**: 1 = write, 0 = read.
- **Bits 6–4**: reserved, must be 0.
- **Bits 3–0**: the register index, offset / 4 from the register map (0–12).

Write (W = 1). The master sends the register value in the four data bytes, **MSB first** (bits 31–24 in the first data byte). The slave commits the assembled value to the addressed register **only once the fifth byte is complete**. Whatever the slave drives on `MISO` during a write is meaningless; the master discards it.

Read (W = 0). At the end of the command byte the slave **latches** the addressed register’s current value **once**, into its response shift register. The four data bytes then clock that value out on `MISO`, MSB first, while the master sends dummy bytes `0x00`. The latch-once rule is not an implementation detail but part of the contract: the four bytes always belong to *one* coherent sample of the register, taken at a single instant — `STATUS` in particular must never be stitched together from two separate moments.

Abort. If `ss_active` goes low before the fifth byte is complete, the transaction is abandoned **with no side effects**: nothing is committed, no trigger fires, and the slave returns to idle, ready for the next falling edge on `SS`. That is what makes the slave self-healing after a glitch or a master that resets.

Invalid commands. A read of a reserved index (13–15) gives `0x00000000`; a write to one is ignored. A command byte with any reserved bit (6–4) set is invalid in the same way: no read is latched and no write is committed, and the slave still consumes the whole five-byte frame before returning to idle. Neither case is an error the slave reports — there is no error channel at this layer. Enforcing the rule about reserved *command* bits is `spi_reg_bridge`’s job; `register_bank` never sees more than a 4-bit index, never a reserved command.

`reg_addr` **holds** the decoded index until the next command byte decodes a new one: it is a latched value, not a pulse following the byte in progress. The testbench checks the index once all five bytes have been consumed, so a bridge that clears `reg_addr` when it returns to idle passes the first cases and fails on that one.

Verifying it

The bridge reads no register at all and therefore needs only `spi_def`:

```
cd bridge
ghdl -a --std=93 spi_def.vhd spi_reg_bridge.vhd spi_reg_bridge_tb.vhd
ghdl -e --std=93 spi_reg_bridge_tb
ghdl -r --std=93 spi_reg_bridge_tb --assert-level=error
```

The testbench models `spi_slave` instead of instantiating it: it pulses `rx_valid` once per byte, framed by `ss_active`, so that you can write and check the bridge before any SPI waveform is involved. Four cases: a write, a read byte by byte, an abort on `ss_active`, and a command byte with a reserved bit set. Cases 3 and 4 are the two that make this module more than a byte counter — read their comments before you start, not afterwards.

19.3 Designing `can_spi_node`

`can_controller` is **not** the entity that goes onto the board, and it is worth being precise about that. Its interface is at register level: `tx_id` is 11 bits, `tx_dlc` 4, `tx_data` 64, plus `tx_req` and five status outputs. Nothing on a DE0-CV offers an interface that wide, and in the finished system those inputs come from the driver's registers rather than from a human hand.

Chapter 18 built the layer making them reachable, and section 19.2 the transport in front of it. The last module the book writes is therefore `can_spi_node.vhd`: the top level tying `spi_slave`, `spi_reg_bridge`, `register_bank` and `can_controller` together into one node, and exposing exactly two things to the outside world, SPI and the CAN bus.

Unlike every other module in the project it comes with no module-level testbench of its own, and that is deliberate: every block inside it already has one, and what remains — pins, wiring and voltage levels — is exactly what a simulation cannot model. The board is its testbench, and the bring-up steps in section 19.6 are how you read it. (There is a system testbench handed out, `can_spi_node_tb`, driving two complete nodes from modelled SPI masters; it is the only one in the book that drives `sclk`, `mosi` and `ss` as real waveforms.)

Interface

Port	Dir.	Type	Meaning
<code>clock</code>	in	<code>std_logic</code>	The board's 50 MHz oscillator. The design's only clock.
<code>reset_n</code>	in	<code>std_logic</code>	Raw, active-low reset, straight from a pin. Asynchronous; this level synchronises it.
<code>sclk</code>	in	<code>std_logic</code>	The SPI clock from the master. Never used as a clock.
<code>mosi</code>	in	<code>std_logic</code>	Out from the master, into the slave.
<code>ss</code>	in	<code>std_logic</code>	Chip select, active low. Frames the transaction.
<code>miso</code>	out	<code>std_logic</code>	Out from the slave, into the master.
<code>rx_bus</code>	in	<code>std_logic</code>	The CAN bus's level, straight from the pin and asynchronous. <code>can_controller</code> synchronises it itself.
<code>tx_bus</code>	out	<code>std_logic</code>	The bit this node wants to drive.
<code>bus_en</code>	out	<code>std_logic</code>	Whether it is driving at all.

The three bus ports are the digital-side interface a real CAN transceiver presents (chapter 10), and that is why they leave the design as three separate signals rather than one wire. Turning them into a single physical wire is a board-level job, and section 19.4 is about that.

The order is SPI's four wires as a group, `sclk/mosi/ss/miso`, and then the CAN bus's three. `miso` therefore sits among the SPI ports and not with the other outputs, so that the four pins stand in the same order as in the wiring table in section 19.6. That matters: `can_spi_node_tb` binds positionally like everything else, and `miso` and `rx_bus` have the same type, so swapping them passes analysis without a word and gives a node that never answers.

There are no generics. Everything the design needs that a pin cannot deliver is either a constant in `can_def` or a register the driver writes.

Behaviour: structural, and that is the point

`can_spi_node` has no state machine, no protocol knowledge and almost no logic at all. It has exactly three jobs:

- **Instantiate the four blocks and wire them in a row.** `spi_slave` turns SCK/MOSI/SS into bytes; `spi_reg_bridge` turns bytes into register accesses; `register_bank` turns register accesses into controller-level signals; `can_controller` turns them into a frame. Every boundary is one of the contracts already verified by a testbench handed out, so if the chain misbehaves the fault is in a connection here, not in a block.
- **Synchronise the reset.** A single `meta_prev` instance takes the raw `reset_n` and produces the `reset_s2_n` every subblock takes. `can_controller` is the exception that proves the rule: it owns its reset synchroniser internally, alongside the one for `rx_bus`, so this level feeds it the raw `reset_n` and synchronises only for the SPI-side blocks. That makes three `meta_prev` instances in the whole design — two inside `can_controller`, one here.
- **Let the controller-facing ports go straight across.** `register_bank`'s `tx_id`, `tx_dlc`, `tx_data` and `tx_req` to `can_controller`, and its `tx_done`, `rx_id`, `rx_dlc`, `rx_data`,

`rx_valid` and `error` back. Port to port, nothing in between. Chapter 18 designed the two interfaces to mirror each other precisely so that this works out.

If a rule in this file ever has to mention a CAN field or an SPI byte, it belongs in another file. That is the test to apply while you write it.

19.4 Getting the bus out onto a wire

`tx_bus` and `bus_en` are a *pair*, not one signal: the design says “drive dominant”, “release”, never “drive recessive”. Two nodes’ push-pull outputs tied to one wire would fight each other the moment one drove low and the other high. Two ways to make the shared wire behave:

With a transceiver. Which is what a real node uses: `tx_bus` goes to its TXD, its RXD comes back as `rx_bus`, and the transceiver does the open-drain work in the analogue domain.

Without one. With two DE0-CVs wired directly together for the two-node demo, the FPGA’s own I/O does the job. Pick a GPIO pin and drive it open-drain:

```
GPIO_0(2) <= '0' when ((bus_en = '1') and (tx_bus = '0')) else 'Z';
rx_bus    <= GPIO_0(2);
```

with a pull-up restoring '1' while every node has released the wire — the FPGA’s weak pull-up enabled on that pin in the Pin Planner, or an external resistor. That is section 10.2’s wired-AND rule made physical, in a single concurrent assignment. `GPIO_0(0)` and `GPIO_0(1)` are worth driving with `tx_bus` and `bus_en` unregistered as well, purely so that a logic analyser can see what the node *intended* beside what the wire actually did; every unused index is driven 'Z'.

Nothing here is synchronised, for the same reason `reset_n` is not synchronised before it reaches `can_controller`: the controller owns its own asynchronous inputs, so the pin is run straight to the `rx_bus` port.

The pin indices are a convenience in the Pin Planner, not a requirement. Any three pins on the header work, as long as two connected boards agree; see appendix B.

19.5 Optional: a board-level wrapper for debugging

All of the above is reachable only over SPI, which means that when the link does not work you have no way of seeing into it. A small wrapper with buttons and LEDs, standing beside `can_spi_node` rather than replacing it, is worth an hour if the bring-up gets stuck — and the three decisions it forces are worth understanding whether or not you build it.

The wrapper drives `register_bank`’s ports directly from buttons and switches and latches its status outputs onto LEDs:

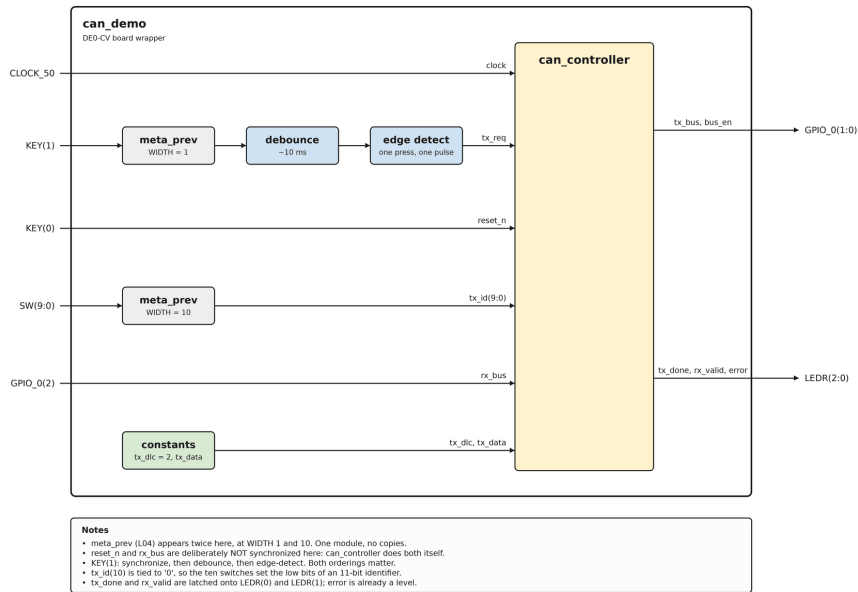


Figure 19.1 A board-level wrapper: buttons and switches into the register bank, the status bits out onto LEDs.

- **A held button has to become a one-cycle pulse**, and not just for neatness. A button held for a few milliseconds is hundreds of thousands of clock cycles at 50 MHz. Feed that straight into `tx_req` and the controller starts a new frame the moment the previous one ends, over and over — and every pass through the start state wipes error before anything has read it. So: **synchronise** the raw pin with a `meta_prev`, **debounce** the synchronised level with a counter accepting a new value only once it has held for about 10 ms, and **then** edge-detect it. All three, in that order: edge-detecting before the debounce gives one pulse per bounce, and debouncing before the synchronisation feeds the counter a signal that was never safely sampled. This is the same conversion `register_bank` performs for `TX_SEND` (chapter 18), and it is why the bank makes the wrapper optional rather than necessary.
- **A switch bank is the judgement call**. Ten switches crossing into the clock domain are synchronised bit by bit, which removes the metastability but not the *skew*: two switches flipped at the same time can land one clock apart, so `tx_id` could briefly read a value that was never on the switches. That is acceptable only because a human sets them and then leaves them alone. Connect the same ten bits to something that changes every cycle and a bit-by-bit synchroniser is entirely the wrong tool — that takes a handshake, or an encoding where only one bit changes at a time. Worth saying outright, since “just widen the synchroniser” is a habit that bites sooner or later.

- **Status LEDs need latching**, since `tx_done` and `rx_valid` are 20 ns pulses. That is the same problem `register_bank` solves for the driver, and solving it twice in two places is exactly how the two copies drift apart — so drive the LEDs from the bank's STATUS bits rather than from the controller's pulses, and the wrapper stays a wrapper.

19.6 Bring-up

The bring-up has two halves: the CAN side between two boards, and the SPI side between a microcontroller hat and the board. Take them in that order. The CAN half needs no microcontroller at all, so it can be run while the hat is still on the soldering bench, and it rules out a whole class of faults before there is any SPI wire to suspect.

Once the design passes simulation in full, synthesise and program `can_spi_node`: New Project Wizard, device 5CEBA4F23C7N, Pin Planner, Compile, Programmer. The steps are in appendix B, which is the authoritative reference for that flow, right down to the add-on for Cyclone V devices and to getting the USB-Blaster visible to the Programmer. Note that `CLOCK_50` is the one pin you may not choose freely.

The CAN half: two boards on one bus

Two boards, their bus lines on `GPIO_0(2)` tied together into **one** wire with a common ground, and the open-drain driving from section 19.4 makes the shared wire a wired-AND.

1. **One board alone.** The bus line carries nothing but the pull-up, so the node reads back exactly the level it drives: a loopback. `tx_bus` and `bus_en` should toggle field by field on a logic analyser via `GPIO_0(0)/GPIO_0(1)`, `tx_done` pulse at the end of EOF, and error never go high.
2. **Two boards, one wire.** Press a button on one board and an LED should light on the other. That is the first time the design meets two *independent* oscillators; in `can_controller_tb` the nodes shared one clock.
3. **Arbitration for real.** Have both boards transmit at once and watch the one with the lower identifier win, and the loser back off.

A loopback is also where section 10.7's 70 % sample point quietly earns its keep. The node's own signal now takes a real round trip, out through a pad, along the wire, back in, and through the `rx_bus` synchroniser's two flip-flops, at a cost of a handful of clock cycles. With `TICKS_PER_BIT = 50` and `SAMPLE_TICK = 35`, the node reads the bus 35 ticks into a bit it began driving at tick 0, so its own echo has long since settled and the arbitration monitoring never misreads it. Sampling at the bit period's *start* would instead read the echo of the previous bit, and the arbitration monitoring (sent recessive, read dominant, chapter 17) would report a false loss at the first recessive bit following a dominant one.

A loopback deliberately cannot show the node *receiving* that frame. `can_controller` latches `role = '1'` for the whole frame and `rx_shift_reg` runs only while `role = '0'`

(chapter 17), so a transmitter never decodes its own traffic; that is exactly why the system testbench needs two nodes.

The second board closes the gap section 19.1 calls the largest. The testbench's two nodes share one clock: resync still phases them at every SOF, but their timers can never *drift apart* mid-frame. Two boards have two crystals, running at 50 MHz only within the tolerance the components are specified to, so their bit periods genuinely differ, and the receiver free-runs on its own timer for the rest of the frame after its single resync at SOF. Honest arithmetic keeps expectations in check: at typical crystal tolerances of a few tens of ppm the drift over a frame is a small fraction of a tick, so what that demo visibly proves is that the whole chain works across genuinely independent clocks from an arbitrary starting phase; how much drift the design would tolerate before the sample point wanders off the bit is the calculation in Exercise 12.4.

The SPI half: a microcontroller hat against the board

The wiring, per node. It is not yours to choose: it is in appendix D, since the parallel class writes its driver against the same table.

Signal	AVR32DB28	DE0-CV
SCK	PC2	sclk
MOSI	PC0	mosi
MISO	PC1	miso
SS	PC3	ss
I/O supply	VDDI02	3.3 V
Core supply	VDD	(the hat's own 5 V)
GND	GND	GND

No level shifter is needed, and that is a choice and not a simplification: the AVR32DB28's PORTC is a voltage domain of its own, supplied from VDDI02 rather than VDD, so it is put on the DE0-CV's 3.3 V while the core stays at 5 V. Every level the FPGA sees is then a level it is specified for, and the 3.3 V the FPGA drives back is read against a 3.3 V threshold rather than a 5 V one. SPI0 is moved there with PORTMUX.SPIROUTEA = ALT1.

Three requirements on the hat, and they have to be in place before it is etched.

- **PC0–PC3 are reserved.** They are SPI0's ALT1 mux and must carry no other peripheral. No LED, no button, and above all no decoupling capacitor on SS — a button with 100 nF on the pin makes the SS edge unusable at 1 MHz.
- **Nothing else on PORTC.** The whole port sits in the VDDI02 domain at 3.3 V, so a 5 V peripheral there would both stop working and feed 5 V into a 3.3 V domain.
- **VDDI02 is a track of its own.** It must not be tied to VDD on the board, but brought out to a pin where the DE0-CV's 3.3 V is connected, with 100 nF near the chip.

Whether the domain is separate at all is moreover controlled by `MVSYS_CFG` in `FUSE.SYS_CFG1`: in single-supply mode `VDDIO2` is tied internally to `VDD`, `PORTC` becomes a 5 V port, and the wiring above then puts 5 V straight into inputs that are not 5 V tolerant — on a hat that looks exactly like a correctly built one. Check the fuse before the first connection. It is the one fault in this chapter that can cost a board.

The steps are taken in this order, and in no other. Every step rules out a source of failure, so that whatever still does not work after step n is certainly in step $n + 1$:

4. **SPI loopback on the hat alone**, with `PC1` connected to `PC0` and the FPGA disconnected. Proves that the master works, and along the way that `PORTMUX` and `VDDIO2` are set correctly.
5. **Register echo**: write `TX_ID`, read it back. Proves that the whole transaction chain carries.
6. **STATUS polling**: read `STATUS` and see that bit 0 is set after reset.
7. **One node transmits on command**: write a frame and trigger `TX_SEND`, and watch `STATUS` bit 0 go low and back — now triggered over SPI instead of by a button.
8. **Two nodes, the whole chain**: one transmits on an SPI command, the other receives and acknowledges with `RX_ACK`. That is the system.

What remains. By the end of the session both halves have run on real hardware. What remains is not the wiring but the **counterpart**: your node has answered a test program written to succeed, not a driver written by somebody else against the specification alone. That is why the protocol specification's timing requirements — the 60 ns around `SS`, the rule about what `MISO` carries during the command byte, and the abort rule — are written as requirements and not as advice: a test program that never does anything unexpected never confirms them, and a driver that does will fail you on them.

19.7 How production CAN controllers go further

This book's controller is deliberately simplified: a teaching design rather than a certifiable one. The list below is this book's version of it; chapter 7 of *CAN: the bus, the frame and the controller* sets out the same list from the protocol side, together with the block overview and the three limitations reaching all the way up into the driver's code, and chapter 6 of *CAN: the bus, the frame and the controller* describes the error handling — error frames, error counters, error-passive and bus-off — that this design lacks entirely.

Notable gaps, worth naming explicitly:

- **Standard data frames only**: the wire format is a bit-exact standard CAN data frame, RTR included, but there are no 29-bit extended identifiers (the receiver does not even check IDE, so an extended frame would be misinterpreted until its CRC check failed), and a remote frame (a recessive RTR) is rejected with error instead of being answered with the requested data.

- **No error frames and no bus-off recovery:** a real CAN node counts transmit and receive errors and enters progressively more restricted states (error-passive, bus-off) to protect a healthy bus from a broken node; this book's `can_controller` merely aborts the frame in progress locally.
- **No bit or form error detection:** this transmitter compares what it sent against what the bus read only during arbitration (chapter 17). A real controller monitors every bit it sends, flags a bit error the moment the bus disagrees outside the arbitration field and the ACK slot, and on reception checks that the tail's fixed fields have their prescribed values.
- **No retransmission on a missing acknowledgement:** a real controller retries automatically. This design does not even notice, since the transmitter never reads the ACK slot back.
- **No acceptance filtering and no receive buffering:** every frame on the bus lands in the same single set of `rx_*` ports, and a second frame arriving before the driver has read the first simply overwrites it. Production controllers offer identifier filters in hardware and a receive FIFO or a set of mailboxes. This is the gap the parallel class's driver feels most directly, and chapter 18 shows exactly where it becomes visible at register level.
- **A fixed bit timing model:** this book samples at a fixed 70 % point; production controllers expose configurable propagation and phase segments and a synchronisation jump width, tuned to bus length and node count.
- **An aborted reception (after lost arbitration) is simply discarded,** rather than seamlessly continuing to receive the frame that won; a real node goes on receiving whoever won the bus. This design's node waits out the rest of the frame in `STATE_IDLE` (section 16.1) and picks up from the next one.
- **A simplified bus-idle rule:** this design requires eleven recessive bit periods, the same number real CAN asks of a node joining bus traffic, before any node may leave `STATE_IDLE`. Real CAN is finer-grained: a node that has just taken part in a frame may transmit again after the 3-bit intermission following EOF, and a node that has been bus-off waits out 128 occurrences of eleven recessive bits before it may join again.

19.8 The design, seen in the rear-view mirror

Every chapter from chapter 10 onwards added exactly one responsibility, never duplicated anywhere else:

Block	Owns	Built in
<code>can_def</code>	Shared constants and subtypes every other block agrees on	chapter 10
<code>meta_prev</code>	Getting an asynchronous signal safely into this clock domain, with no idea what the signal means	chapter 12
<code>bit_timer</code>	<i>When</i> to sample or shift, with no idea what is being sent	chapter 12
<code>crc15</code>	The CRC-15 computation, one bit at a time, with no idea what a “frame” is	chapter 13
<code>tx_shift_reg</code> <code>rx_shift_reg</code> <code>can_controller</code>	/ Bit stuffing and destuffing, with no idea what an ID or a data byte is The only block that knows the actual CAN frame format, arbitration included	chapter 14–15 chapter 11 (ports), chapter 16–17 (behaviour)
<code>register_bank</code>	Pulses to pollable levels, and the register map’s semantics; no idea what SPI or CAN is	chapter 18
<code>spi_slave</code>	Bytes off a serial wire, safely into the 50 MHz domain; no idea what a register is	handed out, read in chapter 18
<code>spi_reg_bridge</code>	The five-byte transaction; no idea what any individual register means	chapter 19
<code>can_spi_node</code>	The connections, and the pins the bus lives on. No logic of its own	chapter 19

That is the whole node: four small blocks with one responsibility each and one shared package composed inside `can_controller`, and then three more layers in front of it, each knowing only the one below it. `can_spi_node` stands outside them all, since adapting to a particular board is a different matter from speaking CAN, or SPI, or registers.

Read the table top to bottom and it is also a list of what would have to change to reach a different system. Swap `spi_slave` and `spi_reg_bridge` for an I2C pair and everything beneath them is untouched. Swap `can_controller` for another protocol engine and the register map moves but the transport does not. That is what the boundaries bought.

The register map in summary. Every port on `can_controller`’s register side corresponds to a register the parallel class’s C++ driver reads or writes; the remaining five (`clock`, `reset_n` and the three bus signals) are physical and no driver ever sees them. The correspondence is no longer a plan, as it was when section 11.6 first drew this table — `register_bank` is the module that realises it, so read the right-hand column as “the register the bank presents at that index”:

can_controller port	Register in the map
tx_id, tx_dlc, tx_data	TX_ID, TX_DLC, TX_DATA_LO/HI
tx_req, tx_done	TX_SEND, one bit in STATUS
rx_id, rx_dlc, rx_data	RX_ID, RX_DLC, RX_DATA_LO/HI
rx_valid	one bit in STATUS, cleared via RX_ACK
error	ERROR_FLAGS

That correspondence is where the driver work begins: everything the parallel class writes against is in appendix D, and nothing in this design's VHDL.

19.9 Summary

What you should be able to do now

- Implement the transaction FSM per the protocol specification, including the abort rule.
- Explain why a read latches the register value once, and what goes wrong without that rule.
- Compose the whole node into a top level and verify it in simulation.
- Wire two DE0-CVs together onto a shared bus, and a microcontroller as master over its MVIO domain, and debug the chain systematically from the bottom up.
- Say what requirements the SPI connection puts on the hat's design, and why they cannot be fixed after the fact.
- Say what simulation has proved and what only hardware can prove.

Questions to test yourself with

1. Why is the register value latched once at the end of the command byte instead of being read while the data bytes are shifted out? Which register suffers most from that rule being broken?
2. SS goes high after three bytes. What has happened to the transaction, and what has changed in the node?
3. A write to a reserved index. What does the bridge do, and why does it consume all five bytes anyway?
4. Why is there no error channel in the transaction layer?
5. Which bring-up step would reveal a swapped MISO and MOSI, and which would reveal a VDDI02 that was never connected?
6. The hat puts an LED on PC2. Which step fails first, and why is that a fault you cannot program your way out of?
7. What did steps 1 to 3 prove that steps 4 to 8 did not have to prove again?
8. What has the simulation proved about the design, and what has it been unable to prove?

Final presentation

Every group shows, in about fifteen minutes: a run of `make build-project` with every testbench green; a short walkthrough of `can_spi_node`, what every block does and where the boundaries run; the bring-up, as far as you got, on real hardware; and `CONTRIBUTORS.md` plus a quick tour of the repository's history, branches, PRs and reviews.

The final presentation is the project's second and last fixed checkpoint. The project is handed in the same day; see the checklist in appendix C.

The bring-up is not graded. Getting to step 3 of eight is a good result; what is graded is the simulation behind it.

Looking ahead

Chapter 20 is the book's second practical exam, with tasks on the project's modules. After it the course is over.

The controller you have built is handed over to the parallel class, which writes the C++ driver against the same registers over the same SPI transport. The shared contract is appendix D, and nothing else.

19.10 Exercises

Exercise 19.1, 19.2 and 19.7 consolidate section 19.1. Exercise 19.3 to 19.6 consolidate section 19.2 to 19.7 and appendix D, which is the authority everywhere it and an exercise disagree.

Exercise 19.1 A third node, and a new arbitration outcome

SIMULATION

a) Extend `can_controller_tb.vhd` with a third node, C, sharing the same `bus_line`. Give C the ID `0x002` and have it request transmission on the same cycle as node A (`0x000`) and B (`0x001`). Confirm (by simulation, not just prediction) that A still wins, and that *both* B and C detect an arbitration loss.

b) Node A's `0x000` is the lowest identifier a standard frame can carry, so no choice of ID lets C beat it. Raise A's ID to `0x010`, and then choose an ID for C that makes C the winner while B (`0x001`) still loses. Justify your choice with section 10.6's arbitration rule, then confirm the new winner in simulation. One consequence to notice before your assertions surprise you: with A on `0x010`, B's `0x001` beats A as well, so in that round **A loses too** and raises error; only C completes.

Exercise 19.2 A feature from a real controller, as a design sketch

REASONING

Pick **one** entry from section 19.7's list of what real CAN controllers do that this design does not. Sketch, without necessarily implementing it in VHDL (in prose, plus an addition to the state diagram or a new port list if that helps), what would have to change in `can_controller` to support it. State which existing block or blocks (`bit_timer`, `crc15`, `tx_shift_reg`/`rx_shift_reg`, or `can_controller` itself) would have to change, and which could stand exactly as they are.

Exercise 19.3 `spi_reg_bridge`

VHDL

Write your own `spi_reg_bridge.vhd` from section 19.2 and appendix D. Verify it:

```
cd bridge
ghdl -a --std=93 spi_def.vhd spi_reg_bridge.vhd spi_reg_bridge_tb.vhd
ghdl -e --std=93 spi_reg_bridge_tb
ghdl -r --std=93 spi_reg_bridge_tb --assert-level=error
```

- a) Get all four cases to pass. Cases 3 and 4 are the two that make this module more than a byte counter — read their comments before you start, not afterwards.
- b) Now break it deliberately. Remove every use of `ss_active` and count bytes modulo five instead. Cases 1 and 2 still pass. Say which check in case 3 fails, quote the value the error message reports, and explain where those bytes came from.
- c) A bridge ignoring the reserved command bits also passes cases 1 and 2. Explain why the protocol specification puts the reserved-bit rule in *this* layer rather than in `register_bank`, given that the bank never sees more than a 4-bit index.
- d) The read path latches `reg_rdata` **once**, at the end of the command byte. Say what `register_bank` would have to do instead if the bridge reread it for every data byte, and why that would be the wrong place to push the rule down into the bank. Put the module back.

Exercise 19.4 Composing `can_spi_node`

VHDL

Write the top level from section 19.3: `spi_slave`, `spi_reg_bridge`, `register_bank` and `can_controller`, plus a `meta_prev` for the reset.

a) Write it, and run `can_spi_node_tb` against it. Four testbenches handed out already cover everything *inside* the file; say which four, and then say the one kind of fault `can_spi_node_tb` can fail that none of the four can. (The clue is the positional port binding.)

b) `can_spi_node_tb` holds 200 ns of quiet around every SS edge, where appendix D requires at least 60 ns. It is therefore deliberately kind to the timing requirements, and no testbench handed out presses them. Copy it, shrink the quiet period step by step towards 60 ns, and find out at what value your node stops answering.

Then say **why** exactly there, counted in clock cycles, and where in `spi_slave` the limit sits. Compare the measured value with the 60 ns the specification promises: do you have margin, or is your node right at the edge? This is the one place in the book where you *measure* a margin rather than read one, and it is worth doing carefully.

c) Count the `meta_prev` instances in the finished design and say what each one synchronises. Then say why `can_spi_node` hands `can_controller` the *raw* `reset_n` rather than the `reset_s2_n` it gives every other block, and what would go wrong if it synchronised it twice.

d) `tx_bus` and `bus_en` leave the design as two separate ports rather than one wire. Explain what has to happen between them and a physical wire shared with a second board, and what would fail if you assigned `tx_bus` straight to a pin and tied two boards together.

Exercise 19.5 What the board showed

REASONING

The bring-up ran in eight steps, in two halves: three on two DE0-CVs and five with the hat as master. This exercise is about what they established and what they did not. It needs no hardware of your own.

a) The steps are ordered so that each rules out a class of faults before the next depends on it. For each of steps 1, 4 and 5 (single board, loopback on the hat, register echo), say the one thing it proves that the step before did not. Then say which of the eight steps `can_spi_node_tb` had **already** established in simulation, and which only hardware can establish.

b) Two wiring faults to tell apart. Say which step fails first if MISO and MOSI are swapped, and which fails first if VDDI02 was never connected, leaving PORTC unsupplied. Explain why they fail at different steps, and what that difference lets you conclude without touching a meter. Also say what `MVI0.STATUS` would have answered in the second case, and why that answer is faster than the meter.

c) The test program handed out is written to succeed: it holds SS low in good time, never sends a sixth byte and never aborts mid-transaction. Say which of the protocol specification's rules — the 60 ns around SS, MISO during the command byte, or the abort rule — **none** of the

eight steps would have revealed as broken, and why that one is therefore the most expensive to have wrong when a driver you did not write meets the node.

d) A single node can never make `rx_valid` pulse, however its pins are wired, which is why the session put a frame across **two** boards. Say why a node can never receive its own frame (the reason is chapter 17's role), and what the second board therefore proves that neither a loopback nor `can_controller_tb` can.

e) The two boards run on two independent 50 MHz oscillators. Section 19.1 calls the shared clock in `can_controller_tb` the testbench's **largest** gap for exactly that reason. Be precise about what changes: `bit_timer.resync` already does real work in simulation, so say what the two independent clocks add that no shared-clock simulation can show, and roughly how far apart the two oscillators could drift before a frame stopped arriving intact (Exercise 12.4 did that calculation; reuse it). Compare that limit with a typical crystal's tolerance and say whether the demonstration really puts it to the test.

f) `rx_bus` comes from outside the FPGA, and `can_spi_node` passes it on unsynchronised to `can_controller` deliberately, since `can_controller` contains its own `meta_prev` (chapter 12). Explain what could go wrong if that instance were removed, why the failure would be considerably harder to diagnose than a miswired line, and why no amount of simulation in this book would ever reproduce it.

Exercise 19.6 Following a `send()` end to end

REASONING

a) Follow a single frame request all the way down. Start from the five bytes on MOSI writing 0x1 to `TX_SEND`, and follow them through `spi_slave` (bytes), `spi_reg_bridge` (a register write) and `register_bank` (a one-cycle `tx_req`) to `can_controller`'s ports. Name what every layer hands on to the next. Then do the same upwards, for how the driver finds out that the frame is done.

b) Four layers are three more than `can_controller_tb` needs. For every boundary, name the testbench handed out that pins it down, and say what would be untested if that testbench did not exist.

c) Polling is all this design offers: there is no interrupt output anywhere. Say what would have to be added to give the driver one, from which existing signal it would be derived, and in which layer it would have to live. Then explain why it would have been hopeless for a driver to poll a `tx_done` lasting one cycle, and which module made sure it is not.

d) A driver writes `TX_ID`, then `TX_DLC`, then `TX_SEND`, as three separate transactions. Between the second and the third the bus goes idle and another node transmits. Say whether anything the driver has written is at risk, and which rule in the register map makes your answer true.

Exercise 19.7 Life after an abort

SIMULATION

Section 19.1 lists “no node ever transmits again after an abort” as something the system testbench does not cover. This exercise covers it, and shows what `crc15`’s `clear` input was for.

- a) Extend `can_controller_tb.vhd` with a third case. Give both nodes time to return to idle after case 2 has run, that is, after node B has lost arbitration and abandoned its frame mid-identifier, and then have B send a frame of its own (ID 0x123, DLC 2, any payload) with A receiving. Confirm in simulation that A receives it: A’s `rx_valid` pulses, A’s error stays low, and the ID comes back correctly.
- b) Now remove the clear: change `crc_clear` to a constant '0' and rerun. Cases 1 and 2 still pass. Case 3 does not. Say which node raises error, at which state, and explain why it is *not* that node that has the bug.
- c) Work out what B’s `crc15` register holds at the moment it aborts in case 2, and why no amount of correct behaviour afterwards ever returns it to zero. (The clue is chapter 13’s property that generating and checking are the same operation: what does it assume about the register at a frame’s *beginning*?)
- d) How many frames must node B send before it works again, with the clear still removed? Explain your answer, and say what a driver polling `STATUS` and `ERROR_FLAGS` over SPI would observe each time, using chapter 18’s rule for how the error bit is latched.
- e) Chapter 16 puts the clear in `STATE_START` rather than in `STATE_CRC_WAIT` or on the way out of an abort. Give one reason each why the other two placements are worse.

Practical exam 2: CAN modules

Three hours, individual, at the computer. Covers the project's modules: bit timing, CRC, bit stuffing, frame sequencing, register semantics and the SPI transaction.

IN THIS CHAPTER

- Why an individual exam follows a group project.
- The exam's format: writing tasks with a testbench handed out, and reading tasks on paper.
- The four kinds of task: variants, repairs, a single register, reading.
- What you are allowed to use — including the group's own project code.
- How to prepare, what the exam measures, and where the design goes afterwards.

This is the course's second and last examination. Like chapter 9 it contains no new theory; the chapter says what the exam measures, what shape the tasks take and how to prepare. Marks and grade boundaries are in appendix C.

The exam covers **the project's modules**, that is, chapter 10 to 19: the bit timing, the CRC, the bit stuffing in both directions, the frame sequencing, the register semantics and the SPI transaction. Part I's material has not gone away — every module in the project is built from the synchronous process template, a counter, a shift register and a Moore machine — but the emphasis is on part II.

20.1 Why an individual exam on a group project

The project is assessed as a group and accounts for half the course. This exam measures something else: that every individual understands what the group built, not only the part they wrote themselves.

That is not a distrust of the way of working but a consequence of it. A group of four or five splits eleven modules between them, and that split is exactly what makes the project feasible in ten sessions. The downside is equally predictable: whoever wrote `crc15` can get away without ever having needed to understand why `bit_timer` has two pulses, and whoever wrote the register bank without having thought about what the stuffing does to the CRC.

The tasks are therefore drawn from *all* parts of the project, regardless of who wrote which module.

There is a practical consequence of this worth taking seriously long before the exam: **the code review during the project is the preparation**. A group that genuinely reviews each other's PRs, instead of approving them unread, has already read everything the exam asks about. Chapter 15's code review seminar is the occasion to practise that; it is also the occasion that pays off here.

20.2 Format

The same shape as chapter 9: independent tasks, unrelated to each other and ordered roughly by increasing difficulty.

Most are **writing tasks** — a self-checking testbench handed out and a description of the module it drives, with port order, types and behaviour. The task is to write the module so that the testbench passes.

Besides those there are **reading tasks**: a timing diagram or an SPI transaction to interpret on paper. They have no testbench, for exactly the reason that makes them worth asking — reading a waveform is a skill in itself, and it is not examined by writing VHDL. They are marked against an answer key, and partly correct answers earn partial marks the same way partly correct logic does.

The four kinds of task

Variants. A `bit_timer` with a different sample point or a different bit rate. A stuffing detector counting a different number. A CRC with a different polynomial. The module is familiar; the one point where it differs from the project's is the point that requires you to have understood it.

Repairs. A given state machine or a given shift register that is *almost* right. Find the fault using the testbench's output and fix it. It is chapter 14's and 15's working method put to the test: read what the testbench reports, and work backwards from there to the line that caused it.

A single register. One register from the register bank, with its set and clear rule, isolated from the rest. Chapter 18's difference between a pulse and a sticky level is the whole task.

Reading. A timing diagram or an SPI transaction to interpret: what is on the bus, and what should have been there?

Partly correct logic earns partial marks, so never hand in an empty file because the module did not get finished; the same goes for a half-interpreted timing diagram. The marking reads the code as well, not only the testbench's outcome: a module that passes by doing something other than what was asked does not earn full marks.

20.3 Permitted aids

Allowed:

- All the course material: this book, the lectures' READMEs and every appendix. The register map and the protocol specification (appendix D) belong there of course; several tasks are written against them.
- **The group's own project code.** You built it; it would be strange to pretend otherwise. The tasks are written so that copying does not help — the variants differ at precisely the point that requires you to have understood the module, and the repairs have no counterpart in the repository at all.
- Your own computer with GHDL, and CircuitVerse if you want to draw a circuit or a waveform before you write it. That is often faster, not slower.

Not allowed: communication with others in any form; language models and other tools that generate or explain VHDL for you; and searching the internet for ready-made solutions. The line is not drawn at who presses the keys but at whose understanding is being tested.

The full and authoritative list of permitted aids is in appendix C; if anything here says otherwise, that one applies. If you are unsure whether something is allowed: ask before the exam, not during it.

20.4 Before the exam

- Go through the modules in the project that *somebody else* in the group wrote. That is where the gaps are, and it is the one piece of advice in this list that is hard to follow the day before.
- The self-check questions at the end of chapter 12 to 19 are deliberately written as revision material. If you can answer them all without looking anything up, you are ready.
- Read the testbenches handed out through once more. They are the clearest thing there is about exactly what every module promises, and a writing task is always easier for somebody who knows how a testbench states its requirements.
- Redraw `bit_timer`'s two pulses and `can_controller`'s state diagram on a sheet of paper, from memory. Those two pictures carry more of the project than any single file does.
- Make sure GHDL works on your machine *before* you arrive. A broken toolchain is no reason for extra time.

20.5 What the exam measures

- That you can turn a specification in words into a correct module, with the right port order.
- That you understand the bit timing: why the sample point sits where it does, and which pulse does what.

- That you can reason about bit stuffing in both directions — inserting and removing are the same rule seen from two sides, and both sides have to keep the CRC out of it.
- That you can tell a pulse from a sticky level, and know which layer is responsible for which.
- That you can read a testbench's output and find your way back to the cause.

20.6 After the course

The controller goes on: the parallel class writes the C++ driver against the same registers over the same SPI transport, and the node travels on to the CAN labs in the next embedded systems course. What holds the two halves together is the register map and the protocol specification in appendix D — not the code on either side. That is the whole point of an interface: two groups that have never read each other's source can still meet on a bus, because the specification between them was written before either of them started.

Over twenty chapters the book has gone from a gate on paper to a CAN node a microcontroller can talk to. Every pattern along the way is common property in digital design and carries over unchanged into whatever you build next:

- the two-flip-flop synchroniser (chapter 4), which every asynchronous input passes through,
- the clocked process template (chapter 3), which every sequential module in the book is written with,
- the Moore machine (chapter 8), as an enumerated type and a case,
- the shift register and the counter (chapter 6) as building blocks rather than as special cases, and
- finally the split into blocks with one clear responsibility each (chapter 11), which is the only reason eleven modules can be written by five people in parallel and still fit together.

If you want to go further than the design in this book — to a controller with error counters and ACK checking, to verification beyond a hand-written testbench, or to the next protocol built with the same method — appendix E points out what there is to read next.

Simulation with GHDL

This is the book's permanent reference for simulation. Every chapter that verifies a module against a testbench points here instead of repeating the steps. Section 2.7 is the gentler introduction, written for the first time you run a testbench at all; this appendix is the one you come back to, and it covers the group project's build in particular.

A.1 Why simulate

Chapter 1 to 8 verify every design by hand, in CircuitVerse or directly on the DE0-CV. That works for small circuits with a handful of inputs a human can toggle.

It stops working as soon as a design's correctness depends on exact timing relationships across several cycles between several modules, which is the group project's situation throughout.

- A **testbench** is a small VHDL program, handed out, that drives a design's inputs and checks its outputs automatically.
- **GHDL** is the tool that compiles and runs it, entirely in software, with no FPGA involved.
- You are never asked to write the testbench that *verifies* a module you build. Every one of those is handed out; you run it and read what it reports.
- A few exercises invite you to write a short *ad hoc* rig, twenty lines feeding your own bit sequence into a module and reporting what comes out. That is exploratory, and always optional in form: nothing you are assessed on depends on a testbench you wrote yourself.

A.2 Installing GHDL

On Ubuntu or WSL:

```
sudo apt-get update && sudo apt-get install -y ghdl
```

Confirm that it worked:

```
ghdl --version
```

GHDL 3.x or later. The course's CI runs the same GHDL commands as below, so what passes locally passes in CI too.

A.3 Where the files live

The group project's VHDL lives in **two flat directories at the repository root**: the modules you write, alongside the testbenches handed out (`*_tb.vhd`) that check them. There are no per-session directories and nothing to copy between them; `can_def.vhd` in particular is written *once*, in chapter 10, and every later controller module reads that one file.

The split between the two directories is the protocol boundary: everything in `controller/` knows something about CAN and nothing about SPI, and everything in `bridge/` the other way round.

controller/	<code>can_def.vhd</code>	ch 10, yours
	<code>can_controller.vhd</code>	ch 11, yours (ports only at first, grows through ch 17)
	<code>meta_prev.vhd</code>	ch 12, yours
	<code>bit_timer.vhd</code>	ch 12, yours
	<code>crc15.vhd</code>	ch 13, yours
	<code>tx_shift_reg.vhd</code>	ch 14, yours
	<code>rx_shift_reg.vhd</code>	ch 15, yours
	+ the six <code>*_tb.vhd</code> handed out	
bridge/	<code>register_bank.vhd</code>	ch 18, yours
	<code>spi_reg_bridge.vhd</code>	ch 19, yours
	<code>can_spi_node.vhd</code>	ch 19, yours
	+ <code>spi_slave.vhd</code> , <code>spi_def.vhd</code> , <code>register_bank_tb.vhd</code> ,	
	<code>spi_reg_bridge_tb.vhd</code> , handed out	

The two directories initially contain only the files handed out: eight testbenches, plus `spi_slave.vhd` and `spi_def.vhd`. Every chapter from chapter 10 adds a module, with the exception that `can_controller.vhd` is written early (chapter 11) as an entity with an empty architecture and then *grows*: chapter 16 fills in its transmit path and chapter 17 its receive path, without any new file appearing for either. That is why `can_controller_tb`, the only testbench exercising the whole controller, is gated on more than the files existing; section A.5 returns to that.

A.4 The three GHDL steps

Every VHDL design (and testbench) goes through the same three steps, in this order:

1. **Analyse** (`ghdl -a`): parses a `.vhd` file, checks it for syntax and type errors, and registers its design units (entities, architectures, packages) in a work library. Dependencies have

to be analysed before the units that use them: `can_def.vhd` before every module that says use `work.can_def.all`;; and a module before its own testbench.

2. **Elaborate** (`ghdl -e`): resolves a particular top-level entity's complete instantiation hierarchy into something executable.
3. **Run** (`ghdl -r`): actually simulates it.

For a module with a testbench handed out, for example chapter 12's `meta_prev`, which reads no package at all and therefore needs only its own two files:

```
cd controller
ghdl -a --std=93 meta_prev.vhd meta_prev_tb.vhd
ghdl -e --std=93 meta_prev_tb
ghdl -r --std=93 meta_prev_tb --assert-level=error
```

Every other controller module reads `can_def`, so the package comes first:

```
cd controller
ghdl -a --std=93 can_def.vhd bit_timer.vhd bit_timer_tb.vhd
ghdl -e --std=93 bit_timer_tb
ghdl -r --std=93 bit_timer_tb --assert-level=error
```

`bridge/` works the same way, with one wrinkle: `register_bank` reads `can_def`, which is a directory away.

```
cd bridge
ghdl -a --std=93 ../controller/can_def.vhd register_bank.vhd register_bank_tb.vhd
ghdl -e --std=93 register_bank_tb
ghdl -r --std=93 register_bank_tb --assert-level=error
```

- `--std=93` selects VHDL-93 throughout the course. Every testbench handed out terminates itself cleanly, by setting a done signal the clock generation process checks and then calling `wait`;; rather than relying on any “stop the simulator” procedure from a later standard.
- `--assert-level=error` tells GHDL to stop the simulation, with a non-zero exit code, at every failed check reported at level error or higher. **Always pass it when you run a project testbench.** The next section says why it is not optional here.

A.4.1 Why `--assert-level=error` matters, and where it does not

GHDL stops on a failed assertion only when the assertion's severity reaches the assert level, and GHDL's default level is failure. The book's two halves sit on opposite sides of that boundary, which is worth knowing since it explains why the flag looks optional in part I and is not in part II:

	Level	Without the flag	With the flag
The chapters' testbenches	failure	stops, exits 1	stops, exits 1
The project's testbenches	error	prints, carries on, exits 0	stops, exits 1

For a project testbench, then, omitting the flag turns a real failure into a message scrolling past above a zeroed exit code, which is exactly what a “silent pass” looks like. Pass it every time and the difference stops mattering.

--stop-time=10ms is worth adding for the same kind of reason. A testbench waiting for an event your module never produces does not *fail*; it hangs. The stop time turns that into a run that ends. It is not in the commands above because the project's testbenches all bound their own waits, but it costs nothing and section 2.7 makes it a habit.

Every testbench reports only pass or fail (and, on failure, the reason), nothing else. A passing run ends with a line like this:

```
bit_timer_tb.vhd:148:9:@7312ns:(report note): bit_timer: all checks passed!
```

A failing run stops early with an (assertion error) message naming exactly which check failed and why. Read that message; it is written to say specifically what went wrong.

A.5 Building everything at once

make build-project, from the repository root, does all of this automatically for every testbench whose modules you have written. It is useful for confirming that your whole tree still passes, but it is the understanding of the three individual steps above that lets you debug a single module quickly during a session.

Since the project is built up one module at a time, make build-project **skips** every testbench whose modules do not exist yet, instead of failing on them, and says so:

```
==> meta_prev_tb
controller/meta_prev_tb.vhd:114:9:@311ns:(report note): meta_prev: all checks ↵
↳ passed!
==> bit_timer_tb (skipped: no bit_timer.vhd yet)
```

That is what chapter 12 looks like halfway through: meta_prev.vhd written and passing, bit_timer.vhd still to do. can_def.vhd is not named as missing, since chapter 10 already wrote it.

Skipped is not failed, deliberately. The project runs over ten sessions and most of the tree is missing during most of them; a build glowing red until the last module landed would say nothing useful during any of the previous nine. The number of testbenches actually running therefore grows as you go: none after chapter 10 or 11, two after chapter 12, then one more in each of chapter 13, 14 and 15, one more in chapter 17, one in chapter 18, and the last in chapter 19.

`can_controller_tb` is **gated on more than the file existing**, and that is worth knowing before it puzzles you. `can_controller.vhd` is written in chapter 11 as an entity with an empty architecture and grows through chapter 16 and 17, so from chapter 15 onwards every file the testbench needs exists while the controller still cannot receive a frame. Running it then would fail, twice over: once in chapter 15, and again in chapter 16, which can transmit but not yet receive. Since the testbench works by having one node receive another's frame, what it actually requires is a **receive path**, and that comes in chapter 17. The build script therefore looks inside your `can_controller.vhd` for chapter 17's receive-side CRC gating and reports:

```
==> can_controller_tb (skipped: can_controller.vhd has no receive path yet;
    set CI_BUILD_ALL=1 to run it anyway)
```

That check is a heuristic rather than a real dependency, and it has a failure mode worth naming: if your chapter 17 is correct but spells those expressions differently from the chapter, you get this skip message when the testbench should have run. Nothing is broken; run `CI_BUILD_ALL=1 make build-project` to force every testbench regardless. The check cannot fail the other way round, since the marker cannot appear before a receive path does.

A.6 Cleaning up

GHDL's analysis step creates `work-obj93.cf` and similar files in the directory you ran them from. The build script keeps a `work/` directory of its own instead, so those files do not litter the source tree. If you run the three steps by hand in `controller/` or `bridge/`, clean up with:

```
make clean
```

which removes the generated `work/` directories, any stray `work-obj*.cf`, and the waveform and executable files a hand run leaves behind.

Quartus Prime Lite and the DE0-CV

This is the book’s tool reference for synthesis and hardware. Through chapter 1 to 18 it is **teacher-facing**: the Quartus and DE0-CV work is demonstrated from the front, starting with the brake assistant in chapter 1, and no participant needs Quartus or a board of their own.

Chapter 19 is the exception, and it is the one that counts. The bring-up is demonstrated first and then handed over: every group synthesises and programs its own `can_spi_node` and demonstrates its own two-node link. So read section B.3 to B.7 as participant-facing — you need section B.3 and B.4 to create the project and add the source files, and section B.6 is the compilation producing the `.sof` that section B.7 programs — and read section B.9 *before* you connect anything, since that is the section deciding whether the Programmer can see the board at all.

Nothing that is **assessed** requires the hardware (appendix C). The bring-up is assessed on the simulation behind it, not on whether the wire worked on that particular day. Everything described here ends on real FPGA hardware, verified by hand. That is a different layer from the exercises, which you verify on your own laptop with a self-checking testbench under GHDL (appendix A).

B.1 What you need

- **Quartus Prime Lite** (the free edition): the manufacturer’s FPGA tool, covering synthesis, place and route (“fitting”), and programming file generation.
- **Cyclone V device support**: a separate add-on package. The DE0-CV’s FPGA, 5CEBA4F23C7N, is a Cyclone V device, and Quartus does not install every family’s support by default.
- **USB-Blaster driver**: the DE0-CV is programmed over USB using the USB-Blaster protocol. The driver usually installs alongside Quartus, but sometimes needs a manual nudge (section B.8).

- A **Terasic DE0-CV** board and a USB cable.

Quartus Prime Lite exists for both Windows and Linux, and the design steps below are the same on either host. **Getting the board visible to the Programmer is not**, and that is where every host-specific problem in this course lives: section B.9 covers Linux and WSL, section B.8 covers Windows.

A recommendation straight away, since it saves an evening: the course's GHDL toolchain runs on WSL or Ubuntu, but **WSL2 cannot see a USB device at all without extra work** (section B.9). If you are on Windows, install Quartus natively on Windows and keep WSL for GHDL. The two never need to see each other; they only share the .vhd files, which live in the Windows file system anyway.

B.2 Installing Quartus Prime Lite

1. Download **Quartus Prime Lite Edition** from the manufacturer's download portal. Search for "Quartus Prime Lite download": Intel's FPGA business became **Altera** again in 2024, so the pages have moved and older deep links rot. What is stable to search for is the product name, not the URL.
 - Choose the **Lite** edition, which is the free one and needs no licence file. Standard and Pro do not cover Cyclone V for free.
 - Any reasonably recent Lite edition works; the course does not depend on a particular point release.
 - Pick the Windows or Linux installer according to where you intend to run it, and read the warning above if you are on Windows with WSL.
2. During installation, make sure **Cyclone V** is selected among the device families. If Quartus is already installed without it, add the Cyclone V support afterwards, either by running the installer again or via Quartus's own "Additional Software" installer.
3. When the DE0-CV board is connected over USB for the first time, Windows should detect the USB-Blaster and either install the driver automatically or ask you to point out Quartus's own driver files (usually under Quartus's installation directory). If Windows finds no driver automatically, see section B.8.

B.2.1 Two pitfalls in Windows 11

A couple of Windows 11's security defaults can interfere with the installation, or with programming the board later:

- **If the installer does not even open:** find Smart App Control in Windows Security (search for it in the start menu), turn it off and restart.

- If everything installs cleanly but the board still cannot be programmed (section B.7): go to Windows Security → Device security → Core isolation and turn off **Memory integrity**.

Core isolation

Security features available on your device that use virtualisation-based security.

Memory integrity

Prevents attacks from inserting malicious code into high-security processes.

 Memory integrity is off. Your device may be vulnerable. [Dismiss](#)

☐ Off

[Learn more](#)

Figure B.1 Memory integrity under Windows 11's core isolation settings.

Neither setting touches Quartus itself; both are general hardening defaults in Windows 11 that happen to be strict enough to interfere with driver-level USB access.

B.3 Creating a new project

1. Open Quartus and start the **New Project Wizard** (File → New Project Wizard).
2. Choose a working directory and a project name. Quartus uses the project name as the default name of the top-level entity too, so let it match your VHDL entity's name to save overwriting it later (for `or_gate.vhd`: name the project `or_gate`).
3. Choose **Empty Project** as the project type. The course uses neither Quartus's IP catalog nor any project template.
4. Skip adding files at this step (section B.4 covers that), unless you already know exactly which file it is; either is fine.
5. On the device selection page, choose exactly the device on the DE0-CV: filter on the **Cyclone V** family, then find and select the device `5CEBA4F23C7N`.
6. Skip the EDA tools page (leave it at "none"); the course runs its simulations directly in GHDL rather than through Quartus.
7. Finish the wizard.

B.4 Adding your VHDL

- Add the source file: Project → Add/Remove Files in Project..., browse to your .vhd file and add it.
- If the design has more than one file (a top-level entity plus one or more subcomponents), add all of them. Quartus resolves the instantiation hierarchy automatically as long as every file is in the project.
- Confirm that the top-level entity is set correctly: Project → Set as Top-Level Entity, with your main file selected. For a single-file design that happens automatically.

B.5 Assigning pins in the Pin Planner

Every port in your entity needs a physical pin on the FPGA before Quartus can produce a working programming file; otherwise there is no path from the design's ports to a real switch or LED on the board.

1. Run Processing → Start → Start Analysis \& Elaboration first, then open the Pin Planner via Assignments → Pin Planner. In a brand new project the port list is empty until Quartus has elaborated the design and knows its ports.
2. Each of the entity's ports appears as a row. Fill in the **Location** column with the pin designation of the switch or LED you want to use.
3. **A switch or an LED is a free choice. A clock is not.** For SW, KEY and LEDR, any convenient ones will do: the design does not care which switch drives which input, only that every port reaches one. CLOCK_50 is a specific pad on the device and cannot be moved. On the DE0-CV it is **PIN_M9**, the 50 MHz oscillator. Every design from chapter 3 onwards is clocked, so from that chapter this is the one pin number worth knowing without looking it up. Confirm it against your own board's files (step 4) before you rely on it.
4. **Do not copy the pin table out by hand.** Ten LEDR pins transcribed from a PDF are ten chances to transpose two digits, and the failure it gives is the nastiest one in section B.8: a clean compilation that behaves incomprehensibly.
 - The DE0-CV's system CD contains a DE0_CV.qsf where every board pin is already named and assigned.
 - Assignments → Import Assignments..., point out that file, and every CLOCK_50, SW[n], KEY[n], LEDR[n] and GPIO_0[n] assignment lands at once.
 - Name your entity's ports to match those signal names and there is nothing left to assign. That is why chapter 19's bring-up names its ports CLOCK_50, KEY, SW, LEDR and GPIO_0 rather than something prettier.
 - The Pin Planner (and the board's pin tables in the user manual) is still the way to check an individual pin, or assign one the .qsf file does not cover.

5. Set the **I/O Standard** column if Quartus does not infer a sensible default for the pin you chose. The DE0-CV's general I/O is 3.3 V LVTTTL/LVCMOS. Importing the board's .qsf (step 4) sets this too, which is one more reason to prefer that route.
6. Save the assignments (they are written into the project's .qsf file). Every change here requires recompilation (section B.6) before it takes effect; Quartus does not retrofit a .sof compiled before the change.

B.5.1 A clock needs a timing constraint too

Assigning CLOCK_50 a pin tells Quartus *where* the clock comes in. It does not say how fast it runs, and until you say so the timing analysis has nothing to check against and reports an unconstrained design. Add a .sdc file to the project with one line:

```
create_clock -name CLOCK_50 -period 20.000 [get_ports CLOCK_50]
```

20 ns is 50 MHz. With that in place, the timing analysis in Processing → Start Compilation becomes meaningful and the report gives you a real Fmax, which is the number section 5.4 sends you here to read. Without it, Fmax is missing, or not to be trusted.

B.6 Compiling the design

Processing → Start Compilation (or ► in the toolbar) runs the whole flow: analysis and synthesis, place and route, timing analysis, and assembly into a programming file.

Read the compilation report when it is done, in particular the **Warnings**: an unconnected port, an incomplete sensitivity list, or a latch inferred where you meant combinational logic all turn up there as warnings rather than errors. They are easy to miss if you only check that the tick went green.

A successful compilation produces a .sof file (SRAM Object File) in the project's output_files directory. That is the one section B.7 downloads to the board. It is volatile: it configures the FPGA's SRAM directly and disappears when the power is removed. The course has no step that burns the configuration permanently.

B.7 Programming the DE0-CV

1. Connect the DE0-CV to the computer over USB and switch it on.
2. Open the Programmer: Tools → Programmer.
3. Select the USB-Blaster under **Hardware Setup**. It should be found automatically if the driver installed correctly (otherwise section B.8).
4. Make sure the .sof file from section B.6 is in the list and that its **Program/Configure** checkbox is ticked. If it is not there, use Auto Detect or add the file by hand.
5. Click **Start**. The progress bar should reach 100 % and report success.

6. Test the design directly on the board: toggle the switches you assigned in section B.5 and confirm that the LED follows the entity's truth table.

B.8 Troubleshooting

The USB-Blaster does not appear in the Programmer. Check Device Manager in Windows for an unknown device or one with a driver error, and manually point out the USB-Blaster driver under Quartus's installation directory.

The board cannot be programmed, with no clear error. Turn off Memory integrity under Core isolation on Windows 11 (section B.2.1). It is by far the most common cause once the driver is installed.

The installer does not start. Check Smart App Control under Windows Security (section B.2.1).

The compilation passes but the design misbehaves on the board. Check the assignments in the Pin Planner (section B.5) *before* you suspect your VHDL. A swapped or missing pin assignment gives a board that compiles cleanly and behaves incomprehensibly, since it is not wired the way you think.

The fitter reports unassigned pins as errors. Some project settings treat that as fatal rather than as a warning. Either assign every port in the Pin Planner, or, for a forgotten unused port early in development, tell Quartus how to treat the rest: Assignments → Device → Device and Pin Options... → Unused Pins. For the DE0-CV, “As input tri-stated (with weak pull-up)” is the safe value. The default in some flows drives unused pins low, which on a board where pins reach headers and peripherals is a way of fighting another driver. Do not leave ports silently unassigned and expect Quartus to guess.

B.9 Programming from Linux and from WSL

Section B.8 is Windows. On Linux the driver is no driver at all: the Programmer talks to the USB-Blaster through libusb, and needs permission to do so.

Linux, natively

Out of the box the Programmer shows no hardware, since the device node is owned by root. Give it a udev rule with the lines below in /etc/udev/rules.d/51-usbblaster.rules:

```
# Altera USB-Blaster
SUBSYSTEM=="usb", ATTR{idVendor}=="09fb", ATTR{idProduct}=="6001", MODE="0666"
SUBSYSTEM=="usb", ATTR{idVendor}=="09fb", ATTR{idProduct}=="6002", MODE="0666"
SUBSYSTEM=="usb", ATTR{idVendor}=="09fb", ATTR{idProduct}=="6003", MODE="0666"
# USB-Blaster II
```

```
SUBSYSTEM=="usb", ATTR{idVendor}=="09fb", ATTR{idProduct}=="6010", MODE="0666"
SUBSYSTEM=="usb", ATTR{idVendor}=="09fb", ATTR{idProduct}=="6810", MODE="0666"
```

Reload the rules and then unplug and replug the board:

```
sudo udevadm control --reload-rules && sudo udevadm trigger
```

Quartus caches its hardware list in a background daemon, so if the Programmer still shows nothing, restart it:

```
killall jtagd 2> /dev/null
jtagconfig          # should now list the USB-Blaster and the device chain
```

`jtagconfig` is the quickest way to tell a permissions problem from a cable problem: if it lists the board, the hardware is fine and the problem is inside Quartus; if it does not, stop and solve that before you touch the Programmer.

WSL2

This is the case the course's own toolchain runs straight into, so it is worth saying outright: **WSL2 has no USB passthrough**. The board is invisible to a WSL2 installation however many `udev` rules you write, since the kernel never sees the device. There is no setting to change. Two ways out:

- **Run Quartus natively on Windows** (recommended). Keep WSL for GHDL, which needs no hardware at all. Your `.vhd` files live in the Windows file system and both tools read them. That is the route the course assumes, and the one with the fewest moving parts.
- **Forward the device with `usbipd-win`**, if you specifically want Quartus inside WSL. From an administrator PowerShell on the Windows side:

```
winget install usbipd
usbipd list          # find the USB-Blaster's BUSID
usbipd bind --busid <BUSID> # once per device, persists
usbipd attach --wsl --busid <BUSID>
```

The device then appears inside WSL and the `udev` rule above applies. It has to be reattached after every unplug and every WSL restart, and Quartus's graphical interface also needs WSLg or an X server. Doable, but three more things that can go wrong on a presentation day.

The group project

This is the project's specification, collected in one place: the way of working, the structure, the modules' interfaces, the test plan and the assessment. Chapter 10 to 19 are the walkthrough; this is the contract, and where the two differ what is written here applies.

The task is to design, simulate and run a simplified CAN controller in VHDL, and make it reachable from a microcontroller over SPI. The parallel class writes the C++ driver against the same registers, so what is built is one half of a real hardware and software system, not an exercise ending in the simulator.

When the project is finished the group has implemented a **bit timer** producing sample and bit-end pulses, a **CRC-15 engine** that both computes and verifies the checksum, a **transmit shift register** that serialises and inserts stuff bits, a **receive shift register** that deserialises and removes them, a **CAN controller** whose state machine sequences the frame's fields and handles arbitration, a **register bank** translating the controller's pulses into pollable registers, an **SPI bridge** making the registers reachable from outside, and a **top level** tying it all together into a node a microcontroller can talk to.

C.1 Way of working

The project is carried out in groups of **4–5 students**. The group is jointly responsible for the whole project, and the way of working follows industrial practice.

C.1.1 Git and GitHub

- Create a private repository and invite every group member and the supervisor as collaborators.
- Work in feature branches. One branch per module is a reasonable starting point.
- Commit often, with messages that say what the change does and not merely that it exists.
- **Never push directly to main.** All code reaches main through a pull request. Protect the branch in GitHub's settings so that the rule does not depend on everybody remembering it.

C.1.2 Code review

- Every PR is reviewed by **at least one other group member** before it is merged.
- The reviewer checks: that the port order matches the interface table, that the module's testbench passes, that no values are hardcoded past `can_def`, and that every entity's interface and every important internal signal is commented.
- Put review comments in GitHub, not in chat or verbally. A review that does not exist in writing has not happened.
- A review that just says “looks good” is no review. Ask at least one real question, even when the code is good: *why* it does what it does is more often interesting than *that* it does it.

C.1.3 Division of work

Everybody is to write VHDL and everybody is to review VHDL. The division may be uneven in quantity, not in kind. Document the division as you go in `CONTRIBUTORS.md` at the repository root.

C.2 How the project is run

The project has **no milestones**. Every chapter from chapter 10 onwards states a *target*, phrased as “you should be roughly here”. The targets are not requirements and are not graded.

That is a deliberate choice and not generosity. After `can_def.vhd`, the four leaf modules `bit_timer`, `crc15`, `tx_shift_reg` and `rx_shift_reg` have **no dependencies on each other**, and each has a testbench of its own handed out. A group of 4–5 can therefore build them in parallel, in any order. Fixing an order would only get in the way of that.

But the dependency graph is not flat, and it is worth planning around:

```

        meta_prev                                (reads no package at all)
can_def  → bit_timer, crc15, tx_shift_reg, rx_shift_reg (the four leaves, parallel)
                                     ↳ can_controller (ch 16 transmit, ch 17 receive)
can_def  → register_bank                        (independent, buildable from ch 10)
spi_def  → spi_reg_bridge                       (independent, buildable from ch 10)
spi_slave                                (handed out, not written by the group)
    can_controller + register_bank + spi_reg_bridge + spi_slave → can_spi_node (ch 19)
```

Two things follow from it:

- **can_controller is the critical path.** It has no internal parallelism, it spans two sessions, and its testbench is a system testbench that either passes or does not — no partial green. Whoever is to own it should start reading chapter 16 early, and the group should expect it to take the most time of anything.

- **register_bank** and **spi_reg_bridge** are two full work streams that can start straight away. One needs only `can_def`, the other only `spi_def`, and both have testbenches handed out. Nobody has to wait for the controller to have something to do, and a group that defers them to chapter 18 and 19 has put its tightest work last.

Two checkpoints are compulsory:

When	What
Chapter 15	Code review seminar. Every group shows a merged PR and a review the group wrote, and talks about one thing in the way of working that has not worked.
Chapter 19	Final presentation and hand-in. Green testbenches, a walkthrough of <code>can_spi_node</code> , bring-up on hardware as far as the group got, and the repository's history.

What is assessed is the end result and the way of working: the design, the Git history, the PRs and the reviews. **Not the pace.** A group that is behind halfway through the project and hands in a complete, well-reviewed node has done the project right.

C.3 Project structure

Two flat directories, with the modules beside the testbenches that check them:

```
controller/  can_def.vhd, meta_prev.vhd, bit_timer.vhd, crc15.vhd,
              tx_shift_reg.vhd, rx_shift_reg.vhd, can_controller.vhd
              + the six *_tb.vhd handed out
bridge/      register_bank.vhd, spi_reg_bridge.vhd, can_spi_node.vhd
              + spi_slave.vhd, spi_def.vhd, register_bank_tb.vhd,
              spi_reg_bridge_tb.vhd, handed out
ci/          build_project.sh, copied from the course repository
Makefile     with the build-project target
CONTRIBUTORS.md
.gitignore
```

The boundary between the two directories is the protocol: everything in `controller/` knows about CAN and nothing about SPI, and **no module in either directory knows about both**. `register_bank` is the middle: it knows neither the CAN frame format nor SPI but only register semantics, and reads `can_def` purely for the port types.

The files handed out are fetched from the course repository and added unchanged. The group does not write `spi_slave.vhd`: it is transport, not the course's subject, and it is read in chapter 18 instead.

C.4 Naming conventions and coding style

All the code and all the comments in the code are written in **English**. So are the instructions.

- Module and signal names in snake_case, for example timer_enable.
- Constants in capitals, for example TICKS_PER_BIT.
- std_logic and std_logic_vector for everything that is wires; natural for counters.
- No hardcoded values. Everything that is protocol lives in can_def.
- Comment every entity’s interface and every important internal signal.
- Keep the interfaces stable. The testbenches are handed out and bind positionally.

Positional port binding. Every testbench handed out binds to its module **positionally**, and so does every instantiation inside can_controller. The contract is therefore the **order and the types** of the ports, exactly as the tables in section C.6 and C.7 list them. The *names* of the ports are the group’s own.

Swap two ports of the same type and nothing complains during analysis; the design simply misbehaves in simulation. That is the price of letting the interface table be the single source of truth.

C.5 The shared package can_def

Written in chapter 10, in controller/can_def.vhd, and read by everything else.

Constants: CLOCK_FREQ_HZ, BIT_RATE_HZ, TICKS_PER_BIT, SAMPLE_TICK, ID_WIDTH, DLC_WIDTH, DLC_MAX, CRC_WIDTH, DATA_WIDTH, MAX_RUN, CRC_POLY.

Subtypes: byte_t, id_t, crc_t, dlc_t, data_t.

Target values for a DE0-CV with a 50 MHz system clock and CAN at 1 Mbit/s:

Constant	Value	Comment
CLOCK_FREQ_HZ	50_000_000	The system clock.
BIT_RATE_HZ	1_000_000	1 Mbit/s.
TICKS_PER_BIT	CLOCK_FREQ_HZ / BIT_RATE_HZ	50 clock cycles per CAN bit. Compute it, do not write it.
SAMPLE_TICK	TICKS_PER_BIT * 7 / 10	The sample point, about 70 % into the bit period.
ID_WIDTH	11	Standard identifier.
DLC_WIDTH	4	DLC is four bits.
DLC_MAX	8	At most eight data bytes.
DATA_WIDTH	DLC_MAX * 8	64 bits of payload.
CRC_WIDTH	15	CAN’s CRC-15.
MAX_RUN	5	The five rule for bit stuffing.
CRC_POLY	"100010110011001"	Corresponds to $x^{15} + x^{14} + x^{10} + x^8 + x^7 + x^4 + x^3 + 1$.

That TICKS_PER_BIT and SAMPLE_TICK are *computed* and not typed in matters: if somebody changes the bit rate, everything should follow.

C.6 The modules in controller/

The chapters describe the modules in detail; this is the port order collected, since that is what the contract is.

meta_prev (chapter 12)

One generic, `WIDTH` (natural := 1), and three ports: `clock` (in), `async_in` (in, `std_logic_vector(WIDTH-1 downto 0)`), `sync_out` (out, the same type). The two-flip-flop synchroniser from chapter 4, written once with a width generic. It needs no reset: after two clock cycles the contents are replaced anyway. Instantiated twice inside `can_controller`, for `reset_n` and for `rx_bus`.

bit_timer (chapter 12)

#	Port	Direction	Type	Meaning
1	clock	in	std_logic	System clock.
2	reset_s2_n	in	std_logic	Synchronised, active-low reset.
3	enable	in	std_logic	Counts while '1', holds while '0'.
4	resync	in	std_logic	Forces the counter back to tick 0.
5	sample	out	std_logic	Pulse at the sample point.
6	bit_done	out	std_logic	Pulse at the end of the bit period.

A counter from 0 to `TICKS_PER_BIT-1` pulsing `sample` at `SAMPLE_TICK` and `bit_done` at the end of the period. `resync` is used once per frame, at SOF; in between the timer free-runs.

crc15 (chapter 13)

#	Port	Direction	Type	Meaning
1	clock	in	std_logic	System clock.
2	reset_s2_n	in	std_logic	Synchronised, active-low reset.
3	clear	in	std_logic	Clears the register without a reset.
4	enable	in	std_logic	Integrates data at this edge.
5	data	in	std_logic	The current bit.

#	Port	Direction	Type	Meaning
6	crc	out	crc_t	The register's current value.
7	valid	out	std_logic	'1' when the register is all zeros.

A 15-bit register with feedback. The same engine both generates and checks: the transmitter feeds in its bits and reads out `crc`; the receiver feeds in the same bits *plus* the received CRC, and `valid` goes high if everything matches. No mode switch is needed. `clear` is needed all the same: an aborted frame leaves rubbish in the register.

tx_shift_reg (chapter 14)

#	Port	Direction	Type	Meaning
1	clock	in	std_logic	System clock.
2	reset_s2_n	in	std_logic	Synchronised, active-low reset.
3	load	in	std_logic	Loads data/bit_count and puts out the first bit the same cycle.
4	data	in	byte_t	The group, with its valid bits at the top.
5	bit_count	in	std_logic_vector(3 downto 0)	Number of valid bits, 1–8.
6	shift	in	std_logic	One pulse per CAN bit, from <code>bit_timer.bit_done</code> .
7	tx_bit	out	std_logic	The bit to drive next bit period.
8	stuff	out	std_logic	'1' on the very shift that puts out a stuff bit.
9	done	out	std_logic	Pulses when the group has been fully sent.
10	bit_valid	out	std_logic	'1' the cycle a new bit was put out.

Shifts out MSB first, counts identical bits in a row, and inserts a stuff bit of the opposite value after `MAX_RUN` identical ones. **The stuffing state survives reloads:** the rule applies to the bit stream, not to the individual group. `stuff` and `bit_valid` exist for `can_controller`'s sake: the CRC is fed on `bit_valid` and not `stuff`, since a stuff bit is not part of the checksum.

rx_shift_reg (chapter 15)

#	Port	Direction	Type	Meaning
1	clock	in	std_logic	System clock.
2	reset_s2_n	in	std_logic	Synchronised, active-low reset.
3	sample	in	std_logic	One pulse per CAN bit, from <code>bit_timer.sample</code> .
4	rx_bus	in	std_logic	The bus level, valid at the sample point.
5	bit_count	in	std_logic_vector(3 downto 0)	Number of <i>real</i> bits to collect, 1–8.
6	enable	in	std_logic	Decides whether sample pulses are acted on.
7	data	out	byte_t	The collected group, right-justified.
8	valid	out	std_logic	Pulses when <code>bit_count</code> real bits have been collected.
9	stuff_error	out	std_logic	Pulses if an expected stuff bit does not invert.
10	real_bit	out	std_logic	The destuffed bit that was accepted.
11	real_bit_valid	out	std_logic	'1' when a real bit was accepted, '0' for a discarded stuff bit.

The transmitter's mirror image. After `MAX_RUN` identical bits in a row the next bit *must* be the opposite; if it is, it is discarded and the count starts over, and if it is not, that is a `stuff_error`.

can_controller (chapter 11, 16, 17)

The top level of the CAN half, and the only block that knows the frame format. Fifteen ports, in exactly this order, every input first:

1 clock	4 tx_id	7 rx_bus	10 bus_en	13 rx_data
2 reset_n	5 tx_dlc	8 tx_done	11 rx_id	14 rx_valid
3 tx_req	6 tx_data	9 tx_bus	12 rx_dlc	15 error

The frame sequence a transmission goes through: SOF (1 bit), the arbitration field (11-bit identifier + RTR), the control field (IDE, r0, DLC = 6 bits), the data field (0–64 bits according to DLC), CRC (15 bits + delimiter), ACK (slot + delimiter) and EOF (7 bits). Every field gets a load state of its own before its shift state.

The bus is modelled as open drain: bus_en decides whether the node drives at all, tx_bus what it drives. The bus value is '1' only if no node pulls it down. During the ACK slot the transmitting node releases the bus, and a receiver with a valid CRC pulls it down. **The transmitter never reads the ACK slot back**; a frame nobody acknowledges completes exactly like an acknowledged one, and tx_done pulses all the same.

Arbitration: during the arbitration field every transmitted bit is compared against what is actually on the bus. If the node sends a recessive bit and reads back a dominant one it has lost: it sets error, pulses tx_done, leaves the transmitter role and goes idle. It does **not** receive the frame that won.

That tx_done pulses here too is deliberate: the pulse means “the transmission attempt is over and the port is yours again”, not “the frame arrived”. error says which. Without that pulse a caller has no edge to wait for, and the register bank never gets its TX-ready bit back.

error goes high for four reasons: lost arbitration, stuff_error, a failed CRC check, and a received remote frame. It stays high until the next transmission or reception starts. **tx_done** and **rx_valid** are pulses of exactly one clock cycle, which is precisely what makes the register bank necessary.

C.6.1 Known limitations the driver notices

The simplifications against real CAN are listed in full in section 19.7, and from the protocol side in chapter 7 of *CAN: the bus, the frame and the controller*. Three of them show all the way out in the register map and therefore have to be stated here too, since the parallel class writes code against them — they are the same three *CAN: the bus, the frame and the controller* highlights, and both classes read that book:

- **A lost arbitration cannot be told from a successful transmission on STATUS bit 0.** Both give the bit back, since the controller pulses tx_done when it aborts as well. That is deliberate: the bit means “the port is free”, not “the frame arrived”. A driver that wants to know the outcome reads ERROR_FLAGS after bit 0 has come back. This was once a real bug, and it is worth knowing why it is not one any more. If bit 0 were set only by tx_done, and tx_done pulsed only at EOF, the bit would be left low after every lost arbitration and the node could not transmit again until it was cleared by the reset button — on a two-node bus, that is, in normal operation. Three things keep it from arising again, and they overlap deliberately: the controller pulses tx_done at aborts too (chapter 17), the bank sets bit 0 on a rising edge of error while the bit is low as well (chapter 18), and TX_ABORT exists as an escape hatch. can_controller_tb case 2 and register_bank_tb cases 10–11 check the three.
- **error is one bit for four causes.** The driver can see *that* something went wrong, never *what*. Retransmission logic distinguishing a lost arbitration from a broken CRC cannot be written against this interface.

- **No buffering on the receive side.** The controller holds one frame. If the next arrives before the previous has been read out, it is overwritten. The register bank passes the limitation on rather than hiding it; see appendix D.

C.7 The modules in `bridge/`

`register_bank` (chapter 18)

Sixteen ports, in this order:

1 clock	5 reg_wdata	9 rx_data	13 tx_id
2 reset_s2_n	6 tx_done	10 rx_valid	14 tx_dlc
3 reg_index	7 rx_id	11 error	15 tx_data
4 reg_write	8 rx_dlc	12 reg_rdata	16 tx_req

Translates the controller's one-cycle pulses into sticky, pollable levels, and register writes into events. The semantics in full are in appendix D; it is the contract with the parallel class and must not be changed unilaterally. `reg_rdata` is **combinational**: latching a read value is the bridge's responsibility, not the bank's.

`spi_reg_bridge` (chapter 19)

Ten ports, in this order: `clock`, `reset_s2_n`, `ss_active`, `rx_data`, `rx_valid`, `reg_rdata`, `tx_data`, `reg_addr`, `reg_wdata`, `reg_write`.

Implements the transaction from appendix D: one command byte plus four data bytes, always exactly five. On a read the **register value is latched once** at the end of the command byte; on a write nothing is committed until the fifth byte is complete. If `ss_active` goes low before that, the transaction is abandoned entirely, with no side effects. `reg_addr` **holds** the decoded index until the next command byte decodes a new one.

`can_spi_node` (chapter 19)

Nine ports, in this order:

1 clock	4 mosi	7 rx_bus
2 reset_n	5 ss	8 tx_bus
3 sclk	6 miso	9 bus_en

A purely structural top level. Instantiates `spi_slave`, `spi_reg_bridge`, `register_bank` and `can_controller`, plus a `meta_prev` for the reset. No functional logic here.

The order is SPI's four wires as a group, `sclk/mosi/ss/miso`, and then the CAN bus's three. `miso` therefore sits among the SPI ports and not with the other outputs, so that the four pins stand in the same order as in the wiring table. That matters: `can_spi_node_tb` binds

positionally like everything else, and `miso` and `rx_bus` have the same type, so swapping them passes analysis without a word and gives a node that never answers.

C.8 Test plan

The group writes no testbenches in this project. Nine are handed out, and they are the contract: seven module testbenches and two system testbenches.

Testbench	Checks
<code>meta_prev_tb</code>	Two flip-flops of delay, for both one bit and a vector.
<code>bit_timer_tb</code>	sample and <code>bit_done</code> at the right tick; <code>resync</code> ; <code>enable</code> .
<code>crc15_tb</code>	CRC values for known sequences; <code>valid</code> ; <code>clear</code> .
<code>tx_shift_reg_tb</code>	MSB-first shifting, stuffing after five identical bits, the done timing.
<code>rx_shift_reg_tb</code>	Deserialisation, destuffing, <code>stuff_error</code> , <code>real_bit</code> .
<code>register_bank_tb</code>	Eleven cases, from the reset state to <code>TX_ABORT</code> and the two routes back to TX ready.
<code>spi_reg_bridge_tb</code>	Four cases: a write, a read byte by byte, an abort on <code>ss_active</code> , and a command byte with a reserved bit set.

`can_controller_tb` puts two nodes on a shared wired-AND bus and checks that one receives the other's frame, and that arbitration is resolved correctly when both transmit at once. It is skipped by the build until the controller has a receive path (appendix A).

`can_spi_node_tb` does the same thing one layer out: two complete nodes on the same bus, but every node driven by a modelled SPI master on its four pins. It plays back the protocol specification's worked example through the whole chain and follows a frame from an SPI write on one node to an SPI read on the other. Seven cases: the reset state, the worked example, invalid command bytes and reserved indices, `TX_ABORT`, the abort rule, extra bytes under the same `SS`, and finally the transmission and the reception.

It is the **only** testbench in the course that drives `sclk`, `mosi` and `ss` as real waveforms. `spi_reg_bridge_tb` feeds the bridge finished bytes and never touches a pin, so everything between the wire and the byte — the synchronisers, the edge detection on `SCK`, the `ss_active` framing — is exercised here or nowhere.

C.8.1 Verification of your own

`can_spi_node_tb` is deliberately kind to the timing requirements: it holds 200 ns of quiet around every `SS` edge, where the protocol requires at least 60 ns. That is right for a testbench meant to separate logic errors from timing errors, but it also means that **no** testbench handed out presses the specification's timing minima.

That is the verification the project asks the group to do itself: copy `can_spi_node_tb`, shrink the quiet period towards 60 ns, and find out at what value the node stops answering and why. The answer is in `spi_slave`'s synchronisers, and it is the only place where the

course lets you *measure* a margin instead of reading one. Exercise 19.4(b) asks the question in full.

C.8.2 Checklist before hand-in

- All nine testbenches handed out pass, `can_controller_tb` and `can_spi_node_tb` included. **Check that they really ran and were not skipped**, see the box below.
- `make build-project` goes green from a clean checkout.
- The port order matches the tables in section C.6 and C.7.
- No values hardcoded past `can_def`.
- Every entity's interface is commented.
- `CONTRIBUTORS.md` is up to date.
- `main` contains no commits that did not go through a reviewed PR.

Skipped is not passed. `can_controller_tb` is gated on whether `can_controller.vhd` contains chapter 17's CRC gating, found by a text search for the internal signal names the chapter uses. If you write a correct receive path with different names, the testbench is reported as *skipped*, and since *skipped* is never failed the checklist looks satisfied while the course's pass/fail testbench never ran. So run

```
CI_BUILD_ALL=1 make build-project
```

at least once before hand-in, and hand in that outcome.

C.9 Examination and grades

The course is examined through **one group project** and **two practical exams**. 100 marks in total.

Element	Where	Form	Marks
Practical exam 1, sequential networks	chapter 9	Individual, 3 h	25
Group project, a CAN node in VHDL	chapter 10–19	Group of 4–5	50
Practical exam 2, CAN modules	chapter 20	Individual, 3 h	25

Grade	Requirement
F	Below 50 marks, or a failed project.
P	At least 50 marks in total and a passed project.
D	At least 80 marks in total and a distinction-level project.

The project is therefore a gate in both directions. It accounts for half the course, it is where most of the work lies, and a failed project cannot be compensated for with two strong exams.

C.9.1 The project's 50 marks

Part	Marks	What is assessed
The design	30	The modules work per the specification and the testbenches pass.
The way of working	12	Git history, PRs, the quality of the reviews, the seminar, CONTRIBUTORS.md.
The presentation	8	The final presentation: that the group can explain what it built and why.

The grade is given to the group. If the effort is markedly uneven, individual grades may be adjusted; that is the only reason CONTRIBUTORS.md is to be kept up to date as you go and not written the evening before.

A **passed project** requires a **complete node**: all nine testbenches handed out must pass, that is, `CI_BUILD_ALL=1 make build-project` green throughout. The controller alone is not enough, and the reason lies after the course: the node travels on to the CAN labs in the next embedded systems course, where a driver has to reach it over SPI. A controller without a register bank and a bridge cannot be reached from any driver at all, and neither can a node whose parts work separately but not together.

Distinction additionally requires the verification of your own in section C.8.1, that the code is documented throughout and stylistically consistent, and that the review culture is visible in the repository's history.

Bring-up on real hardware is **not** assessed. A broken kit or a broken connection must not cost marks. Everything that is assessed can be run in GHDL on an ordinary laptop.

C.9.2 The practical exams

Both are individual, three hours, at the computer with GHDL. The form is the same in both: independent tasks, unrelated to each other and ordered roughly by increasing difficulty.

Most of the tasks are **writing tasks**: a self-checking testbench handed out and a description of the module it drives, with port order and behaviour. Chapter 20 additionally has **reading tasks**: a timing diagram or an SPI transaction to interpret on paper. They have no testbench, for exactly the reason that makes them worth asking — reading a waveform is a skill in itself, and it is not examined by writing VHDL. They are marked against an answer key, and partly correct answers earn partial marks the same way partly correct logic does.

Partly correct logic earns partial marks: never hand in an empty file because the module did not get finished, and the same goes for a half-interpreted timing diagram. The marking reads the code, not only the testbench's outcome; a module that passes by doing something other than what was asked does not earn full marks.

Chapter 9 covers chapter 1 to 8, with the emphasis on sequential networks: registers, counters, shift registers, timers, synchronisers and simple Moore machines. A practice paper is handed out in advance. Chapter 20 covers the project's modules: bit timing, CRC, bit stuffing in both directions, frame sequencing, register semantics and the SPI transaction.

C.9.3 Permitted aids

This section is the **authoritative** list. If anything else says otherwise anywhere else, what is written here applies.

Allowed at both exams:

- All the course material: this book, the lectures' READMEs and every appendix. At chapter 20 the register map and the protocol specification (appendix D) belong there of course; at chapter 9 they are available too, but are not examined.
- All the VHDL you wrote yourself during the course, and at chapter 20 the group's project code as well.
- Your own computer with GHDL, and CircuitVerse. At both exams: drawing a circuit before you write it is often faster.

Not allowed:

- **Communication with others**, in any form.
- **Tools that generate or explain VHDL for you**: language models, code assistants and the like, whether they write the code or merely reason about it. The line is not drawn at who presses the keys but at whose understanding is being tested.
- **Searching the internet for solutions**. CircuitVerse runs in a browser, so the computer is online, and that assumes you keep to the course material under your own steam. Looking up GHDL's or VHDL's documentation is allowed; searching for a finished module is not.

If you are unsure whether something is allowed: ask before the exam, not during it.

That the group's project code is allowed at chapter 20 is a deliberate choice. You built it. The tasks are instead written as variants and repairs, so that they differ from the project's modules at precisely the point that requires you to have understood them.

C.9.4 Re-examination and absence

Both exams can be retaken. The project is handed in once, with the possibility of a resubmission after word from the supervisor: a resubmission is marked against a pass, not against a distinction.

The code review seminar in chapter 15 and the final presentation in chapter 19 are the project's two compulsory checkpoints. Missed attendance is made up by agreement with the supervisor. Other sessions have no attendance requirement; the group project, on the other hand, has a group dynamic that does not survive somebody being absent, which is a different and stronger reason to come.

C.10 Practical tips

- Draw the timing diagram before you write the shift registers. Most faults there are one-cycle timing errors, not logic errors, and they show up on paper before they show up in the simulator.

- Start with `bit_timer` and `crc15` if you are unsure where to start. They are the simplest, they depend on nothing, and they can be finished completely.
- Read the testbenches handed out. They say more precisely what every module promises than any prose does, and they are written to be read.
- Test the stuffing carefully, in both directions. It is the project's most common source of failure.
- Keep `can_spi_node` structural. As soon as logic creeps into the top level it becomes impossible to test in isolation.
- Talk to the parallel class early, and do it about the register map rather than about your code.

Shared documentation. Two documents are shared with the software group and are the contract between the classes: the register map (what the registers mean) and the protocol specification (how they are reached), both reproduced in appendix D. If anything in them changes, both classes have to know about it before the change is merged. The documents are updated *before* the code, never after.

The register map and the SPI protocol

This is the contract at the middle of the project, and it is written for **two classes at once**: this course builds the FPGA side as an SPI slave (chapter 18 and 19), and the parallel class builds the microcontroller side as master. Both halves are written against this document, not against each other's code. **If an implementation and this appendix disagree, the implementation is wrong.**

The contract says nothing about CAN as a protocol; it assumes it. *CAN: the bus, the frame and the controller* is the description, and its chapter 7 is the one explaining why this layer is needed at all: a controller signalling with one-cycle pulses cannot be polled over a serial bus.

Two things are kept apart, and it is worth knowing which is which. The *register map* (section D.1 to D.4) says **what** the registers mean. The *transaction format* (section D.6 to D.8) says **how** they are reached over the wires.

An older version of this project had the driver reach the registers via memory-mapped I/O at the base address 0xFF200000. That assumes a processor with an address bus out to the FPGA fabric, which the microcontroller in this system does not have. The registers are reached over SPI, and the **Index** column below, not the offset, is what goes in the command byte.

D.1 The registers

Index	Register	Offset	Access	Description
0	STATUS	0x00	R	Bit 0: TX ready. Bit 1: RX valid. Bit 2: Error.
1	TX_ID	0x04	R/W	11-bit transmit ID (bits 10–0).
2	TX_DLC	0x08	R/W	Data Length Code (bits 3–0, value 0–8).
3	TX_DATA_LO	0x0C	R/W	Transmit data bytes 0–3 (byte 0 = MSB).
4	TX_DATA_HI	0x10	R/W	Transmit data bytes 4–7 (byte 4 = MSB).
5	TX_SEND	0x14	W	Write 0x1 to trigger a transmission.
6	RX_ID	0x18	R	Received identifier (bits 10–0).
7	RX_DLC	0x1C	R	Received DLC (bits 3–0).
8	RX_DATA_LO	0x20	R	Received data bytes 0–3.
9	RX_DATA_HI	0x24	R	Received data bytes 4–7.
10	RX_ACK	0x28	W	Write 0x1 to acknowledge and empty the RX buffer.
11	ERROR_FLAGS	0x2C	R/W	The error register; write 0x0 to clear.
12	TX_ABORT	0x30	W	Write 0x1 to force TX ready back after an aborted transmission.

Indices 13–15 are reserved: a read gives 0x00000000, a write is ignored.

Index is Offset / 4. The offset is still in the table because the driver's constants are written in that form, but it is the index that goes out on the wire.

D.2 STATUS

Bit	Meaning	Set by	Cleared by
0	TX ready	reset; the controller's tx_done pulse; a rising edge on error while the bit is low; an accepted write to TX_ABORT	an accepted write to TX_SEND
1	RX valid	the controller's rx_valid pulse, which also latches RX_*	a write of 0x1 to RX_ACK
2	Error	a rising edge on the controller's error level	a write of 0x0 to ERROR_FLAGS

The point of the whole layer is in that table. The controller signals with one-cycle pulses, 20 ns at 50 MHz. A driver polling over SPI sees the system a few microseconds at a time at best, so a pulse is gone thousands of cycles before the next poll arrives. The register bank turns the pulses into **sticky, pollable levels with an explicit clear**.

D.3 Semantics, register by register

- **TX_SEND** triggers a single tx_req pulse when a value with **bit 0 set** is written and STATUS bit 0 is set. The other bits of the written word are ignored. If STATUS bit 0 is low the write is ignored: a transmission is in progress. A write with bit 0 low does nothing. A read gives 0x00000000, since the register stores nothing but *does* something. Writes are edge events at this layer, so a repeated write is a repeated request, never a held level.
- **RX_ACK** clears STATUS bit 1. Requires bit 0 set; other bits are ignored. A read gives 0x00000000.
- **TX_ABORT** sets STATUS bit 0 back to 1, and does nothing else. Only a write with **bit 0 set** does anything; a read gives 0x00000000. The register does not reach the controller — a controller that has aborted a transmission is already back at idle and can transmit again; it is only the bank's sticky bit that is stuck. This is the escape hatch: in a correct design it should never be needed, since bit 0 already has two automatic routes back. A driver that finds it needs this has found a bug worth reporting.
- **ERROR_FLAGS** is a latch-and-clear-on-zero construct, not a general register: only a write where **the whole word is zero** clears. 0x000000FF therefore does not clear, even though bit 0 is set. Bit 0 is the latch and mirrors STATUS bit 2; bits 31–1 read as 0. The name is plural for historical reasons.
- **Masking on write.** TX_ID stores bits 10–0 and TX_DLC bits 3–0; surplus bits read back as zero. A write of 0xFFFFFFFF therefore reads back as 0x000007FF and 0x0000000F respectively. TX_DATA_LO/HI store all 32 bits.
- **No validation.** A TX_DLC of 0xF is stored and passed on to the controller as it is. Rejecting dlc > 8 is the driver's responsibility. Doing it on both sides is exactly how the two copies of the rule drift apart.
- **No buffering on the receive side.** If a new frame arrives before the previous has been acknowledged it is overwritten, and the newest wins. That is a deliberate, documented choice and not a bug; a real controller has a receive queue.

D.3.1 Details a driver needs

The points below belong to the contract as much as the tables above.

- **Every register is 32 bits wide.** All five bytes of a transaction are always transferred, even for a register with only four meaningful bits.
- **STATUS bits 31–3 read as 0.** The bank's read multiplexer zeroes everything above bit 2, so a comparison against 0x1 is safe.
- **ERROR_FLAGS does not distinguish causes.** The four causes — lost arbitration, stuffing error, failed CRC and a received remote frame — all give the same single bit.
- **RX_DATA_LO/HI beyond RX_DLC read as 0x00.** The controller clears its data accumulator at every frame start, so bytes above the received length never carry residue from an earlier frame: a frame with DLC = 2 following one with DLC = 8 reads as two bytes of data and

six zeros. **Mask against RX_DLC all the same** — the zeros are the controller’s guarantee, not the frame’s contents, and a driver copying eight bytes reports six bytes that were never sent.

- **TX_DATA_LO/HI beyond TX_DLC are never sent**, so they need not be cleared.
- **Writes to STATUS, RX_ID, RX_DLC and RX_DATA_LO/HI are silently ignored**. The transaction completes normally; no error is reported.
- **There is no timeout to derive**. The map does not say how long a transmission is, so a driver that wants to give up has to choose its own timeout. A guide value: a maximum frame is about 130 bits, and at 1 Mbit/s therefore about 130 μ s, plus waiting for an idle bus.
- **A lost arbitration restores STATUS bit 0**, exactly as a successful transmission does. A driver therefore does not have to distinguish the outcomes in order to transmit again: poll bit 0, and read ERROR_FLAGS to find out how it went.

What “the transmission is over” means here. STATUS bit 0 coming back says that the transmit port is free again, not that the frame was delivered. A lost arbitration gives back the same bit as a clean transmission. ERROR_FLAGS is what tells them apart, and the ordinary polling loop is therefore: wait for bit 0, then read ERROR_FLAGS to decide whether the frame should be resent. Section C.6.1 lists the other simplifications against real CAN.

D.4 Data byte order

tx_data and rx_data are 64 bits wide, with data byte 0 in the most significant bits:



The names LO and HI refer to the register pair’s order, not to bit significance. TX_DATA_LO carries the *first* four data bytes.

D.5 Counterparts in the hardware

Register	Port on <code>can_controller</code>
TX_ID, TX_DLC, TX_DATA_LO/HI	<code>tx_id</code> , <code>tx_dlc</code> , <code>tx_data</code>
TX_SEND	<code>tx_req</code> (a pulse)
STATUS bit 0	<code>tx_done</code> (a pulse), latched into a level
RX_ID, RX_DLC, RX_DATA_LO/HI	<code>rx_id</code> , <code>rx_dlc</code> , <code>rx_data</code>
STATUS bit 1	<code>rx_valid</code> (a pulse), latched into a level
STATUS bit 2, <code>ERROR_FLAGS</code>	<code>error</code> (a level), latched on a rising edge

The translation between the two columns is exactly what `register_bank` does, and nothing more.

D.6 Electrical parameters and framing

Parameter	Value
Roles	The microcontroller is SPI master, the FPGA slave.
Mode	SPI mode 0 (CPOL = 0, CPHA = 0): SCK idles low, both sides sample on the rising edge.
Bit order	MSB first, in every byte.
SCK frequency	≤ 1 MHz (the AVR32DB28 runs at 4 MHz and divides by 4). See section D.8 for why.
SS	Active low. Framing signal: one transaction per low period.
Voltage	The AVR32DB28's core runs at 5 V, the DE0-CV at 3.3 V, and no wire passes through a level shifter. The four SPI wires sit on P0RTC, the AVR DB's second supply domain, with VDDI02 supplied from the DE0-CV's 3.3 V rail.

D.6.1 Pin assignment

Both sides are fixed here. The microcontroller side uses SPI0's **ALT1** pin mux on the AVR32DB28, which puts the peripheral on PC0–PC3; the FPGA side is free, and this section spends that freedom on making the boards interchangeable.

ALT1 is no arbitrary choice. P0RTC is the AVR DB's **MVIO** domain: it is supplied from VDDI02 rather than from VDD, so tying VDDI02 to the DE0-CV's 3.3 V rail puts all four SPI wires in a 3.3 V domain while the core goes on running at 5 V. Every level the FPGA sees is then a level it is specified for, and the 3.3 V the FPGA drives back is read against a 3.3 V threshold rather than a 5 V one. On SPI0's default mux the same four signals would sit on PA4–PA7 in the VDD domain, and the bench would need a level shifter to be safe. That is why the mux is part of the contract rather than every group's own choice.

The microcontroller side therefore owes two things beyond the pin numbers, and both belong to the driver class: `PORTMUX.SPIROUTEA` has to select `ALT1`, and `VDDIO2` has to be powered before `PORTC` does anything at all. `MVIO.STATUS` reports whether it is.

This constrains the board, not just the firmware. The hat carrying the AVR32DB28 is built for this system, and the pin table below places three requirements on its layout. `PC0–PC3` carry `SPI0` and nothing else — no LED, no button, and above all no debouncing capacitor on `SS`, which at 1 MHz would make the framing edge unusable. Nothing else sits anywhere on `PORTC`, since the whole port is a 3.3 V domain as soon as `VDDIO2` is. And `VDDIO2` is routed as a net of its own out to a pin, decoupled with 100 nF near the package, rather than tied to `VDD` on the board. None of the three can be fixed in firmware once the board is etched.

A single fuse decides whether any of the above is true. `MVIO`'s dual-supply mode is selected by `MVSYS_CFG` in `FUSE.SYS_CFG1`. In single-supply mode `VDDIO2` is tied internally to `VDD`, `PORTC` becomes a 5 V port, and the wiring in this section then drives 5 V into FPGA inputs that cannot take 5 V — while the bench looks exactly like a correct one. Confirm the fuse against the datasheet before the first node is connected, not after the first board has behaved oddly.

Signal	AVR32DB28	DE0-CV port	DE0-CV pin
<code>SCK</code>	<code>PC2</code>	<code>sclk</code>	<code>GPIO_0(4)</code>
<code>MOSI</code>	<code>PC0</code>	<code>mosi</code>	<code>GPIO_0(5)</code>
<code>MISO</code>	<code>PC1</code>	<code>miso</code>	<code>GPIO_0(6)</code>
<code>SS</code>	<code>PC3</code>	<code>ss</code>	<code>GPIO_0(7)</code>
I/O supply	<code>VDDIO2</code>	—	3.3 V
Ground	<code>GND</code>	—	<code>GND</code>

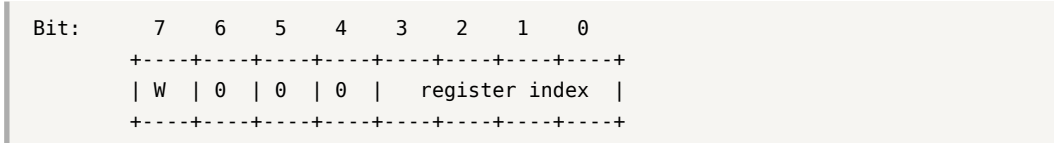
`GPIO_0(0)` to `GPIO_0(2)` are reserved for the CAN side — `tx_bus`, `bus_en` and the wired-AND bus line — and `GPIO_0(3)` is deliberately left unused as a gap between the two groups, so that a cable loom sitting one pin off does not short anything between the domains.

Why fix them at all, when Quartus does not care? Nothing in the design breaks if a group picks other pins; the point is what happens at the bench. With an assignment shared by both classes, any microcontroller node fits any FPGA board: a cable loom can be built once instead of per pair, and a software group whose partner board is not ready can borrow a working one instead of falling back on simulation. That last case is not hypothetical — the two courses hold their bring-up and their final presentations the same week.

A group *may* use other pins, but only if the board and the node it is paired with agree, and only if it is written down where the other class can see it. An undocumented pin change shows up as a node that answers register reads and never sends a frame, which is an expensive way to discover a wiring convention.

D.7 Transactions

Every transaction is exactly **5 bytes** long: one command byte, then four data bytes. SS falls before the command byte and rises after the fifth byte; it has to stay low for the whole transaction.



- **Bit 7 (W):** 1 = write, 0 = read.
- **Bits 6–4:** reserved, must be 0.
- **Bits 3–0:** the register index, offset / 4 from section [D.1](#) (0–12).

Write (W = 1).. The master sends the register value in the four data bytes, **MSB first** (bits 31–24 in the first data byte). The slave commits the assembled value to the addressed register only once the fifth byte is complete. Whatever the slave drives on MISO during a write is meaningless; the master discards it.

Read (W = 0).. At the end of the command byte the slave **latches** the addressed register’s current value **once**, into its response shift register. The four data bytes then clock that value out on MISO, MSB first, while the master sends dummy bytes 0x00. The latch-once rule is not an implementation detail but part of the contract: the four bytes always belong to *one* coherent sample of the register, taken at a single instant — STATUS in particular must never be stitched together from two separate moments.

Abort.. If SS rises before the fifth byte is complete, the transaction is abandoned **with no side effects**: nothing is committed, no trigger fires, and the slave returns to idle, ready for the next falling edge on SS. That is what makes the slave self-healing after a glitch or a master that resets.

Invalid commands.. A read of a reserved index (13–15) gives 0x00000000; a write to one is ignored. A command byte with any reserved bit (6–4) set is invalid in the same way: no read is latched and no write is committed, and the slave still consumes the whole five-byte frame before returning to idle. Neither case is an error the slave reports — there is no error channel at this layer. Enforcing the rule about reserved *command* bits is spi_reg_bridge’s job; register_bank never sees more than a 4-bit index.

D.8 Timing requirements

- The slave **never** uses SCK as a clock. All three inputs (SCK, MOSI, SS) are synchronised into the 50 MHz domain with two flip-flops each, and SCK edges are *detected*, not clocked on (chapter 4 for why, chapter 18 for the module that does it). spi_slave builds its

synchroniser in place rather than instantiating `meta_prev`, so that `spi_slave` depends on nothing in `controller/`.

- Two-flip-flop synchronisation plus edge detection costs a few 50 MHz cycles per edge; at $SCK \leq 1$ MHz there are ≥ 50 system cycles per SPI bit, an order of magnitude of margin. The protocol therefore guarantees correct operation only up to 1 MHz; faster may work, and is deliberately not promised.
- MISO changes on SCK's falling edge and is sampled by the master on the rising one, as mode 0 prescribes.
- The master has to raise SS between transactions. Back-to-back transactions under a single low SS are not supported, and attempting it is harmless: **a transaction begins only on the first byte after SS has fallen**, so a sixth byte arriving while SS is still low is discarded rather than taken for a new command byte.

D.8.1 Setup and hold around SS

Since SS is synchronised and then edge-detected rather than used directly, it needs a few system clock cycles to take effect. At 50 MHz one cycle is 20 ns, and the numbers are:

Parameter	Minimum	Why
SS low before the first rising SCK edge	60 ns (3 cycles)	the frame-start edge is detected on the third system clock edge after SS falls
SS high between transactions	60 ns (3 cycles)	the same detector has to see the line return before it can be rearmed
SS low after the last SCK edge	60 ns (3 cycles)	so that the last byte is registered before the frame is torn down

Allow 100 ns for each of them in a driver and none of this is close. An AVR toggling SS through `PORTC.OUTCLR` and `PORTC.OUTSET` around a byte written to `SPI0.DATA` is already far slower than that, so the limits matter only if the master drives SS from hardware, or runs much faster than the AVR32DB28 does.

There is **no** minimum gap between bytes: bytes may come back to back within a transaction, and the slave does not care how long the master pauses between them, as long as SS stays low.

D.8.2 MISO outside a read

- MISO is driven **low** while the slave is deselected, it is not released to high impedance. The design assumes a single slave on the bus. Adding a second would require making this output tri-stated.
- The byte the master clocks out **during the command byte** is meaningless: the slave loads its response shift register at the command byte's *end*, so what leaves during it is whatever happened to remain. Discard it.
- During a write all four MISO data bytes are meaningless too.

D.9 Worked example

Sending a frame (id 0x123, dlc 2, data AA BB) is five transactions, and then a poll:

```
Write TX_ID      : 81 00 00 01 23
Write TX_DLC     : 82 00 00 00 02
Write TX_DATA_LO : 83 AA BB 00 00
Write TX_DATA_HI : 84 00 00 00 00
Write TX_SEND    : 85 00 00 00 01
Read STATUS     : 00 xx xx xx xx  -> MISO gives 00 00 00 00 while the transmission runs,
                                   then 00 00 00 01 once tx_done has set TX ready again.
```

Note 83 AA BB 00 00: byte 0 in the most significant position. That is the packing on both sides of the wire — the driver assembles it, `register_bank` presents `TX_DATA_LO` as the *first* four bytes, and `can_controller` reads `tx_data` with byte 0 in bits 63–56. The diagram in section D.4 is the picture of that, and `register_bank_tb` checks it.

Worth reading off the same example: the first four MISO bytes of a write are meaningless, and so is the MISO byte returned *during* any command byte. Discard it.

Changing this contract. The register map and the protocol are a contract between two classes that cannot compile against each other. If anything changes, both sides have to know about it before the change is merged. In practice: changes are discussed with both supervisors first, and the documents are updated *before* the code, never after.

Further reading

This book ends with a CAN node a microcontroller can talk to, and it ends there deliberately: the design is simplified at several deliberate points (section 19.7), and the driver on the other side of the SPI link is written by somebody else. This appendix says where the basics the book assumes can be revised, where the protocol itself is described, which standards the design relates to, and where to go next.

E.1 The basics

The book assumes the basics of digital electronics and revises them only where the language calls for it. *Digital Electronics* (<https://github.com/qrotech-academy/digital-electronics>) is the book to turn to when they need brushing up. It is freely available, and it ends roughly where this book begins: with a state machine built from gates and flip-flops, simulated in the same CircuitVerse.

Six chapters plus appendices, and each chapter corresponds to something this book takes for granted:

Ch.	Subject	Here in this book
1	Logic gates, and the transistors and the delay inside them	section 1.1, section 5.4
2	Boolean algebra, De Morgan, number systems and codes	section 1.2
3	Karnaugh maps and minimisation	section 2.1, section 8.2
4	Combinational building blocks: multiplexer, decoder, comparator, adder	section 2.4
5	Flip-flops, registers, metastability and synchronisation	chapters 3 and 4
6	Counters, timers and state machines	chapters 6, 7 and 8
A	Answers to the exercises	—
B	CircuitVerse	section 1.3

The appendices after B are two written papers with model answers. They are the quickest way to test whether the prerequisites are in place before chapter 1, and sitting one of them

against the clock shows more clearly than a read-through which of the chapters above are worth revising.

E.2 The protocol reference

CAN: the bus, the frame and the controller is the book this one leans on every time CAN as a *protocol* comes up. It is short and written for both halves of a CAN system: the one building the controller in hardware and the one writing the driver in software. It therefore mentions neither course nor class, and it is deliberately independent of which of the two courses is reading it.

It exists in two editions with the same chapter and section numbering, so every reference in this book holds for both: in English at <https://github.com/Yrgo-26/can-book/blob/main/en/can-en.pdf>, and in Swedish, as *CAN: bussen, ramen och kontrollern*, at <https://github.com/Yrgo-26/can-book/blob/main/sv/can-sv.pdf>.

Seven chapters plus an appendix, and the chapter division is this book's part II seen from the protocol side:

Ch.	Subject	Here in this book
1	The bus: topology, dominant and recessive, wired-AND, termination	section 10.2
2	The frame: the fields, the identifier, RTR, DLC, the acknowledgement	section 10.4
3	Bit stuffing and CRC-15	section 10.5, chapter 13, 14 and 15
4	Arbitration, bit by bit	section 10.6, section 17.6
5	Bit timing and synchronisation	section 10.7, section 12.2
6	Error handling: error frames, error counters, bus-off	section 19.7, as a gap
7	The simplified controller, and the three limitations that reach all the way up	section 19.7, section C.6.1, appendix D
A	Answers to every exercise	—

Three things make it worth having open beside this book rather than read once:

- **Chapters 1–6 are the requirement specification, bit by bit**, for the controller part II builds. Where this book says “the rule is five identical bits in a row”, that book says why the rule exists and what breaks without it.
- **Chapter 7 describes exactly this design**, from outside: the blocks, what it implements of the standard, what it deliberately leaves out, why a register layer is needed, and the three limitations that reach all the way up into the driver's code. Anyone who wants the whole picture in five pages before chapter 11's architecture reads that chapter first.
- **Four exercises per chapter, all pen and paper, all answered in appendix A**. They test the protocol understanding without a line of VHDL being involved, which makes them the best preparation there is for chapter 20's reading tasks.

The books share an author and are typeset the same way, so a reference looks the same in both. They do number their chapters independently of each other, however: “chapter 3” with no further qualification always means a chapter in *this* book, and a reference to the other is always written out in full.

E.3 Standards and specifications

The book relates to real CAN throughout without being it, and *CAN: the bus, the frame and the controller* is the intermediate step: it describes the standard at the level of detail a designer needs. If you then want to see what the original actually says:

- **Bosch CAN Specification 2.0** (parts A and B) is the base document: the frame formats, the bit stuffing, the CRC polynomial and the arbitration. Part A is the standard frame with an 11-bit identifier, that is, this book’s frame; part B adds the extended 29-bit form.
- **ISO 11898-1** is the standardisation of the same thing, with the data link layer and the physical signalling interface. ISO 11898-2 is the high-speed transceiver, that is, the device that would sit between tx_bus/bus_en and a real differential bus.
- **IEEE 1076-1993** is VHDL-93, the language standard - -std=93 selects. It is worth reading as a reference work rather than cover to cover; the packages `ieee.std_logic_1164` and `ieee.numeric_std` are the two parts you have used throughout the book.

The four points where the book’s controller deliberately departs from the standard are collected in section 19.7, and section C.6.1 says which three of them reach all the way out into the register map.

E.4 The tools

- **GHDL’s documentation** covers considerably more than the three commands appendix A needs: waveform output to VCD or GHW, code coverage, and the other language standard flags. **GTKWave** is the usual waveform viewer to pair it with, and the first time you debug a timing bug by *looking* at the signals instead of reading report lines it is worth its installation time.
- **Quartus Prime Lite’s own documentation**, in particular the timing analysis: appendix B settles for one `create_clock`, while a real design also needs input and output constraints and exceptions for crossing clock domains.
- **The DE0-CV’s user manual** from the board manufacturer has the complete pin table, the peripherals and the schematic. It is the answer key when `DE0_CV.qsf` does not cover something you want to use.
- **CircuitVerse** is free and runs in the browser. It is still the fastest way to try a small combinational idea, and chapter 9 and 20 allow it for exactly that reason.

E.5 Further into digital design

This is a book about one system, not about the whole subject. Four directions that lie close by and for which material is not lacking:

- **Timing analysis and clock domains in earnest.** Chapter 4 gives the two-flip-flop synchroniser and section 5.4 a first Fmax. The next step is setup and hold analysis, FIFOs between clock domains, and Gray-coded pointers — that is, what you do when a whole bus, and not a single bit, has to pass between two clocks.
- **Verification beyond a hand-written testbench:** self-checking models, random stimuli and coverage metrics. VHDL's OSVVM and UVVM are the two common frameworks, and the step from there to SystemVerilog and UVM is the one industry takes.
- **Other interfaces, with the same method.** The book builds CAN; UART, SPI, I²C and Modbus all have the same shape — a bit timer, a shift register, a sequencer — and building one more of them with the book's patterns is the fastest way to notice how much of chapter 11's split was common property.
- **Processor design.** A small RISC core is the natural next step for anyone who found the state machine in chapter 16 the most interesting part: the same building blocks, an instruction decoder instead of a frame decoder, and a register file interface instead of a bus.

What all four have in common is that they rest on the same five patterns chapter 20 lists. That is the real takeaway from this book: not CAN, but that you can build the next protocol too.